

ANTSDR 开发手册

Matlab & Simulink 篇

Rev. 1.1



1. 版本记录

版本	时间	描述
Rev. 1.0	2021.6.23	第一版发布
Rev. 1.1	2021.7.28	增加了硬件支持包的离线安装

2. 版权声明:

XME/IME/SDR 系列核心模块、评估板及其相关知识产权由微相科技有限公司所有。

本文档由微相科技有限公司版权所有，并保留一切权利。在未经微相公司书面许可的情况下，不得以任何方式或形式来修改、分发或复制本文档的任何部分。

3. 免责声明:

产品中所提供的程序源代码、软件、资料文档等，微相科技不提供任何类型的担保；不论是明确的，还是隐含的，包括但不限于合适特定用途的保证，全部的风险，由使用者来承担。

前 言

软件无线电 SDR 的定义是大多数的物理层功能都使用数字信号处理算法实现，理想的 SDR 接收机的组成部分是很少量的硬件前端、天线和高速 GHz 的采样器，采样器能够捕获大范围的无线信号并转换为数字信号。任何的解调、同步、解码、解密都是用软件来实现，这个设备的处理速度必须非常快。

目前大多数智能手机及类似的设备都拥有多达 8 种无线模块，用于处理不同频段的信号，比如 WiFi(2.4GHz)、LTE(800MHz)、GSM(900MHz)、UMTS(2.1GHz)、GPS(1.5GHz)、蓝牙(2.4GHz)、NFC(13.56MHz)和 FM(100MHz)，将来可能还要包括 IoT 和电视频段 UHF(400MHz)。最终的解决方案是用一个单一的 SDR 以 GHz 的带宽来采样，并且把这些信号进行数字化，把从基带到 3GHz 的所有信号都直接用软件来解调。

软件无线电的主要特点如下：

具有很强的灵活性：软件无线电可以通过增加软件模块，很容易增加新的功能。可以与其他任何电台通信，并且可以作为其他电台的射频中继。为了减少开支，可以根据所需功能的强弱，取舍选用合适的软件模块。

具有较强的开放性：软件无线电由于采用了标准化模块化的结构，其硬件可以随着器件和技术的发展而更新或扩展，软件也可以随着需要而不断升级。

由于软件无线电这一概念的提出，就受到了全世界无线电领域的广泛关注。由于软件无线电所具有的灵活性开放性等特点，使其不仅在军、民无线通信中获得了使用，在其他领域，比如电子战、雷达、智能家电等领域得到推广。

目 录

1.	版本记录.....	2
2.	版权声明:	3
3.	免责声明:	3
前	言.....	4
第 1 章	ANTSDR 简介	11
1.1	基本信息	11
1.2	基本结构	11
1.3	ANTSDR 的应用	13
第 2 章	安装硬件支持包.....	14
2.1	安装驱动	14
2.2	安装 Matlab	19
2.3	安装硬件支持包	19
2.4	硬件安装包离线下载	22
2.5	测试 ANTSDR 连接	25
第 3 章	Matlab & Simulink 的基本使用.....	31
3.1	Matlab 介绍	31
3.1.1	Matlab 的界面	31
3.1.2	Matlab 命令行使用	31
3.1.3	在命令行中使用函数	32
3.1.4	获取某个函数的帮助信息	33
3.2	Matlab 脚本	34
3.3	Matlab 函数	36
3.4	Matlab 绘图	38
3.5	Matlab 的数组、矩阵和结构	41
3.5.1	Matlab 中的数组	41
3.5.2	Matlab 当中的矩阵	42

3.6 Matlab 系统对象	45
3.6.1 创建系统对象	45
3.6.2 使用系统对象	46
3.7 Simulink 简介	47
3.7.1 新建 Simulink 工程	47
3.7.2 Simulink 当中的变量和参数	57
3.7.3 观察频域图像	60
3.7.4 数据类型	64
3.7.5 从文件输入和输出到文件	66
3.8 小结	69
第 4 章 信号的调制与解调	70
4.1 实数信号和复数信号	70
4.1.1 简单的三个正弦信号在频域的表示	71
4.2 幅度调制	73
4.2.1 抑制载波的双边带调幅(AM-DSB-SC)与解调	74
4.2.2 正交幅度调制与解调(QAM)	76
4.2.3 使用复数来表示 QAM	78
4.3 小结	82
第 5 章 AM-DSB-TC 调制与非相干解调	84
5.1 常规调幅原理	84
5.1.1 单音基带信号调制	84
5.1.2 基带信号的调制	86
5.2 使用 Simulink 仿真常规调幅	87
5.3 常规调幅的非相干解调原理	90
5.3.1 一般情况下的包络检波	90
5.3.2 实数解调的优化	91
5.3.3 复包络检波	91

5.4	常规调幅的非相干解调仿真	92
5.5	使用 ANTSDR 发送音频信号	94
5.6	使用 ANTSDR 接收并解调信号	96
5.6.1	ANTSDR 进行下变频	96
5.6.2	使用 ANTSDR 解调音频信号	97
5.7	总结	100
第 6 章	常规调幅的相干解调.....	101
6.1	相干解调和载波同步	101
6.1.1	有误差的解调	101
6.1.2	Simulink 仿真有误差的解调.....	102
6.2	PLL 简介	104
6.2.1	鉴相器	105
6.2.2	环路滤波器	109
6.2.3	NCO/VCO.....	113
6.2.4	设计一个 2 类 PLL.....	115
6.3	常规调幅的相干解调	124
6.4	ANTSDR 相干解调 AM-DSB-TC.....	126
6.5	总结	128
第 7 章	AM-DSB-SC 调制	129
7.1	AM-DSB-SC 调制的原理	129
7.1.1	AM-DSB-SC 调制单音信号	129
7.1.2	AM-DSB-SC 调制多个单音信号	130
7.2	使用 Simulink 仿真 AM-DSB-SC	131
7.3	AM-DSB-SC 解调	132
7.4	使用 Costas 环解调 AM-DSB-SC 信号	133
7.5	ANTSDR 进行 AM-DSB-SC 调制与解调.....	135
7.5.1	使用 Costas 环进行解调	135

7.5.2 直接得到基带信号	136
7.6 总结	136
第 8 章 实现 AM 的多路复用	138
8.1 音频信号 AM 复用	138
8.2 AM FDM 仿真	140
8.3 音频信号 AM 解复用	142
8.4 音频信号 AM 解复用	143
8.5 使用 ANTSDR 完成 AM FDM 的调制与解调	144
8.6 总结	146
第 9 章 FM 调制与解调（微分器）	147
9.1 FM 调制的数学原理	147
9.1.1 FM 信号产生的过程	147
9.1.2 FM 调制一个单音信号	148
9.1.3 FM 调制一个基带信号	149
9.1.4 宽带调频与窄带调频	149
9.2 使用微分的方式对 FM 信号进行解调	158
9.2.1 使用求导的方式对 FM 进行解调	158
9.2.2 下变频信号到复数基带	159
9.2.3 使用复数微分鉴别器解调 FM	160
9.2.4 数字复数微分鉴别器	163
9.3 使用 ANTSDR 发送 FM 信号	164
9.3.1 ANTSDR 发送产生 FM 信号原理	164
9.3.2 使用 ANTSDR 发送 FM 信号	166
9.4 ANTSDR 使用差分鉴相器解调 FM 信号	169
9.5 总结	171
第 10 章 FM 非相干解调（延迟器）	172
10.1 复数延迟鉴别器原理	172

10.2 ANTSDR 复数延迟解调	174
10.3 总结	175
第 11 章 FM 调频立体声	176
11.1 FM 立体声的生成原理	176
11.2 FM MPX 信号仿真	178
11.3 使用 ANTSDR 发送立体声信号	179
11.4 ANTSDR 解调 FM 立体声	181
11.5 ANTSDR 同时调制和解调 FM 信号	183
11.6 总结	184
第 12 章 FM 频分复用	185
12.1 FM 频分复用原理分析	185
12.2 使用 ANTSDR 发送 FM FDM 信号	187
12.3 使用 ANTSDR 接收 FM FDM 信号	189
12.4 总结	191
第 13 章 数字通信基础.....	192
13.1 数字调制	192
13.1.1 数字相位调制 PSK	192
13.1.2 数字幅度调制	194
13.1.3 Bit 与符号的映射与反映射	195
13.2 符号映射与反映射仿真	197
13.2.1 QPSK 映射与反映射	197
13.2.2 16QAM 的映射与反映射	205
13.3 脉冲成型	210
13.3.1 根升余弦滤波器的仿真	211
13.3.2 根升余弦滤波器对符号之间的影响	214
13.4 粗频率校正	217
13.5 符号同步	221

13.6	载波同步	224
13.7	使用 ANTSDR QPSK 调制与解调.....	228
13.8	总结	235
第 14 章	使用 QPSK 发送 ASCII 码.....	236
14.1	打开官方 QPSK 仿真模型.....	236
14.1.1	发射模块	237
14.1.2	信道模块	239
14.1.3	接收模块	239
14.2	使用 ANTSDR 来发送消息	242
14.3	总结	244
总	结.....	245
参	考.....	246

第1章 ANTSDR 简介

1.1 基本信息

ANTSDR 是由微相科技(上海)有限公司推出的基于 AD936x 系列芯片的一个开源 SDR 平台。该系列 SDR 设备的灵感来源于 ADI 官方推出的 SDR 主动学习模块 ADALM-PLUTO。微相科技在官方所提供的固件源码基础上,对固件进行移植,使得 PlutoSDR 的固件能够运行在 ANTSDR 上,与此同时在 ANTSDR 上,也对 PlutoSDR 的固件进行了修改,相较于 PlutoSDR 能够提供更强大的性能,方便用户的使用和集成。

关于 ANTSDR 的固件源码,可以到 MicroPhase 的 Github 当中获取,链接地址如下: <https://github.com/MicroPhase/antsdr-fw>.

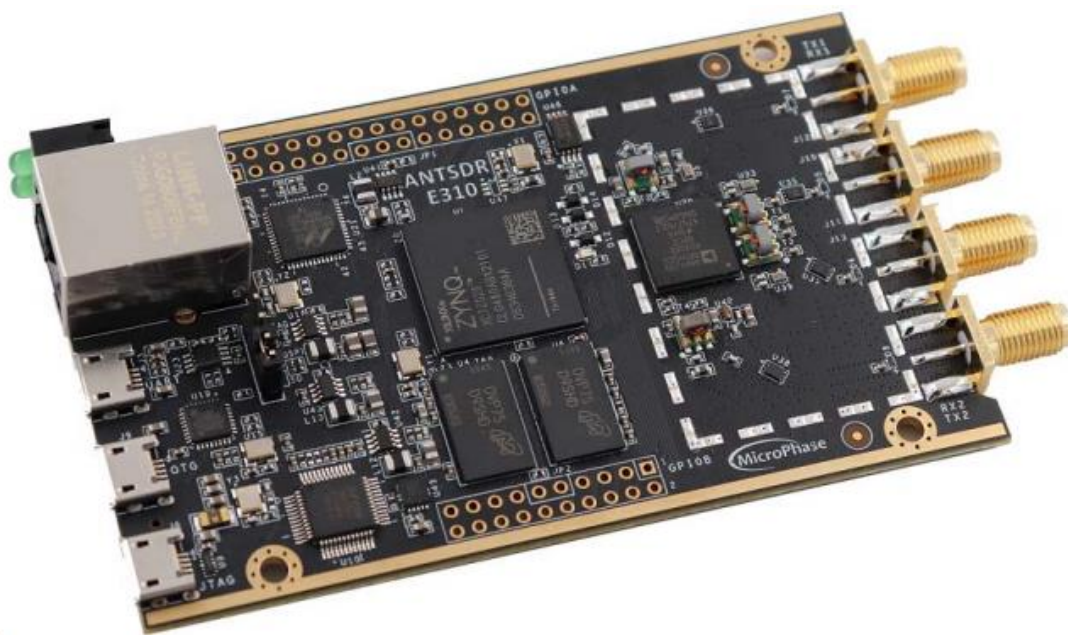


图 1-1 ANTSDR E310 实物图

1.2 基本结构

ANTSDR 的基本硬件结构如图 1-2 所示

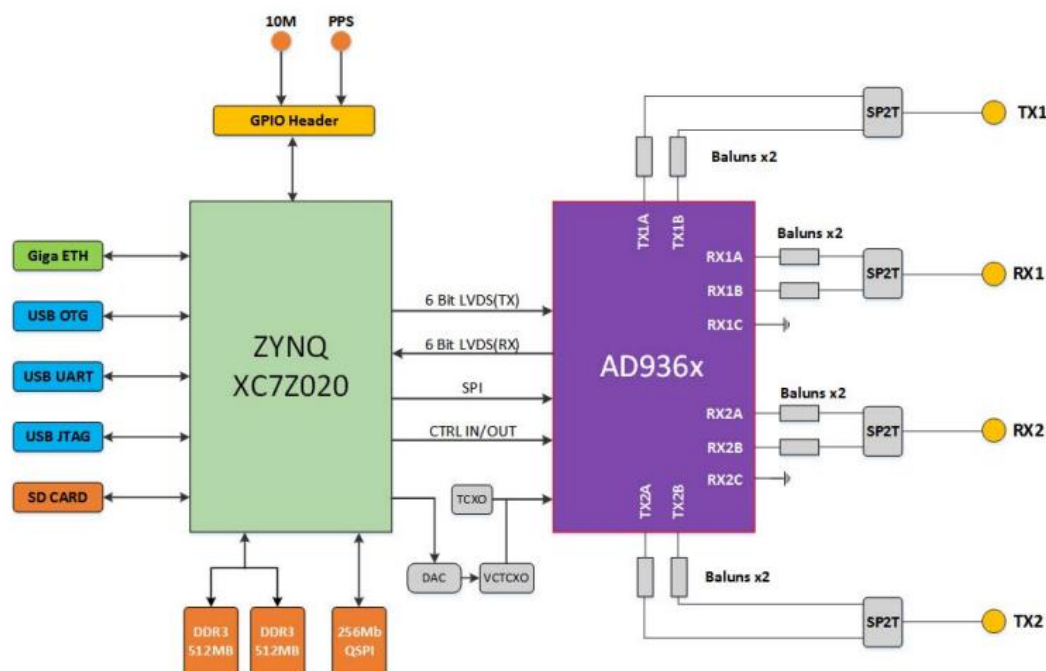


图 1-2 ANTSDR 硬件结构

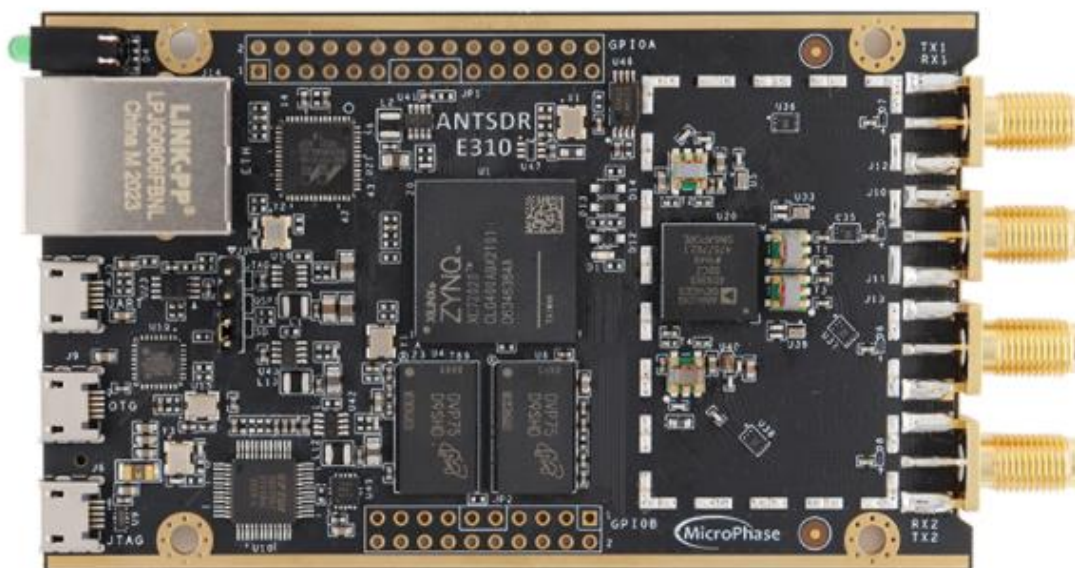


图 1-3 ANTSDR 硬件接口

从图中可以看到，在 ANTSDR 设备上，最主要的有两大部分，一个是射频 IC，采用的是 ADI 推出的 AD936x 系列芯片，ANTSDR 上可以使用 AD9363 和 AD9361 进行替换。一个是 BBP 单元，采用的是 Xilinx 推出的 ZYNQ7020，该芯片相较于 PlutoSDR 上的 BBP 单元能提供更多的逻辑单元，方便用户设计和实现自己的基带处理算法。

从 ANTSDR 的射频前端可以看到，在微相科技对射频前端信号采用了分频段的处理方式，这样能够有效的减小在高频范围内的信号衰减。

ANTSDR 上含有丰富的外设，具体包括千兆以太网，USB OTG, USB Uart, USB Jtag 和 SD Card。除此之外还引出了外围的 GPIO 以便用户可以连接扩展设备。千兆以太网和 USB OTG 能够提供高速的数据传输，通过该接口能够将 ANTSDR 接入到丰富的 SDR 软件社区，从而实现 SDR 的真正实力，作为一个通用 SDR 平台，只需要在软件上进行设计和实现不同的功能，就能够通过 ANTSDR 完成不同类型的信号的调制与解调。USB Uart 和 USB Jtag 能够帮助基带开发人员实时地对 ANTSDR 进行调试，开发和设计基带处理算法。SD Card 的增加，能够方便用户在 ZYNQ 上运行功能更完善的 Linux 系统，ANTSDR 能够进一步的集成到用户的产品当中。

1.3 ANTSDR 的应用

使用 ANTSDR，用户能够很方便地接入到各种 SDR 软件当中，可以尝试各种有趣的无线电实验。比如在 GNU Radio 和 Matlab&Simulink 中，用户可以在软件上实现通信和信号处理相关的算法，然后通过 ANTSDR 来实现信号的发射和接收。又或者可以使用开源的软件，来进行诸如广播解调，电视信号解调，飞机追踪等有意思的功能。



图 1-4

第2章 安装硬件支持包

本章将进行 PlutoSDR 硬件支持包的安装，在 Matlab 相关的实验都是在 windows 10 平台下进行的，因此需要安装 win10 下的驱动，这样我们才能够正确地使用到 ANTSDR 硬件设备。

关于 Matlab & Simulink 的下载和安装，用户可以在 Matlab 官网注册并下载安装，在本教程当中不再介绍安装过程。在本文档中所使用的 Matlab 版本是 2018.a。

2.1 安装驱动

PlutoSDR 的 windows 驱动官方安装教程如下：

<https://wiki.analog.com/university/tools/pluto/drivers/windows>

在资料当中也提供了对应的驱动，双击这个 exe 文件进行安装即可，一直点击下一步就能安装好驱动。

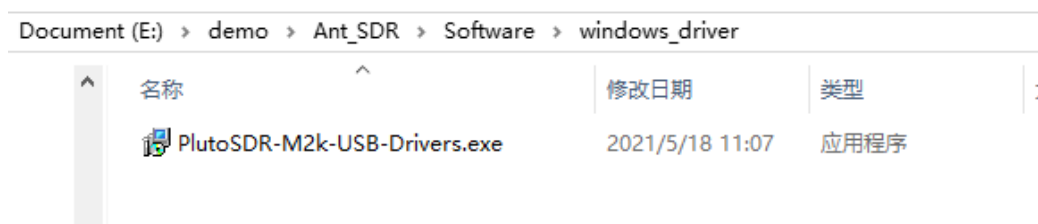


图 2-1 PlutoSDR windows 下驱动

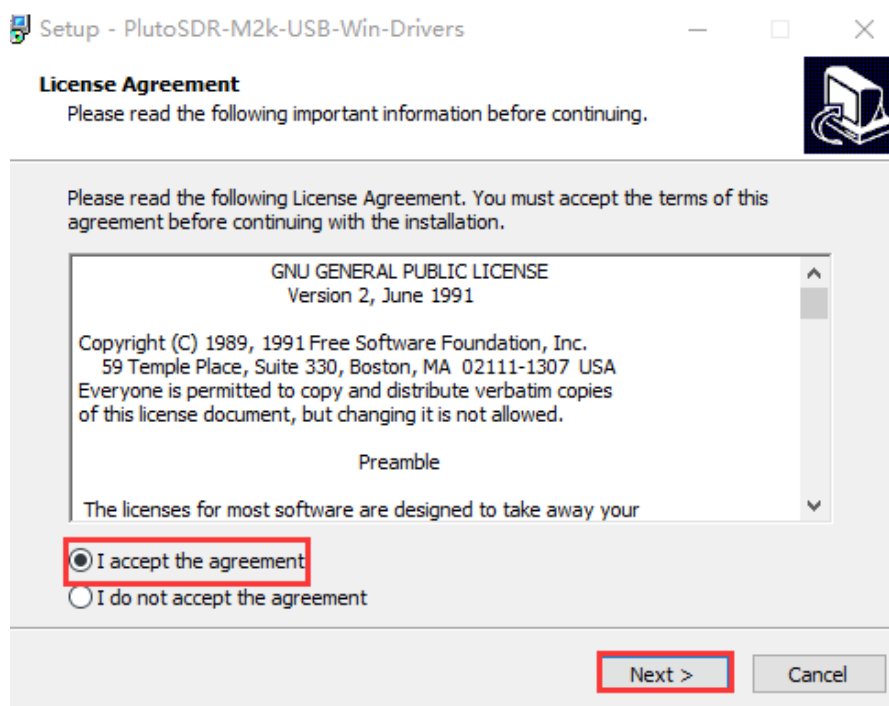


图 2-2

接下来选择自己的驱动安装路径即可。

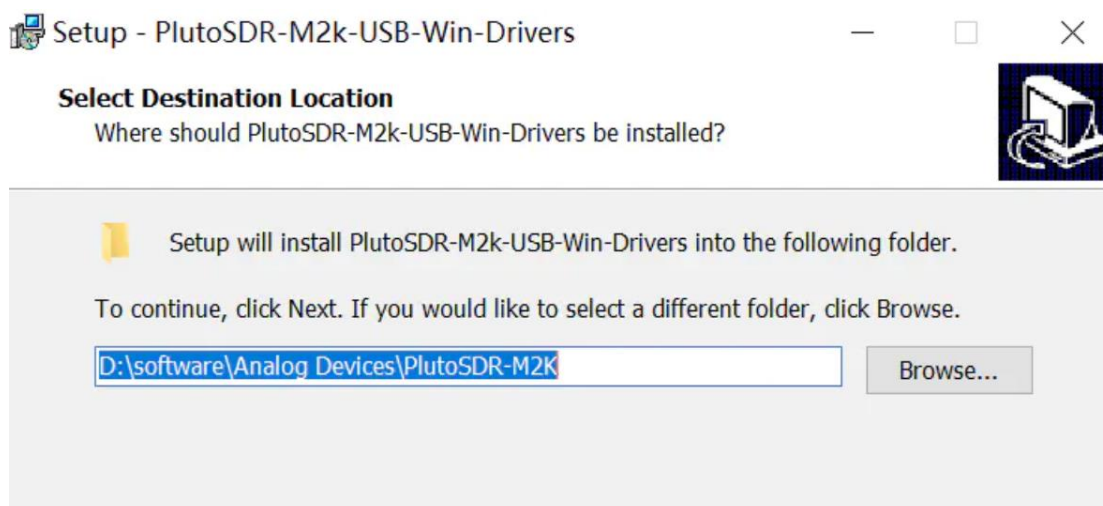


图 2-3 选择安装路径

接下来一直点击下一步即可，知道驱动安装完成。驱动安装完成之后，将 ANTSDR 的供电口和 USB OTG 通过 usb 线连接到电脑上，此时打开设备管理器，能够在设备管理器当中看到如图 2-4 所示设备。



图 2-4 PlutoSDR 显示的设备

ANTSDR 使用的处理器是 ZYNQ，该处理器包含了两个 A9 的核，能够在上面运行操作系统，ANTSDR 移植了 PlutoSDR 的固件，因此我们也能够通过命令行的方式来访问到 ANTSDR 上的硬件设备。

使用串口终端软件，笔者在这里使用的 MobaXterm，通过串口终端能够访问到 ANTSDR 上的资源。

MobaXterm 的安装包也提供在软件当中了，解压之后进行安装即可。

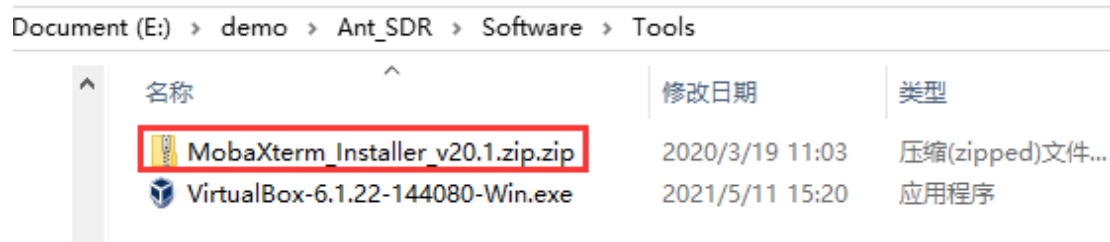


图 2-5

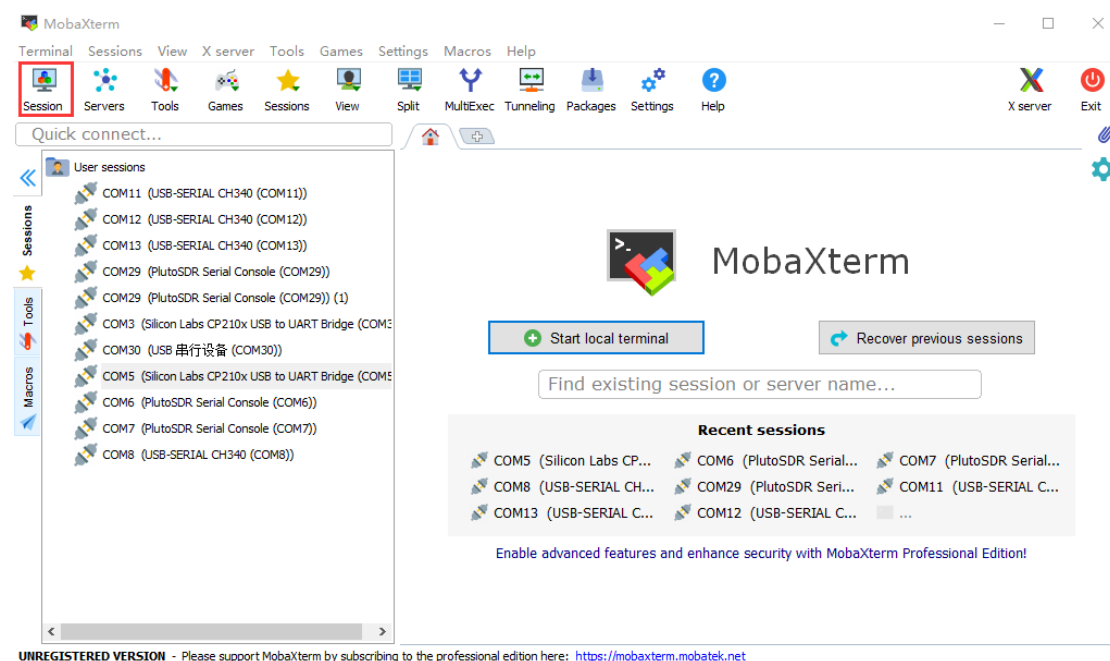


图 2-6 新建连接

打开 MobaXterm 之后，首先需要新建一个连接，点击左上角 Session

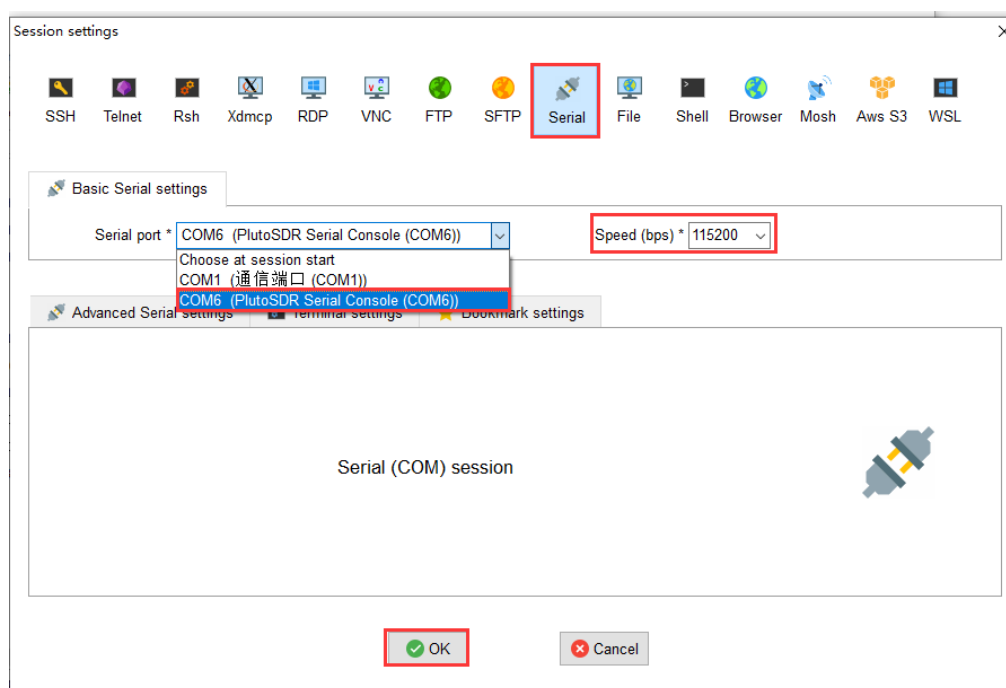


图 2-7 新建立终端连接

这时候使用可以点击刚才创建好的终端连接进入到登录窗口。

登录的用户名为 **root**, 登录的密码是 **analog**。

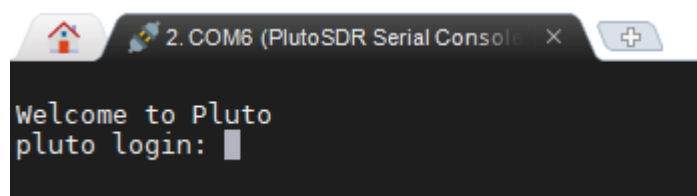


图 2-8 PlutoSDR 登录窗口

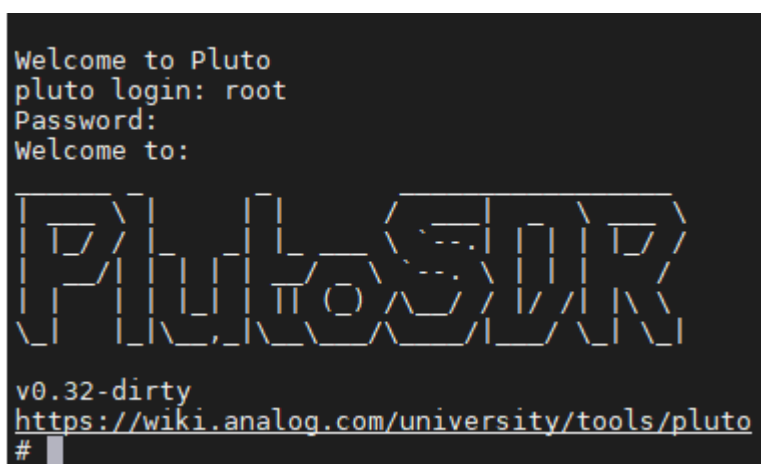


图 2-9 登录之后的界面

登录完成之后的界面如图 2-9 所示。

2.2 安装 Matlab

笔者在这里安装 matlab2018.a 版本，用户可以选择 2018.a 及其以上的版本自行安装。提供的资料当中的 Simulink 仿真模型均是以 2018.a 版本为例进行的。

2.3 安装硬件支持包

Matlab 安装完成之后，想要能够使用 ANTSDR 还需要安装 PlutoSDR 的硬件支持包。在上方菜单栏中选择附加功能，选择获取硬件支持包。

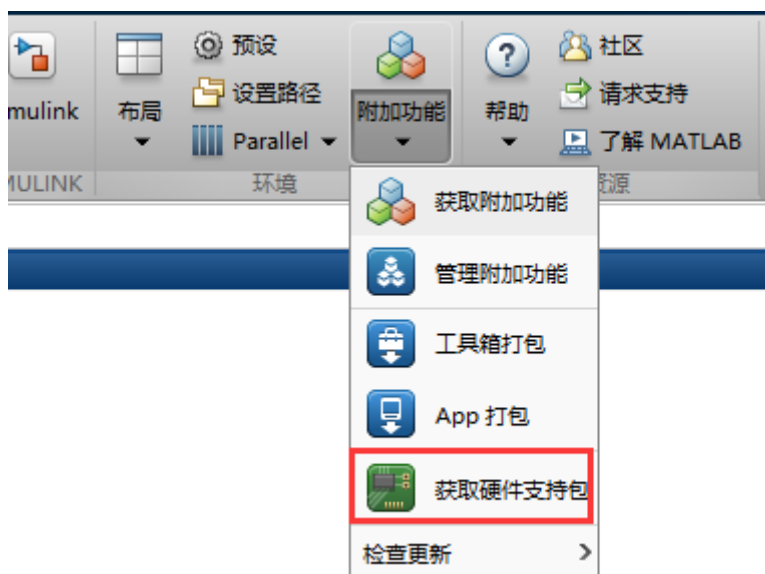


图 2-10 获取硬件支持包

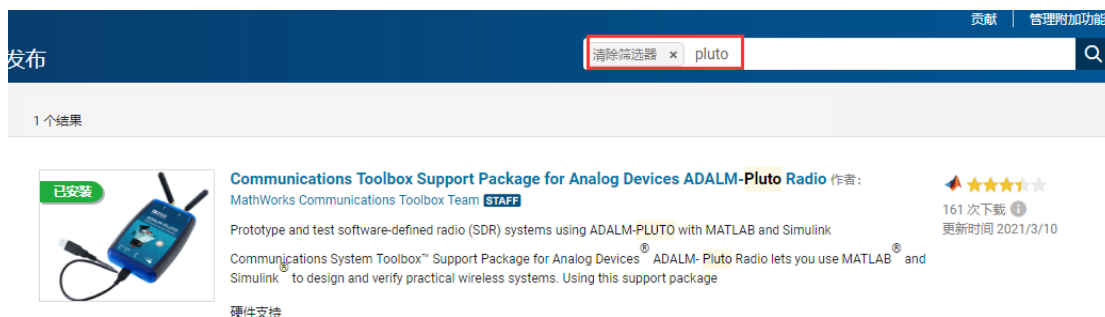


图 2-11 搜索 PlutoSDR

点击 PlutoSDR 通信工具组件进行硬件支持包的安装。



图 2-12 选择安装硬件支持包



电子邮件

尚未创建帐户？立刻创建一个！

登录即表示您同意我们的隐私政策。

下一步

图 2-13

在安装硬件支持包的时候，需要有一个 Mathworks 的账号，用户可以注册一个 Mathworks 的账号，然后登录即可

MathWorks 辅助软件许可协议

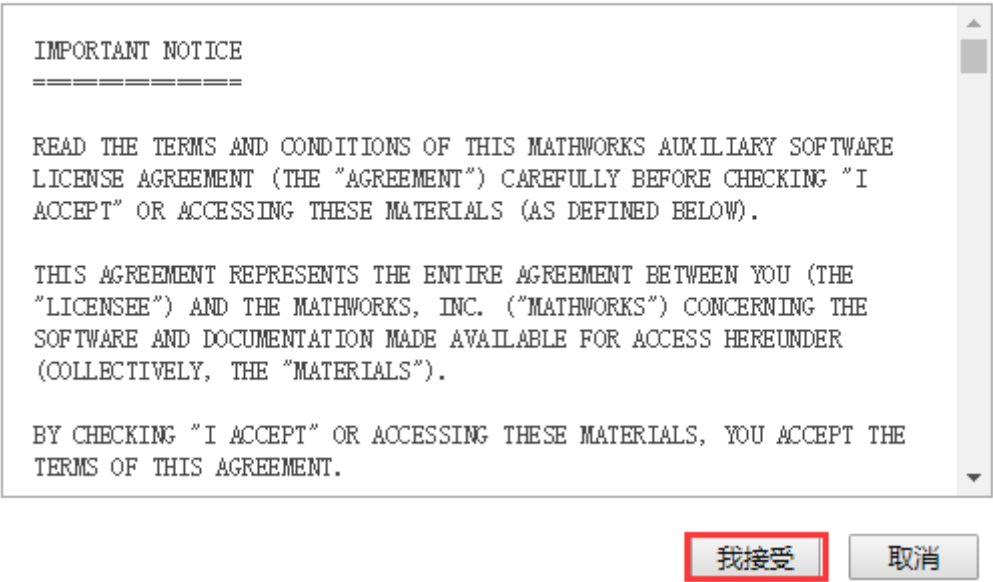


图 2-14

在之后点击下一个就会开始下载和安装 PlutoSDR 所需要的硬件资源了。

第三方软件

点击“下一步”即可安装可能包含开源软件的以下软件，开源软件可能受通用公共许可证(GPL)条款的限制。

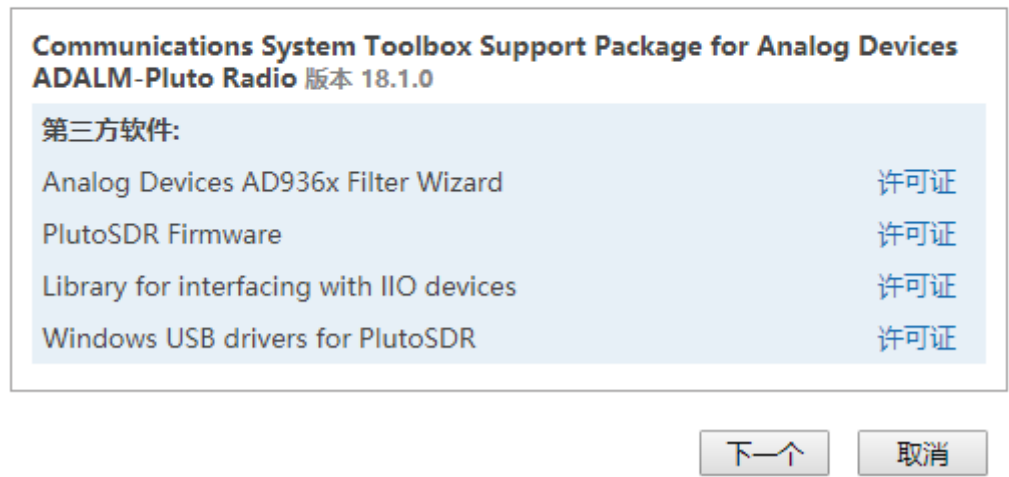


图 2-15

安装完成之后选择稍后设置。

安装完毕

需要对您的硬件支持包进行配置。

您可以选择立即完成配置步骤，或者在以后通过 MATLAB 工具条中的附加功能管理器来执行配置。

立即设置

稍后设置

图 2-16 安装完成

2.4 硬件安装包离线下载

经过用户的反馈，在使用 Matlab 2018.a 安装 Pluto SDR 硬件支持包的时候，可能出现下载失败的情况，因此在这里介绍一种使用离线安装的方式。

在所提供的 Matlab 下载包当中还有一个用于离线安装 PlutoSDR 硬件支持包的压缩包，用户需要将其下载，并解压。

☐	 MATLAB 2018a_win64.zip	   
☐	 R2018a_Hardware_Support_Package.zip	2021-07-28 16:21 zip文件 236.95MB

图 2-17

解压完成之后，使用 matlab 定位到这个解压之后的目录。

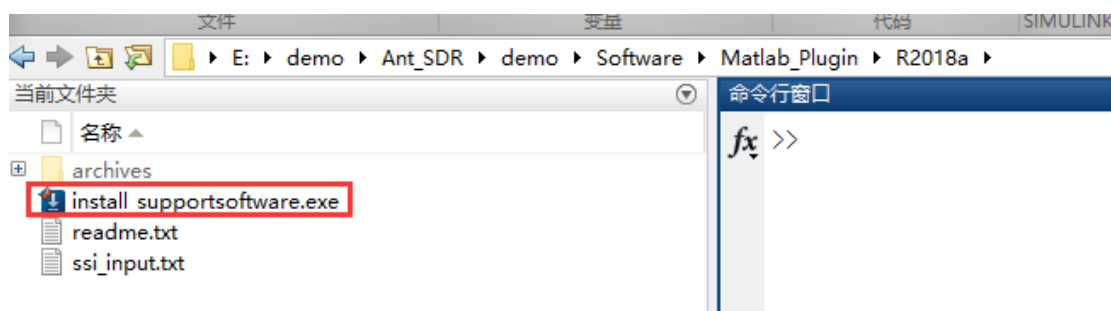


图 2-18

然后双击解压出来的 exe 文件，会弹出一个新的 Matlab 界面，并且随后会弹出硬件支持包的界面，在这里选择安装 PlutoSDR 的硬件支持包即可。

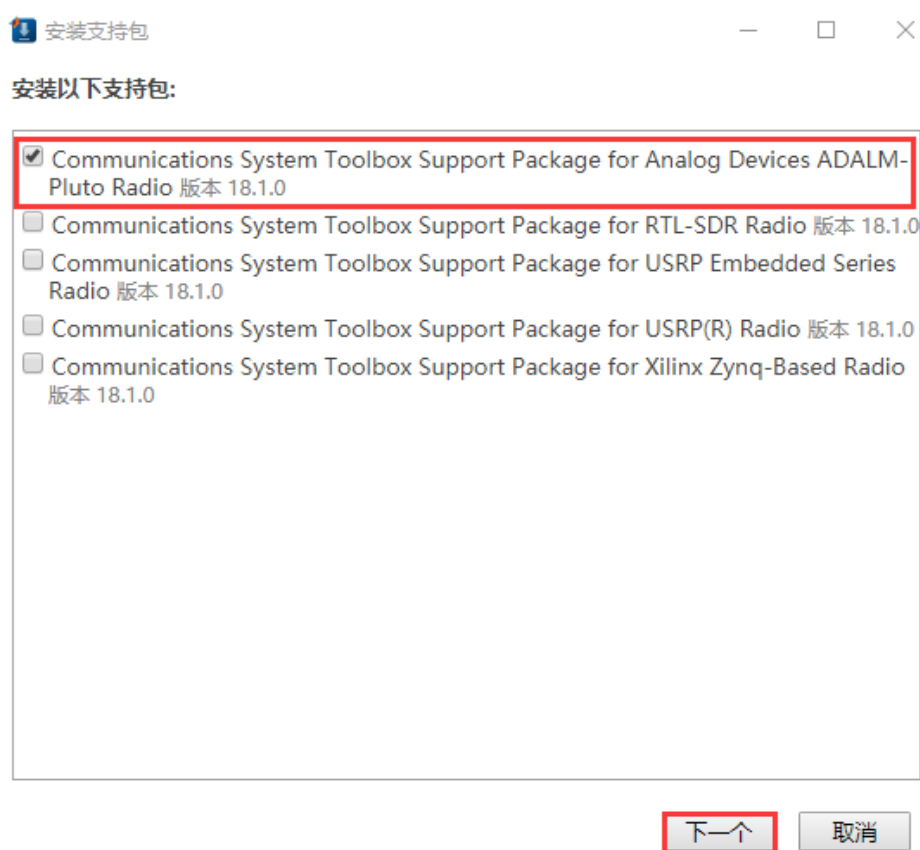


图 2-19 选择安装 Pluto SDR 硬件支持包

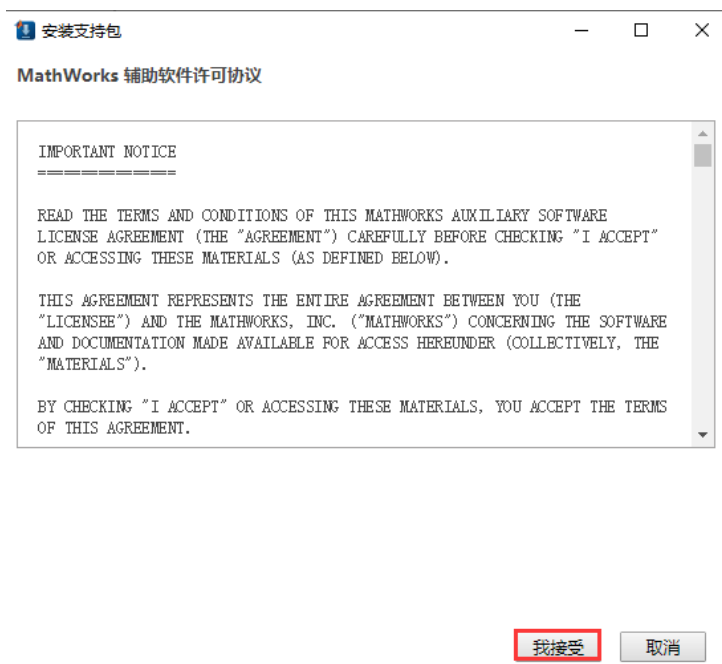


图 2-20

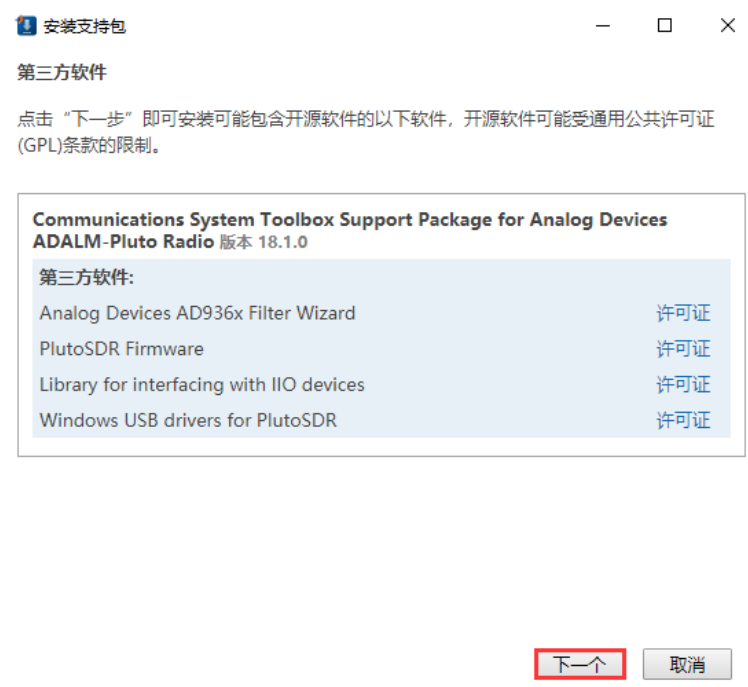


图 2-21

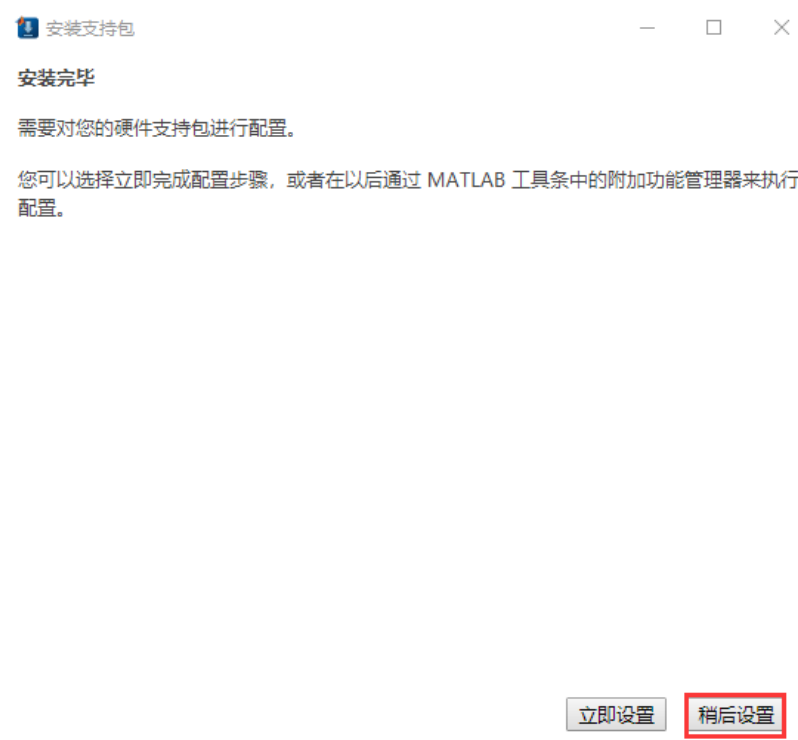


图 2-22

安装完成之后，需要先关闭所有打开的 Matlab，然后重启 Matlab，进行后续的检测步骤。

2.5 测试 ANTSDR 连接

硬件支持包安装完成之后，我们可以测试当前 ANTSDR 和电脑主机的连接情况。

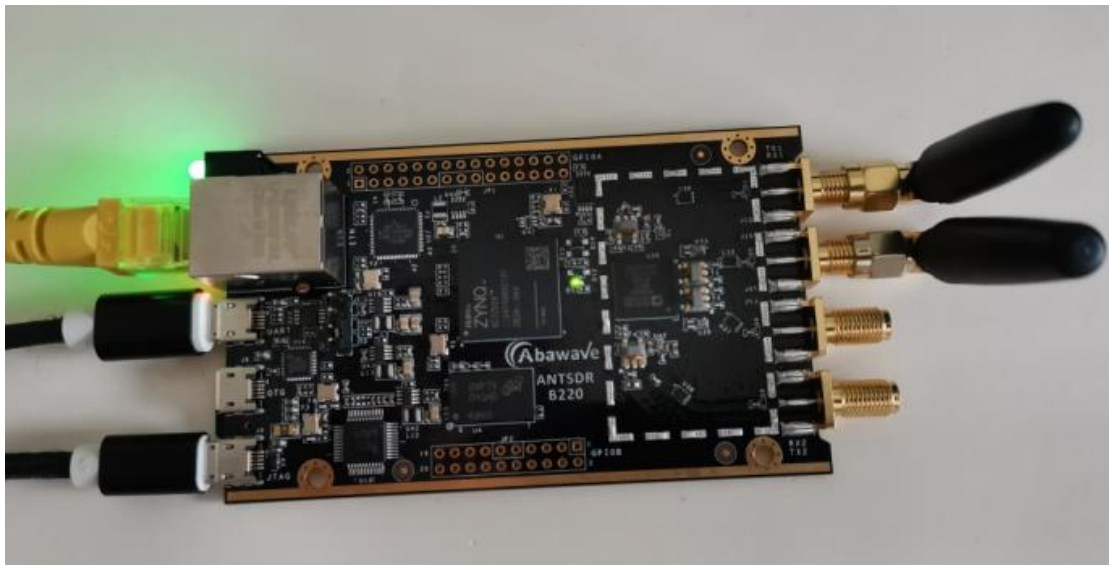


图 2-23

```
https://wiki.analog.com/university/tools/pluto
# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0A:35:00:01:22
          inet addr:192.168.1.10  Bcast:0.0.0.0  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:591777  errors:0  dropped:0  overruns:0  frame:0
          TX packets:967794  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:204995915 (195.4 MiB)  TX bytes:821100354 (783.0 MiB)
          Interrupt:28 Base address:0xb000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0  errors:0  dropped:0  overruns:0  frame:0
          TX packets:0  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

usb0      Link encap:Ethernet  HWaddr 00:05:F7:19:E7:93
          inet addr:192.168.2.1  Bcast:0.0.0.0  Mask:255.255.255.0
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0  errors:0  dropped:0  overruns:0  frame:0
          TX packets:0  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

图 2-24

可以看到,当 ANTSDR 系统启动之后,以太网显示的 IP 地址为 192.168.1.10。为了主机能够正常地访问 ANTSDR,需要将电脑的 IP 地址设置为与 ANTSDR 在同一网段,可以参考开箱检测当中的网络设置。



图 2-25 电脑主机以太网口 IP 设置

设置完电脑主机的 IP 地址之后，可以使用 Matlab 中的 PlutoSDR 的示例程序来测试当前 ANTSDR 的连接情况。打开 Matlab 之后，在右上角搜索框中输入 pluto 进行搜索，然后再弹出窗口中打开一个 Simulink 仿真模型。

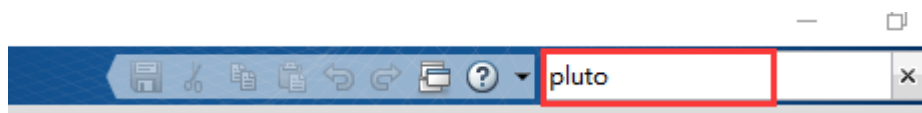


图 2-26

Pluto Transmitter - Transmit data to Analog Devices ADALM-PLUTO radio

The Pluto Transmitter block is a signal source that sends data to an Analog Devices ADALM-PLUTO radio.

文档 > Communications System Toolbox Support Package for Analog Devices ADALM-Pluto Radio > Radio I/O

Pluto Receiver - Receive data from Analog Devices ADALM-PLUTO radio

The Pluto Receiver block is a signal source that receives data from an Analog Devices ADALM-PLUTO radio.

文档 > Communications System Toolbox Support Package for Analog Devices ADALM-Pluto Radio > Radio I/O

QPSK Receiver with ADALM-PLUTO Radio

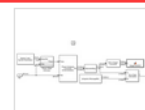
This example shows how to use the ADALM-PLUTO Radio System objects to implement a QPSK receiver.

示例 > Communications System Toolbox Support Package for Analog Devices ADALM-Pluto Radio > Digital Modulation > MATLAB

QPSK Receiver with ADALM-PLUTO Radio

This model shows how to use the the ADALM-PLUTO Radio with Simulink® to implement a QPSK receiver.

示例 > Communications System Toolbox Support Package for Analog Devices ADALM-Pluto Radio > Digital Modulation > Simulink



QPSK Transmitter with ADALM-PLUTO Radio

This example shows how to use the ADALM-PLUTO Radio System objects to implement a QPSK transmitter.

示例 > Communications System Toolbox Support Package for Analog Devices ADALM-Pluto Radio > Digital Modulation > MATLAB

图 2-27 Pluto SDR Simulink 仿真模型

选择 Open Model 就能够打开仿真模型了。

QPSK Receiver with ADALM-PLUTO Radio

R2018a

This model shows how to use the the ADALM-PLUTO Radio with Simulink® to implement a QPSK receiver. The receiver addresses practical issues in wireless communications, e.g. carrier frequency and phase offset, timing offset and frame synchronization. This model receives the signal sent by the QPSK Transmitter with ADALM-PLUTO Radio model. The receiver demodulates the received symbols and outputs a simple message to the MATLAB® command line.

Open Model

Overview

This model performs all processing at complex baseband to handle a time-varying frequency offset, a time-varying symbol delay, and Gaussian noise. To cope with the above-mentioned impairments, this example provides a reference design of a practical digital receiver, which includes correlation-based coarse frequency compensation, symbol timing recovery with fixed-rate resampling and bit stuffing/skipping, fine frequency compensation, frame synchronization and phase ambiguity resolution. The example uses some key algorithms in MATLAB, emphasizing textual algorithm expression over graphical algorithm expression.

Structure of the Example

The top-level structure of the model is shown in the following figure, which includes a ADALM-PLUTO receiver block, a QPSK Receiver subsystem and a BER Display blocks.

图 2-28

QPSK Receiver with ADALM-PLUTO Radio

Note: Before running the QPSK models, first run the companion models for frequency offset calibration.

Open the companion plutoradiofreqcalib Tx model

Open the companion plutoradiofreqcalib Rx model

Open the companion plutoradioQPSKTransmitterSimulinkExample model

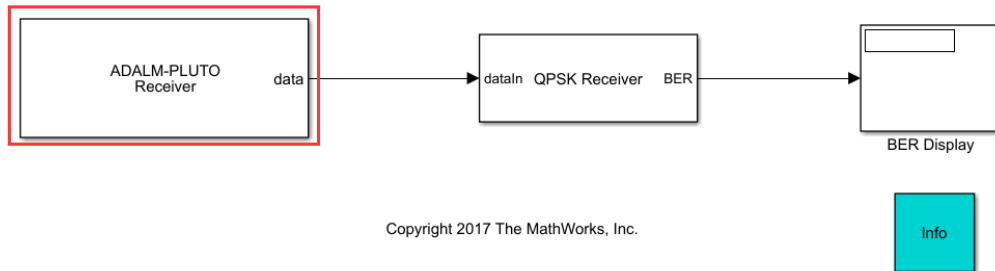


图 2-29 仿真模型

双击 ADALM-PLUTO 接收模块打开设置窗口，在设置窗口中将 Radio ID 设置为

ip:192.168.1.10

然后点击 Apply，再点击 Info，就能够看到当前设备的信息，如图 2-31 所示。

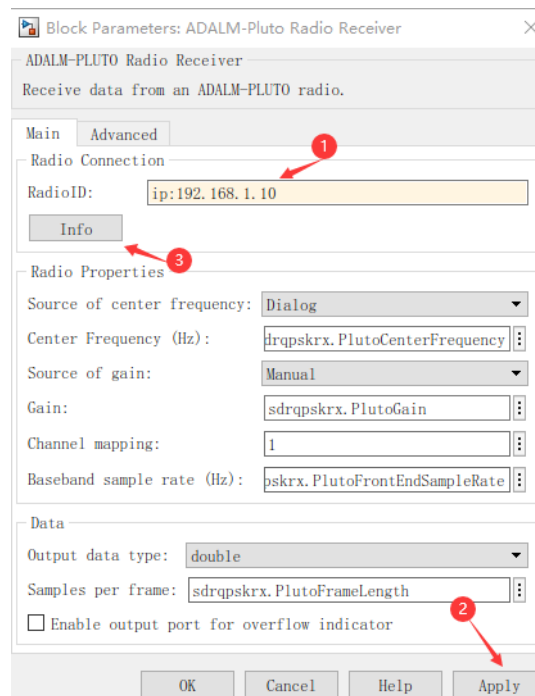


图 2-30

Values going to the SDR device are often quantized by the hardware.
The following values are the actual SDR device settings.

```
        Status: 'Full information'
      CenterFrequency: 915000000
    BasebandSampleRate: 400000
          Gain: 30
    RadioFirmwareVersion: '0.32'
    ExpectedFirmwareVersion: '0.26'
          HardwareVersion: 'A1'
    EnableQuadratureTracking: true
      EnableRFDCTracking: true
    EnableBasebandDCTracking: true
      FrequencyCorrection: 0
```

图 2-31 当前 SDR 设备信息

第3章 Matlab & Simulink 的基本使用

在这一章当中，会简单介绍一下 Matlab 和 Simulink 的基本的使用。

3.1 Matlab 介绍

3.1.1 Matlab 的界面

打开 matlab 之后，可以看到如下的界面：

区域 1 是文件目录的导航栏，可以查看当前文件夹下有什么文件和文件夹。

区域 2 是命令行窗口，在 matlab 的命令窗口里面可以直接输入一些命令，类似于 Linux 终端当中的 terminal。

区域 3 是保存有变量的工作区，在 matlab 工作的时候，产生的各种类型的变量都会保存在这里。

区域 4 是菜单栏，有关 matlab 的设置和一些附加功能的安装都在这个区域中可以找到。

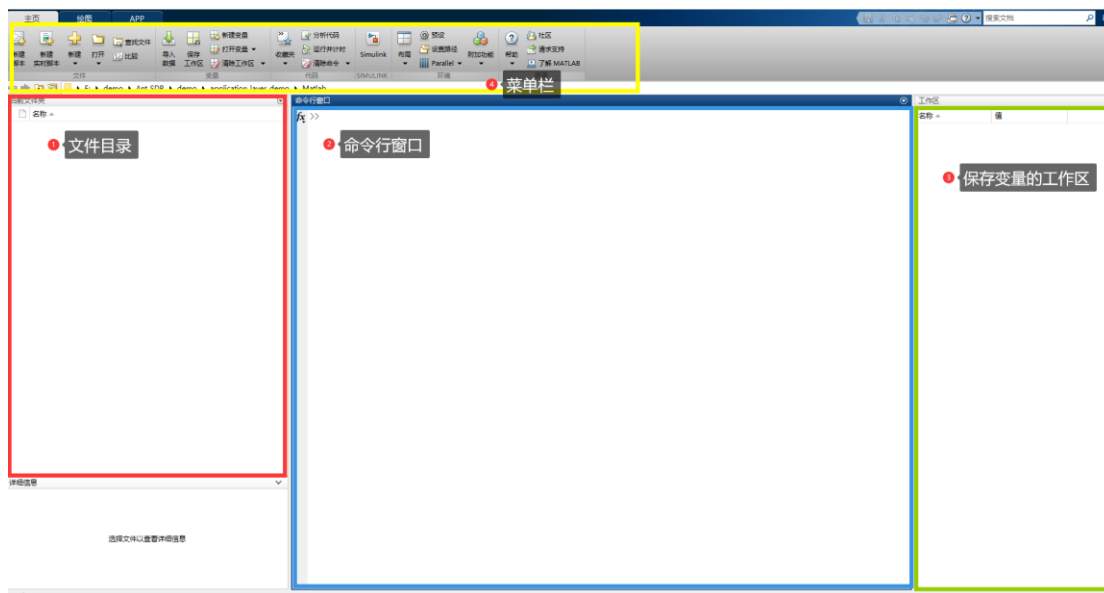


图 3-1

3.1.2 Matlab 命令行使用

在命令行的窗口中可以直接输入 matlab 支持的代码，例如要对 x 赋值为 100，

只需要在命令行中输入

```
x = 100
```

当在命令行中输入该命令的时候，会在工作区中产生一个名为 **x** 的变量，其值为 **100**，并且在命令行中会将该值打印出来。如下图所示：

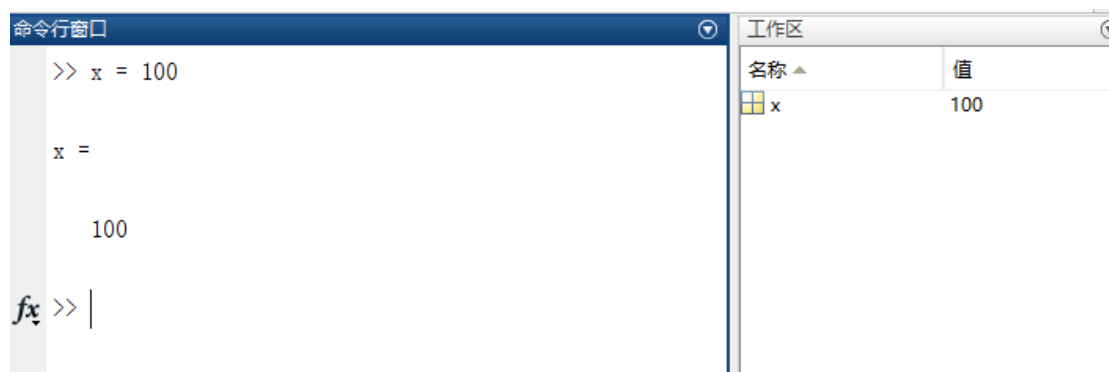


图 3-2 在命令行中输入代码

再创建一个变量 **y**，对 **y** 赋值之后将 **x+y** 的值赋值给 **z**，如下图所示，可以看到，输入上面的代码之后新创建了两个变量 **y** 和 **z** 并且 **z** 的值为 **x+y** 的值。

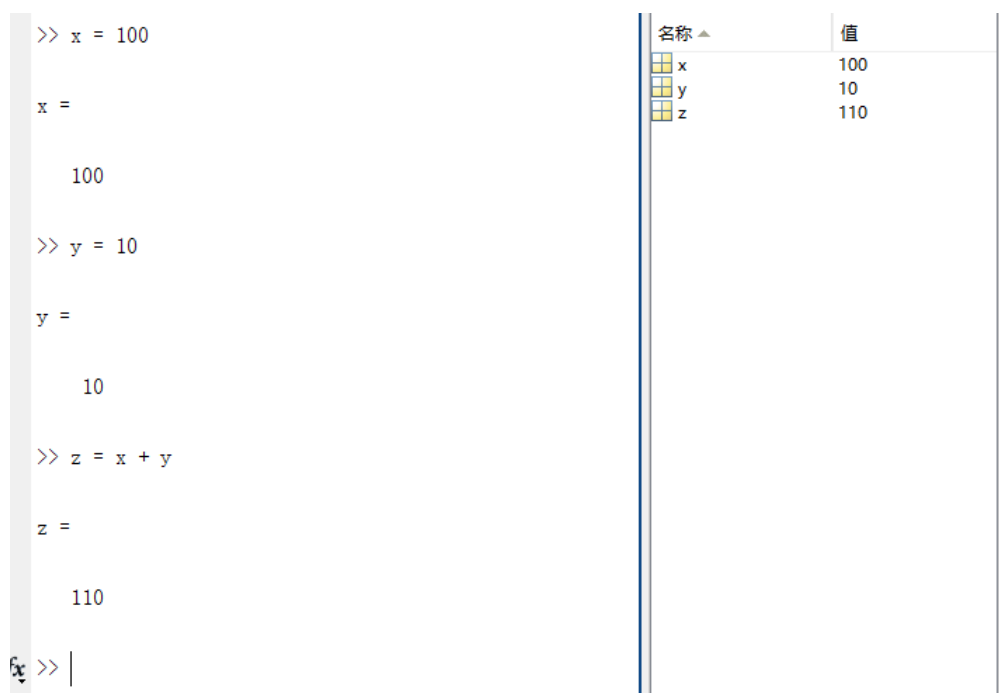


图 3-3

3.1.3 在命令行中使用函数

接下来，在命令行中使用函数来计算 **x/y** 的余数，在命令行中输入如下代码：

```
r = rem(x,y)
```

输入该代码后可以得到 x/y 的结果，当前 x 是 100， y 是 10 所以相除之后余数为 0。你可以更改不同的 x ， y 的值，看看得到的结果是否与预期相符。

3.1.4 获取某个函数的帮助信息

Matlab 中提供了大量的函数供用户使用，用户想要知道某个函数的功能只需要在命令行中输入 `help` 然后加上对应的函数名称即可，例如，在命令行中输入：

```
help rem
```

```
>> help rem
rem - 除后的余数

此 MATLAB 函数 返回用 b 除以 a 后的余数，其中 a 是被除数，b 是除数。此函数通常称为求余运算，表达式为  $r = a - b \cdot \text{fix}(a./b)$ 。rem 函数遵从  $\text{rem}(a, 0)$  是 NaN 的约定。

r = rem(a,b)

另请参阅 mod

rem 的参考页
名为 rem 的其他函数
```

图 3-4 查看函数帮助信息

可以看到命令行中就出现了该函数的简单的描述，若想要查看关于该函数的详细描述，只需要点击图 3-4 中的红色位置，也就是该函数的参考页。Matlab 会打开该函数的使用说明的网页。

你也可以通过输入 `doc` 然后加上对应的函数名，来获取该函数的使用说明，例如，在命令行输入

```
doc rem
```

然后你同样可以进入 `rem` 函数使用方法的详细页面。



图 3-5

3.2 Matlab 脚本

如果你要写更复杂的 Matlab 代码，或者以后要重复使用这些代码，可以使用 Matlab 脚本代替一行行的在命令行里输入指令。脚本文件可以进行编辑、写注释、运行、保存。文件扩展名是.m。

- 1) 首先新建一个脚本。有许多方法可以来创建脚本，简单方式是点击菜单栏中的新建选项。

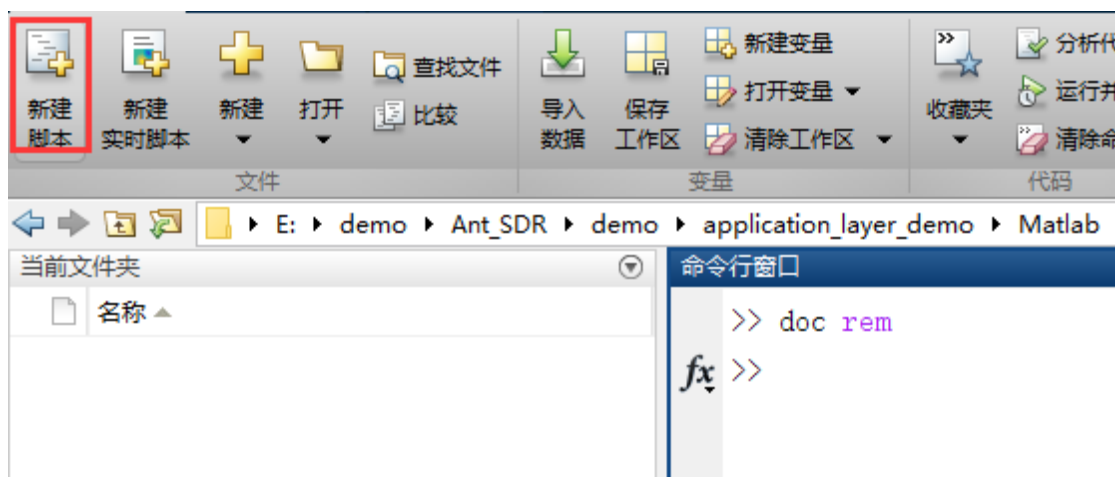


图 3-6

新建脚本之后，会出现一个未命名的脚本文件，点击保存，会弹出文件的保存路径窗口，选择用户自己的文件夹然后将脚本文件保存到对应的目录下就可以了。

保存好之后，可以在脚本中输入如下代码来求得一个数组中每个元素的平均值。

```
a_array = [1 5 -4 9 7 -6 0];
thresh = 3;
% an array of numbers to test
% threshold to test against
a_mean = mean(a_array);
if a_mean > thresh
disp('The average value exceeds the threshold.');
```

```
else
disp('The average value is less than or equal to the threshold.');
```

```
end
disp(['The threshold is ', num2str(thresh), ' and the average value is ',
num2str(a_mean), '.']);
```

%后面是注释符号，后面跟着的语句都是注释的内容。Matlab 会根据文字功能的不同对其自动显示不同的颜色。脚本编写完成之后，就可以点击保存，然后点击执行，就可以执行这个脚本了。运行的结果如下，在 Matlab 的命令行当中打印出了运行的结果信息。

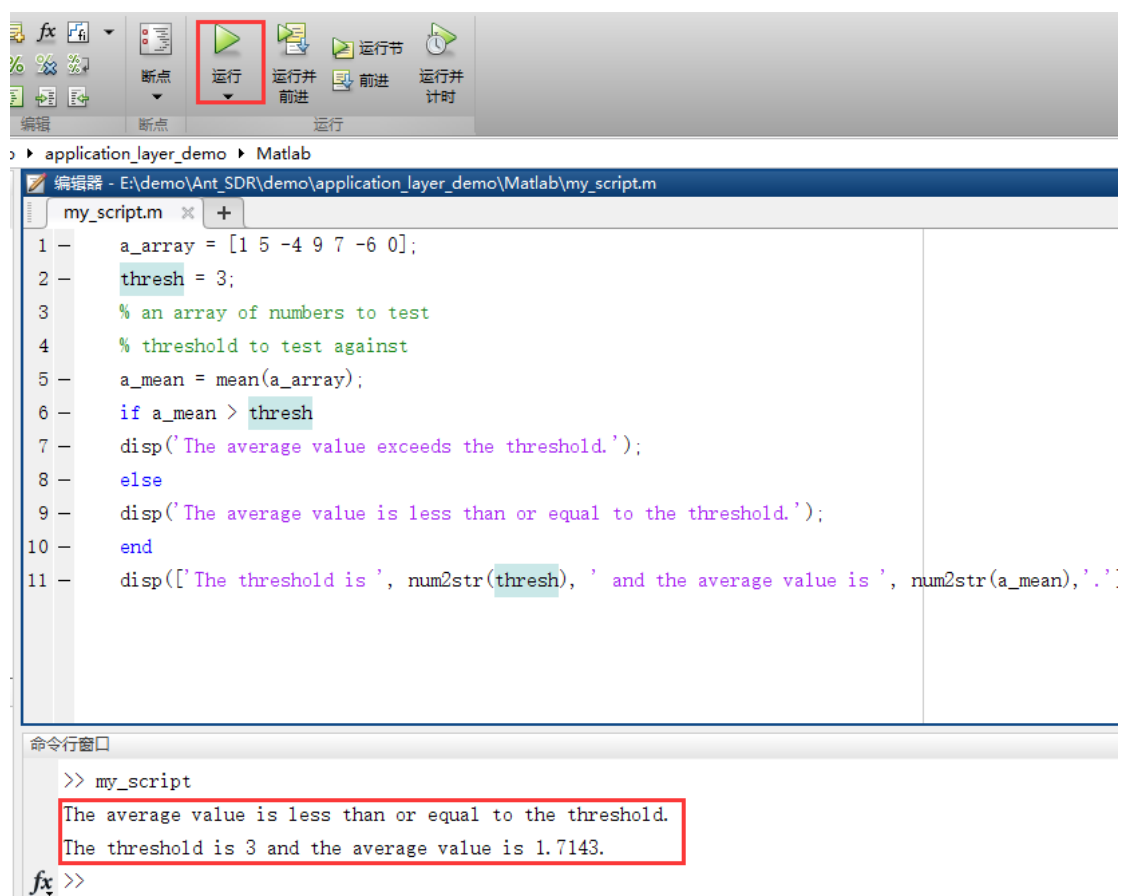


图 3-7 脚本运行结果

3.3 Matlab 函数

尽管 Matlab 中已经为用户提供了大量的函数，但是用户还是有可能需要来实现自定义的函数，因此掌握自定义函数的方法还是比较重要的。

函数可以使用很多不同的输入输出类型。它们也可以重载，也就是说输入的参数类型或数量不同，函数功能不同。

点击新建函数，可以根据模板创建出来一个函数。

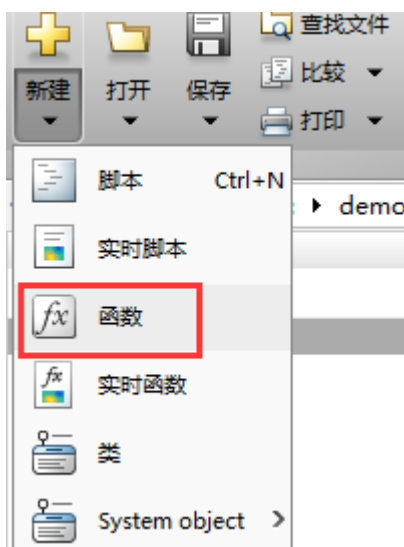


图 3-8

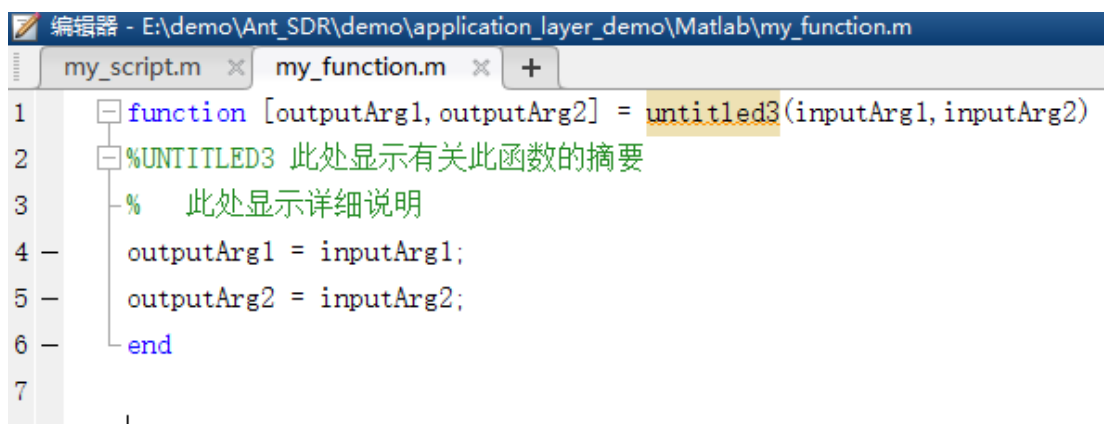


图 3-9 函数模板

函数的模板如图 3-9 所示，function 关键字后面的数字是函数返回的结果，等号之后是函数的名称，括号内的是函数需要传递的参数。

函数的功能是在关键字 function 和 end 之间的内容来实现的。

接下来，把前面那个实现求取数组平均值并于 Threshold 比较的 matlab 代码

用 `function` 的方式来实现，将上面新创建的函数更改为如下形式，然后点击保存。

```
function [result] = my_function(a_array,thresh)
%my_function compare the mean value of input array with the threshold
a_mean = mean(a_array);
if a_mean > thresh
    result = 1;
else
    result = 0;
end
```

创建好函数之后，就可以来调用这个函数了，接下来新建一个 `matlab` 脚本，名称为 `function_call.m`，然后在这个脚本里面调用前面创建的这个函数。

```
% prepare array and threshold
a_array = [1 5 -4 9 7 -6 0];
thresh = 1;
% call the function
result = my_function(a_array, thresh);
if result == 1
    disp('The average value exceeds the threshold.');
```

再次点击运行，即可完成 `function_call` 这个脚本的执行，脚本会去调用 `my_function` 这个函数，然后返回比较的结果，最终会根据返回的结果来打印对应的信息。

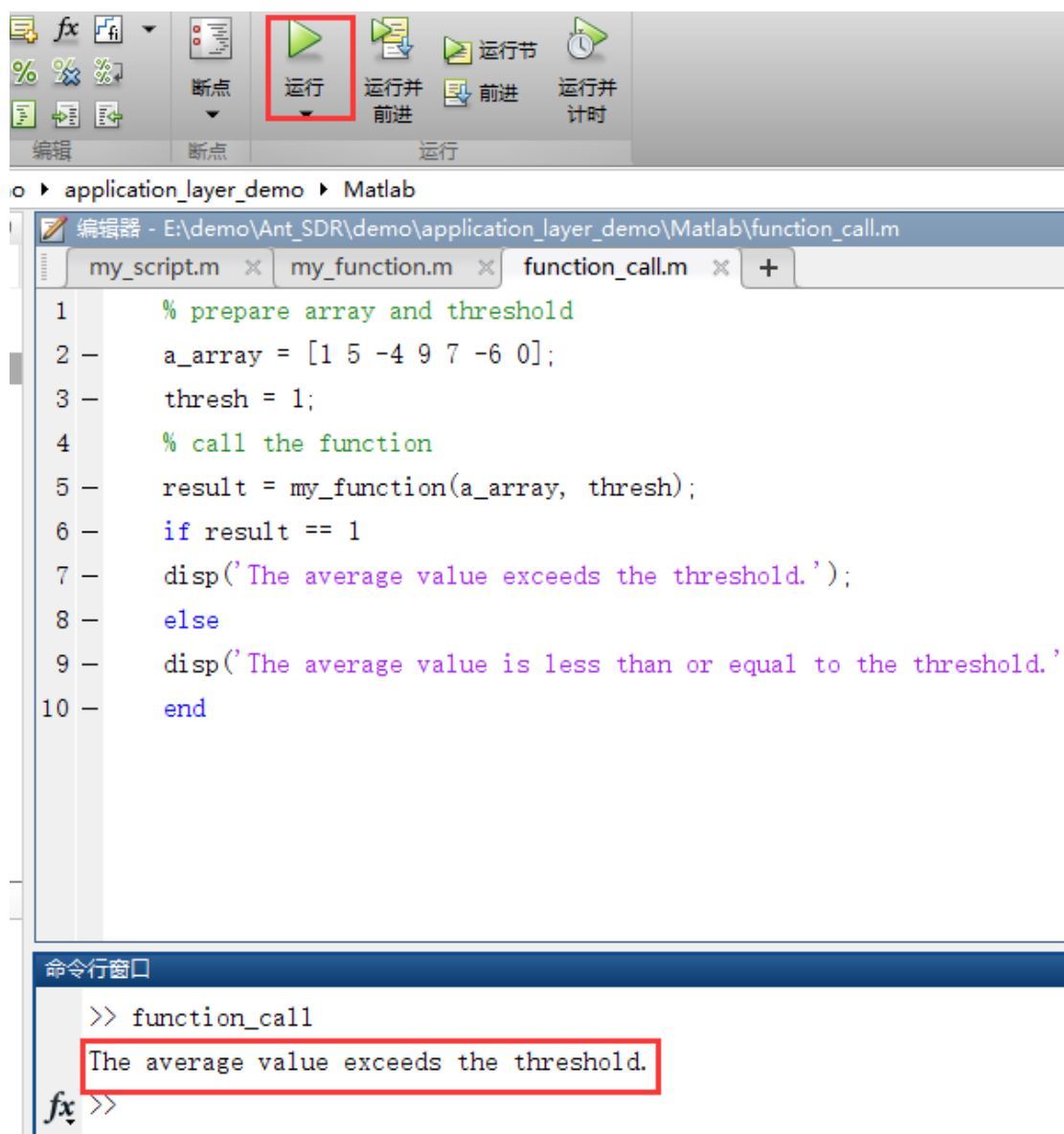


图 3-10 函数调用情况

3.4 Matlab 绘图

Matlab 一个强大的功能就是它能够很容易地绘制出图形，能够帮助用户更加直观地来感受计算的结果。Matlab 的绘图方法很多，绘图效也很多，在这里只介绍基本的绘图方法。

首先新建一个 Matlab 脚本，名称为 `sin_wave_plot.m`，在这个脚本当中，我们会绘制一个正余弦的波形。

```
% script to generate and plot sine and cosine waves
fs = 1000;          % sampling frequency in Hz
f1 = 20;            % frequency of sine and cosine waves in Hz
Ts = 1/fs;
t = [0:Ts:100/fs]; % time vector

sin_wave = sin(2*pi*f1*t); % create a corresponding array of sine samples
cos_wave = cos(2*pi*f1*t); % create a corresponding array of cosine samples

figure(1)           % create a new figure
hold off            % clear previous data in plot
plot(t,sin_wave,'r'); % plot sin_wave in red line
hold on             % keep the data in the plot
plot(t,cos_wave,'b'); % plot the cos_wave in blue line
```

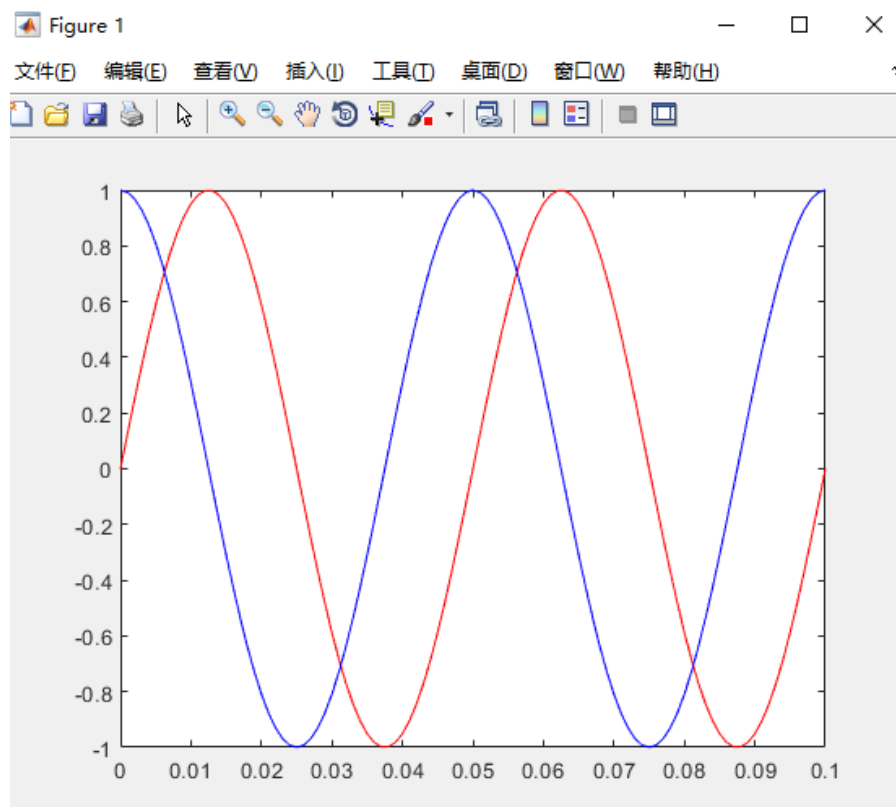


图 3-11 绘图结果

运行上面的脚本会绘制出正余弦的波形，其中正弦信号和余弦信号分别用了不同的颜色标记出来。

上面的图形绘制出来之后，还有许多信息不能显示出来，首先我们可以给上面的图加上坐标轴和标题。你可以在上面的脚本的末尾增加如下代码：

```
xlabel('Time (seconds)'); % add a label to the x (time) axis  
ylabel('Amplitude');      % add a label to the y axis  
title('Sine and Cosine'); % add a title for the plot
```

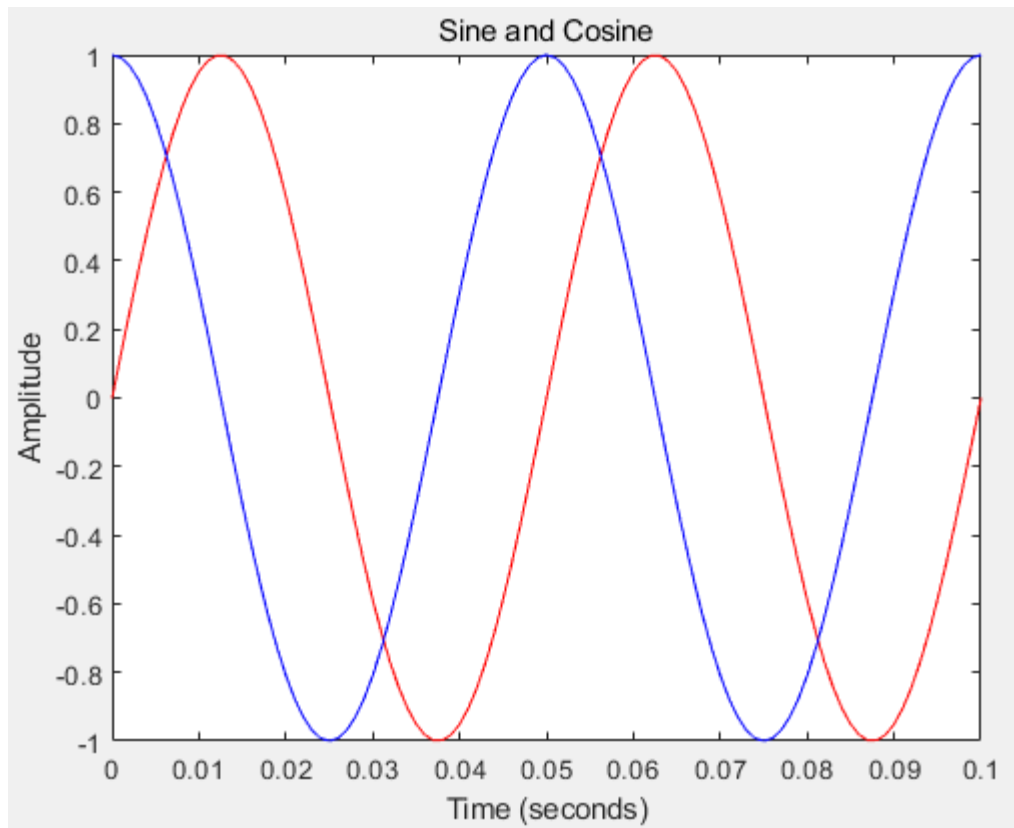


图 3-12 增加了坐标轴和标题之后的图片

现在的图像比其之前要稍微详细一点了，但是从图片上还是不能直观地感受到每个信号代表了什么，所以还可以对图中的信号做标注。你可以在脚本的最后加上如下代码：

```
legend('sin','cos'); % add a legend to the graph  
grid on;             % show the grid  
axis([0 max(t) -1.2 1.2]); % configure the axis scaling
```

运行的结果如图 3-13 所示，更改了坐标轴的范围，并且在图上标注出了信号的名称。

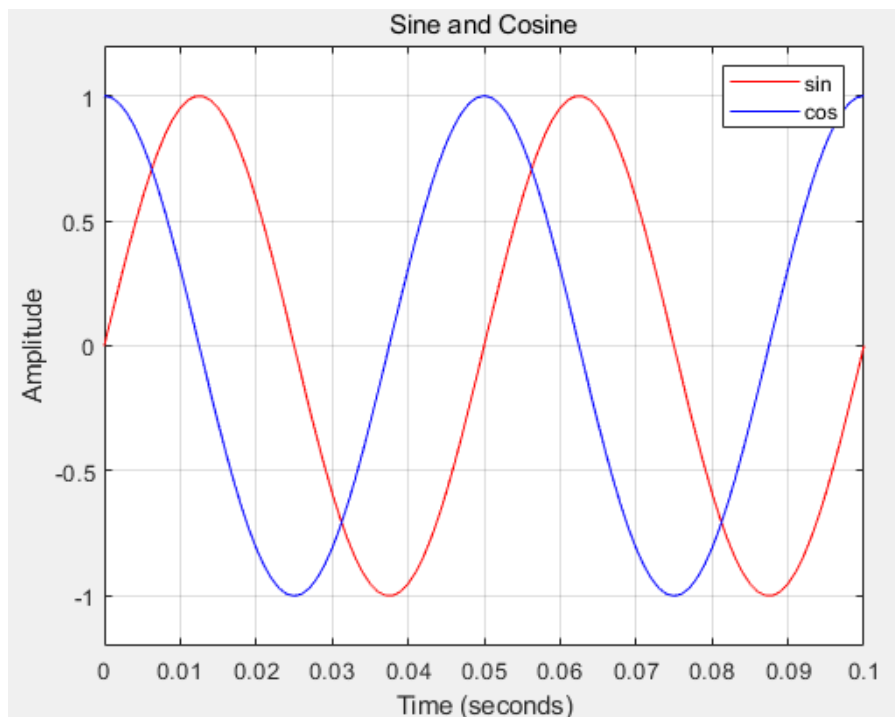


图 3-13

3.5 Matlab 的数组、矩阵和结构

Matlab 是工程计算工具，它很适合处理数组、矩阵、结构型的数据。数组和矩阵对于许多数学上的分析很有用。在这里会介绍一些 Matlab 中基本的结构。

3.5.1 Matlab 中的数组

定义一个数据

在 Matlab 中定义数组十分简单，只需要把数组赋值给一个变量就可以了。行列向量的定义方式如下，你可以在命令行中输入如下，命令查看结果：

```
row_array = [1 2 3 4 5 6]; % define a row vector
column_array = [1; 2; 3; 4; 5; 6]; % define a column vector
```

创建一个全零数组或者全 1 数组

可以使用 matlab 的函数创建一个全为 0 或者全为 1 的数组。

```
my_ones = ones(1,10) % create an array of ones, with 1 row and 10 columns
my_zeros = zeros(1,8) % create an array of zeros, with 1 row and 8 columns
```

确定数组的长度

确定数组长度可以使用 `length` 函数，该函数会返回一个数组的长度。

```
num_ones = length(my_ones) % find the length of array my_ones
num_zeros = length(my_zeros) % find the length of array my_zeros
```

找到数组中的最大值和最小值

可以使用 `max` 函数和 `min` 函数找到一个数组当中的最大值和最小值。

```
max_r = max(row_array) % find the largest value in row_array
min_c = min(row_array) % find the smallest value in row_array
```

数组的转秩

数组的转秩可以使用在数组后加上 “`'`” 或者使用转秩函数来完成。

```
column_array = transpose(row_array) % transpose the first array
column_array = row_array' % transpose the second array
```

3.5.2 Matlab 当中的矩阵

矩阵可以看作是由多个向量组成的，矩阵可以是二维的或者多维的，在这里以二维矩阵来进行演示，介绍一些在 **Matlab** 中和矩阵相关的操作。

定义矩阵

矩阵的定义同数组类似，可以采用直接定义的方式，在矩阵定义的时候，每一行需要使用分号隔开。

```
matrix_a = [1 2 3 4; 5 6 7 8; 9 10 11 12]
matrix_b = [3 3 3 3; 1 2 1 2; 4 4 5 5]
```

```
>> matrix_a = [1 2 3 4; 5 6 7 8; 9 10 11 12]
```

```
matrix_a =
```

```

1     2     3     4
5     6     7     8
9    10    11    12
```

图 3-14 矩阵定义

使用函数来定义矩阵

同数组类似，矩阵也可以采用函数的方式来定义全零、全一矩阵和随机矩阵。

```
zero_matrix = zeros(3,4)
one_matrix = ones(3, 4)
randm_matrix = rand(3, 4)
```

```
zero_matrix =

    0    0    0    0
    0    0    0    0
    0    0    0    0

>> one_matrix = ones(3, 4)

one_matrix =

    1    1    1    1
    1    1    1    1
    1    1    1    1

>> randm_matrix = rand(3, 4)

randm_matrix =

    0.8147    0.9134    0.2785    0.9649
    0.9058    0.6324    0.5469    0.1576
    0.1270    0.0975    0.9575    0.9706
```

图 3-15

矩阵加减法

矩阵在进行加减法的时候，必须得满足矩阵得尺寸是相同，只有两个矩阵是相同大小得时候，矩阵中的元素才能一一对应进行相加。

```
matrix_add = matrix_a + matrix_b
```

```
>> matrix_add = matrix_a + matrix_b
```

```
matrix_add =
```

4	5	6	7
6	8	8	10
13	14	16	17

图 3-16 矩阵相加

矩阵中对应元素相乘

想要使两个矩阵中的对应元素相乘，需要使用到点乘，也就是 `.*` 在乘号的前面需要加上一个点。这样会使两个尺寸相同的元素中对应位置的元素进行相乘，在进行对应元素相乘的时候，同样需要两个矩阵的尺寸一致才可以。

```
matrix_c = matrix_a .* matrix_b
```

```
>> matrix_c = matrix_a .* matrix_b
```

```
matrix_c =
```

3	6	9	12
5	12	7	16
36	40	55	60

```
>>
```

图 3-17 矩阵元素相乘

矩阵相乘

在进行两个矩阵的相乘的时候，类似与在线性代数中的矩阵相乘，但是需要满足第一个矩阵的列数和第二个矩阵的行数相同才能进行两个矩阵的相乘，两个矩阵相乘的结果还是一个矩阵。

```
matrix_mult = matrix_a * matrix_b'
```



```
>> matrix_mult = matrix_a * matrix_b'

matrix_mult =

    30    16    47
    78    40   119
   126    64   191
```

图 3-18 矩阵相乘

3.6 Matlab 系统对象

许多编程语言都是面向对象的，也就是说它们支持数据对象并对其进行运算。Matlab（以 java 为基础）也是面向对象的。许多 MathWorks 的工具盒（提供 Simulink 模块和 Matlab 函数）都使用一种特殊的对象，叫作系统对象，允许使用动态系统，即可以在 Matlab 里也可以在 Simulink 里初始化。这些动态系统根据时变的输入值来输出不同的信号。它们保存着参数值，包括当前参数和过去的参数，所有这些都用在将来的运算里。接下来会以一个滤波器的设计，来介绍系统对象。

3.6.1 创建系统对象

在这里首先创建一个滤波器，需要使用到 matlab 中的 DSP 工具包当中的组件，使用之后会为我们创建一个系统对象。在 Matlab 命令行中输入：

```
obj_filter = dsp.FIRFilter('Numerator', fir1(50,0.25,'low'));
```

```
>> obj_filter = dsp.FIRFilter('Numerator', fir1(50,0.25,'low'));
>> obj_filter

obj_filter =

    dsp.FIRFilter - 属性:

        Structure: 'Direct form'
    NumeratorSource: 'Property'
        Numerator: [1×51 double]
    InitialConditions: 0

显示 所有属性
```

图 3-19 创建一个滤波器

`fir1` 函数设计一个阶数是 50 的低通滤波器，截止频率 0.25（归一化的频率，实际是 $0.25 \cdot f_s / 2$ ）。这些系数会指定滤波器的分子，滤波器命名为 `obj_filter`。

可以使用 `fvtool` 来查看生成的滤波器的幅频响应。

```
fvtool(obj_filter);
```

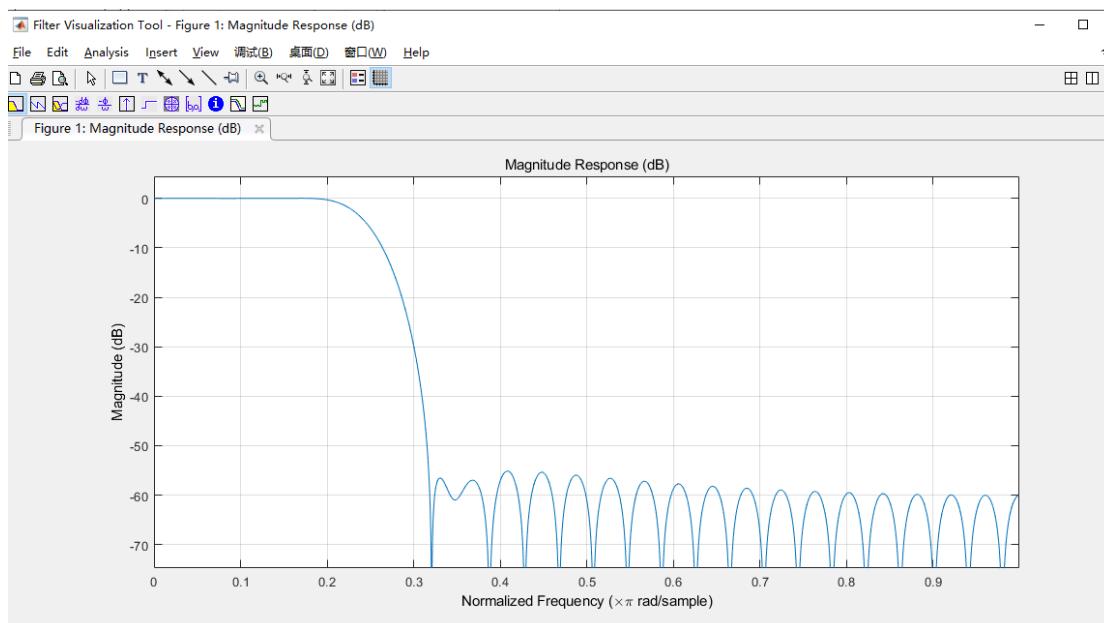


图 3-20 滤波器的幅频响应

3.6.2 使用系统对象

众所周知，滤波器是用来对信号进行滤波的，有去除噪声的作用，在这里我们首先生成一个随机信号，然后使用上面所设计的滤波器来对其进行滤波。

新建一个脚本 `fir_object_test.m`

```
x = randn(125,1); % generate a Gaussian White Noise signal
obj_filter = dsp.FIRFilter('Numerator', fir1(50,0.25,'low')); % create a low pass filter object
y = step(obj_filter,x); % filter the signal with the lowpass filter
% create a figure
figure
grid on
hold on
xaxis = 1:100;
% plot the signals
plot(xaxis,x(1:100),'r');
plot(xaxis,y(26:125),'b'); % note the 25 sample delay
```

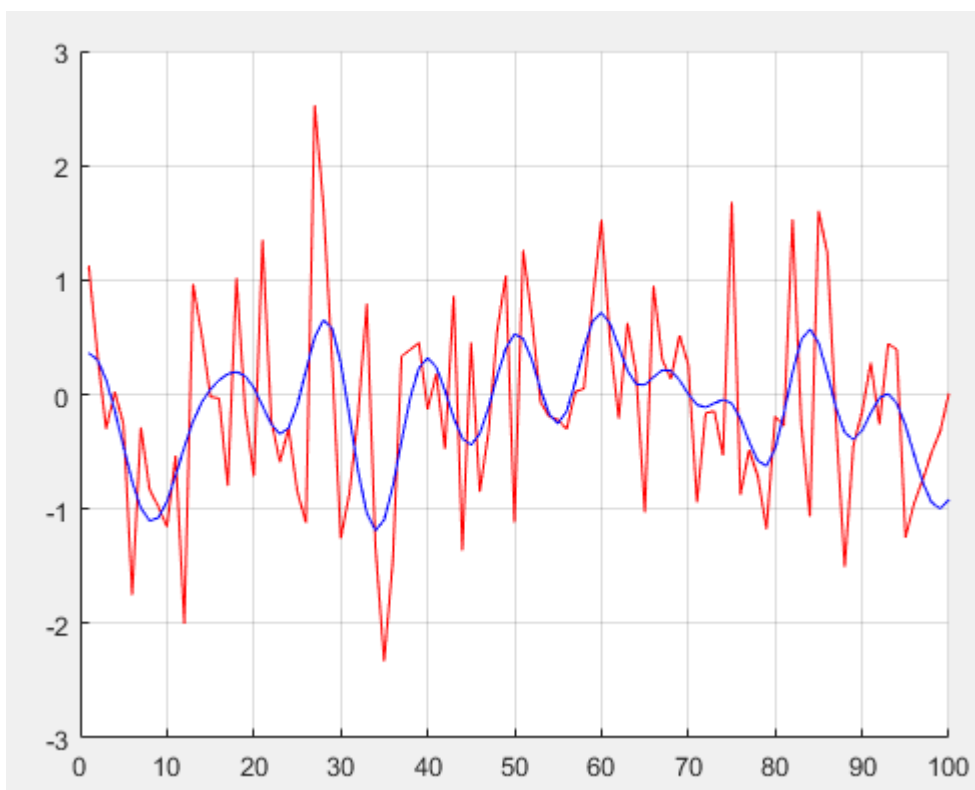


图 3-21 运行结果

可以看到，经过创建好的低通滤波器之后，消除了其中的噪声。使得经过滤波后的信号，相较于原始信号更加平滑。

3.7 Simulink 简介

Simulink 是一个图形化、以模块为基础的工具，集成了 Matlab 组件。有时候设计更适合以模块为基础，而不以代码为基础，设计者也可能有自己的偏好。还要指出的是 Simulink 和 Matlab 集成在一起，有一些机制可以实现模块和代码共同工作。比如，Matlab 代码可以放在 Simulink 的模块里。也可以在模型加载的时候自动执行 Matlab 代码，也可以自定义变量作为模型的参数（因为 Simulink 和 Matlab 共享 workspace）。这样可以方便地使用 Matlab 和 Simulink。

3.7.1 新建 Simulink 工程

在本章中会创建一个简单的 Simulink 工程，然后简单介绍一下 Simulink 库。

新建 Simulink 模型

首先在菜单栏中点击 Simulink，进入 Simulink 的界面。

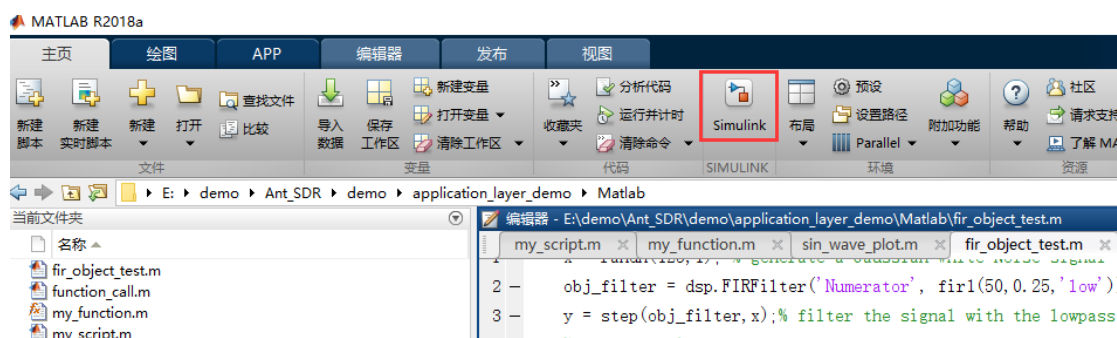


图 3-22

然后选择 Blank Model，开始搭建一个新的 Simulink 模型。

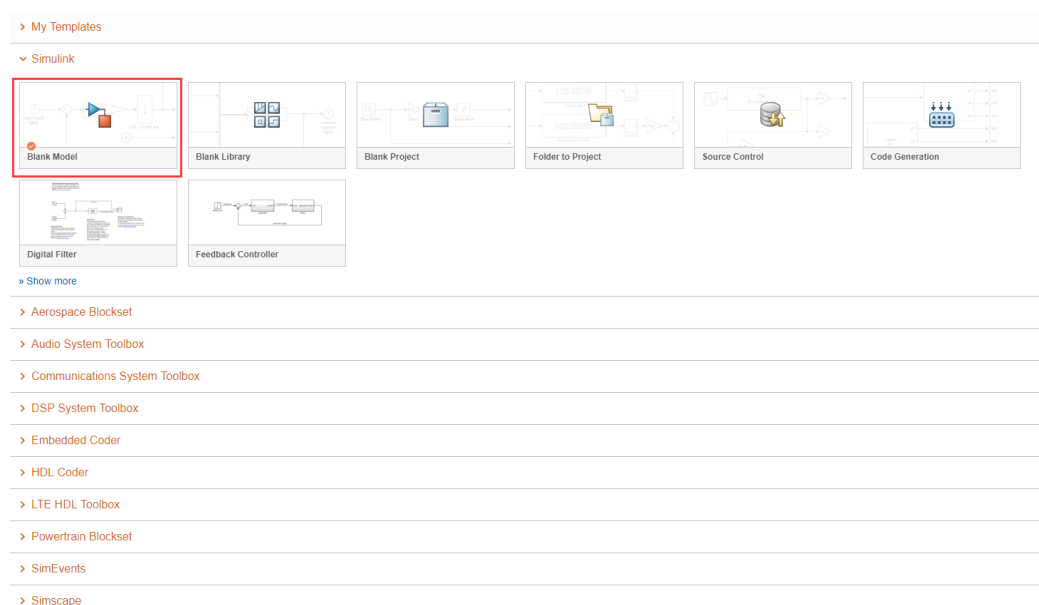


图 3-23

此时新建了一个 Simulink 模型，可以看到此时还没有命名，点击保存，将 Simulink 模型保存到用户的目录中。接下来，就开始设计第一个 Simulink 的仿真模型。首先点击 Simulink 的仿真库，如图 3-24 所示。

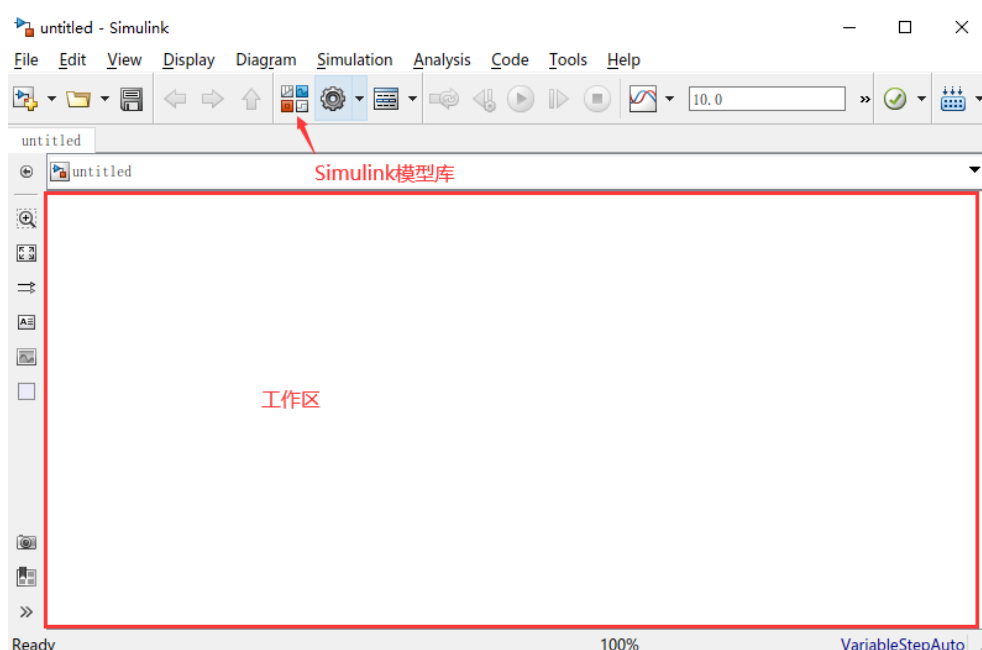


图 3-24

在 Simulink 仿真库当中有不同的模块，在进行 SDR 设计的时候，主要需要使用到的 4 中不同的工具包如下：

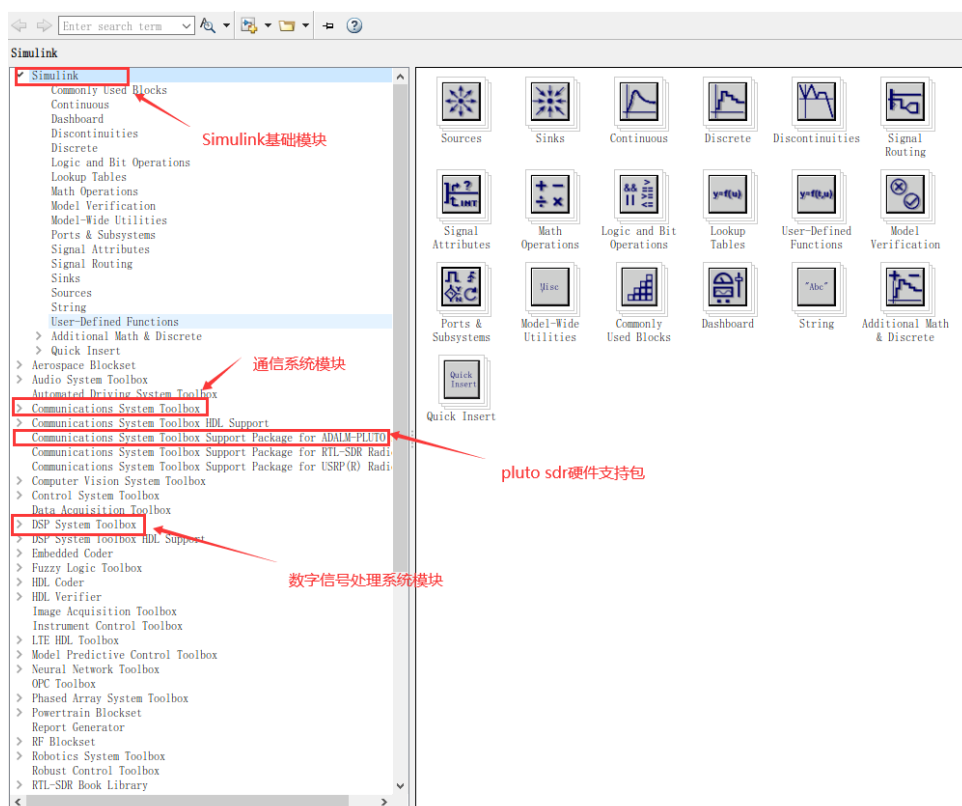


图 3-25 Simulink 常用支持包

在这里，首先设计一个最简单的模型，其分为三个部分，一个信号源模块，一个中间延时模块，和一个接收信号的 time scope 模块。

首先加如信号源模块到 Simulink 仿真模型当中。

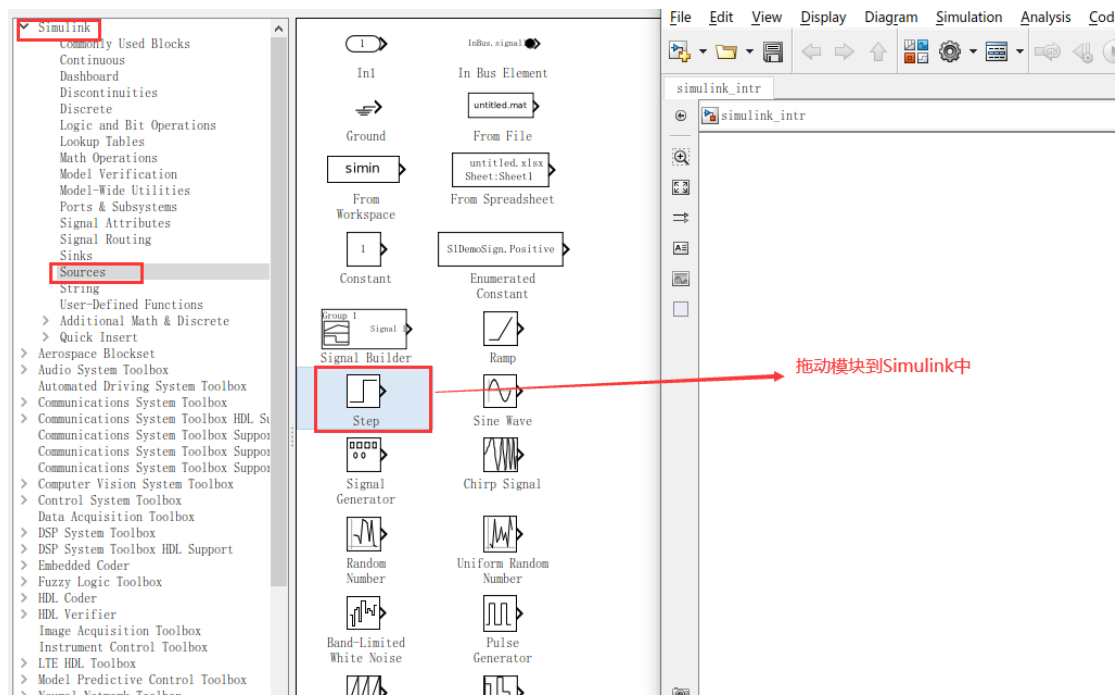


图 3-26

然后以同样的方式，加如延时模块和 time scope 模块。

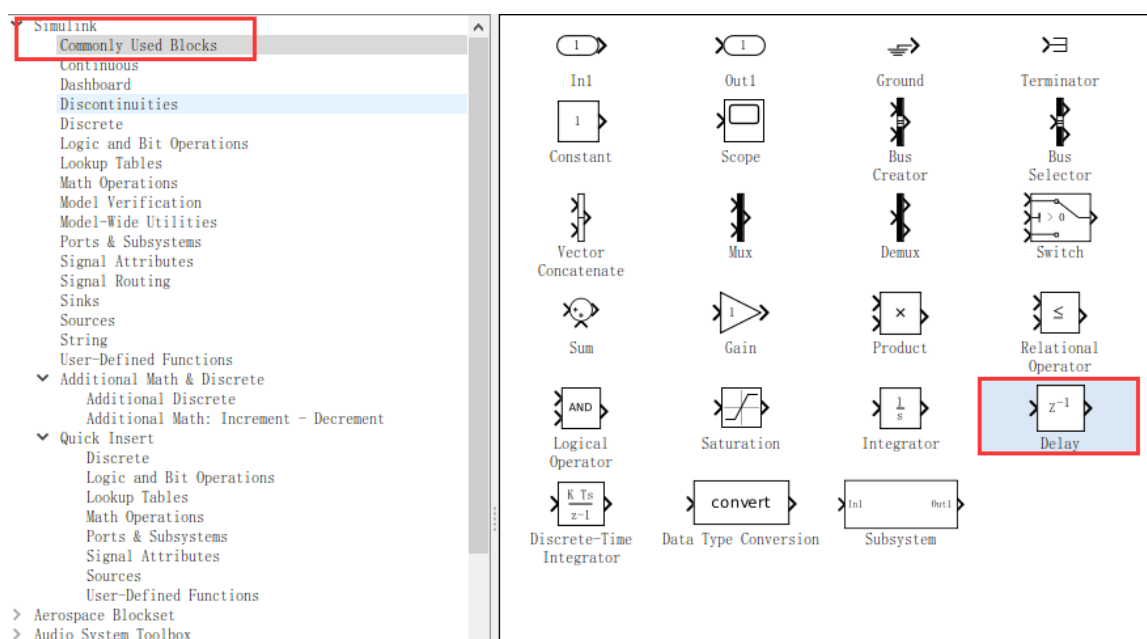


图 3-27

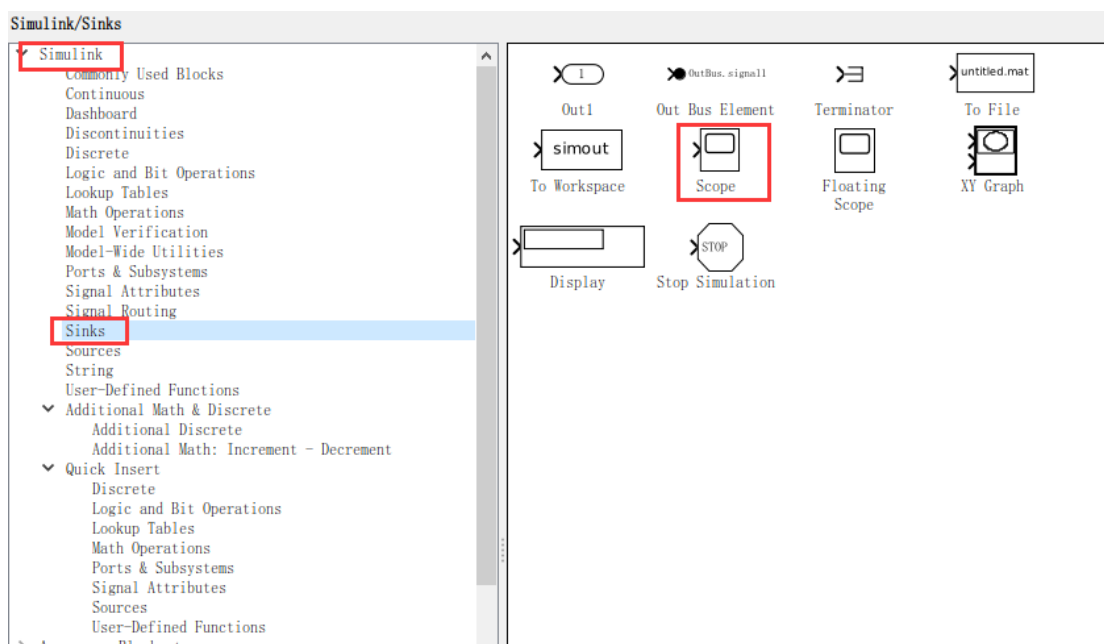


图 3-28

在 Simulink 中按照如下方式将这几个模块进行连接：

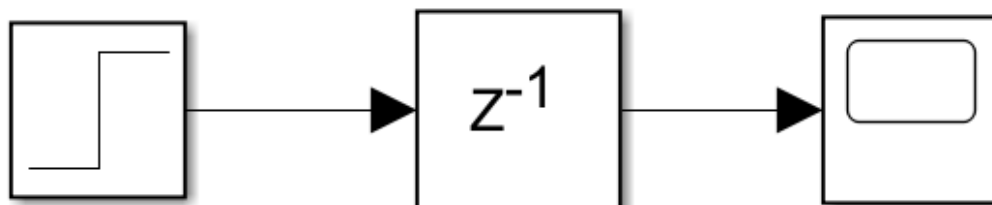
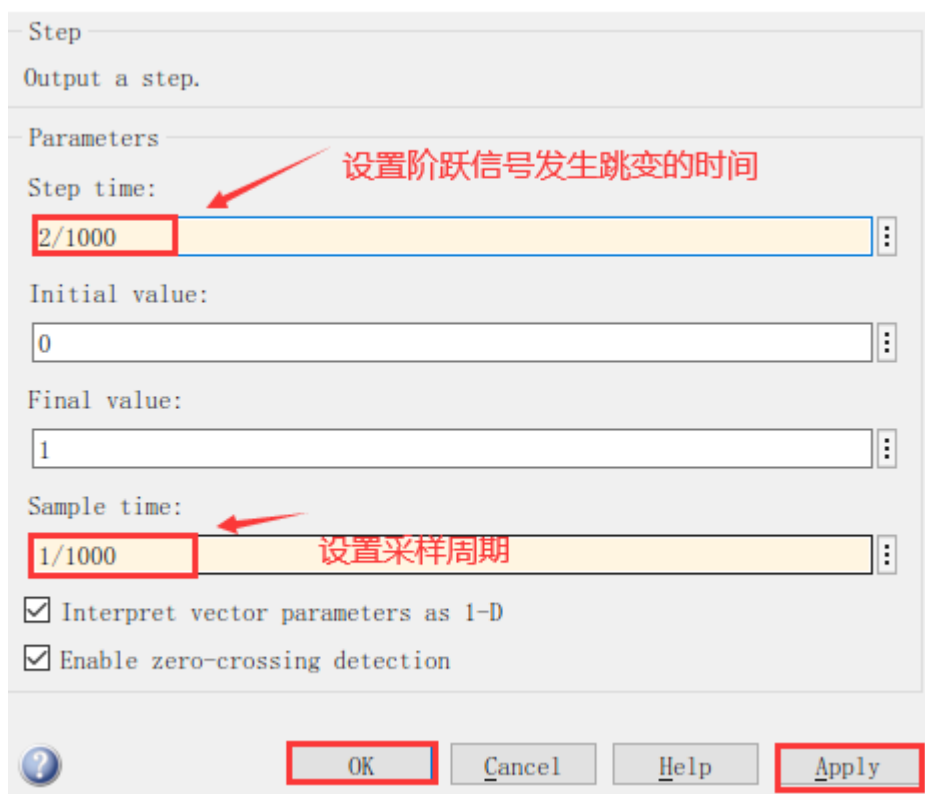


图 3-29

接下来读各个模块进行设置，首先双击阶跃信号模块，对其进行设置。在阶跃信号模块当中，需要设置阶跃信号发生跳变的时间，和阶跃信号的初始值和结束值还有采样周期等如图 3-30 所示。

然后设置延时模块，设置需要延时几个采样周期。



Step

Output a step.

Parameters

Step time: 2/1000 设置阶跃信号发生跳变的时间

Initial value: 0

Final value: 1

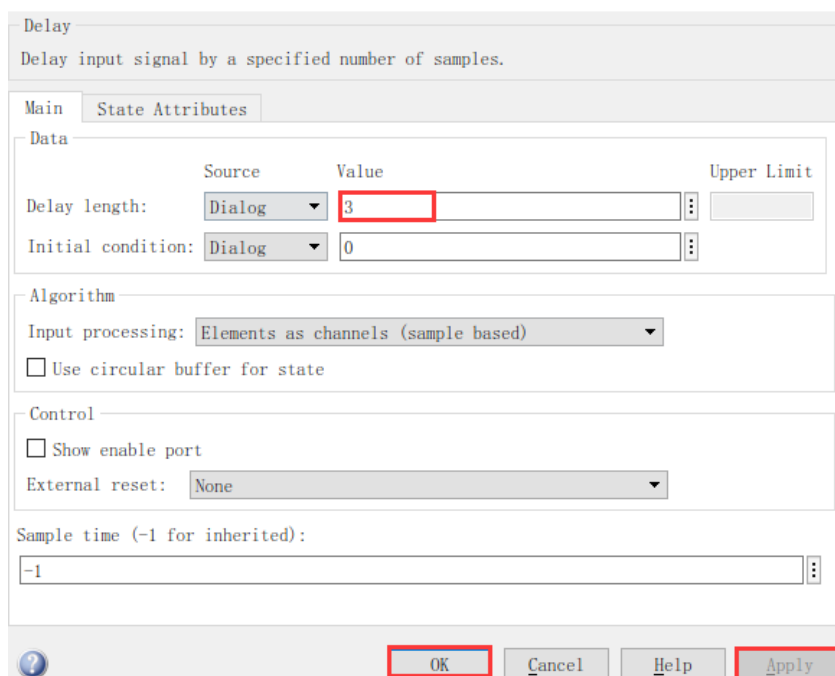
Sample time: 1/1000 设置采样周期

☒ Interpret vector parameters as 1-D

☒ Enable zero-crossing detection

? OK Cancel Help Apply

图 3-30 阶跃信号模块设置



Delay

Delay input signal by a specified number of samples.

Main State Attributes

Data

	Source	Value	Upper Limit
Delay length:	Dialog	3	
Initial condition:	Dialog	0	

Algorithm

Input processing: Elements as channels (sample based)

☐ Use circular buffer for state

Control

☐ Show enable port

External reset: None

Sample time (-1 for inherited): -1

? OK Cancel Help Apply

图 3-31 延时模块设置

接下来就可以设置仿真时间和开始进行仿真了。设置仿真时间先在上方的菜单栏中设置，如图 3-32 所示。然后点击仿真按钮即可开始仿真。

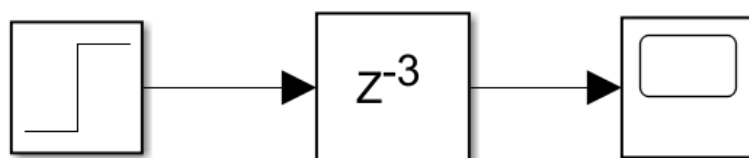
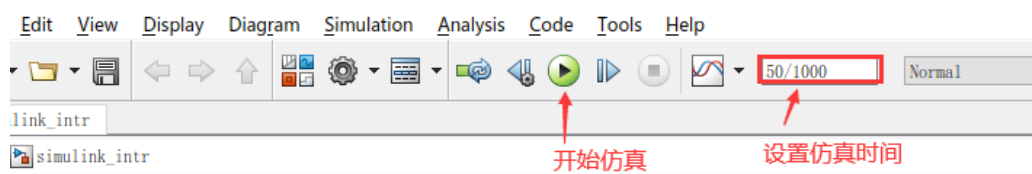


图 3-32

双击打开 time scope 可以查看仿真的信号，可以看到产生了一个阶跃信号，接下来，我们对 time scope 进行设置，从而更好地来分析时间序列。点击菜单栏当中的 view → style。

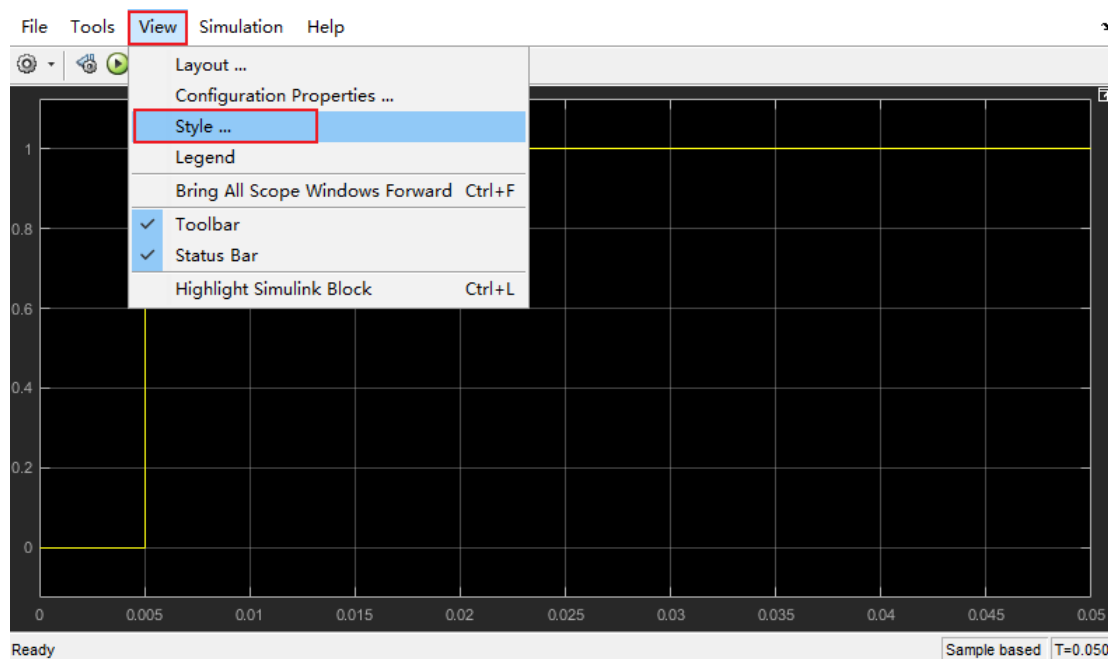


图 3-33 仿真结果

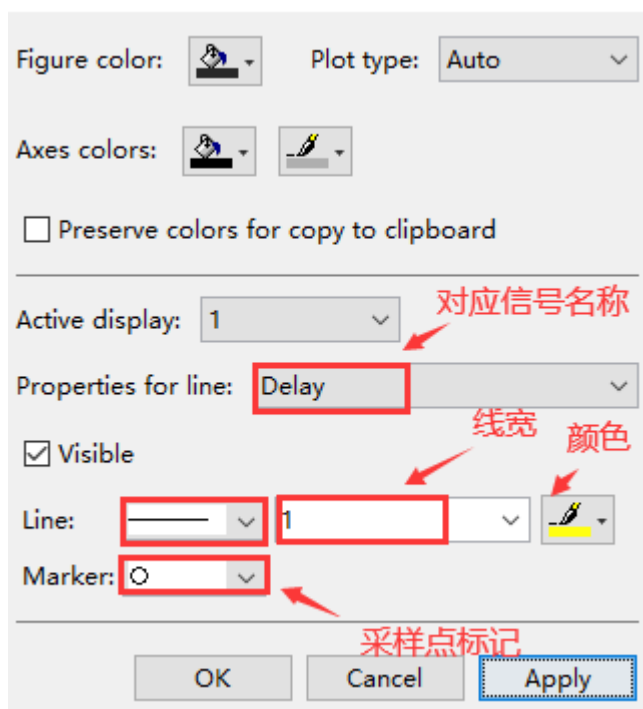


图 3-34

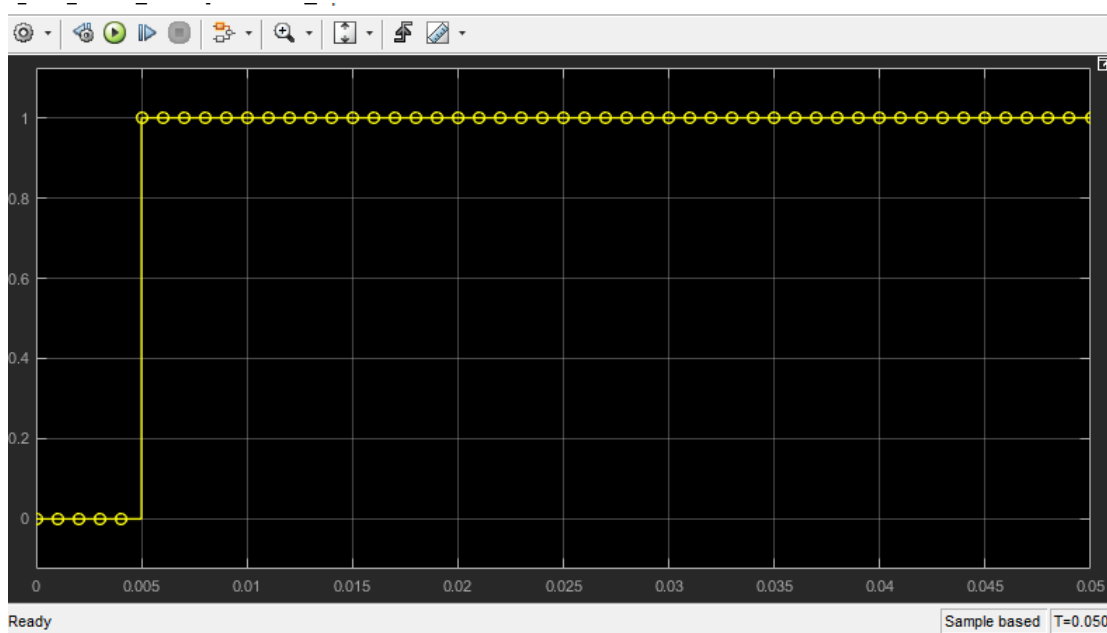


图 3-35 修改设置之后的结果

接下来再次修改 time scope 模块，让其可以同时显示多个信号，点击菜单栏设置，设置输入的信号为 2。

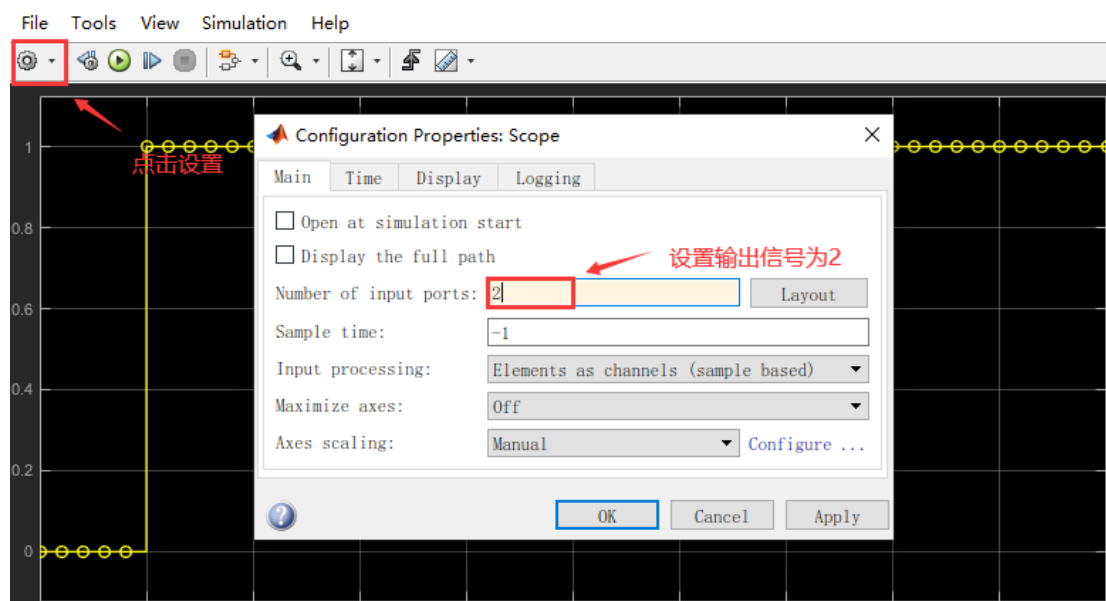


图 3-36

更改过后可以看到 Time scope 模块多出了一个信号输入接口，将这个信号输入接口与未经过 delay 的信号源输出相连。

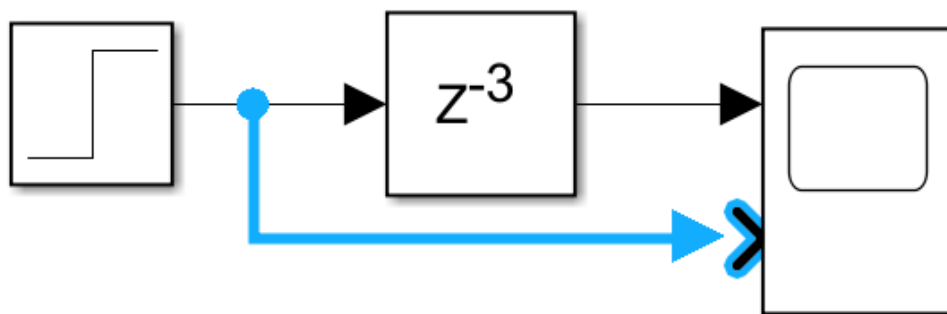


图 3-37

然后再次运行仿真，观察结果，在这里，可以对输入新的输入信号进行和之前同样的操作，让其以采样点的形式展示。然后选择设置，选中显示信号的名称。

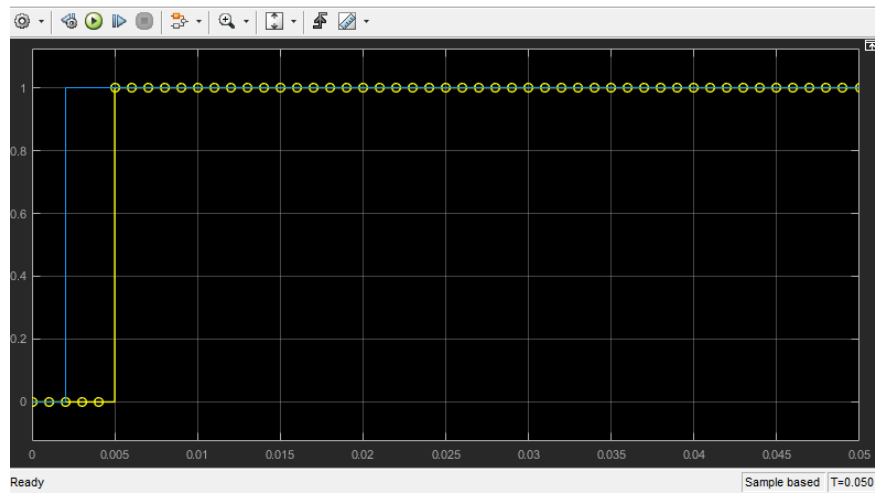


图 3-38 输入更改为 2 之后的仿真结果

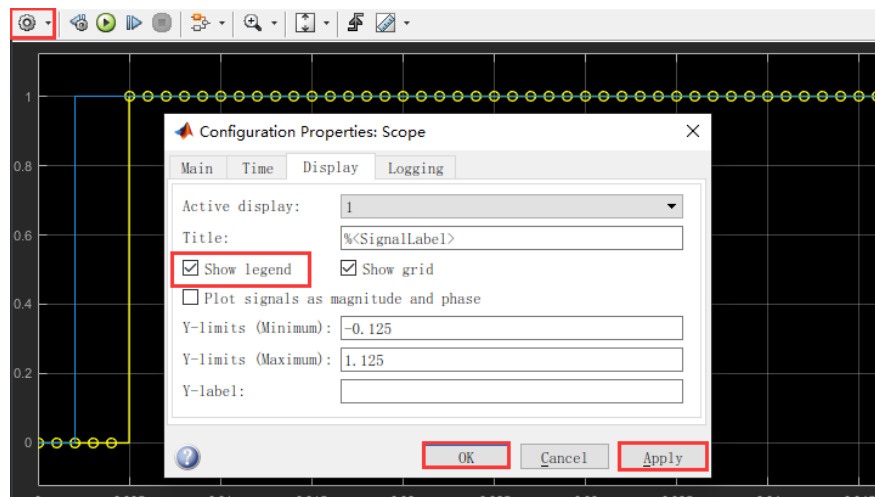


图 3-39 显示不同信号信息

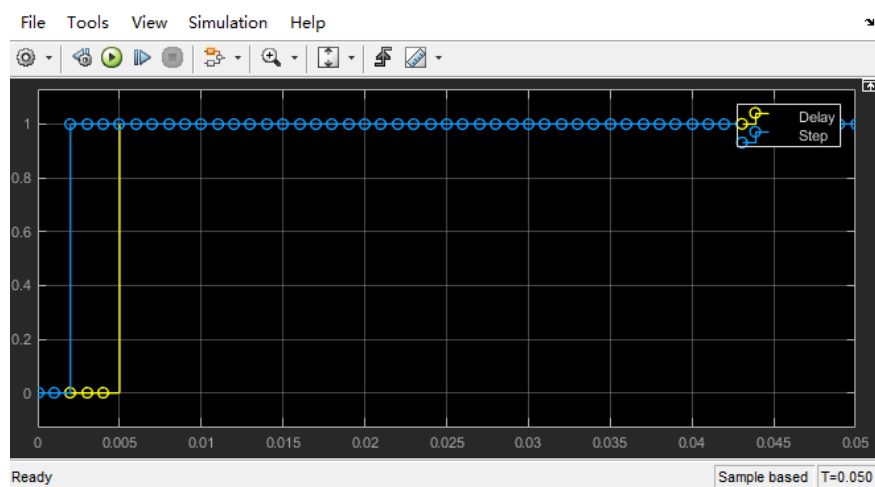


图 3-40

可以看到修改之后,能够清晰地得到输入与输出信号之间的关系。Time scope 模块,除了可以让信号都显示在同一图表外,还可以对几个信号单独显示,点击上方 **view** → **layout**, 可以设置显示的类型。

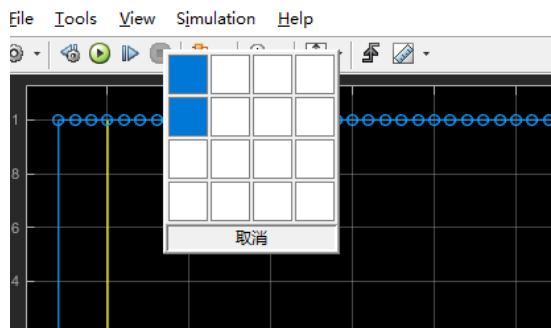


图 3-41

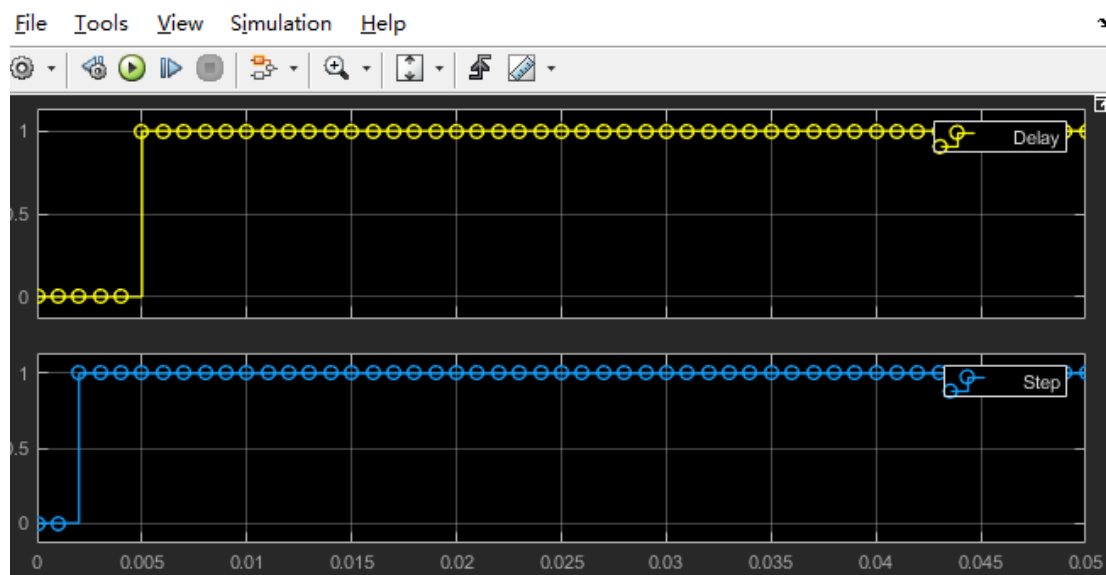


图 3-42

3.7.2 Simulink 当中的变量和参数

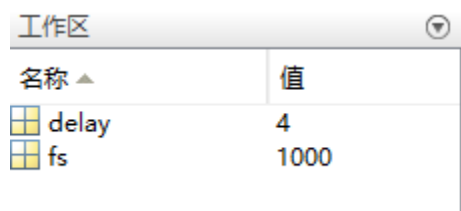
参数化是一个很重要的方面,使模型拥有了灵活性和重用能力。变量可以创建到 **Matlab Workspace**, 然后在 **Simulink** 模块里设置值(变量名可以直接输入到参数盒里,代替数字值),也可以填到别的地方,比如 **Simulation Stop Time**。

另一个有用的技术是将 **Simulink** 仿真的输出数据写入 **MATLAB Workspace**, 它创建一个新变量(或一组变量)。随后可以使用这些新变量来生成模拟输出的图表,或者对模拟结果进行其他分析。

在这个例子当中,我们还是以上一个 **Simulink** 模型为基础,将 **Simulink** 模型复制一份,然后更改其名字为 **step_delay.slx**。本次实验将在上一个例子的基础上

进行修改。

首先，在 Matlab 的命令行中，创建一个变量，用于表示采样率。直接在命令行中输入 `fs = 1000`; 这个表示 `fs` 的是采样率为 1000Hz。然后输入 `delay = 4`，表示 `delay` 的值是 4。



名称	值
delay	4
fs	1000

图 3-43

然后回到 simulink 当中，修改其中的参数，使得 Simulink 模型能够参数化。

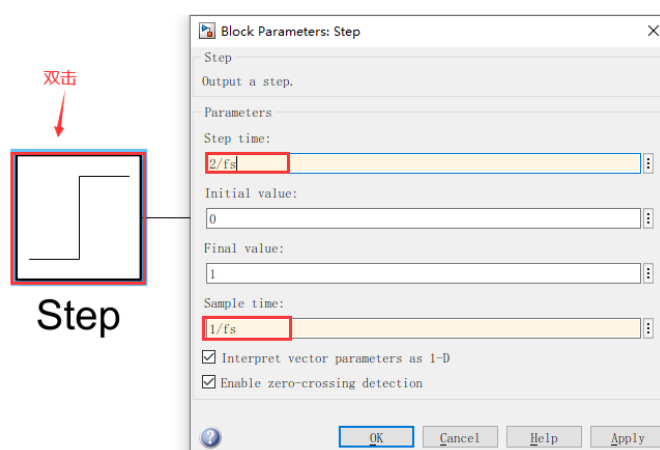


图 3-44 修改

然后再修改 `delay` 的值：

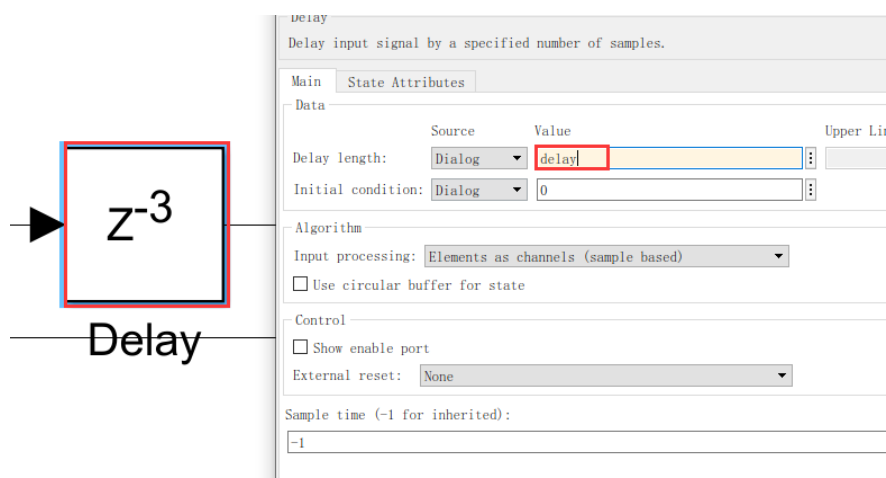


图 3-45

最后修改仿真时间了再次运行程序：

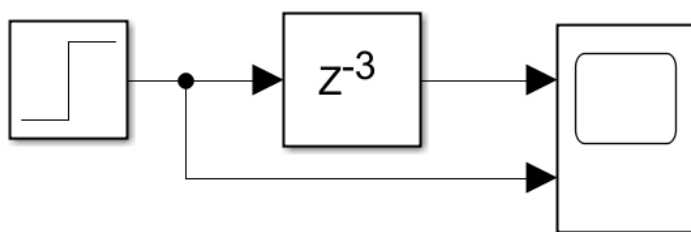
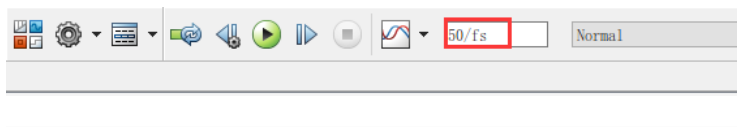


图 3-46

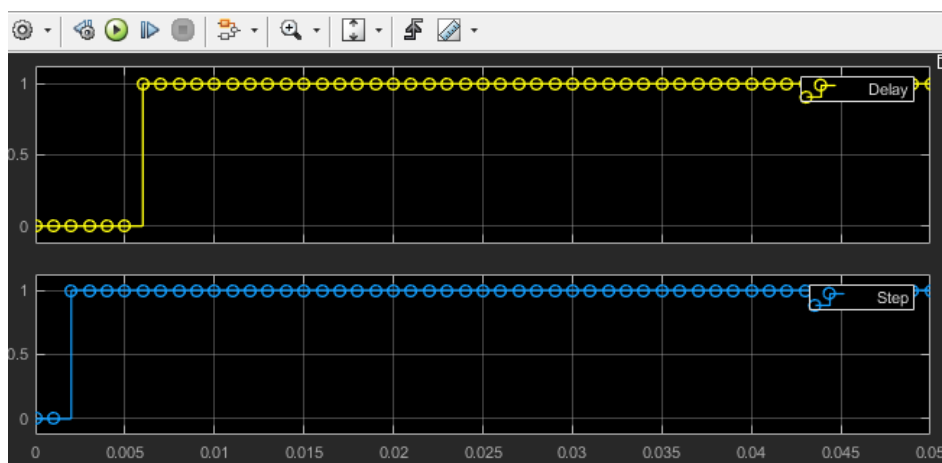


图 3-47

然后再 matlab 的命令行当中，修改 delay 的值，修改之后再次运行仿真观察结果。

```
>> delay = 10  
  
delay =  
  
10
```

图 3-48

可以看到更改 delay 值过后，仿真得到结果延时也变大了。

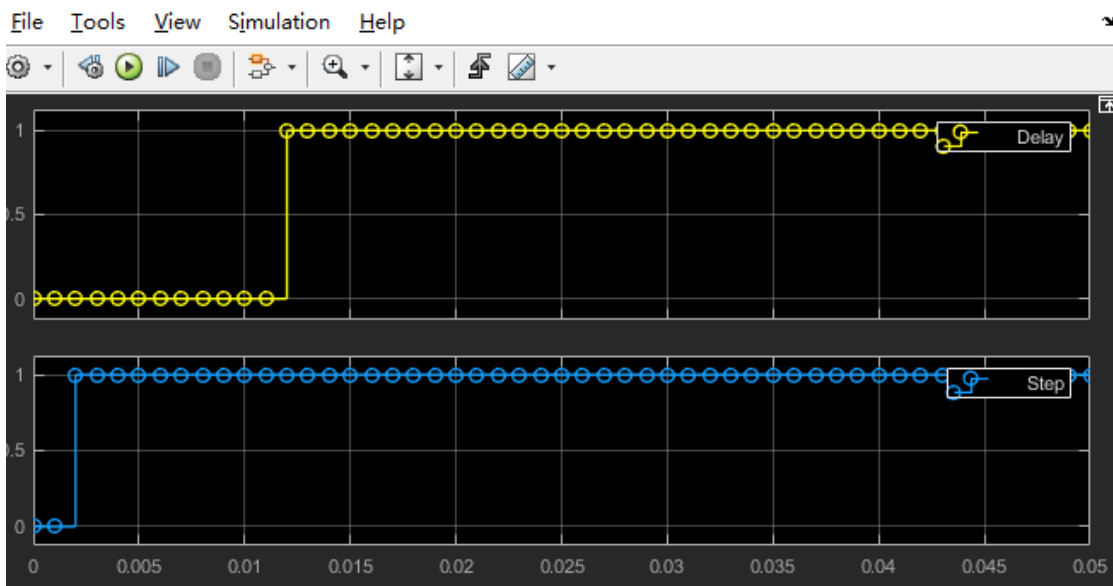


图 3-49 增大延时值之后的仿真结果

3.7.3 观察频域图像

在前面介绍了 Simulink 的基本的使用方法，并用 time scope 观察了时域的信号，在数字信号处理当中，除了观察时域波形外经常会在频域中来观察信号。在接下来的例子当中，会使用频谱仪来观察信号。

步骤	操作
1	打开 Matlab
2	打开 Simulink
3	新建一个 Simulink 模型 spectrum.slx

然后依次添加信号源模块，加法器模块和频谱分析仪模块。

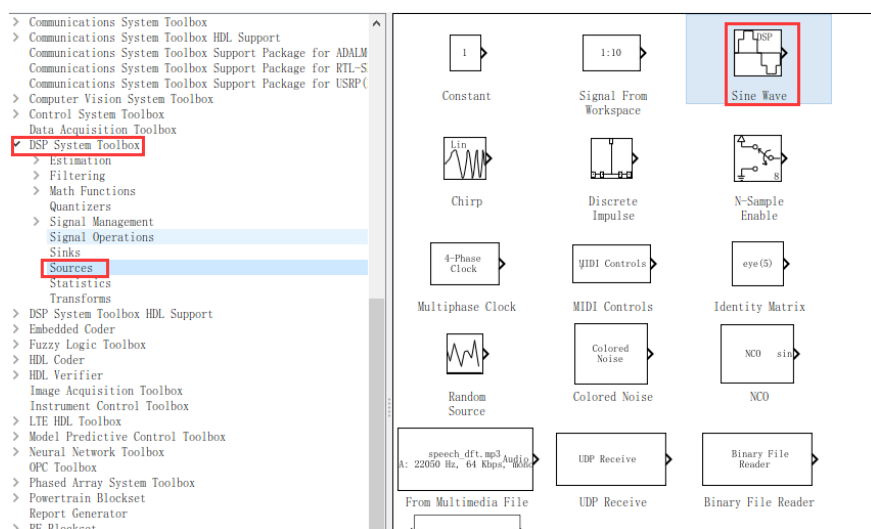


图 3-50 正弦信号发生器

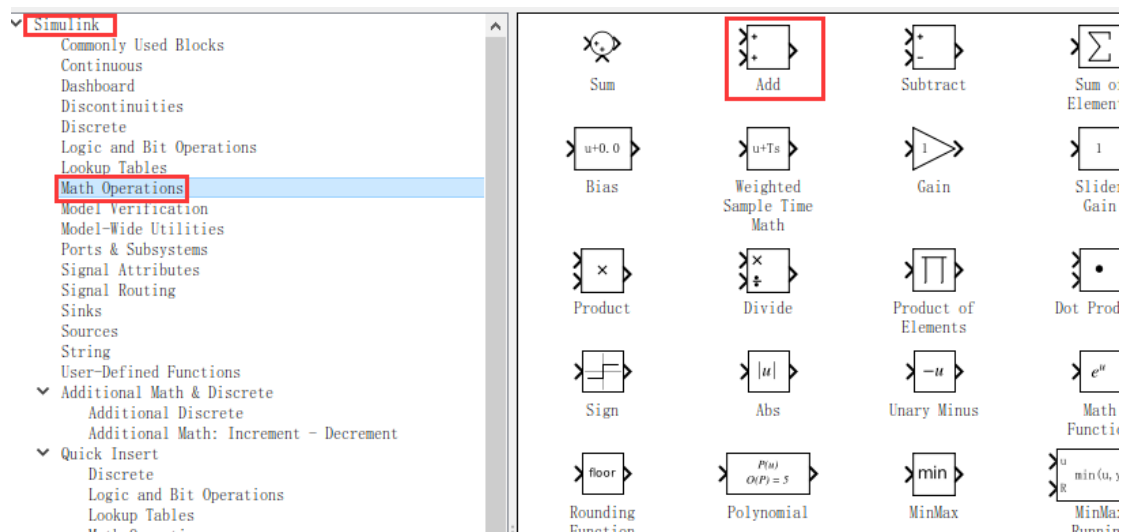


图 3-51 加法器模块

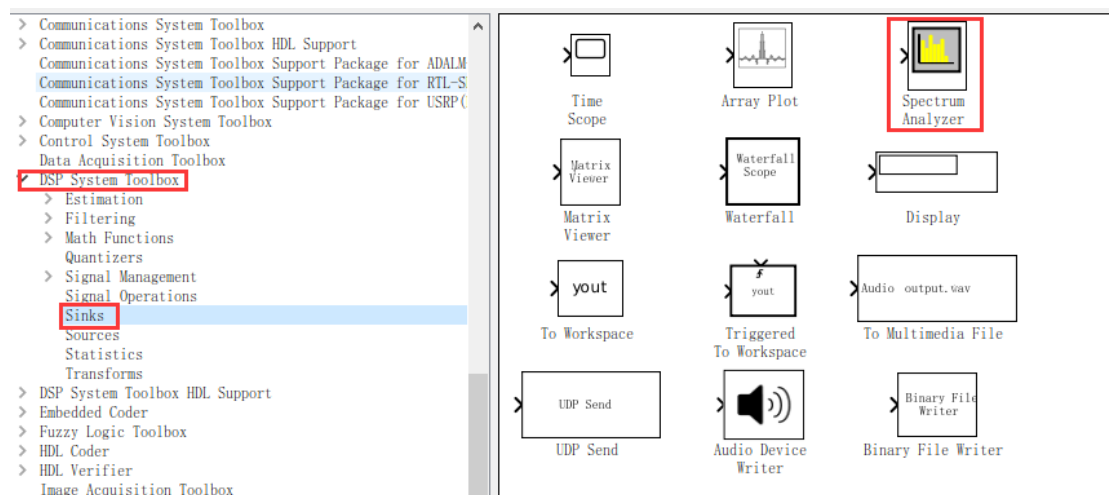


图 3-52 频谱分析仪模块

按照下图的方式依次添加上面几个模块：

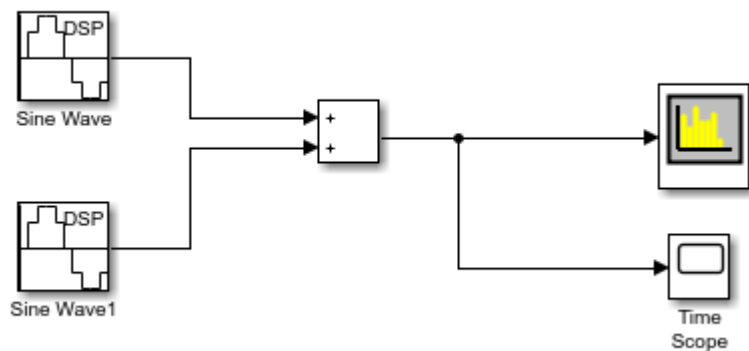


图 3-53

接下来设置输入的两个信号的频率，双击打开其中一个信号发生器。

设置振幅为 1，

频率为 20Hz，

信号的初始相位为 0，

采样模式为离散，

输出的类型为实数，

生成正弦信号的策略为基于三角函数，

采样周期设置为 1/1000

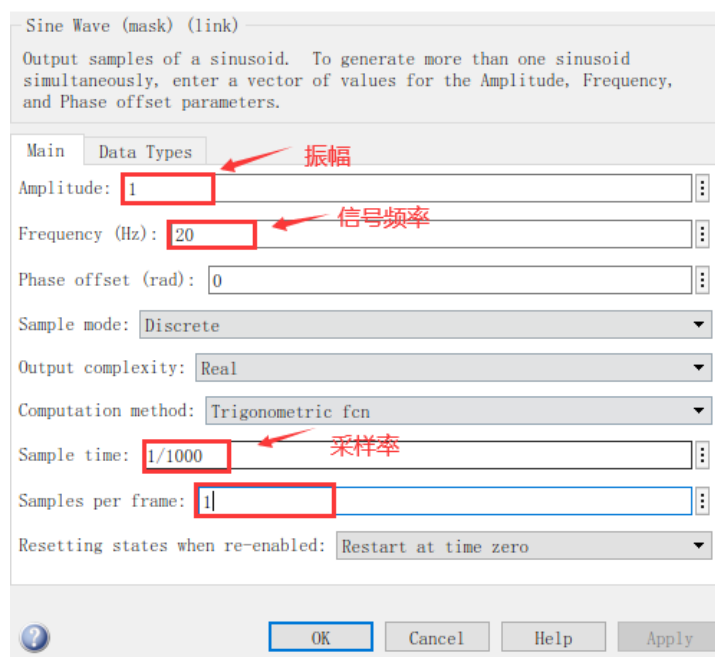


图 3-54 正弦信号发生器设置

对于另外一个信号发生器采取相同的操作，

设置振幅为 1，

频率为 50Hz，

信号的初始相位为 $\pi/2$

这样就产生了两个振幅和初始相位都不同的信号。然后通过加法器之后，送给频谱分析仪进行观察。

设置仿真时间为 3s，然后点击开始仿真，就能够观察仿真结果了。

仿真的结果如下图所示：

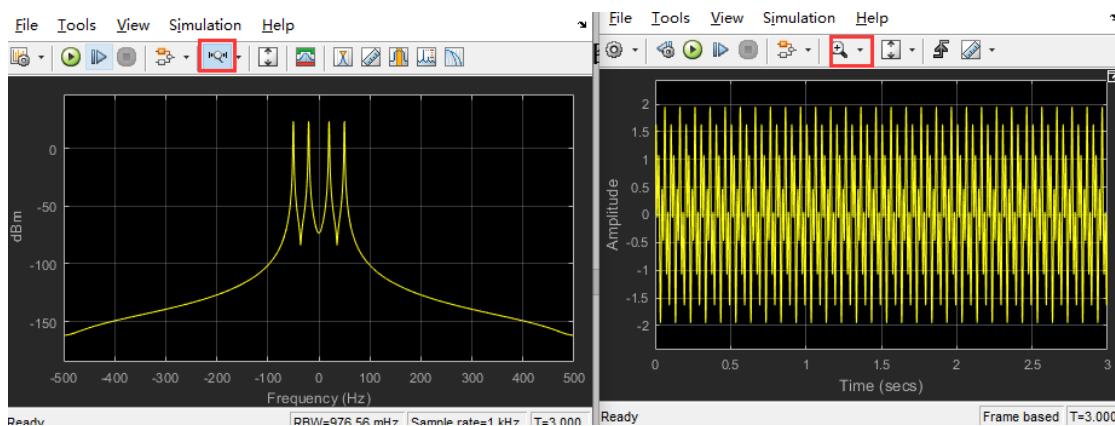


图 3-55 仿真结果

使用上图中的红色位置的放大镜，可以将放大图中的波形，从频谱上可以看到，在 20Hz 和 50Hz 处有较强的信号，这也正对应着前面的两个输入信号。从时域波形上很不易看出来的信号的信息，在时域中很直观地就表现了出来。你也可以在这个实验地基础上，尝试其他类型的输入，然后来观察输出的结果。

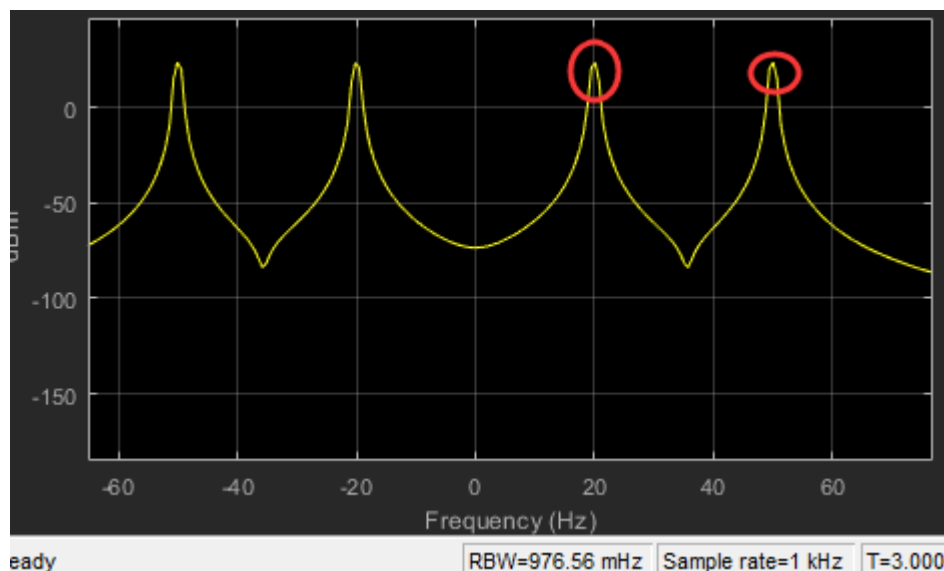


图 3-56

3.7.4 数据类型

Matlab 和 Simulink 中可以使用多种多样的数据类型，默认情况下一般是双精度浮点型的。大多数 Simulink 源模块都会输出浮点类型的数据。常见的数据类型如下：

数据类型	描述
Double	双精度浮点
Single	单精度浮点
Boolean	布尔型
Uint8, Uint16, Uint32	无符号整型（8bit, 16bit, 32bit）
Int8, Int16, Int32	有符号整型（8bit, 16bit, 32bit）
Fixdt(s,n,f)	定点数，s 表示符号位 1 为有符号数，0 为无符号数，n 数据位宽，f 为浮点数位宽

这些类型也可以用表示复数的数据类型，若用来表示复数的时候，一般会在数据类型之后加上(c)来表示复数。

复数在数字信号处理值中经常会使用到，虽然我们所处的世界是真实的，但是复数能够帮我们更好地来认识这个世界地规律。接下来将会搭建一个 Simulink

模块，来完成实数和复数的转换。

打开资料当中的模型 `real_to_complex.six`, 打开的模型如下图所示：

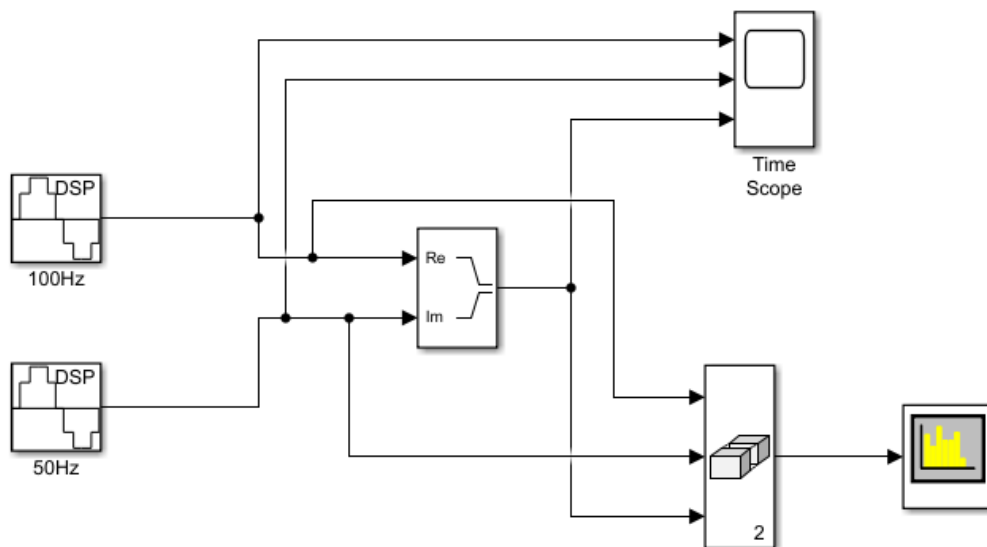


图 3-57 实部虚部合成复数仿真模型

在这个仿真模型当中，有两个信号源分别产生 100Hz 和 50Hz 的正弦波，然后将这两个信号通过复数合成器，一个信号作为实部，另一个信号作为虚部，通过 `real-imag to complex` 这个模块之后可以产生一个复数信号。然后使用 `time scope` 和 `spectrum analyser` 对信号进行观察，仿真的结果如下：

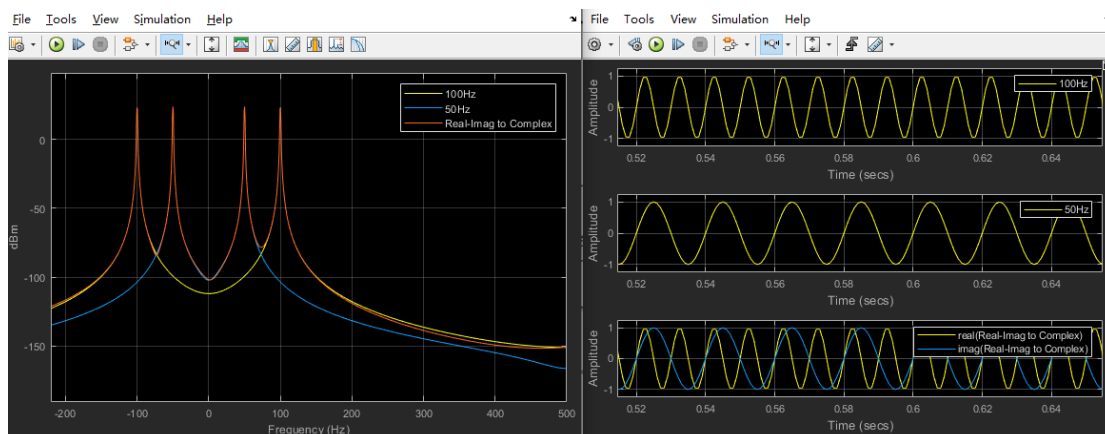


图 3-58 实数复数转换仿真结果

从仿真结果中可以看到，生成的复数中同时包含了两个频率的分量，从频域波形中可以直观地看出，橘红色的波形是复数信号的频域波形，从这里可以看到这个信号包含两个频率的分量。

3.7.5 从文件输入和输出到文件

在使用 Simulink 的时候,经常需要把需要的文件导入到 Simulink 中作为数据源,又或者,需要把 Simlink 仿真的结果导出到文件当中,进行进一步的分析。

接下来的两个例子,会分别完成从 Simulink 导出数据到 Matlab 和从读取音频文件到 simulink。

3.7.5.1 导出文件到 Matlab

打开 `write_mat_file.slx` 仿真模型,可以看到这个简单的模型的结构如下图所示:

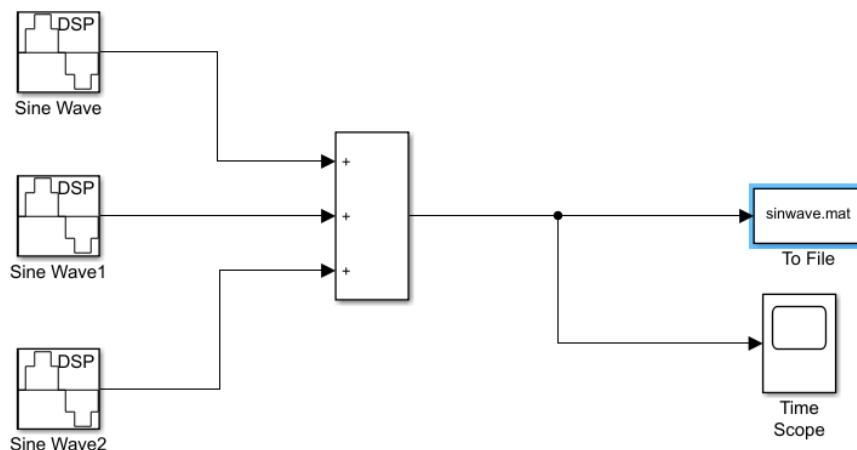


图 3-59 导出数据仿真模型

可以看到这个仿真模型当中,有 3 个正弦波的信号发生器,其频率分别为 20Hz,30Hz,50Hz。通过加法器相加之后,给到 To File 模块,To file 模块能够实现将数据导出到 Matlab 中,双击这个模块查看其配置。

可以看到这个模块有可以设置导出的文件的名称,然后是导出的在这个文件中的变量的名字。

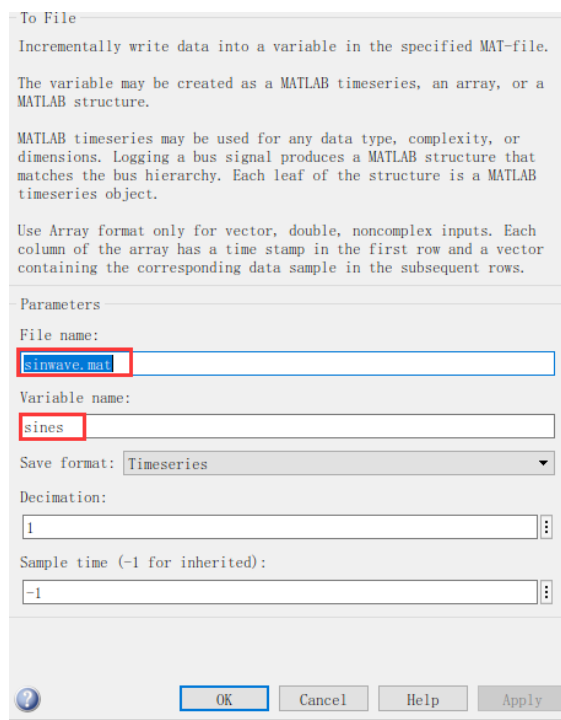


图 3-60

仿真运行完成之后，可以在 matlab 的文件导航栏中看到新生成的一个.mat 格式的文件。

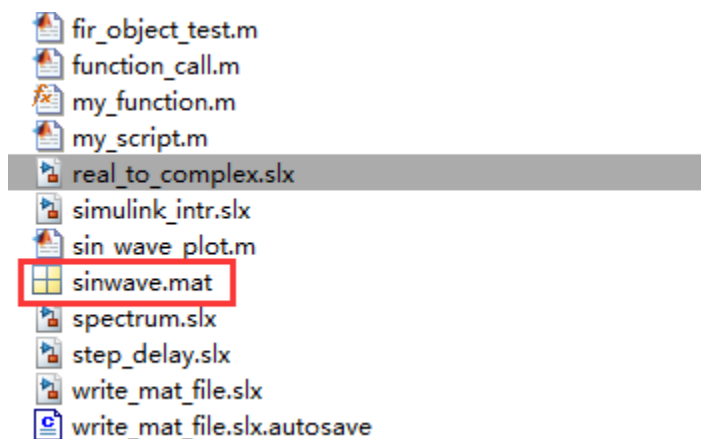


图 3-61 导出的文件

此时双击这个文件，或者在命令行中输入 `load('sinwave.mat')`，都能够将文件加载到工作区当中，可以看到会加载一个变量到工作区，并且这个变量的名字，正式前面在导出文件的时候设置的名字。

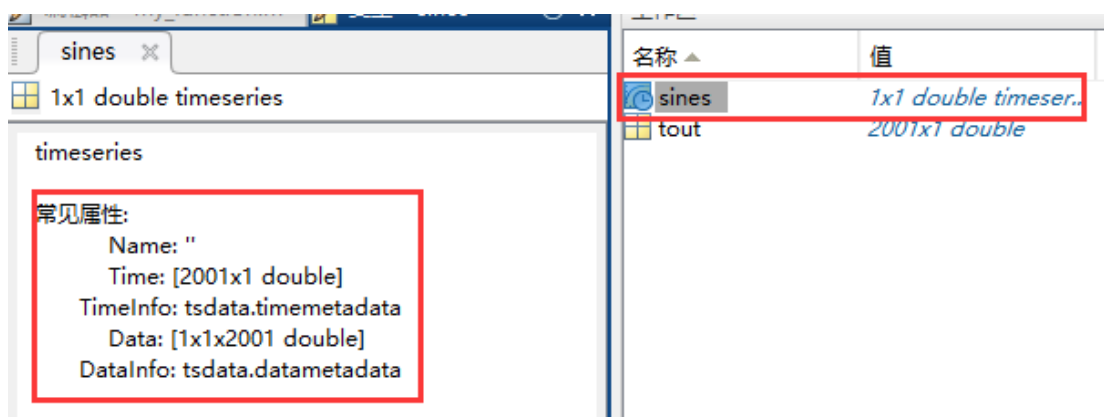


图 3-62

在命令行中输入 `sines.plot` 可以将导出的变量进行绘制。

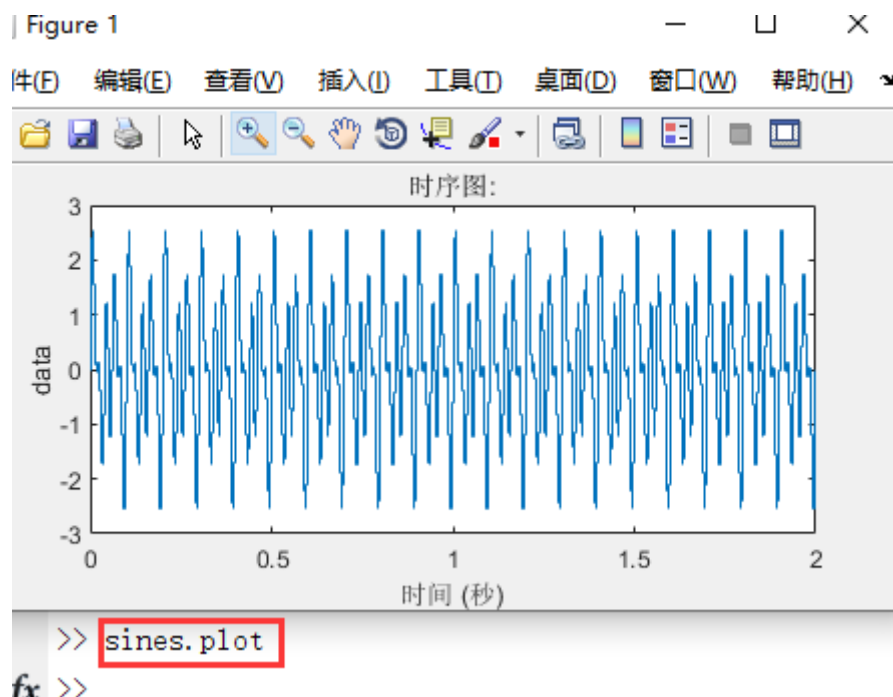


图 3-63 绘制出时序图

3.7.5.2 导入文件到 Simulink

前面介绍了，如何将文件从 Simulink 中导出，在这里介绍一下如何从外部导入音频文件到 simulink 中，因为在之后可能会多次导入音频文件，所以在这里以导入一个音频文件为例子，介绍如何使用 From file 这个模块。

打开仿真模型 `read_aduio_file.slx`:

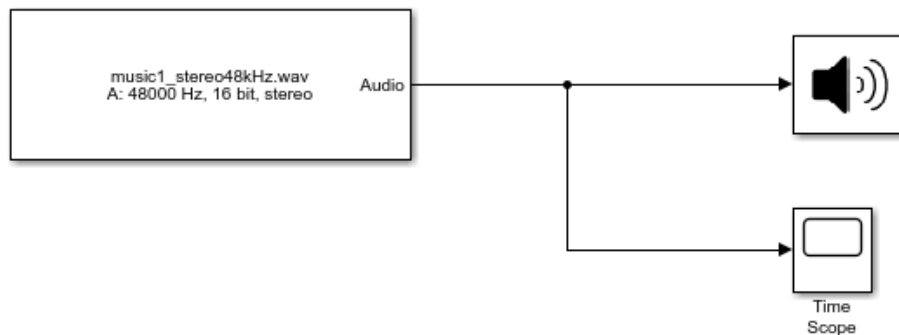


图 3-64

该仿真模型会通过 **From Multimedia File** 这个模块从保存有音频文件的目录下导入文件，然后将导入的音频文件播放出来，这样就能够听到音乐了。

3.8 小结

在这一章当中，首先介绍了 **Matlab** 的简单的使用方法，这些都是最基本的用法，当然关于 **Matlab** 的其他使用方法，用户可以查看相关的资料或者书籍，进行进一步的了解。

然后还介绍了 **Simulink** 的使用方法，在之后的内容当中 **Simulink** 多次使用后会到，在这里也只是简单介绍了 **Simulink** 的使用方法，有了这些基础，在之后介绍调制解调的相关操作时，会更加得心应手。

第4章 信号的调制与解调

在前面介绍过，ANTSDR 采用的射频芯片为 AD936x,这颗芯片进行数据无线信号传输的时候，采用的是 IQ 两路数据，由 ad936x 采集到的数据，又可以称作“复基带数据”。我们所处的世界是真实的，实际上只存在实数信号，复数信号或者虚部分量在真实的世界中是不存在的，然而，在描述信号的调制与解调的时候，使用复数能够很方便来表示调制解调的过程。

在本章中，会介绍有关实数和复数信号的一些基本的概念，然后会简单介绍一下调制解调的概念，最后会介绍在模拟和数字通信中非常重要的正交调制。

4.1 实数信号和复数信号

在模拟和数字通信中，经常使用复数来表达信号，这只是为了简化表达和计算。联系复数和实数的经典公式就是欧拉公式：

$$\begin{aligned} e^{j\omega t} &= \cos \omega t + j \sin \omega t \\ e^{-j\omega t} &= \cos \omega t - j \sin \omega t \end{aligned} \quad 4-1$$

通过重新整理 $\cos$ 和 $\sin$ 的表达式，可以使用复指数来表示 $\cos$ 和 $\sin$

$$\begin{aligned} \cos \omega t &= \frac{e^{j\omega t} + e^{-j\omega t}}{2} \\ \sin \omega t &= \frac{e^{j\omega t} - e^{-j\omega t}}{2j} \end{aligned} \quad 4-2$$

可以看到通过欧拉公式，很容易就将实数信号和复数信号相联系起来了。从这里也可以很直观的看出，一个实数信号在频域中有负的频率分量。如前面在进行频谱仪的使用的时候，来观察单音信号的频率的时候，可以看到信号具有负的频率分量。

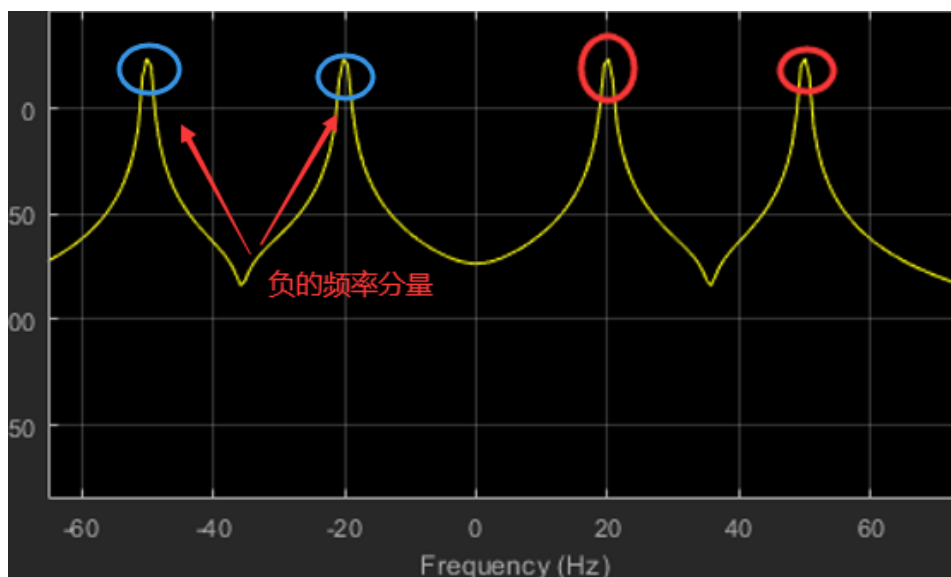


图 4-1 正弦单音信号同时具有正负频率分量

4.1.1 简单的三个正弦信号在频域的表达

考虑这么三个单音信号，频率分别为 100Hz, 200Hz 和 300Hz，对应的幅度分别是 10、1 和 4。将这三个信号叠加，可以得到。

$$s(t) = 10 \cos 2\pi 100t + \cos 2\pi 200t + 4 \cos 2\pi 300t \quad 4-3$$

对于这样一个信号，其时域图大概如下：

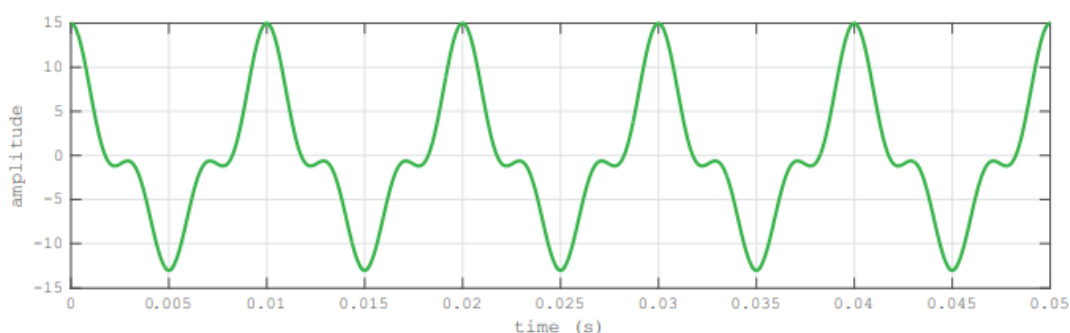


图 4-2 信号时域图

从时域图上，很难看出信号的特征，但是在频域上，却可以很容易地就分析出这个信号地特点。

从频域上来分析信号的特征时，只需要绘制出幅度-频率和相位-频率的响应图，就能够很直观地看到信号的特性。在知道信号的表达式的时候，分析信号的

幅频响应是比较简单，但是大多数时候，信号的表达式都是未知的，这个时候使用频谱分析仪，就能够很容易地帮助我们来分析信号了。

4.1.1.1 在 Simulink 中完成三个信号和的信号分析

在上一章当中，介绍使用频谱分析仪的时候，就已经分析了两个信号的幅频响应，在这里再一次来实现一下这个 demo，从而能够完成上面所说的三个信号之和的分析。

打开仿真模型 `time_to_frequency_domain.slx`，可以看到本次实验的仿真模型如下：

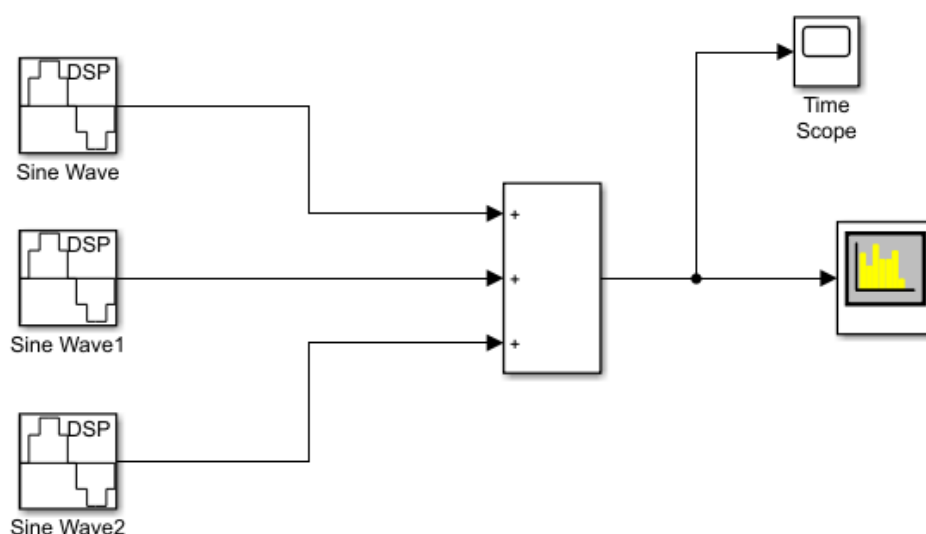


图 4-3 仿真模型

点击仿真开始运行，可以同时观察 time scope 和 spectrum 当中的结果。

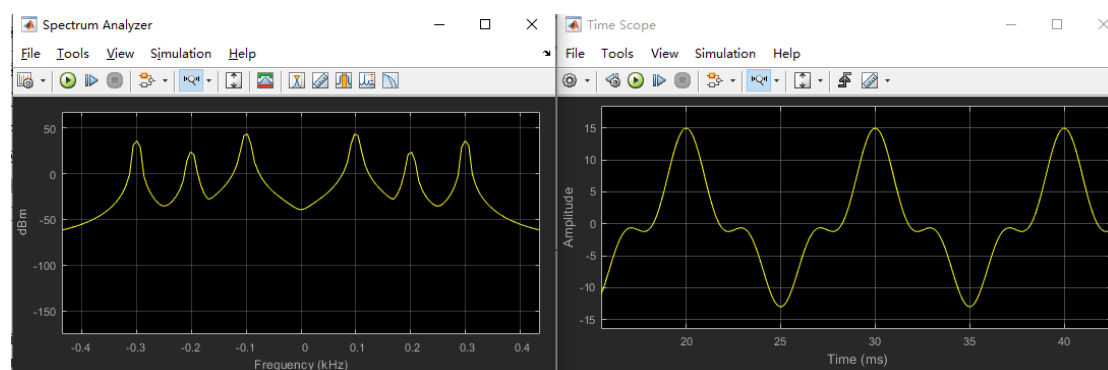


图 4-4 仿真结果

这里需要注意的是，spectrum 的纵坐标的单位是 dBm，以对数的形式显示的，所以在对应频率的幅度是以对数的形式展示的。

但还是很容易就从频域波形上看出来，信号的主要的频率分量。右边是显示

的时域波形,从时域波形上分析,很难得出详细的信息,但是从频域就很容易了。

4.1.1.2 三个信号之和在复频域的表达

虽然上面已经很直观地看到了这三个信号在复频域的结果,但是在这里我们还是稍微分析一下这个结果是如何产生的。

将这个信号用复指数表示如下

$$\begin{aligned}
 s_I(t) &= 10\cos(2\pi 100t) + \cos(2\pi 200t) + 4\cos(2\pi 300t) \\
 &= 10\left(\frac{e^{j2\pi 100t} + e^{-j2\pi 100t}}{2}\right) + 1\left(\frac{e^{j2\pi 200t} + e^{-j2\pi 200t}}{2}\right) + 4\left(\frac{e^{j2\pi 300t} + e^{-j2\pi 300t}}{2}\right) \\
 &= 5e^{j2\pi 100t} + 5e^{-j2\pi 100t} + \frac{1}{2}e^{j2\pi 200t} + \frac{1}{2}e^{-j2\pi 200t} + 2e^{j2\pi 300t} + 2e^{-j2\pi 300t}
 \end{aligned}$$

4-4

由前面的介绍可以知道,正弦信号和余弦信号在频域上同时包含正的频率分量和负的频率分量。所以可以将上面的式子,按照正频率和负频率进行分组。

$$s_I(t) = \underbrace{5e^{j2\pi 100t} + \frac{1}{2}e^{j2\pi 200t} + 2e^{j2\pi 300t}}_{\text{positive frequencies}} + \underbrace{5e^{-j2\pi 100t} + \frac{1}{2}e^{-j2\pi 200t} + 2e^{-j2\pi 300t}}_{\text{negative frequencies}}$$

4-5

然后把正负频率分别绘制到幅频响应图上可以看到,在正负频率上对应的幅度,这也在前面从 Simulink 上仿真得来的结果是一致的。

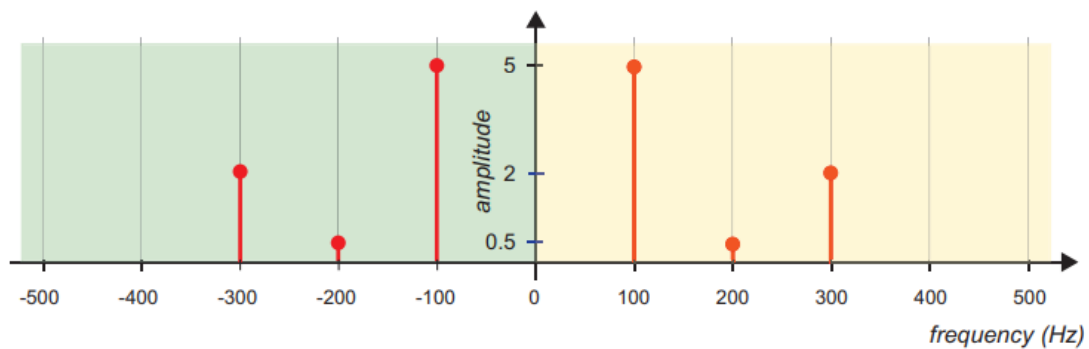


图 4-5

4.2 幅度调制

我们经常用到正交信号,传输两个相位上独立的数据。正交信号的产生过程是,把两个独立的基带信息分别加载到频率相同的正弦波和余弦波的载波上,产生了两个正交分量。这样,两个独立的数据流就可以同时在一个频率上传输了,

它们不会互相干扰。在介绍正交幅度调制之前，先看一下常规的幅度调制。

4.2.1 抑制载波的双边带调幅(AM-DSB-SC)与解调

4.2.1.1 信号调制

一个低频的基带信号 $g(t)$ ，和一个高频载波 $c(t)$ ，的幅度调制过程如图所示。如果 $g(t)$ 只是一个余弦波，比如 $g(t) = A\cos(2\pi f_b t)$ ，那么我们可以用简单的三角运算来算出调制后的信号

$$s(t) = A \cos(2\pi f_b t) \cos(2\pi f_c t) = \frac{A}{2} \left[\cos(2\pi(f_c - f_b)t) + \cos(2\pi(f_c + f_b)t) \right]$$

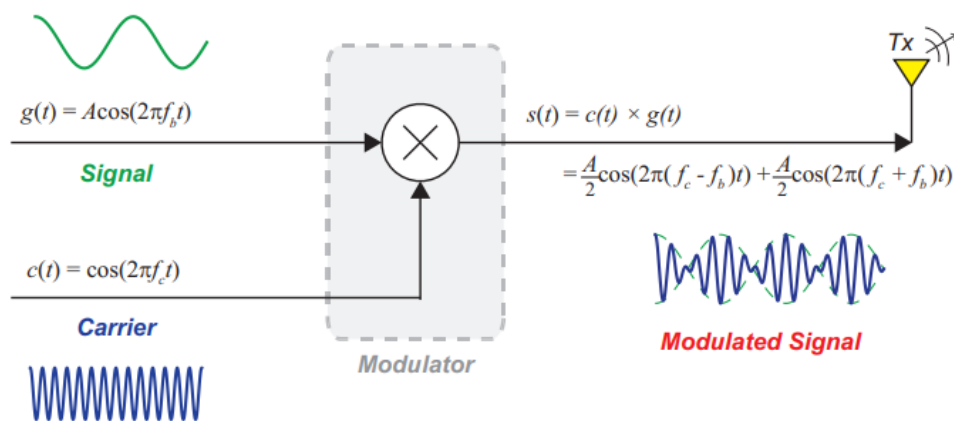


图 4-6 调制单音信号的时域特征

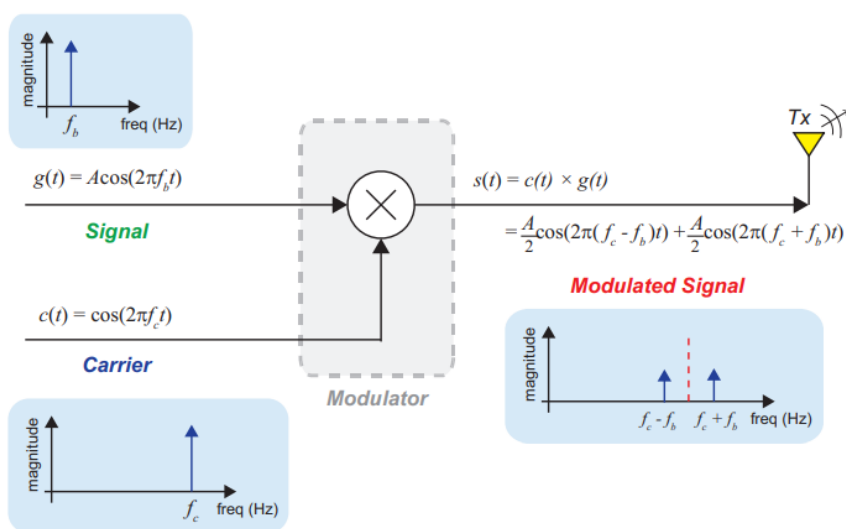


图 4-7 调制单音信号的频域特征

4.2.1.2 信号解调

解调简单的 AM 信号，从表达式上看来很简单，只需要产生一个和调制时同频同相的一个载波，将这个载波和已调信号进行相乘，然后将得到的结果通过低通滤波器就能够将原始的基带信号解调出来了。但是在实际中，想要得到一个同频同相的信号是几乎不可能的，在这里我们以理想的情况来介绍 AM 信号的解调。

理想情况下，可以在接收机产生一个和调制时同频同相的这么一个高频载波信号。

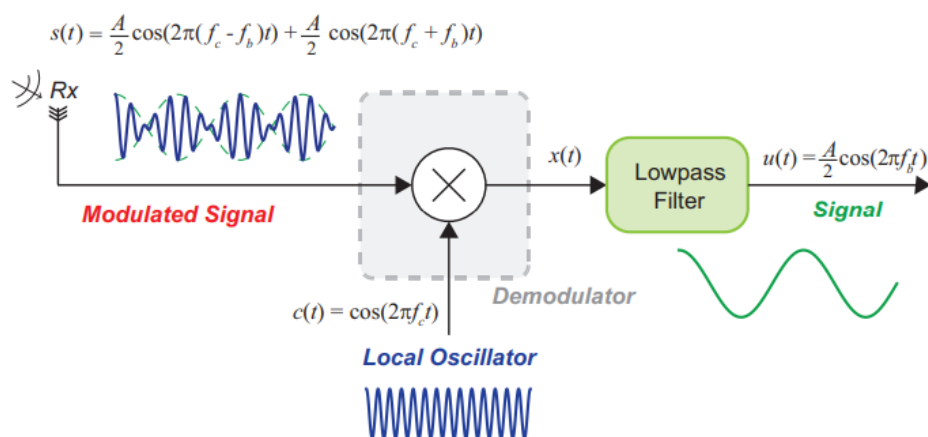


图 4-8

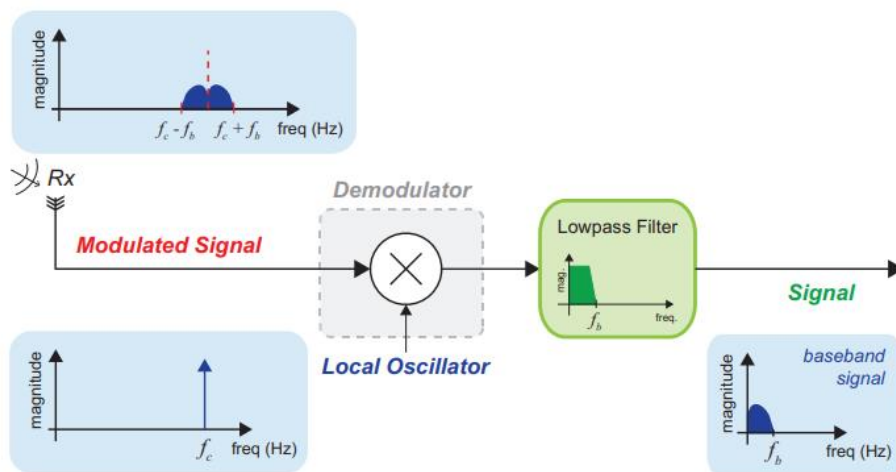


图 4-9 解调 AM 信号的频域分析

从天线接收到的信号为：

$$s(t) = \frac{A}{2} \cos(2\pi(f_c - f_b)t) + \frac{A}{2} \cos(2\pi(f_c + f_b)t)$$

若使用一个完美的正弦载波对其进行解调，那么可以得到如下的结果：

$$\begin{aligned}
 x(t) &= c(t) \times s(t) \\
 &= \frac{A}{2} \cos(2\pi f_c t) \left[\cos(2\pi(f_c - f_b)t) + \cos(2\pi(f_c + f_b)t) \right] \\
 &= \frac{A}{2} \cos(2\pi f_c t) \cos(2\pi(f_c - f_b)t) + \frac{A}{2} \cos(2\pi f_c t) \cos(2\pi(f_c + f_b)t) \\
 &= \frac{A}{4} \cos(2\pi(2f_c - f_b)t) + \frac{A}{4} \cos(2\pi f_b t) + \frac{A}{4} \cos(2\pi(2f_c + f_b)t) + \frac{A}{4} \cos(2\pi f_b t) \\
 &= \frac{A}{2} \cos(2\pi f_b t) + \left[\frac{A}{4} \cos(2\pi(2f_c - f_b)t) + \frac{A}{4} \cos(2\pi(2f_c + f_b)t) \right]
 \end{aligned} \tag{4-7}$$

可以看到，使用一个和发射机相同的载波之后，能够得到一个低频分量和一个高频分量，这个低频分量正是前面发送的基带信号，因此只需要将高频分量进行滤除，就能够得到这个低频分量，从而将基带信号复原。

$$\begin{aligned}
 \Rightarrow u(t) &= \frac{A}{2} \cos(2\pi f_b t) + \left[\frac{A}{4} \cos(2\pi(2f_c - f_b)t) + \frac{A}{4} \cos(2\pi(2f_c + f_b)t) \right] \text{ lowpass filtered terms} \\
 &= \frac{A}{2} \cos(2\pi f_b t) \\
 &= \frac{g(t)}{2}
 \end{aligned} \tag{4-8}$$

4.2.2 正交幅度调制与解调(QAM)

4.2.2.1 正交调制

在前面的 AM 调制中，基带信号的带宽为 $f_b \text{ Hz}$ ，那么当信号被高频载波调制后，将占用的带宽为 $2f_b \text{ Hz}$ ，如果想要提高效率，此时可以采用正交调制的方式。可以同时传输 2 个带宽为 $f_b \text{ Hz}$ 的基带信号，并且其在经过调制之后所占的带宽和只传输一个基带信号所占用的带宽相同。

将这两个带宽都为 $f_b \text{ Hz}$ 的信号，用两个频率相同的载波进行调制，但是这两个载波之间有 90° 的相移(正交的两个信号)，也就是一个基带信号用正弦波调制，另外一个基带信号用预先波调制。

为了简单的介绍，在这里举一个简单的例子，假设现在有两个带宽都为 $f_b \text{ Hz}$ 的基带信号：

$$g_1(t) \quad g_2(t)$$

分别用正弦信号和余弦信号对其进行调制。假设载波的频率为 f_c ，那么载波为

$$\cos 2\pi f_c t \quad -\sin 2\pi f_c t$$

那么采用正交调制这两个信号，得到的调制信号为：

$$y(t) = g_1(t) \cos 2\pi f_c t - g_2(t) \sin 2\pi f_c t \quad 4-9$$

对于这个一条信号，其中使用 $\cos 2\pi f_c t$ 进行调制的分量，称作同相分量也就是 In-Phase 分量，也称作 I 路，使用 $\sin 2\pi f_c t$ 进行调制的分量，称作正交分量也就是 Quadrature 分量，也称作 Q 路。

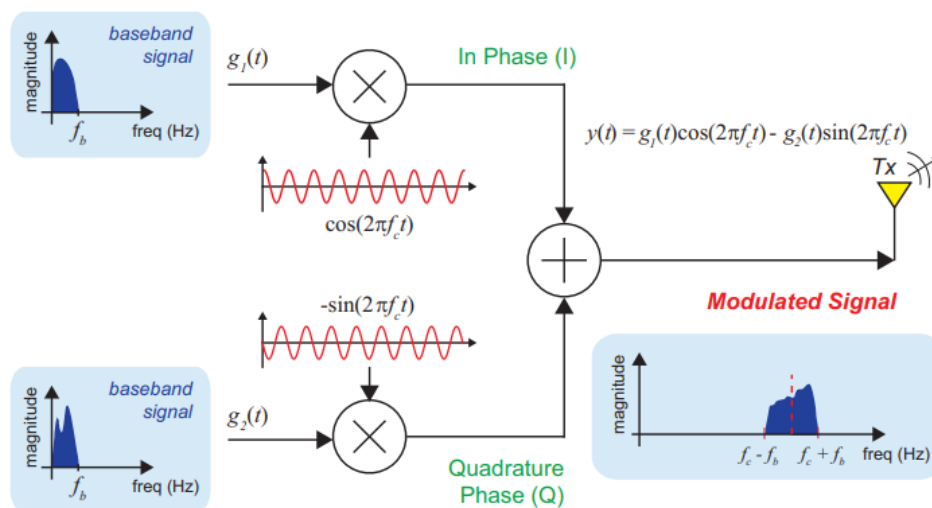


图 4-10 正交调制

4.2.2.2 正交解调

同前面的基本的解调类似，在进行正交解调的时候，同样采取相同的载波即可把信号解调出来，如下图所示：

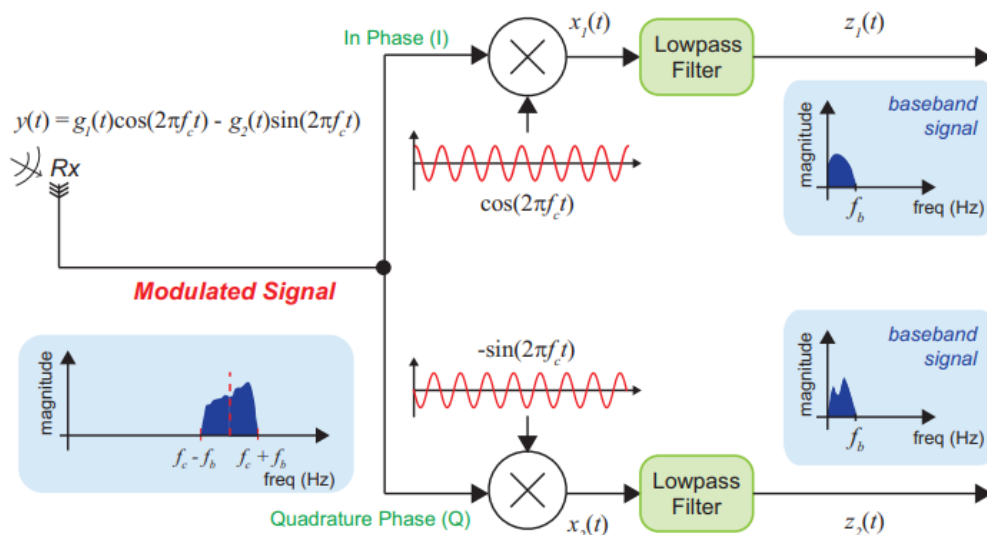


图 4-11 正交解调

对于 I 路信号，使用与发射机同频同相的余弦载波，然后将信号通过低通滤波器，即可得到解调的信号。

$$\begin{aligned}
 x_1(t) &= y(t) \cos(2\pi f_c t) \\
 &= [g_1(t) \cos(2\pi f_c t) - g_2(t) \sin(2\pi f_c t)] \cos(2\pi f_c t) \\
 &= g_1(t) \cos^2(2\pi f_c t) - g_2(t) \sin(2\pi f_c t) \cos(2\pi f_c t) \\
 &= \frac{1}{2} g_1(t) [1 + \cos(4\pi f_c t)] - \frac{1}{2} g_2(t) \sin(4\pi f_c t) \\
 &= \frac{1}{2} g_1(t) + \left[\frac{1}{2} g_1(t) \cos(4\pi f_c t) - \frac{1}{2} g_2(t) \sin(4\pi f_c t) \right] \text{ lowpass filtered terms}
 \end{aligned}$$

4-10

对于 Q 路信号，与 I 路信号类似，采用与发射机同频同相的正弦波进行解调即可。

$$\begin{aligned}
 x_2(t) &= y(t) (-\sin(2\pi f_c t)) \\
 &= [g_1(t) \cos(2\pi f_c t) - g_2(t) \sin(2\pi f_c t)] (-\sin(2\pi f_c t)) \\
 &= [-g_1(t) \cos(2\pi f_c t) \sin(2\pi f_c t)] + g_2(t) \sin^2(2\pi f_c t) \\
 &= -\frac{1}{2} g_1(t) \sin(4\pi f_c t) + \frac{1}{2} g_2(t) [1 - \cos(4\pi f_c t)] \\
 &= \frac{1}{2} g_2(t) - \left[\frac{1}{2} g_1(t) \sin(4\pi f_c t) + \frac{1}{2} g_2(t) \cos(4\pi f_c t) \right] \text{ lowpass filtered terms}
 \end{aligned}$$

4-11

4.2.3 使用复数来表示 QAM

在前面介绍了如何进行正交调制与解调，可以看到在进行调制和解调的过程中，使用到了三角函数，并且提出了正交分量和同相分量的概念，其实在这个过程中，使用复指数信号来表示这个过程会显得更加简单。

假设两个基带信号：

$$g_1(t) \quad g_2(t)$$

频率为 f_c 的载波信号，在这里使用复指数的形式：

$$e^{j2\pi f_c t} = \cos 2\pi f_c t + j \sin 2\pi f_c t$$

4-12

然后用这个信号去调制基带信号：

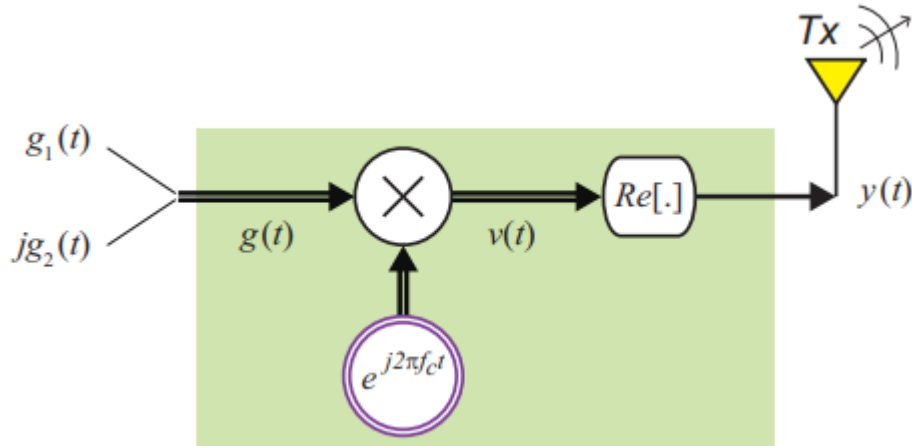


图 4-12 正交调制的复指数形式

$$\begin{aligned}
 v(t) &= g(t)e^{j2\pi f_c t} = [g_1(t) + jg_2(t)]e^{j2\pi f_c t} \\
 &= [g_1(t) + jg_2(t)] [\cos(2\pi f_c t) + j\sin(2\pi f_c t)] \\
 &= g_1(t)\cos(2\pi f_c t) + jg_2(t)\cos(2\pi f_c t) + jg_1(t)\sin(2\pi f_c t) - g_2(t)\sin(2\pi f_c t) \\
 &= \underbrace{[g_1(t)\cos(2\pi f_c t) - g_2(t)\sin(2\pi f_c t)]}_{\text{Real}} + j \underbrace{[g_1(t)\sin(2\pi f_c t) + g_2(t)\cos(2\pi f_c t)]}_{\text{Imaginary}}
 \end{aligned}$$

4-13

可以看到，使用复指数调制的信号的实部正是前面正交见过的经过正交调制过后的信号。其实在实际中，不存在复数，因此我们只需要取调制信号的实部。就可以得到最终的调制信号。

对于采用复指数的方式，来表示正交解调的过程和与上面类似

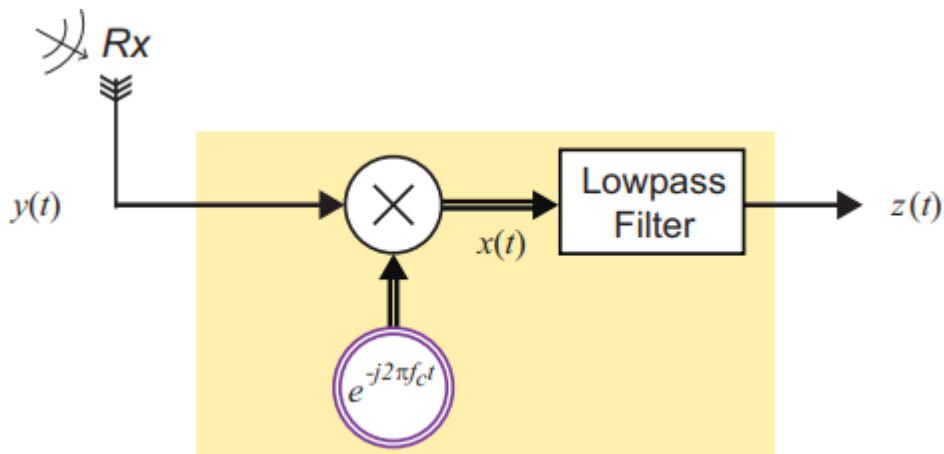


图 4-13 正交解调的复指数形式

$$\begin{aligned}
 x(t) &= y(t)e^{-j2\pi f_c t} \\
 &= \left[g_1(t)\cos(2\pi f_c t) - g_2(t)\sin(2\pi f_c t) \right] e^{-j2\pi f_c t} \\
 &= \left[g_1(t)\cos(2\pi f_c t) - g_2(t)\sin(2\pi f_c t) \right] \left[\cos(2\pi f_c t) - j\sin(2\pi f_c t) \right] \\
 &= \frac{1}{2} \left[g_1(t) + jg_2(t) \right] + \cancel{\frac{1}{2}g_1(t)\cos(4\pi f_c t)} - \cancel{j\frac{1}{2}g_2(t)\cos(4\pi f_c t)} \\
 &\quad - \cancel{j\frac{1}{2}g_1(t)\sin(4\pi f_c t)} - \cancel{\frac{1}{2}g_2(t)\sin(4\pi f_c t)}
 \end{aligned}$$

4-14

一个完整的正交解调的过程如下图所示：

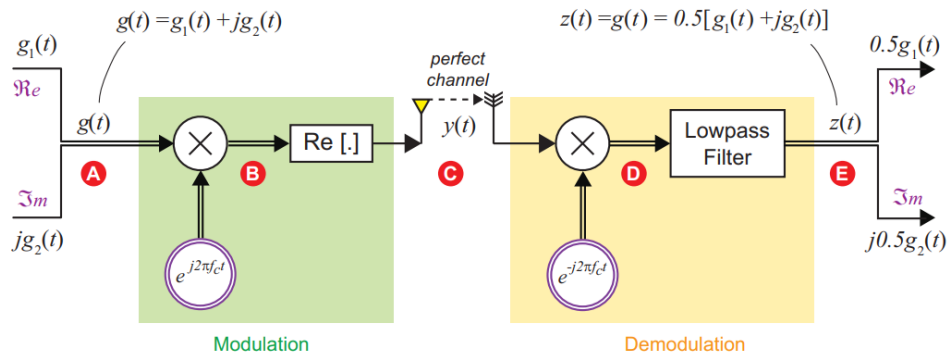


图 4-14

使用复指数来表示一个完整的正交调制解调过程的频谱变化如图 4-15 所示

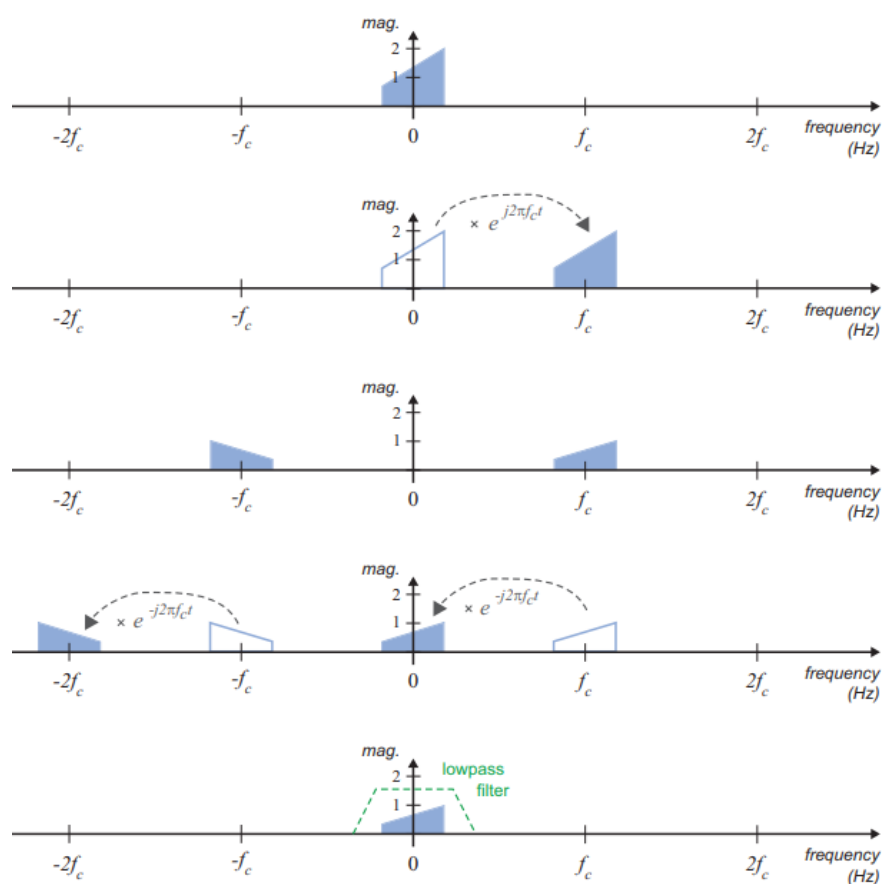


图 4-15 信号调制解调的过程

了解了 QAM 的调制和解调原理之后，接下来，我们将在 Simulink 当中来实现这个调制解调的过程。打开仿真模型 `qam_mod_demod.slx` 这个模型

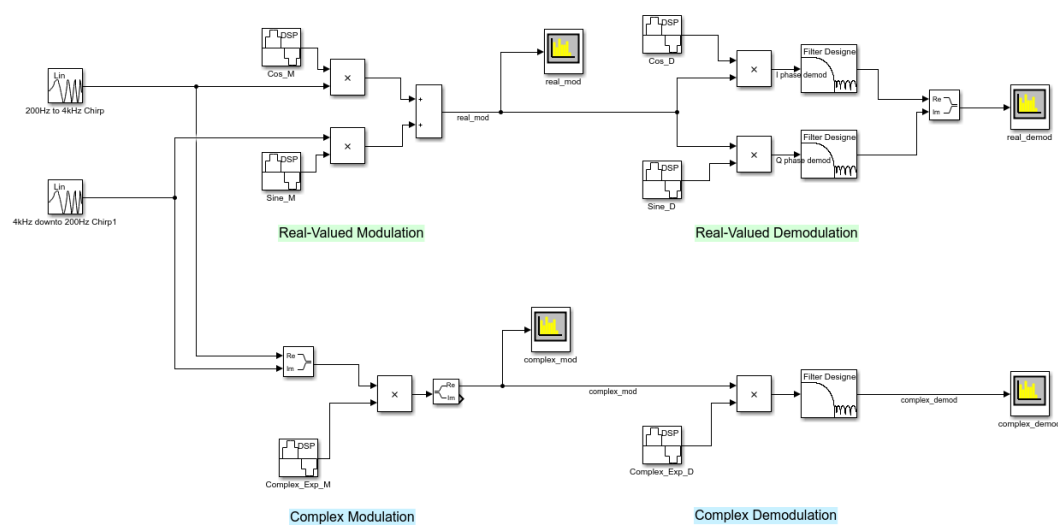


图 4-16 QAM 仿真模型

从仿真模型可以看到，信号源为两个能够生成线性变化的正弦信号的模块。这样能够产生基带信号。在仿真模型的上方，是采用实数的方式进行调制和解调的过程，下方是采用复数进行调制和解调的过程。在调制和解调完成之后，使用了频谱仪来观察结果。

首先我们来看，采用实数进行 QAM 调制和解调的频谱，如图 4-17 所示，从图中可以看到，经过调制过后，基带的 IQ 信号被搬移到了中心频率为 20KHz 的频率附近，然后经过解调之后，又恢复出了基带信号。

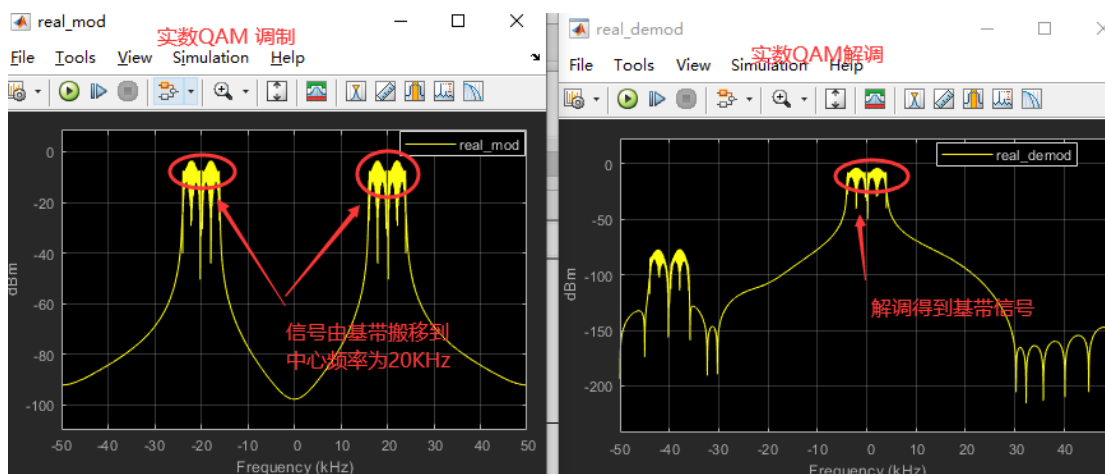


图 4-17 实数 QAM 调制与解调结果

然后再看采用复数的方式进行 QAM 调制和解调的结果，其结果与采用实数 QAM 调制的结果一致。

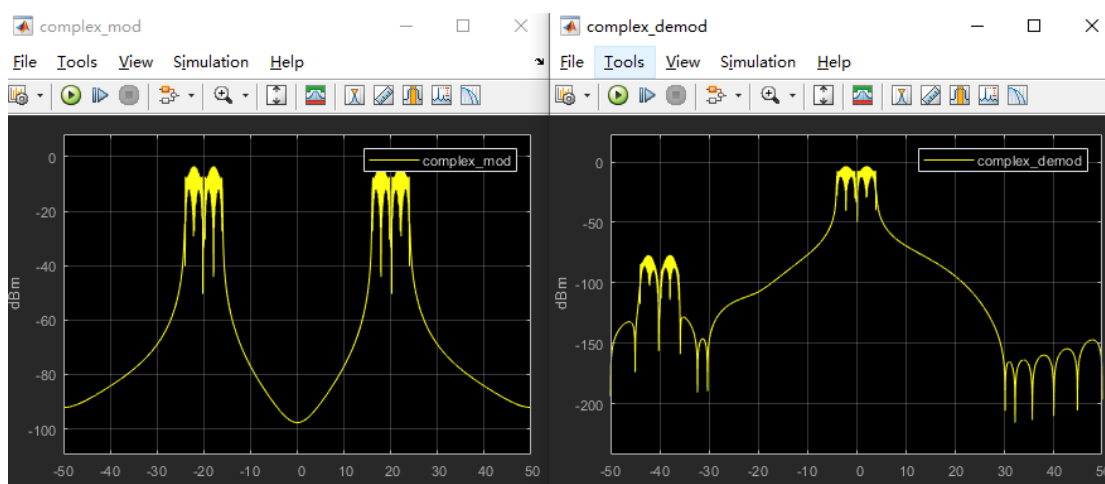


图 4-18 复数 QAM 调制与解调

4.3 小结

在本章中，介绍了有关调制和解调的基本概念。首先介绍了实数信号和复数

信号，在现实世界中的信号都是实数信号，不存在复数信号，但是在信号处理和通信系统中，经常会使用到复数来表示信号，这个能帮助我们更加简单地来理解信号。

然后介绍了常规的调制和解调方法，在这里给出了基本的数学公式，展示了调制和解调背后的数学原理，当然，在这里介绍的都是理想情况下的结果，在实际的信道中，存在各种各样的干扰，实际的调制和解调会比这要复杂，但是最基本的原理是一致的。

最后介绍了正交调制，正交调制是一种能够提高通信系统传输效率的调制方法，在通信系统中被大量的采用，我们的 **ANTSDR** 也正是基于这一种正交调制的架构来实现的。在正交调制中，从实数和复数的角度来进行了分析，在实际的 **QAM** 调制中，可以看作是信号进行复调制之后只取了信号的实部作为已调信号。

第5章 AM-DSB-TC 调制与非相干解调

在前面的章节中，介绍了 Matlab&Simulink 的基本使用方法和基本的调制解调的概念。在本章当中，我们会先介绍 AM 的常规调幅的原理，然后给出响应的仿真。然后会介绍 AM 常规调幅的非相干的解调方法，并对其进行仿真，最后会使用 ANTSDR 使用 AM 常规调幅来发射一个信号，并采用 ANTSDR 使用非相干的方法对信号进行解调。

AM 是最简单的调制方式，AM 最基本的形式，是把原始信息与载波混频，产生一个在频谱上有双边带的信号。这个过程叫作双边带（AM-DSB）调制，它的结果是把原始信息信号从基带上搬移到载波频率。有两种不同的 AM-DSB：不抑制载波的（AM-DSB-TC）和抑制载波的（AM-DSB-SC）。AM 还有单边带调幅（AM-SSB）和残留边带调幅（AM-VSB）。

5.1 常规调幅原理

基带信号 $s_i(t)$ 和直流偏置信号 A_0 相加，然后与频率为 f_c 的载波进行混频得到了已调信号 $s_{am-dsb-tc}(t)$ 。AM-DSB-TC 调制器的结构如图 5-1 所示

$$s_{am-dsb-tc}(t) = (s_i(t) + A_0) \cos 2\pi f_c t \quad 5-1$$

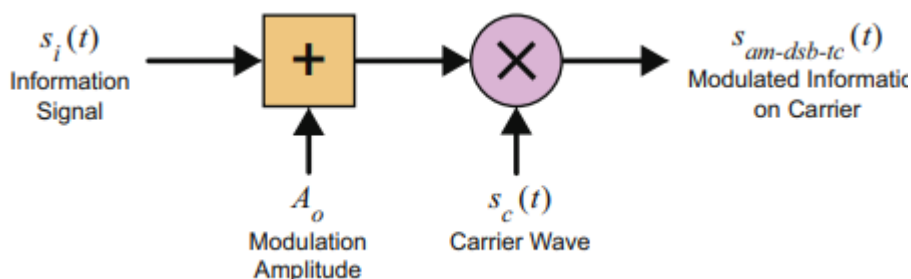


图 5-1 AM 调制器结构

知道了 AM-DSB-TC 调制的基本原理之后，就可以来举个例子看看具体的调制过程是怎么样的，在这里用 AM-DSB-TC 的调制方法，来调制一个单音信号为例。

5.1.1 单音基带信号调制

单音信号的幅度为 A_i 频率为 f_i

$$s_i(t) = A_i \cos(2\pi f_i t) = A_i \cos(\omega_i t) \quad 5-2$$

载波信号的幅度为 A_c 频率为 f_c

$$s_c(t) = A_c \cos(2\pi f_c t) = A_c \cos(\omega_c t) \quad 5-3$$

已调信号为:

$$s_{am-dsb-tc}(t) = \left[A_o + A_i \cos(\omega_i t) \right] A_c \cos(\omega_c t) \quad 5-4$$

在这里可以使用积化和差公式:

$$\begin{aligned} s_{am-dsb-tc}(t) &= A_o A_c \cos(\omega_c t) + \frac{A_i A_c}{2} \cos(\omega_c - \omega_i)t + \frac{A_i A_c}{2} \cos(\omega_c + \omega_i)t \\ &= A_o A_c \cos(\omega_c t) + \frac{A_i A_c}{2} \left(\cos(\omega_c - \omega_i)t + \cos(\omega_c + \omega_i)t \right). \end{aligned} \quad 5-5$$

调制的过程如图 5-2 所示, 可以看到经过 AM-DSB-TC 调制过后, 在已调信号的频谱中产生了三个频率分量, 分别是处于下边带的 $f_c - f_i$ 处于上边带的 $f_c + f_i$ 和载波频率分量 f_c 。在已调信号中, 把包络绘制出来, 这里包络的最大幅度与上方基带信号的最大幅度刚好对应起来。

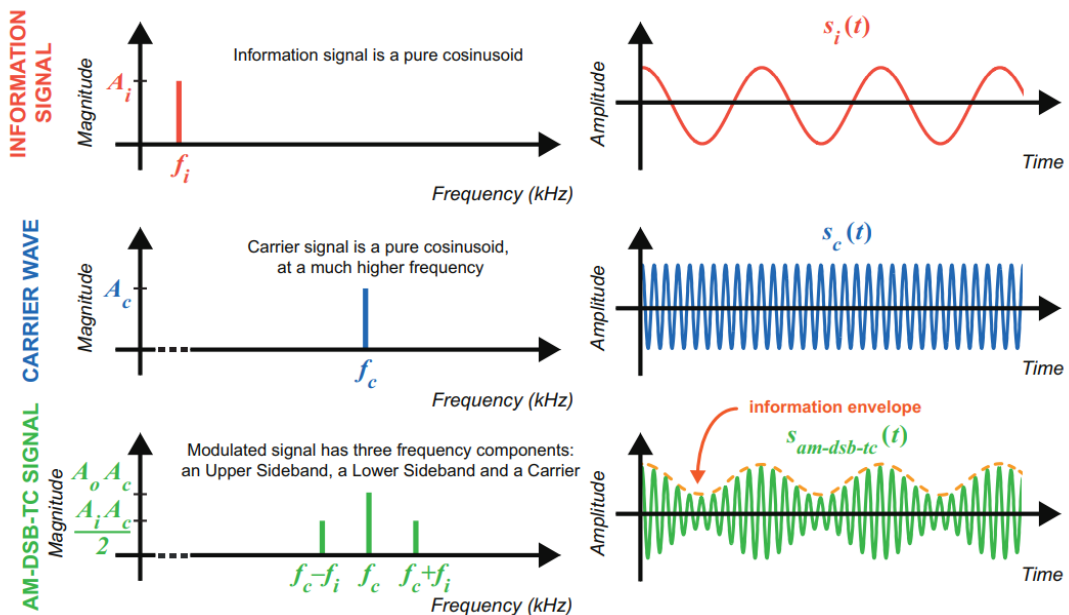


图 5-2 AM 调制过程

接下来需要考虑的是, AM-DSB-TC 在什么时候会出现异常, 还是考虑上面的已调信号的表达式, 可以对其进行简单的变形。

$$\begin{aligned}
 s_{am-dsb-tc}(t) &= A_o \left[1 + m \cos(\omega_i t) \right] A_c \cos(\omega_c t) \\
 &= A_o A_c \cos(\omega_c t) + \frac{A_o A_c m}{2} \left(\cos(\omega_c - \omega_i)t + \cos(\omega_c + \omega_i)t \right) \\
 \text{where: } m &= \frac{A_i}{A_o} .
 \end{aligned}$$

5-6

在这里使用一个调制系数 m 来表示 AM-DSB-TC 的调制程度，这个调制系数就是基带信号的振幅和载波频率的振幅的比值。下图表示的是不同调制系数下已调信号的时域波形，可以看到随着调制系数的增大，已调信号的波形将会失调。所以，当调制系数为 1 的时候称作满调幅。

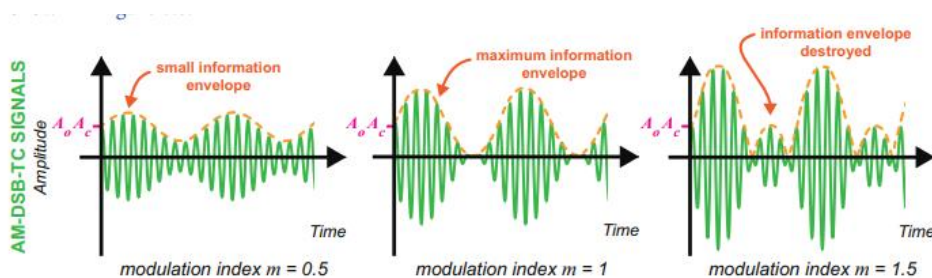


图 5-3 不同调制系数下的时域波形

5.1.2 基带信号的调制

在前面介绍了一个单音信号的常规调幅，在实际中，基带信号一般不会是一个单音信号这么简单，基带信号一般都有一定的带宽。若对基带信号使用常规调幅，其过程和上面的过程基本一致。如下图所示：

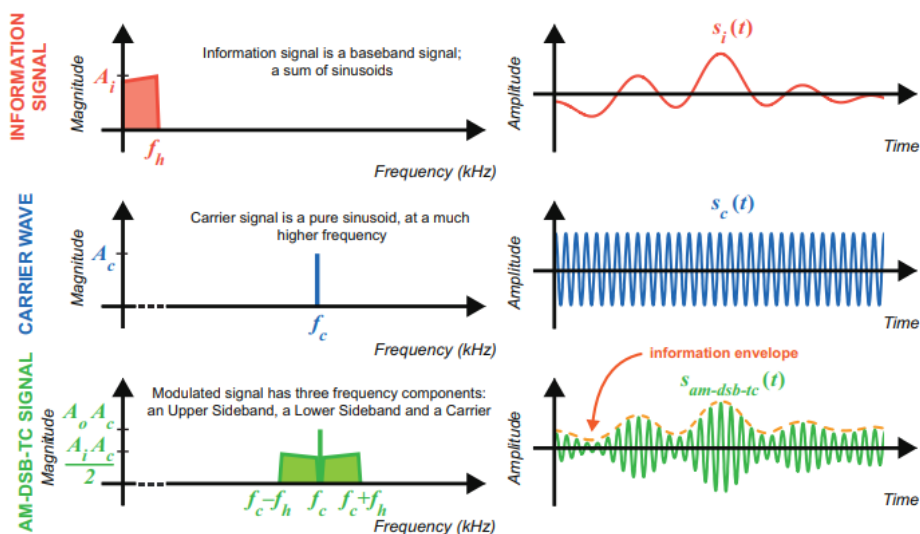


图 5-4 基带信号的常规调幅

由信号与系统的知识可以知道，任何信号都可以用正余弦信号来表示，因此基带信号可以看作是有不同频率的单音信号组成的，这样一想就能够简单地来弄清楚基带信号调制过程中的频域变化。

为了更直观地描述这一过程，我们从基带信号中，任意找出三个频率的单音信号，观察其在调制前和调制后的变化，如图 5-5、图 5-6 所示：

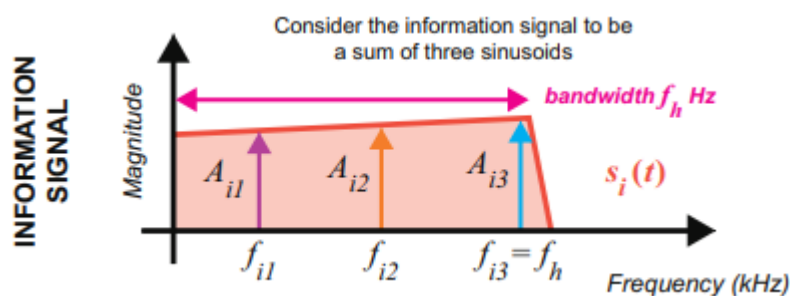


图 5-5 调制前的基带信号

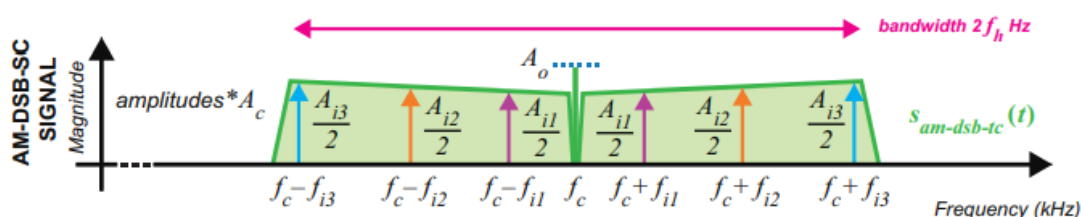


图 5-6 调制过后的基带信号

可以看到使用常规调幅过后，在频域上回生成对称的两个边带。

5.2 使用 Simulink 仿真常规调幅

前面介绍了常规调幅的原理之后，在这里就可以在 Simulink 里面来对这个原理进行简单的仿真了，通过仿真能够使我们对调制过程有更加清楚的认识。

打开仿真模型 `am_dsb_tc_mod.slx`，然后你会看到 am-dsb-tc 的调制的仿真模型，如图 5-7 所示：

可以看到在本这个仿真模型中，有一个手动开关用于选中基带信号源，这个模块可以直接在 Simulink 中搜索 manual switch 将模块添加到 simulink，双击这个模块，就能选中不同输入。

在上方的是一个单音信号的输入，在下方的信号源是三个单音信号的和。

选择信号源之后，将会给这个信号源添加一个直流偏置。

然后将会把这个偏置之后的信号与载波信号相乘，进行信号的调制

最终在示波器和频谱仪中，可以来观察所期望的信号。

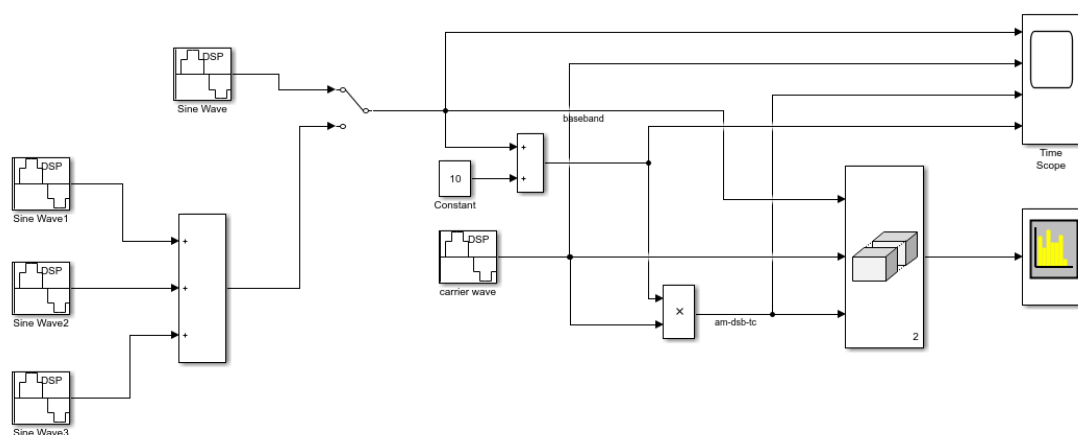


图 5-7 常规调幅仿真模型

首先进行单音信号的调制，选择上方的信号源，运行仿真，可以看到仿真过程中频谱仪展示的结果如下，基带信号为 1 个频率为 2kHz 的单音信号，载波信号为一个频率为 50kHz 的余弦波，经过调制过后的信号，有 3 个频率分量，这个结果与预期一致。

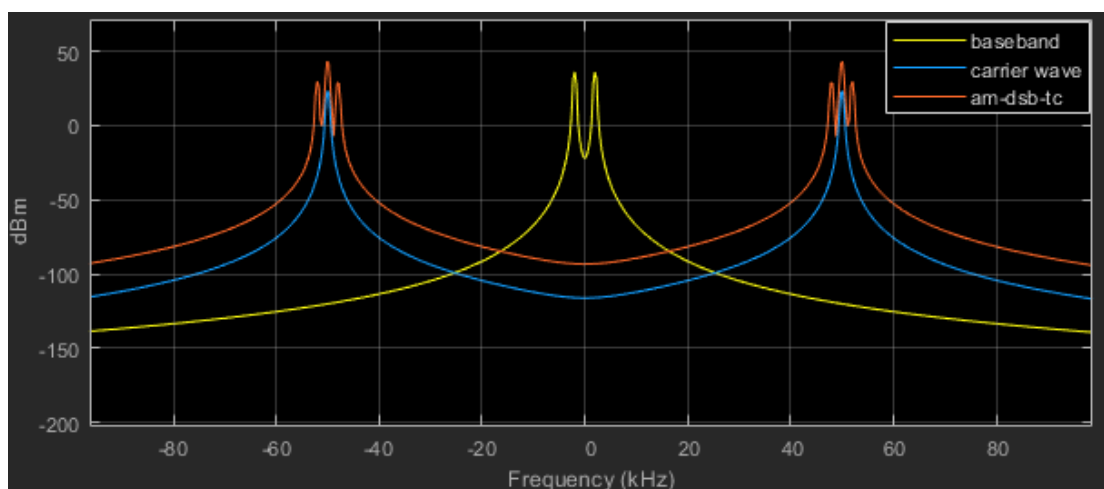


图 5-8 单音信号的调制情况

然后我们还可以观察一下示波器当中的信号，如图 5-9 所示，第一个信号是基带信号的时域图，第二个是载波信号的时域图，第三个是偏置信号和调制信号的时域图。可以看到，一条信号各个正弦波的幅值处，刚好位于偏置载波之上，这个也和预期的结果一致。

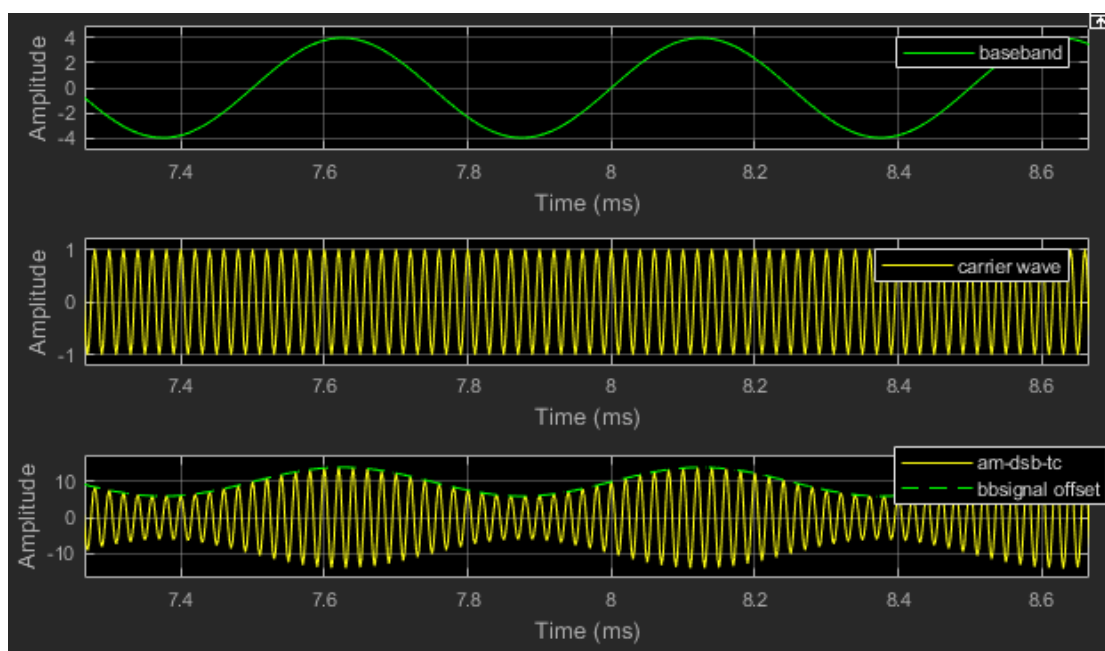


图 5-9 示波器当中的结果

现在将信号源的输入选择为下方的多个单音信号之和，双击 manual switch 模块，然后分别观察频谱仪和示波器的结果。

频谱仪的结果如下：可以看到基带信号是由 3 个频率和振幅都不相同的单音信号组成，经过载波调制之后，将基带信号搬移到了 50KHz 附近。

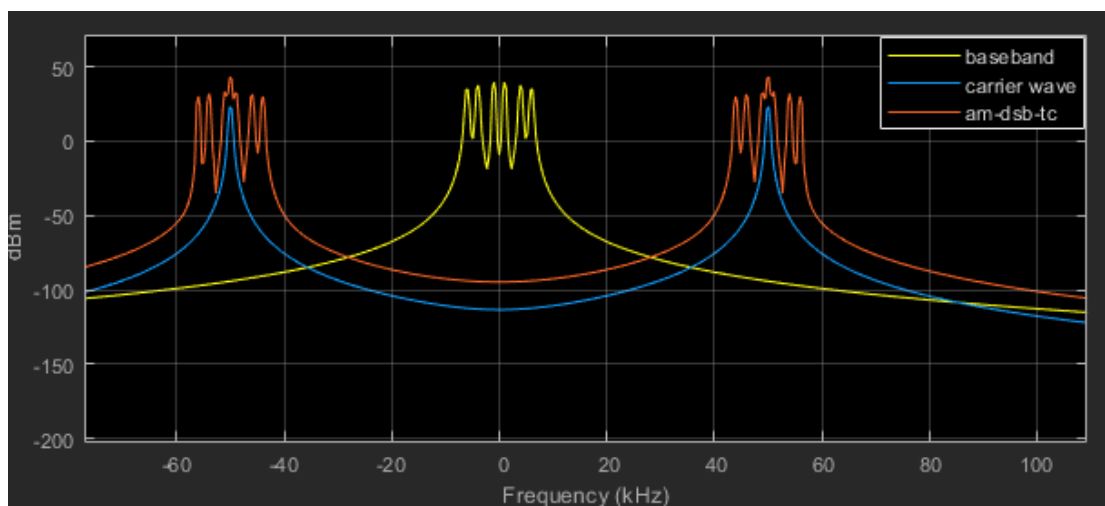


图 5-10 多个单音信号调制过后的频域结果

示波器的结果如下,在时域中, 比较难发现信号的规律, 但是还是能够比较直观地看出信号的包络和调制载波的关系。

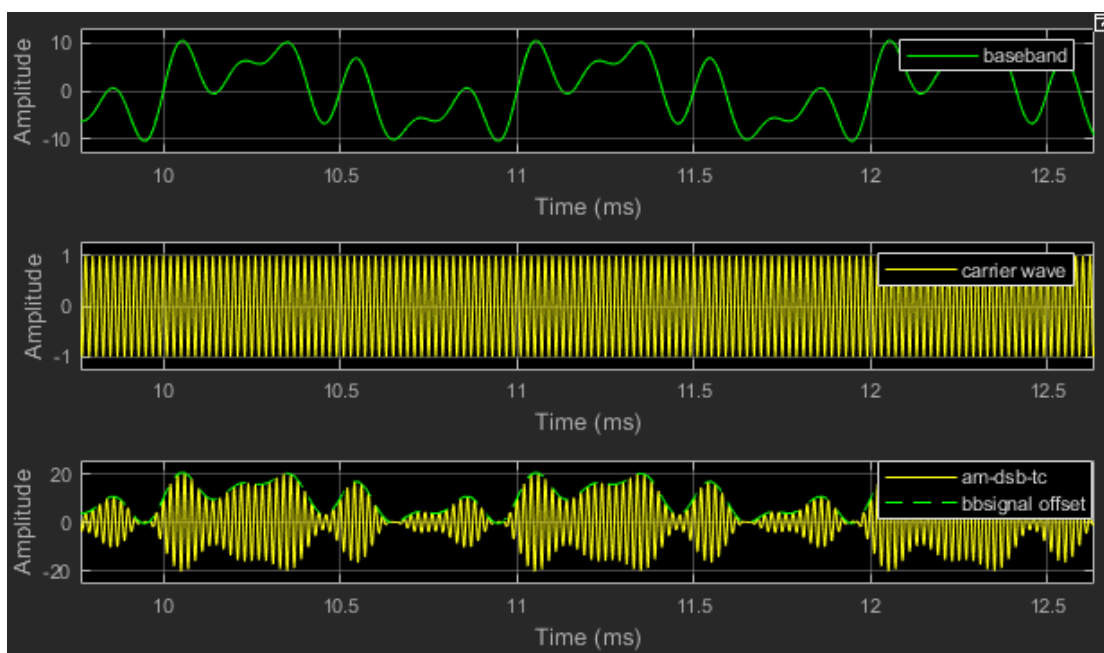


图 5-11 多个单音信号的时域结果

5.3 常规调幅的非相干解调原理

在前面介绍了常规调幅的调制原理，并且通过仿真实现了对常规调幅更加直观的认识，信号调制完成之后，在接收机里面需要对信号进行正确的解调，从而恢复出数据。

常规调幅的一个有点就是，消息信号能够在已调信号的包络中提取出来，从而能够使用非相干的方式进行解调。所谓非相干解调，就是在接收机当中，不需要生成一个和调制载波同频同相的信号就能将基带信号从已调信号中解调出来。

5.3.1 一般情况下的包络检波

在 Matlab 和 ANTSDR 共同作用来实现包络检波的时候，不能够使用模拟的方法，因为 Matlab 接收到的信号是已经经过采样之后的信号，是数字信号。因此在数字信号处理中可以饱和操作来实现包络检波的功能。所谓饱和操作就是使信号的幅度值并使其平滑。一般的结构如下图所示

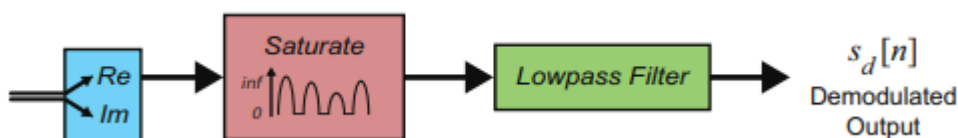


图 5-12 离散时间包络检波

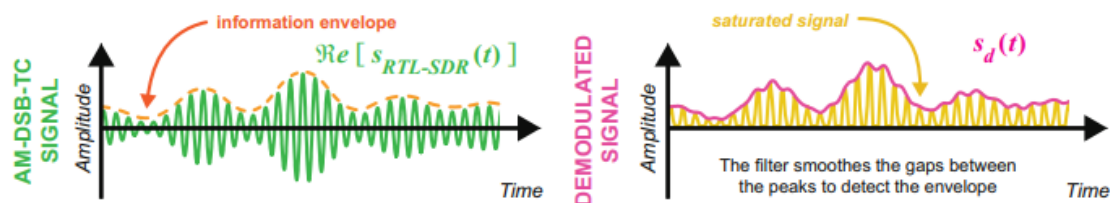


图 5-13 包络检波示意图

为了使低通滤波器能够成功的把波峰平滑化，并检测出信息包络线，AM-DSB-TC 信号的载波频率要相对信息信号的频率尽可能地高。如果载波频率不够高，那么滤波器就很难把它消除，这样输出的信号就会含有噪声。

5.3.2 实数解调的优化

上面的包络检波是在实数域下进行的，在前面也提到过，想要使解调出来的包络平滑，载波的频率需要比基带信号的频率要尽可能的高。我们可以看到在前面的解调过程中，复数部分的包络和正数部分的包络是对称的，但是在解调的使用，只使用了正数部分的包络。若我们现在对信号取绝对值，然后再来求取信号的包络，就相当于增大了载波的频率。这样，更加能够使包络平滑。

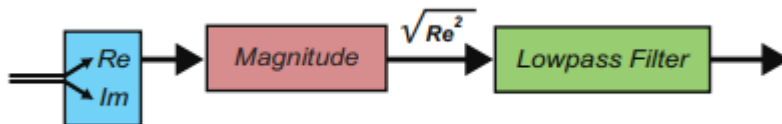


图 5-14 改进包络检波

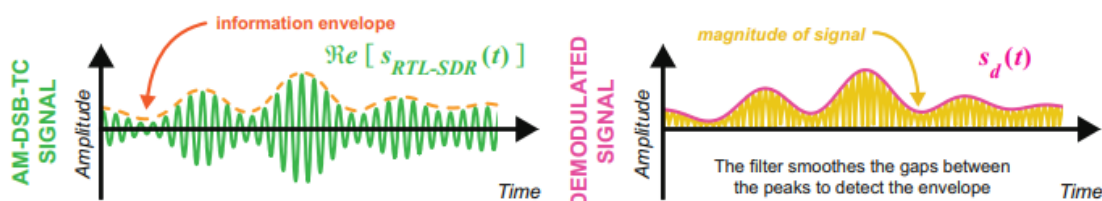


图 5-15

5.3.3 复包络检波

在前面的章节中，介绍了使用复数来表示信号的方法，其实在进行包络检波的时候，使用复包络会使整个过程显得更加直接。

我们使用的 ANTSDR 是在和上位机进行数据传输的时候，使用的是 IQ 两路数据，可以将其看作是复数信号的形式，如下图所示：

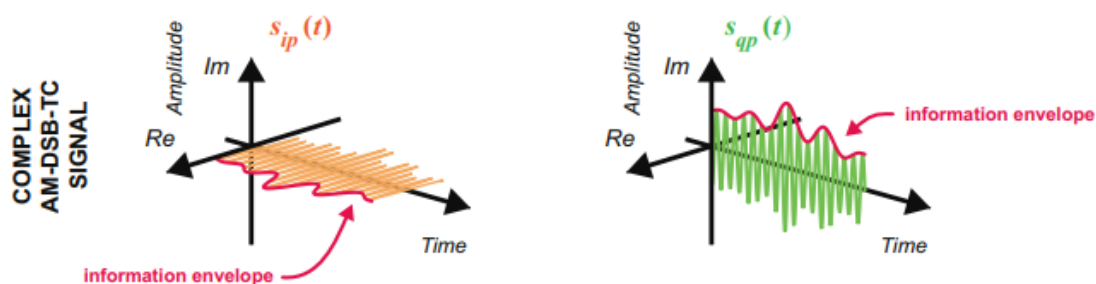


图 5-16 IQ 路包络

在复数空间内，其实可以将这虚部和实部的信号放在同一坐标轴下随着时间的变化，那么信号就可以使看作围绕着时间轴在不停旋转的一个点。

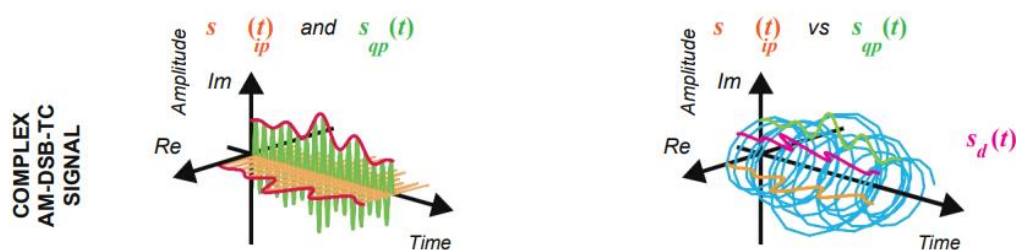


图 5-17

因此，在求取信号的包络的时候，我们只需要求得信号得复包络，便能够得到消息信号。所以包络检波得结构可以进一步简化：

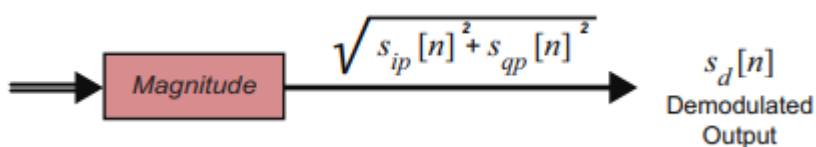


图 5-18 复包络检波

可以看到，求一个复数的包络，只需要知道这个信号的模就可以了，这个也就是相当于求这个复数的绝对值。

5.4 常规调幅的非相干解调仿真

在前面介绍了常规调幅的包络检波，为了能够更加直观地来认识包络检波，可以设计一个 Simulink 模型，来对常规调幅的信号进行解调。

打开仿真模型 `envelope_demod.slx`，可以看到仿真模型如下图所示：

首先基带信号为一个复数信号，复数信号的 IQ 路数据相同，经过一个偏置过后经过载波进行调制。

调制完成之后的信号经过一个求取模(绝对值)的模块,求得复数信号的模之后,将信号通过低通滤波器,最终得到信号的包络。

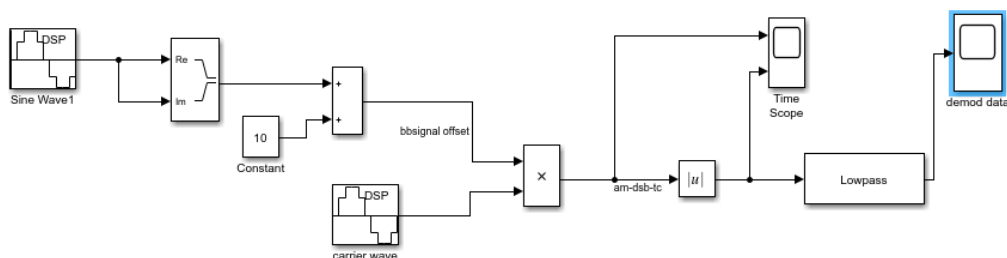


图 5-19 包络检波模型

点击运行仿真,第一个示波器中显示的是已调信号和求绝对值的信号,求运行结果如下图所示。

可以看到,对已调信号进行求模之后,得到的信号刚好就是信号的包络。需要注意的是,从图上看,求模之后的信号就是信号的包络,这是因为我们的信号源只是一个单音信号,因此在这里很直观的就能够得到包络,当基带信号不是一个单音信号的时候,还需要将求模之后的信号通过一个低通滤波器,滤除其高频分量得到的结果才是最终的基带信号。

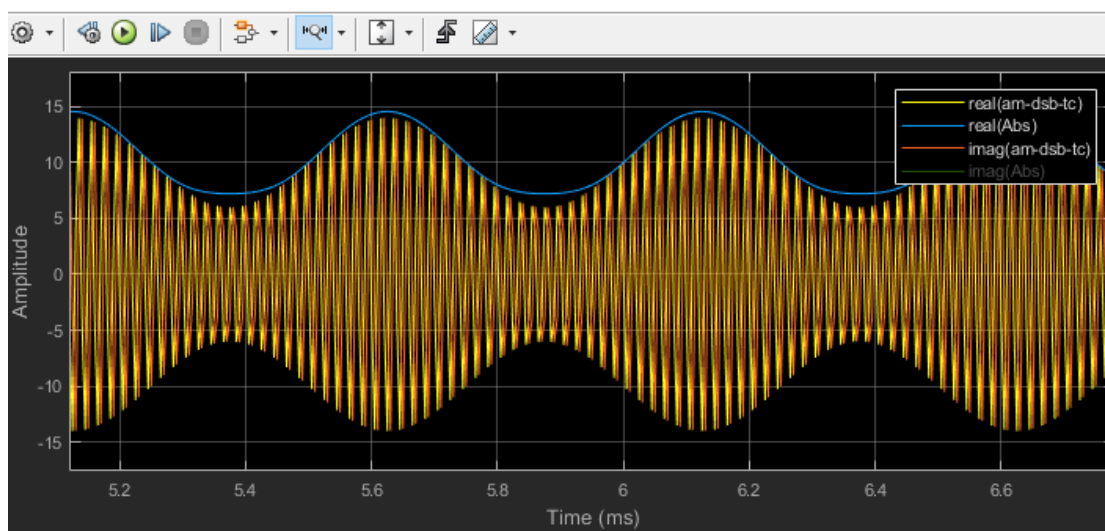


图 5-20

5.5 使用 ANTSDR 发送音频信号

在进行本次实验之前，需要先郑重的声明，使用 SDR 的时候需要遵守当地的无线电法律，切记不要使用大功率的发射机向信道中发射信号。

在前面已经介绍了常规调幅的原理，并且在 Simulink 当中实现了对常规调幅的仿真，和包络检波的仿真。那么现在我们可以来尝试着发送真实的无线信号了。这里会使用到 ANTSDR 作为发射机，将基带信号进行 AM 的常规调幅。

由前面的章节可以知道，ANTSDR 采用的是一个零中频的架构，信号的调制部分都是在 ad936x 中完成的，因此在 Matlab 中，我们只需要为其提供基带数据，然后使用 plutosdr 的发送模块设置需要发送的中心频率就可以了。

打开仿真模型 `antsdr_am_dsb_tc_tx.slx`，如下图所示：

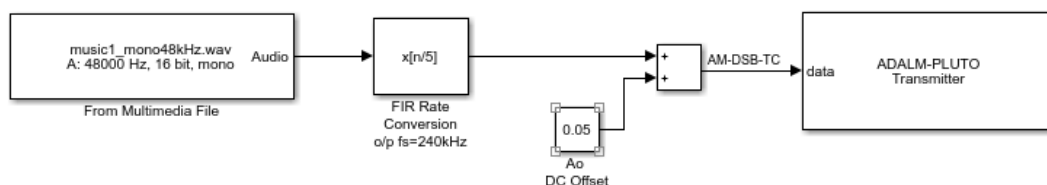


图 5-21 ANTSDR 发射模型

首先，基带信号的数据源，使用读取音频文件的模块，读取在 `audio_source` 文件夹下的音乐，本次实验读取的音频数据的采样率是 48KHz, 16bit 的单声道信号。

读取的音频数据使用一个速率转换模块，将采样率转换到 240kHz。

然后将数据经过一个直流偏置，最终提供给发送模块，将数据发射到真实的信道中。

其中 ADALM-PLUTO 的设置如下：

若用户使用网口连接至 ANTSDR，那么在 RadioID 处需要设置 ID 为：`ip:192.168.1.10`，IP 地址是用户自己设置的网口的 IP 地址，关于网口 IP 地址的设置，可以参考开箱手册。

中心频率设置为 400MHz，这个就相当于再之前进行仿真的时候所设置的载波频率。

Gain 设置为 -5dB，输出增益会影响输出信号的强度，在这里设置为 -5dB。

基带采样率设置为 240KHz，这个采样率就和前面的经过上采样之后的信号的采样率是一样的。

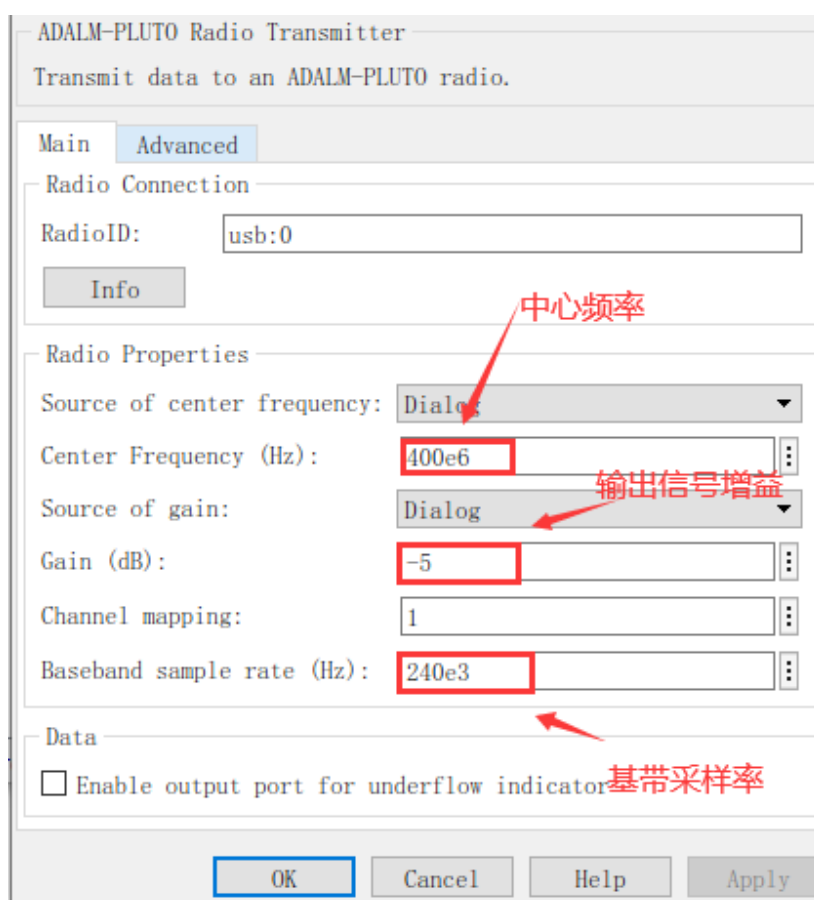


图 5-22 发射机设置

点击开始运行仿真，就可以向外部发射无线信号了。若用户手中有另外一个 ANTSDR 或者其他无线电接收设备比如 RTL_SDR，USRP 等，现在就可以尝试这来接收我们刚刚发出的信号。

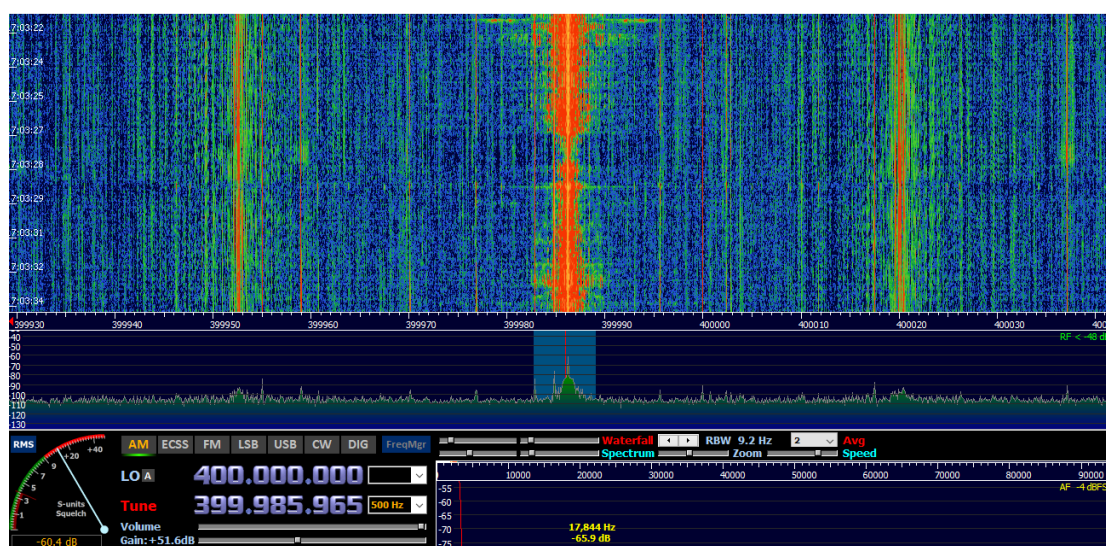


图 5-23 使用 HDSDR 接收到的信号

若用户使用 HSDR 或者 SDR# 将频率调到 400MHz，并且解调方式旋转 AM，那么就on能够听到解调出来的音乐。有关 SDR# 的使用方法，请参考资料当中的《常用 SDR 软件使用方法》。

5.6 使用 ANTSDR 接收并解调信号

在之前我们使用 ANTSDR 发送了音频数据，并将其调制到一个具体的频段上，而且使用其他 SDR 接收机也能够接收该信号并且成功解调。现在我们会使用同一 ANTSDR 来完成同时发射信号和接收信号的工作，并在 Simulink 中对 AM 信号进行非相干的解调。

5.6.1 ANTSDR 进行下变频

在之前的常规调幅的解调当中，使用非相干解调的时候，期望载波的频率越大越好，这样能够使包络检波的时候检测到的信号更加的平滑。这里就需要我们注意一个问题，若是我们直接将接收的频率设置为和发射机一样的频率上，那么我们得到的就是基带的信号，这样就不需要我们再进行处理。但是为了能够演示出使用 Simulink 解调真实信号的过程，我们可以在设置中心频率的时候，让接收机的中心频率相较于发送机的中心频率较小，这样就能够在 ANTSDR 接收到的复数基带信号中，含有一个较高的调制频率，然后我们可以使用前面介绍的包络检波，将这个信号解调出来。

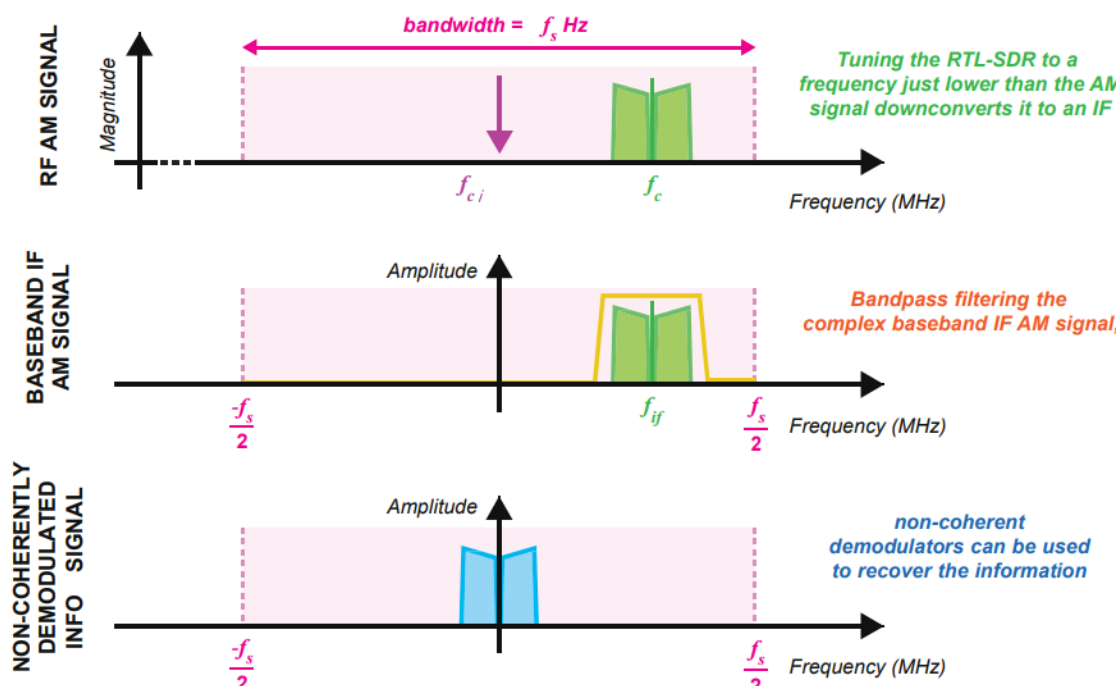


图 5-24 引入频率误差

从图 5-24 中可以很好直观地来解释这一过程，一条信号处于 400MHz，若我们此时使用复数解调的方式，但是接收机也就是 ad936x 的接收的中心频率比 400MHz 小几十 KHz，那么就相当于把信号搬移到了一个中频的位置，比如 40KHz。也就是图中的第二幅图所示。ANTSDR 把这个信号传递到上位机 Simulink 当中，这时候，就可以采用包络检波的方式将信号解调出来了。

5.6.2 使用 ANTSDR 解调音频信号

上面说明了本次实验的一个实现原理，接下来就可以在 Simulink 当中，搭建模型来实现这个操作了。

打开仿真模型 `antsdr_envelope_demod.slx`，仿真模型如图 5-25 所示

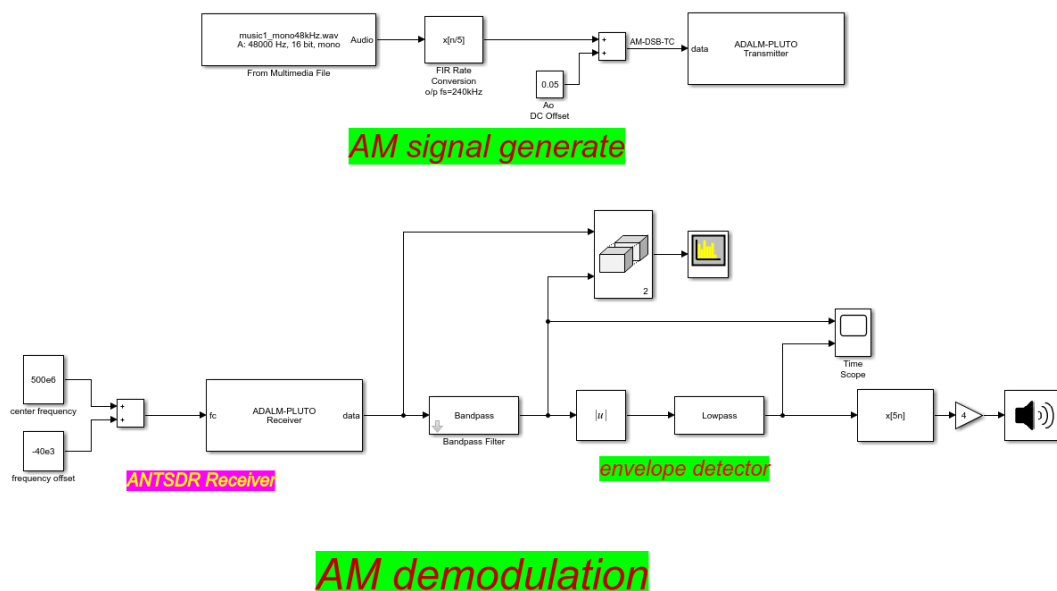


图 5-25 ANTSDR 解调模型

这个解调模型的上方，是和前面的 ANTSDR 发射信号部分是一样的，用于向无线信道中发送信号。在下方的是 AM 的解调模块。

首先是 ANTSDR 的接收模块，该模块的设置如图 5-26 所示。接收机的接收中心频率由外部输入也就是图中的两个常量模块所指定的接收频率，如图 5-27 所示。接收的基带采样频率和发送的基带采样频率保持一致。数据类型为 double，表示的数据范围为-1~1。

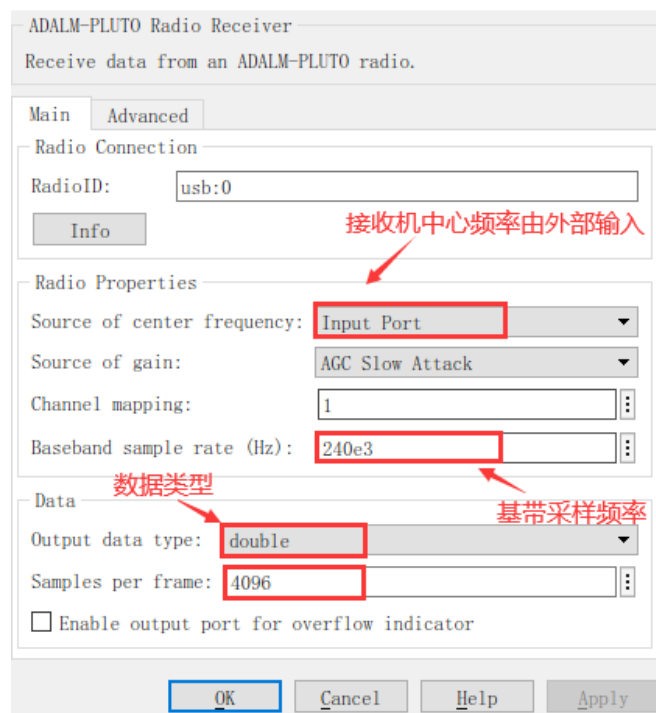


图 5-26 ANTSDR 接收机设置

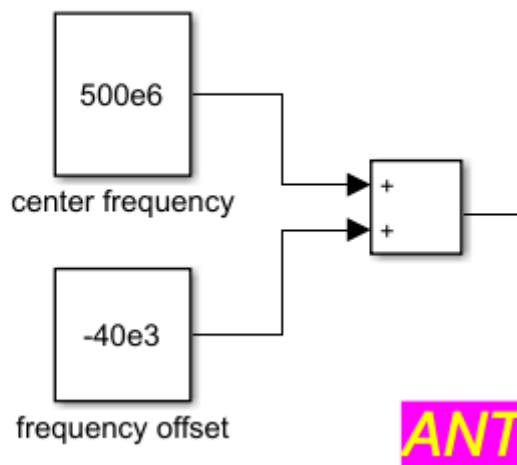


图 5-27 设置的中心频率

接收到信号之后，首先进行的是一个带通滤波，这个操作的目的是，将消息信号进行滤波，降低噪声的影响。

再之后就是进行包络检波，对复数信号进行求模操作，求模之后进行低通滤波，就得到了信号的包络。

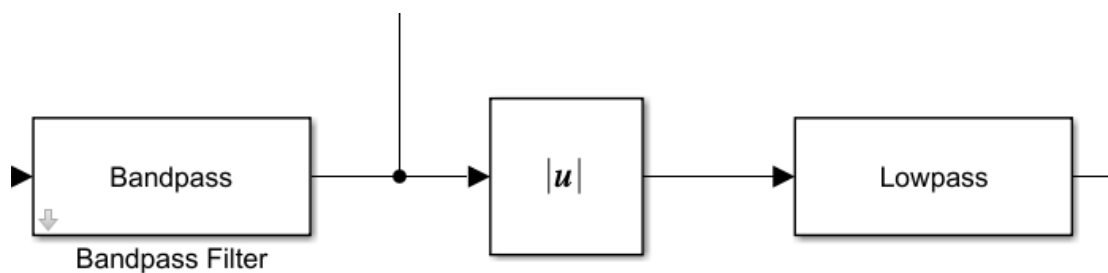


图 5-28 带通滤波与包络检波

最后将得到的包络进行降采样，得到一个采样率较低的信号，然后将这个信号通过音频播放器进行播放，我们就能够听到解调出来的音乐了。

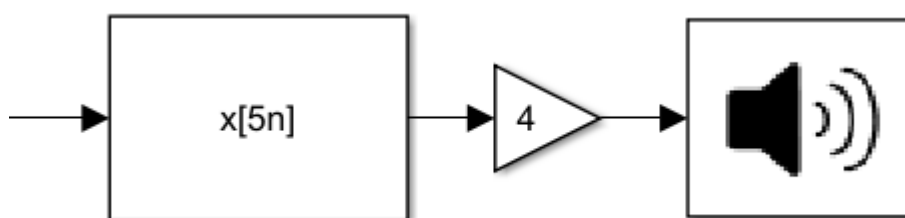


图 5-29 降采样

准备好之后，点击运行开始仿真，我们可以观察频谱仪和示波器的结果。频谱仪显示的是 ANTSDR 接收到的信号，和经过带通滤波之后的信号。

示波器中显示的是，经过带通滤波之后的信号和包络检波的信号。

首先观察频谱仪中的信号，如图 5-30 所示，可以看到在 40KHz 的地方为 AM 的调制信号。

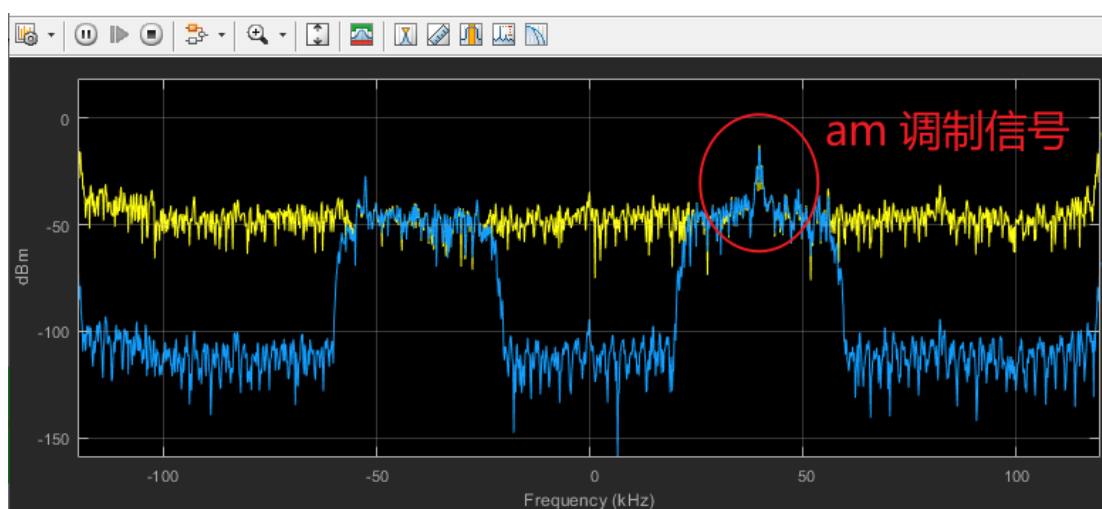


图 5-30 频谱仪中的信号

然后再观察示波器中的信号，如图 5-31 所示，上面的信号为接收到的信号，

下方为经过包络检波之后得到的信号。

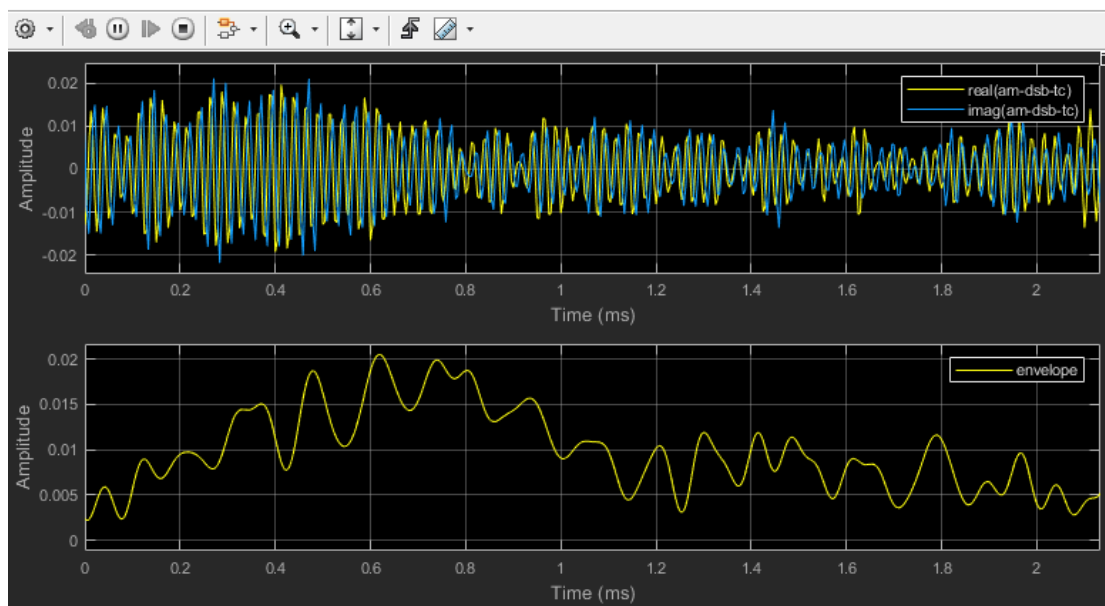


图 5-31 包络检波的结果

与此同时，我们应该能够听到解调之后得到的音乐。

到这里我们就完成了使用 ANTSDR 进行常规调幅和使用包络检波进行解调的整个过程。

5.7 总结

在本次实验当中，我们首先介绍了常规调幅的原理，并给出了常规调幅的仿真，然后介绍了如何使用包络检波来解调常规调幅信号，这里的包络检波是一种非相干的解调方式。

最后我们使用 ANTSDR 进行了 AM 信号的调制和解调，尝试了使用 ANTSDR 来进行在真实的信道中发送和接收信号的这一个过程。在之后的实验当中，我们还会接触到其他的调制解调方法。

第6章 常规调幅的相干解调

在上一次实验当中，我们介绍了常规调幅的调制方法和其相干解调的方法，并且使用 ANTSDR 成功地进行 AM 的调制与非相干解调。对于常规调幅，可以从信号的包络上获取其携带的信号，因此可以采取比较简单的包络检波来实现解调。

在今天将介绍常规调幅的相干解调方法，相干解调应该是应用得最广泛的一种解调方法，在之后的各种调制解调的例子中，还会多次地使用到这个解调方法。

6.1 相干解调和载波同步

6.1.1 有误差的解调

在前面的章节中，我们介绍了调制解调的基本原理，现在我们首先看下面这个图中所展示的发射机和接收机。可以看到，用于解调信号的载波，其相较于发射机的调制载波有一个 Δf 的频率偏移。

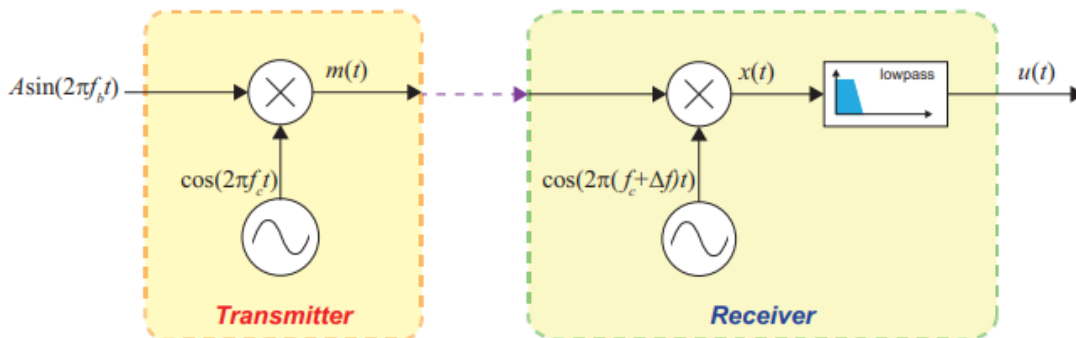


图 6-1 调制解调模型

单音信号的振幅为 A ，频率为 f_b ，调制载波为 f_c ，已调信号可以表示为：

$$\begin{aligned}
 m(t) &= A \cos(2\pi f_c t) \sin(2\pi f_b t) \\
 &= \frac{A}{2} [\sin(2\pi(f_b + f_c)t) - \sin(2\pi(f_b - f_c)t)]
 \end{aligned}$$

在之前的解调过程中，我们使用的都是理想情况下的解调，也就是说，接收用于解调的载波频率与发射机完全一致，但是在实际的接收中，想要产生一个和发射机完全一致的载波信号是十分困难的，因此在这里我们假设解调时接收机会产生一个 Δf 的频率偏移。

将解调的载波，作用在已调信号之上得到的结果如下：

$$\begin{aligned}
 x(t) &= \cos(2\pi(f_c + f_\Delta)t) \times \frac{A}{2} \left[\sin(2\pi(f_c + f_b)t) - \sin(2\pi(f_c - f_b)t) \right] \\
 &= \frac{A}{2} \left[\cos(2\pi(f_c + f_\Delta)t) \sin(2\pi(f_c + f_b)t) - \cos(2\pi(f_c + f_\Delta)t) \sin(2\pi(f_c - f_b)t) \right]
 \end{aligned}$$

使用积化和差公式对上面的式子进行整理，可以得到

$$\begin{aligned}
 x(t) &= \frac{A}{2} \left[\frac{1}{2} \sin(2\pi((f_c + f_\Delta) + (f_c + f_b))t) - \frac{1}{2} \sin(2\pi((f_c + f_\Delta) - (f_c + f_b))t) \right] \\
 &\quad - \frac{A}{2} \left[\frac{1}{2} \sin(2\pi((f_c + f_\Delta) + (f_c - f_b))t) - \frac{1}{2} \sin(2\pi((f_c + f_\Delta) - (f_c - f_b))t) \right]
 \end{aligned}$$

将这个信号通过低通滤波器，滤除高频分量之后可以得到：

$$\begin{aligned}
 u(t) &= \frac{A}{4} \sin(2\pi(2f_c + f_\Delta + f_b)t) - \frac{A}{4} \sin(2\pi(f_\Delta - f_b)t) \\
 &\quad - \frac{A}{4} \sin(2\pi(2f_c + f_\Delta - f_b)t) + \frac{A}{4} \sin(2\pi(f_\Delta + f_b)t)
 \end{aligned}$$

lowpass filtered term (pointing to the first term)
lowpass filtered term (pointing to the last term)

最终我们将在基带得到的信号为：

$$u(t) = \frac{A}{4} \left[\sin(2\pi(f_b + f_\Delta)t) + \sin(2\pi(f_b - f_\Delta)t) \right]$$

可以看到，最终解调出来的信号包含两个单音信号，这两个单音信号相较于原始的消息信号都有 Δf 的频率偏移。

接下来，我们会使用 Simulink 来直观地展示这一过程。

6.1.2 Simulink 仿真有误差的解调

打开仿真模型 `demod_mismatch_sync.slx`，如图 6-2 所示，其中消息信号为 1KHz 的单音信号，调制载波为 10KHz 的余弦波，解调载波为 10.2KHz 的余弦波。下变频之后的信号经过了低通滤波最终得到了输出的信号。我们可以运行仿真，来查看，在输入输出频率不一致的情况下该解调出来的结果。

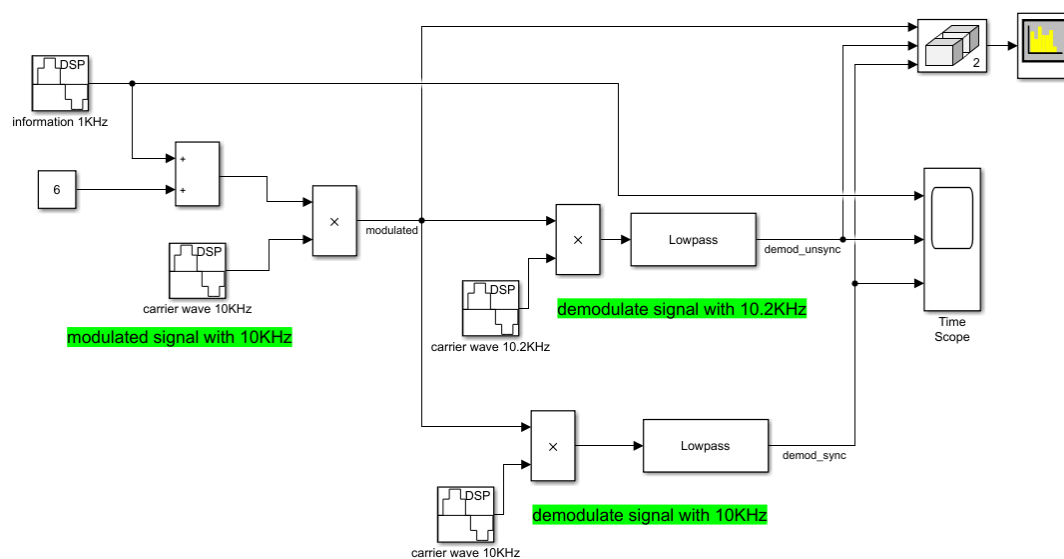


图 6-2 引入误差的解调

若是信号经过了完美的解调，下图中橙色线所示，那么在基带信号处，除了直流分量外，就只有一个 1KHz 的消息信号。若经过了有误差的解调，那么将会在原来的消息信号处得到两个有偏移的信号。

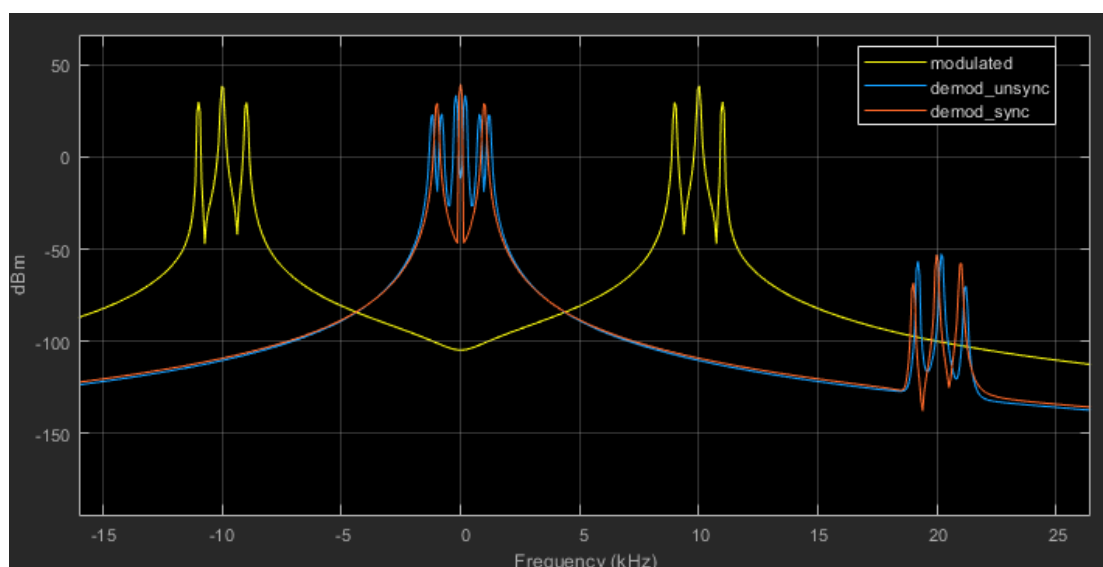


图 6-3 调制信号和解调信号频谱

从示波器中，能够更加直观地将两者的区别表现出来。

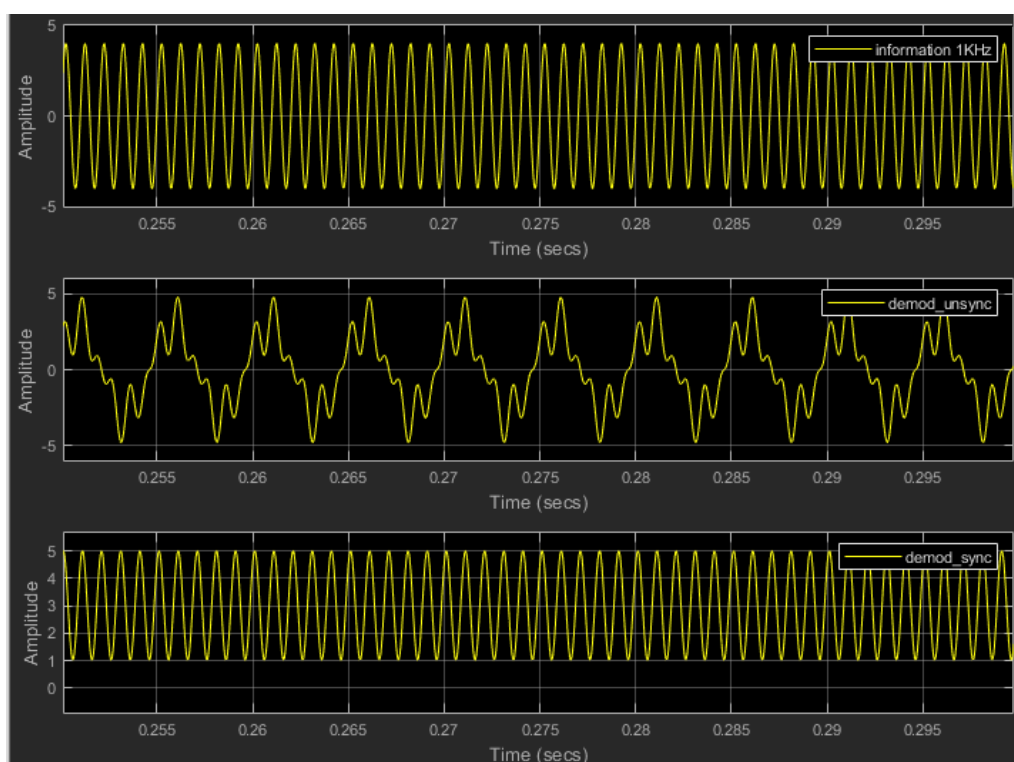


图 6-4 解调之后的时域波形

6.2 PLL 简介

PLL 在解决同步问题中，是一个常用的模块，它能够跟踪频率的变化，抵抗噪声的印象，并且重新生成一个正弦波，这个重新生成的正弦波就是来进行同步解调的关键。有关研究 PLL 的论文和文献有很多，感兴趣的用户可以自行搜索，在本开发手册中，我们之后简单的了解 PLL 的组成和工作原理。

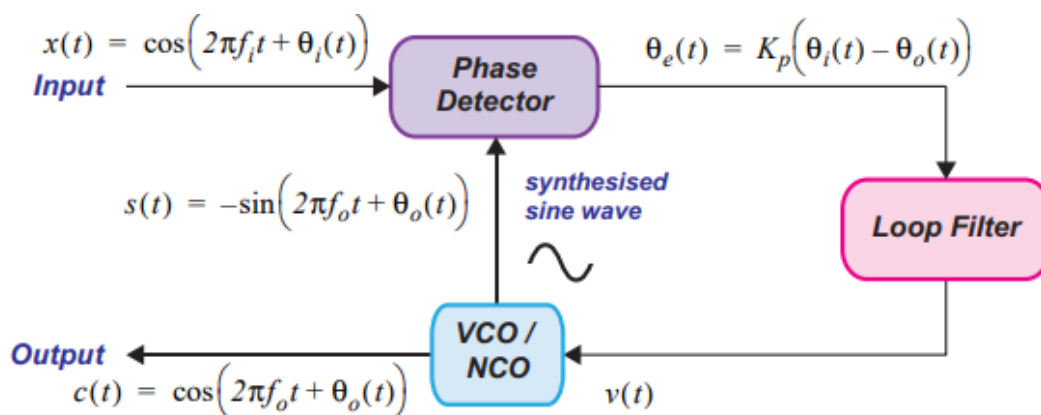


图 6-5PLL 的基本组成部分

一个 PLL 的基本结构如上图所示,主要包含 3 个部分，分别是：

鉴相器：鉴相器用于生成一个误差信号，这个误差信号是由输入信号和 PLL 生成的正弦波两者产生的。用于检测输入信号和生成信号的相位变化。

环路滤波器：环路滤波器会对鉴相器的输出信号进行滤波消除其高频分量，然后将滤波之后的信号提供给压控振荡器或者数控振荡器，用于 PLL 输出信号的频率。

VCO/NCO：压控振荡器或者数控振荡器的作用是产生一个正余弦信号，这个信号能够根据输入的误差信号来动态地调节生成的信号的频率和相位。

6.2.1 鉴相器

6.2.1.1 鉴相器原理

鉴相器的作用是要比较参考的输入信号和本地生成的正弦波（由 VCO 或 NCO 生成）之间的相位差，然后把这个相位差按比例产生另一个信号。我们把这个相位差叫做相位误差信号，对于理想的鉴相器，这个信号的形式如下：其中 K_p 是鉴相器增益， $\theta_i(t)$ 和 $\theta_o(t)$ 分别是鉴相器输入参考信号相位和 PLL 输出信号相位。

$$\theta_e(t) = K_p(\theta_i(t) - \theta_o(t))$$

最简单的鉴相器就是乘法器，将输入的参考信号和 PLL 输出的正弦信号进行相乘，将会得到一个高频分量和一个低频分量，高频分量经过环路滤波将会被滤除，低频分量就是两个信号的相位差。用公式可以很直观地表示这一过程。

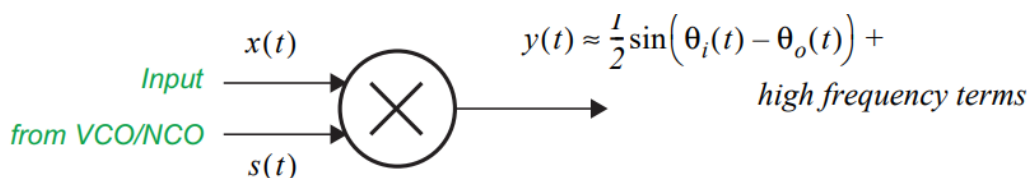


图 6-6 鉴相器结构

假定输入的参考信号是：

$$x(t) = \cos(2\pi f_i t + \theta_i(t))$$

PLL 输出的余弦信号为：

$$s(t) = -\sin(2\pi f_o t + \theta_o(t))$$

将这两者经过鉴相器相乘之后的结果：

$$\begin{aligned}
 y(t) &= x(t) \times s(t) \\
 &= \cos(2\pi f_i t + \theta_i(t)) \times -\sin(2\pi f_o t + \theta_o(t)) \\
 &= \frac{1}{2} \left[\sin((2\pi f_i t + \theta_i(t)) - (2\pi f_o t + \theta_o(t))) - \sin((2\pi f_i t + \theta_i(t)) + (2\pi f_o t + \theta_o(t))) \right] \\
 &= \frac{1}{2} \left[\sin(2\pi(f_i - f_o)t + (\theta_i(t) - \theta_o(t))) - \sin(2\pi(f_i + f_o)t + (\theta_i(t) + \theta_o(t))) \right] .
 \end{aligned}$$

由于输入的其中一个信号是经过 PLL 产生的信号，当 PLL 达到稳定或则正在稳定的过程中，那么产生的这个信号的频率应该是和输入信号的参考频率一致，也就是说 f_i 和 f_o 的大小是一致的。于是鉴相器的输出可以表示为：

$$y(t) = \underbrace{\frac{1}{2} \sin(\theta_i(t) - \theta_o(t))}_{\text{low frequency term}} - \underbrace{\frac{1}{2} \sin(4\pi f_i(t) + \theta_i(t) + \theta_o(t))}_{\text{high frequency term}}$$

图 6-7 鉴相器输出

可以看到在这里产生了一个高频分量和一个低频分量，高频分量的频率大约为参考信号的两倍。低频分量就是两个信号的之间的误差。

当两个信号之间的误差很小的时候，可以对余弦信号采取一个近似，也就是：

$$y(t) \approx \frac{1}{2} (\theta_i(t) - \theta_o(t))$$

这样就能够成功地提取出误差信号了。

6.2.1.2 鉴相器仿真

在上面介绍了鉴相器的基本的原理，接下来就可以来在 Simulink 中搭建一个仿真模型来直观地看一下鉴相器是如何工作的。

打开仿真模型 `phase_detector.slx`，可以看到仿真模型如下图所示：

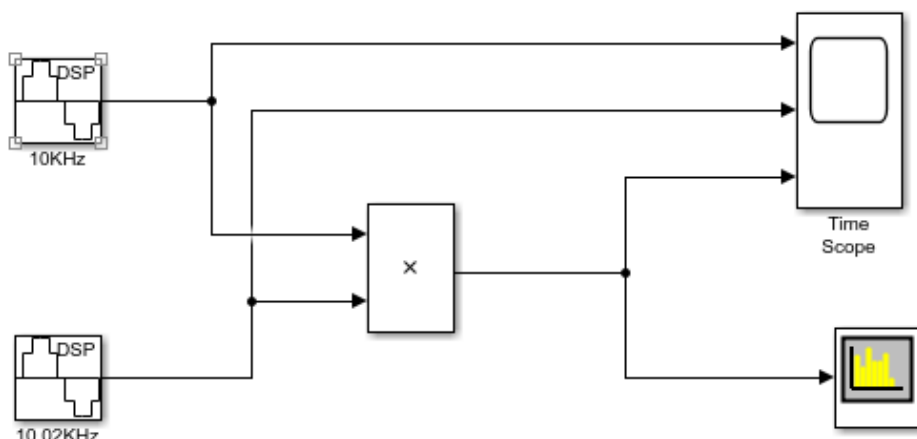


图 6-8 鉴相器仿真模型

输入的有两个信号，分别其中一个频率为 10KHz，另外一个信号的频率为 10.02KHz，将这两个信号经过鉴相器，就能够看到两个信号的频率的差别，首先我们看一下示波器中抓取到的波形，如下图所示：

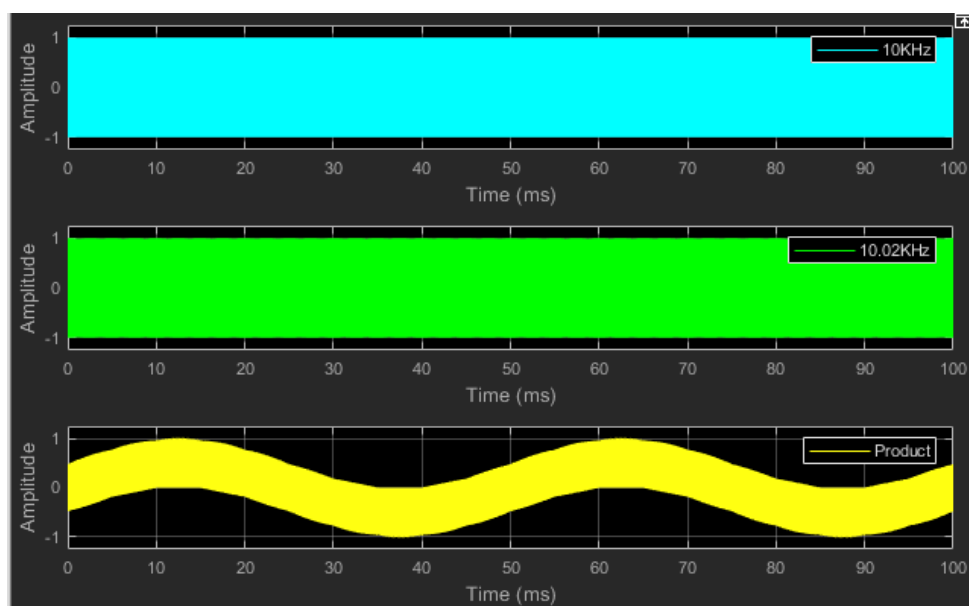


图 6-9 时域波形的整体情况

可以看到，经过鉴相之后，产生了一个信号，从这个信号的整体上来看，呈现了一个频率较低周期信号的形式。这个频率较低的包络，反应的就是两者之间的相差的频率。

若我们放大这个图形来看，将会看到，在生成的信号之后，还有另外的一个高频信号。如图 6-10 所示

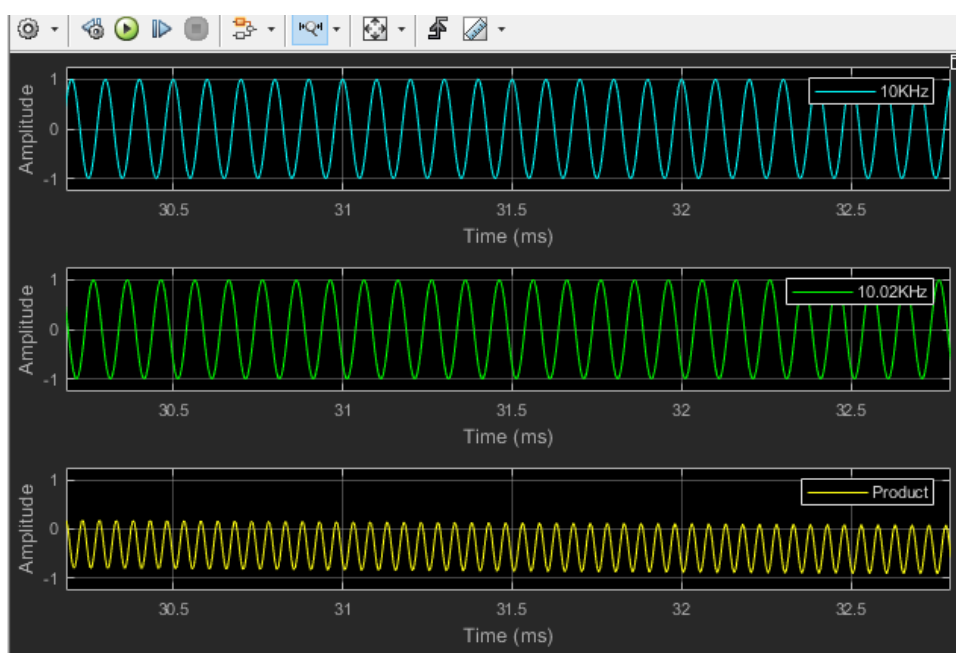


图 6-10 时域波形的局部

我们也能够从频域上更加直观地看出这一过程，使用频谱仪直接观察信号经过鉴相器的信号。

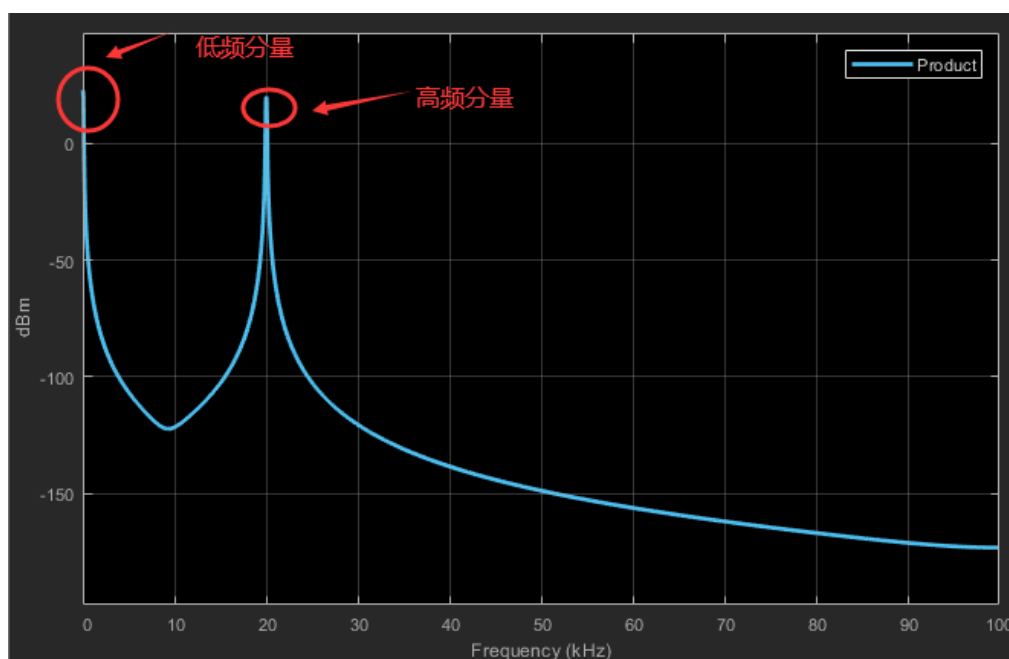


图 6-11 信号的频域分析

从上面的图中可以更加直观地看到两个信号经过鉴相之后，产生了一个高频信号和一个低频的信号。在之后的操作中，只需要将这个高频信号滤除，就能够得到误差信号。

6.2.2 环路滤波器

6.2.2.1 环路滤波器结构

环路滤波器的作用是用来滤除误差信号的高频分量，一般来说环路滤波器是一个低通滤波器，由比例路径和积分路径组成。根据 PLL 当中积分路径的数量，通常又把 PLL 分为 1 类，2 类，3 类 PLL，如下图所示：

PLL Type	Integrators		
	VCO/NCO	Loop Filter	Total
Type 1	1	0	1
Type 2	1	1	2
Type 3	1	2	3

图 6-12PLL 分类

可以看到在当只有 NCO/VCO 这一个积分器时，在环路滤波器中没有积分器的时候，这时候是一个 1 类 PLL。当环路滤波器当中有积分路径时，这时的 PLL 是 2 类或 3 类 PLL。

也就是说环路滤波当中可以只包络比例路径而不包括积分路径。

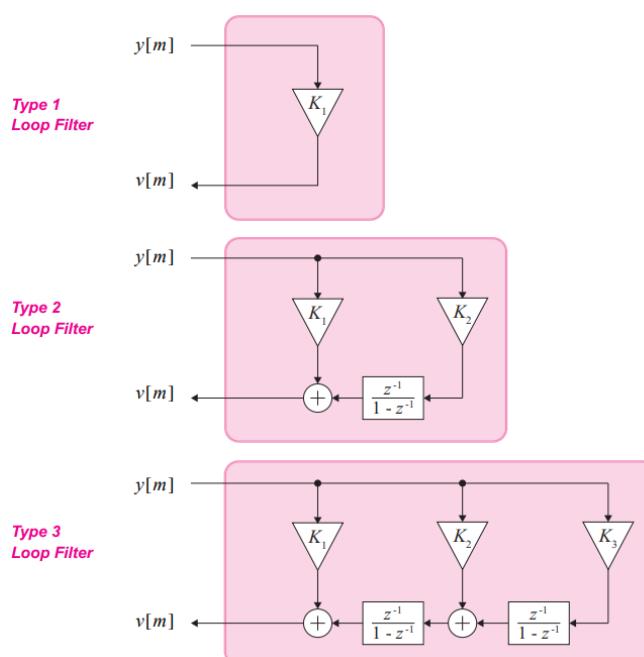


图 6-13 环路滤波器种类和结构

环路滤波器的设计是 PLL 设计中最重要也是最复杂的一部分，在之后设计 PLL 的时候，还会介绍到。

6.2.2.2 仿真环路滤波器

接下来就可以建立一个 Simulink 模型来对环路滤波器进行仿真了。在这里会介绍一个模块合成的方法。新建一个仿真模型 `loop_filter.slx`，首先在 Simulink 模块库中找到 Subsystem 这个模块，并将其添加到仿真模型当中。

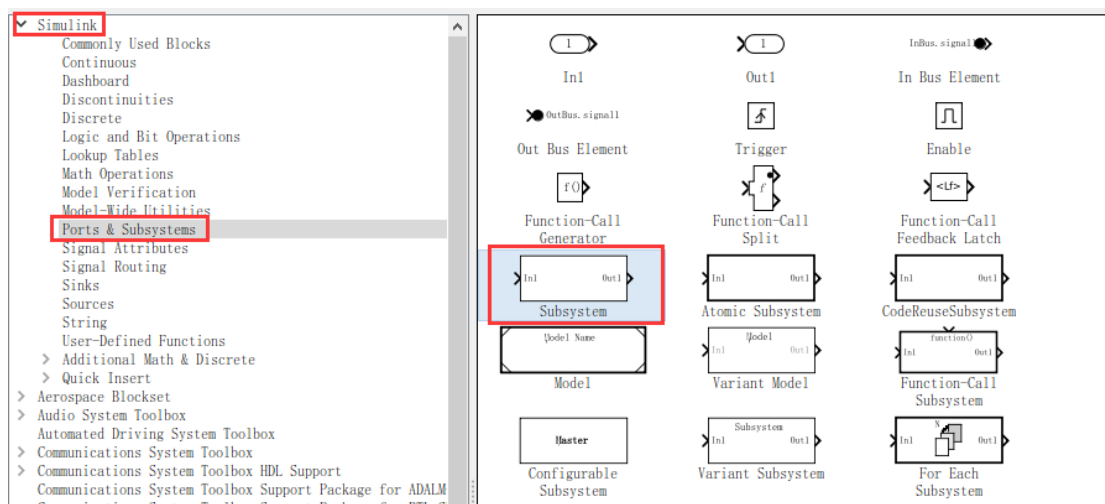


图 6-14 添加 subsystem

添加 subsystem 这个模块之后，双击进入 subsystem 模块的编辑页面。

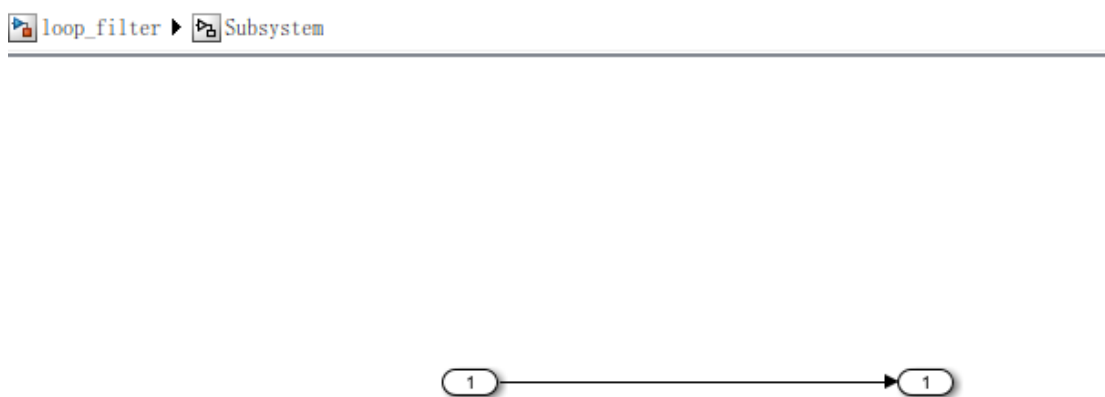


图 6-15

将环路滤波器的主体在这里实现，本次要实现的是一个 2 类 PLL。如下图所示：

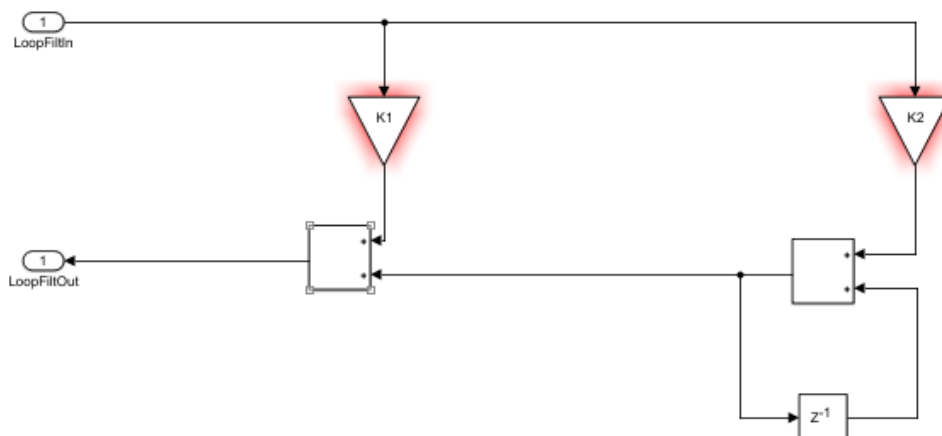


图 6-16 2 类 PLL 设计

在这个子模型中，给先让增益用 K1, K2 来代替，这样在之后就可以使用参数化的方式来更改 PLL。设计完成之后返回上层结果。开始对这个子系统进行封装。选中这个模块之后，点击 **Diagram** → **Mask** → **Create Mask** 这样就进入了封装的界面，或者选中模块之后使用 **CTRL+M** 同样能进入封装界面。

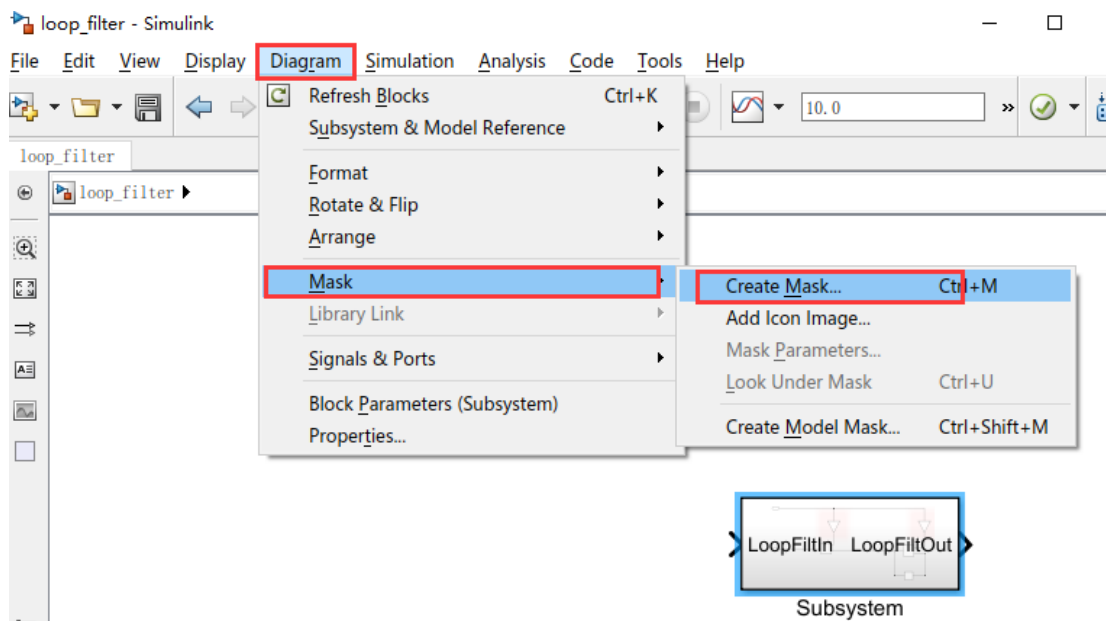


图 6-17 准备封装模块

进入封装界面之后，为这个模块添加两个参数，点击 **edit** 就能添加两个参数，将参数名改为和子模块中两个增益模块相同的参数名。

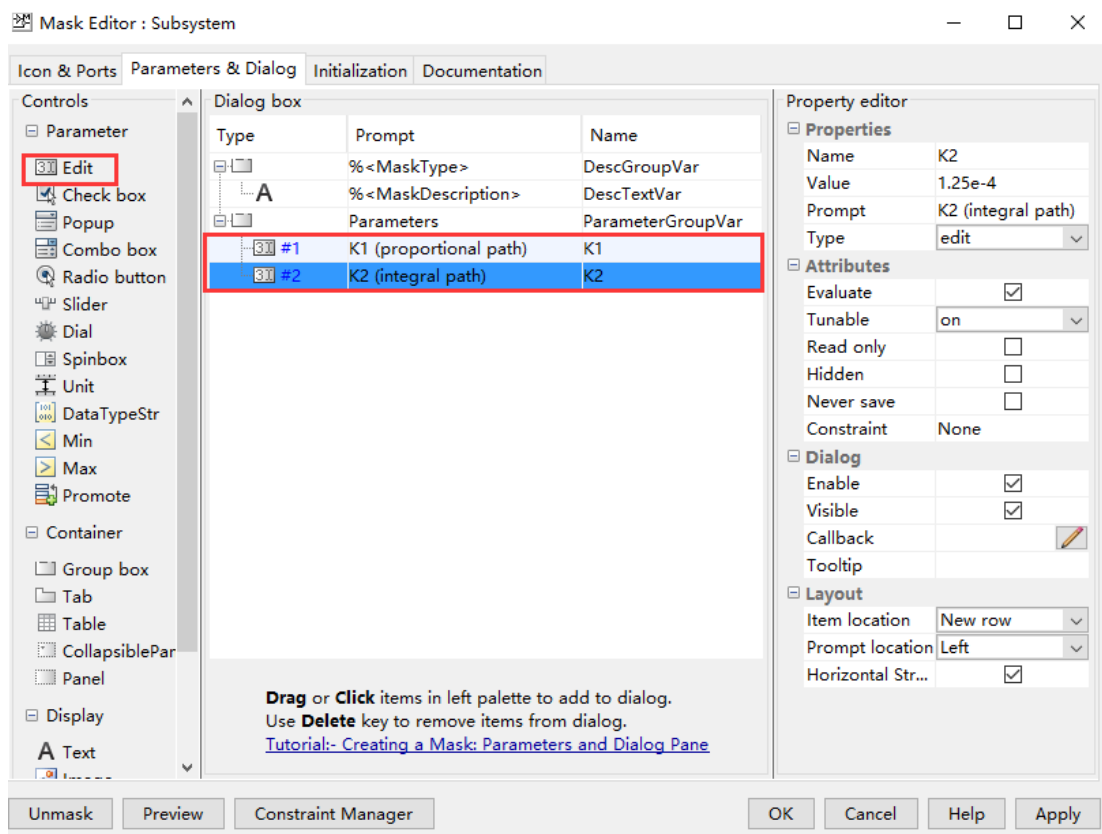


图 6-18

点击 Apply → OK 之后，再次查看这个模块，双击，可以为这个模块定义参数。

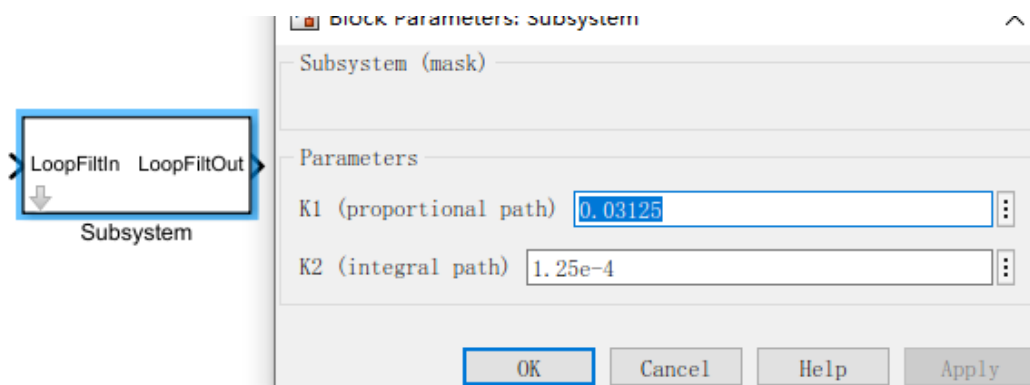


图 6-19 设定参数

然后我们可以把之前的鉴相器输出，提供给环路滤波模块，让环路滤波模块对其进行滤波。仿真模型如下图所示：

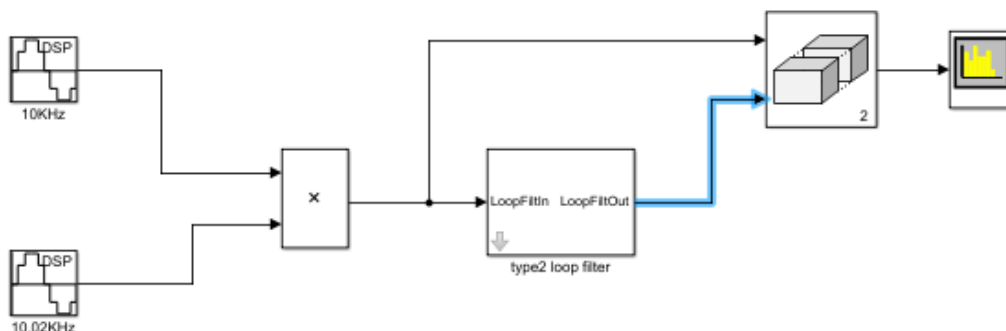


图 6-20 环路滤波仿真模型

点击运行仿真，可以看到仿真的结果如下图所示，可以看到高频分量已经被滤除。

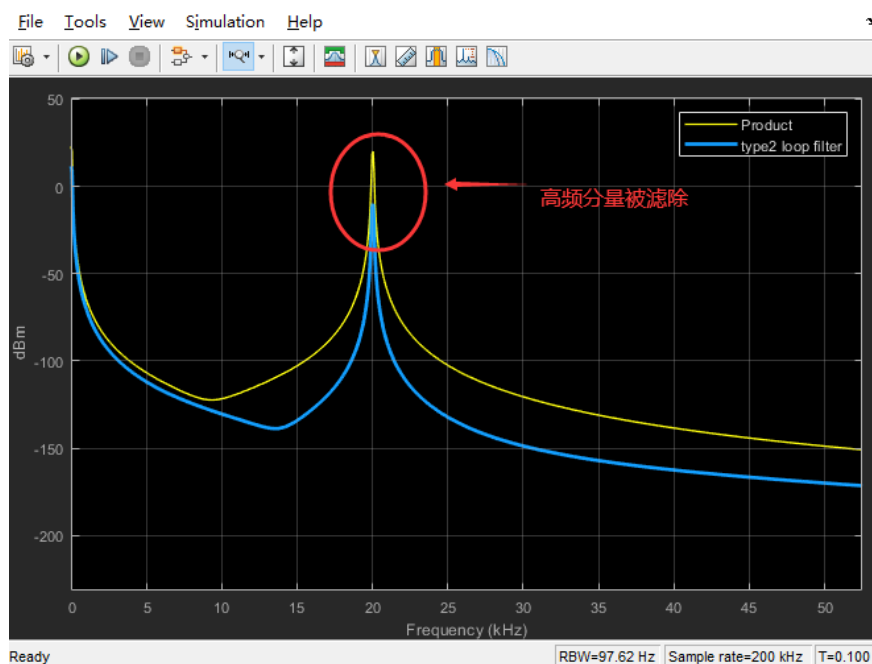


图 6-21 高频分量被滤除

6.2.3 NCO/VCO

PLL 需要一个 VCO 或者 NCO，它们两个的功能类似。尽管 VCO 是模拟器件，我们还是可以在 Simulink 里来仿真 VCO，NCO 本来就是数字的，只在数字信号里使用。VCO 或 NCO 是一个具有标准或静态频率的振荡器， f_o 和一个控制输入(前面介绍的误差信号)可以从静态值向上或向下调整输出频率。

6.2.3.1 压控振荡器 VCO

在 PLL 模型中，经过环路滤波器过滤的相位差作为控制信号 $v(t)$ ，并作为 VCO

的输入。也就是说，相位差越大，产生的修正越大。VCO 的输出信号 $c(t)$ 和 $s(t)$ 分别是频率经过修整的余弦波和正弦波。

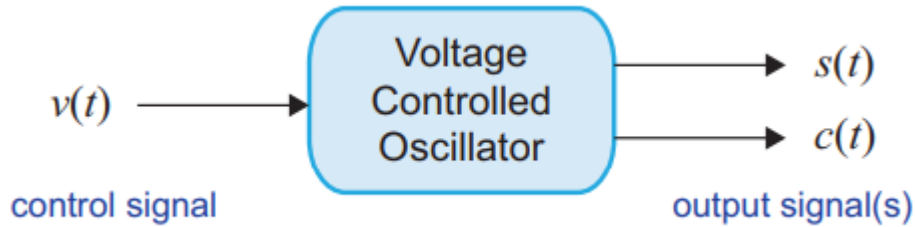


图 6-22 VCO 示意图

其中 VCO 的输出的余弦信号如下：

$$c(t) = \cos(2\pi f_o t + \theta(t))$$

其中 $\theta(t)$ 是输入的误差信号 $v(t)$ 的积分：

$$\theta(t) = k_o \int_0^t v(t) dt$$

其中 k_o 是 VCO 的内部的增益。

6.2.3.2 数控振荡器

NCO 和 VCO 具有类似的结构：

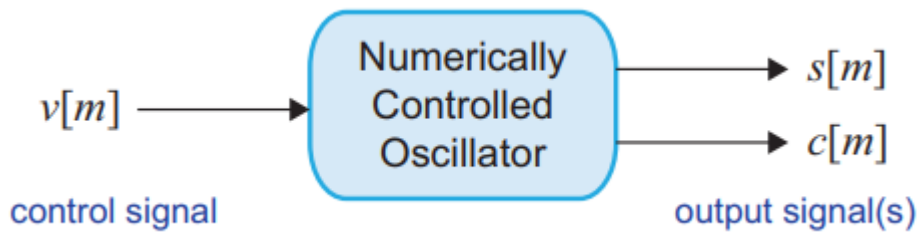


图 6-23

NCO 的输出也和 VCO 相似，都是和输入的误差信号有关。

$$c[m] = \cos(2\pi f_o mT + \theta[m])$$

$$\theta[m] = k_o \sum_{m=0}^M v[m]$$

为了正确理解 NCO 的行为，我们可以看一下它是如何以采样点为基础工作的。NCO 的内部操作经常用步长 $u(t)$ 描述。如果我们假设步长是固定的，那么 NCO 的相位按照每个采样点增加固定的步长如下：

$$\theta[m] = \theta[m-1] + u[m]$$

因此 NCO 的输出在经历固定数量的采样点后完成一个周期 (2π 弧度)。得到相位后，我们把相位对应的幅度计算出来，也就是这个相位值对应的正弦波或者余弦波的值。

从这里可以看出来，步长越大，相位每次增加的量也就越大，累加到一个周期的速率也就更快，从而可以确定，产生的频率也就越高。

PLL 设计的目的是可以追踪动态变化的信号，因此在 PLL 内部其步长是在不停变化的，在这种情况下，我们可以让 PLL 的步长分为两个部分来进行变化，其中一部分为 PLL 的静态步长，动态变化的那一部分，我们将其和输入的误差信号关联起来。

用 $u[m]$ 表示当前的步长， u_q 为静态步长，因此当前的步长就是静态步长和输入的误差信号的和。

$$u[m] = u_q + k_o v[m]$$

由前面的推导，我们可以将 NCO 划分为如下的结构，误差信号进入到 NCO 之后，首先乘上一个增益，然后与静态步长相加作为当前采样点需要变化的步长。使用一个累加器来记录当前的相位变化，由这个相位来产生最终的输出信号。从而实现了由输入的误差信号来控制整个输出的目的。

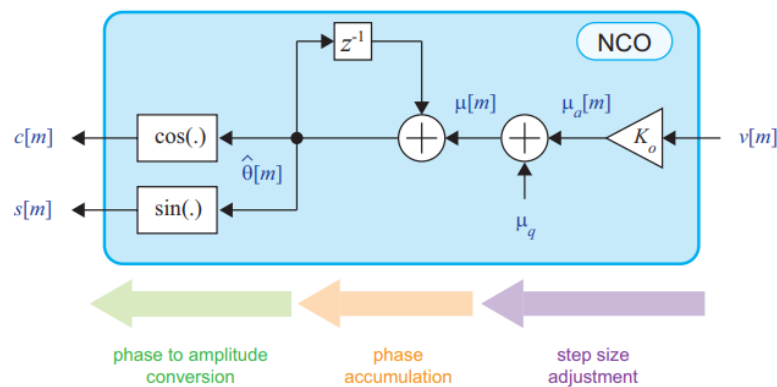


图 6-24

6.2.4 设计一个 2 类 PLL

6.2.4.1 PLL 关键参数

关于 PLL 的设计，由很多资料可以参考，在本章中，不会做过多的说明，本

章中会介绍如何来设计一个 2 类 PLL，并给出其中关键参数的设计。

在设计一个 2 类 PLL 的时候，首先需要考虑的就是一下几个参数

采样周期， T
 NCO 的增益， k_o
 鉴相器增益， k_p
 阻尼系数， ξ
 噪声的带宽， B_n

NCO 可以产生正弦和余弦信号频率可变,使用环路滤波器的输出来改变信号的频率。NCO 的增益一般选择 1。

对于鉴相器而言，若是乘法鉴相器，那么其增益一般选择输入信号振幅的一半，也就是 $\frac{A}{2}$ 。

阻尼系数在 0.5~2 这个范围内都有较好的锁定能力。阻尼系数是 PLL 的一个内部属性,在设计 PLL 之前就需要确定，一般选择阻尼系数为 $\frac{\sqrt{2}}{2}$ 。

PLL 的带宽是一个很重要的参数。比较窄的带宽能够允许更少的噪声信号进入,与此同时需要更多的时间来进行锁定。带宽比较宽的信号能够更快地完成锁定,但是抗噪声性能不好。

有了上面的这些参数，就可以来计算环路滤波器中的两个增益了，这也是 PLL 当中最为复杂的两部分。环路滤波器的增益计算公式如下：

$$K_1 = \frac{4\xi \left(\frac{B_n T}{\zeta + \frac{1}{4\xi}} \right)}{K_o K_p \left(1 + 2\xi \left(\frac{B_n T}{\zeta + \frac{1}{4\xi}} \right) + \left(\frac{B_n T}{\zeta + \frac{1}{4\xi}} \right)^2 \right)}$$

$$K_2 = \frac{4 \left(\frac{B_n T}{\zeta + \frac{1}{4\xi}} \right)^2}{K_o K_p \left(1 + 2\xi \left(\frac{B_n T}{\zeta + \frac{1}{4\xi}} \right) + \left(\frac{B_n T}{\zeta + \frac{1}{4\xi}} \right)^2 \right)}.$$

使用变量来替换一下上面的式子，让其书写更加简化

$$\eta = \frac{B_n T}{\zeta + \frac{1}{4\zeta}},$$

于是可以将计算公式进一步简化：

$$K_1 = \frac{4\zeta\eta}{K_o K_p (1 + 2\zeta\eta + \eta^2)}$$

$$K_2 = \frac{4\eta^2}{K_o K_p (1 + 2\zeta\eta + \eta^2)}$$

当允许通过的噪声的带宽，远比采样频率低的时候，也就是 $B_n T \ll 1$ 的时候，上面的式子可以进一步化简。

$$K_1 \approx \frac{4\zeta B_n T}{K_o K_p \left(\zeta + \frac{1}{4\zeta} \right)}$$

$$K_2 \approx \frac{4(B_n T)^2}{K_o K_p \left(\zeta + \frac{1}{4\zeta} \right)^2}$$

现在，所有的这些有关 PLL 的参数都拥有了，就可以开始来设计一个 PLL 了，在接下来的例子里面，我们会自己来设计一个 PLL 的仿真模型。

6.2.4.2 设计一个 2 类 PLL

在设计一个 PLL 之前，我们先要确定这个 PLL 的参数，这些参数决定了 PLL 最终能否锁定，除此之外还决定了 PLL 的性能。

1) 使用 Matlab 脚本确定参数

新建一个脚本来完成所需要的参数的计算，首先确定系统确定的参数：

采样频率 $f_s = 10\text{MHz}$ ，采样周期就是 $T = 10\text{ns}$

鉴相器增益 k_p : 0.5

NCO/VCO 增益 $k_o=1$
 阻尼系数 ξ : 0.707
 噪声带宽 $B_n = 20\text{K Hz}$
 期望生成的频率 $f_q = 1\text{MHz}$

根据这些参数，我们可以计算出环路滤波器的参数，新建一个 matlab 脚本 `pll_parameters.m` 来计算这些参数：

```
fs = 10e6;           % sampling rate
T = 1/fs;           % sample period
fq = 1e6;           % expected (quiescent) frequency
d = 0.707;          % damping ratio
Kp = 0.5;           % phase detector gain (multiplier)
Ko = 1;             % oscillator gain
Bn = 20e3;          % noise bandwidth

% calculate secondary parameters
%-----
mu_q = (2*pi*fq)/fs; % quiescent step size for NCO

eta = (Bn*T)/(d+(1/d)); % a repeated section of the equations below

K1 = (4*d*eta)/(Ko*Kp*(1+(2*d*eta)+eta^2)); % Loop filter prop coeff.

K2 = (4*eta^2)/(Ko*Kp*(1+(2*d*eta)+eta^2)); % Loop filter integral
coeff.

% for simulation purposes...
%-----
n = 50000;          % number of samples in simulation
ferr = 4500;         % frequency error of input sine (in Hz)
awgn = 0.05;         % variance of AWGN
```

点击运行即可在 Matlab 的工作区生成这些脚本。

接下来就可以来搭建一个 Simulink 模型来验证 PLL 的功能了，首先新建一个 Simulink 模型，然后保存为 `type2_pll.slx`。

2) 使用 Simulink 预加载模式

在上面我们使用的是 Matlab 来生成 PLL 所需要的参数，但是这样有一个比较麻烦的是每次从新打开 Simulink，工作区必须要存在这些参数，才能使 Simulink 的模型正常工作。

在 Simulink 模型中，提供了一些功能，能够使 Simulink 在开始仿真前和结束仿真的时候去执行一定的脚本。

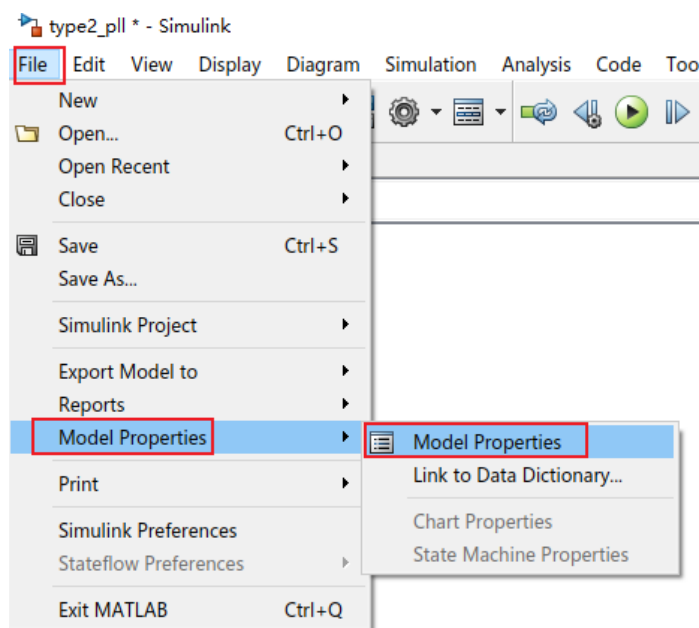


图 6-25 仿真模型属性

在 Callbacks 当中，可以设置在仿真不同阶段可以执行的操作，在这里我们在 PreLoadFcn 里面把先前在 Matlab 脚本中计算的参数，放到这个位置，那么之后每次打开这个仿真模型的时候，都会先来执行这里面的脚本，这样就能够保证每次打开这个仿真模型的时候都已经有了仿真模型所需要的参数了。

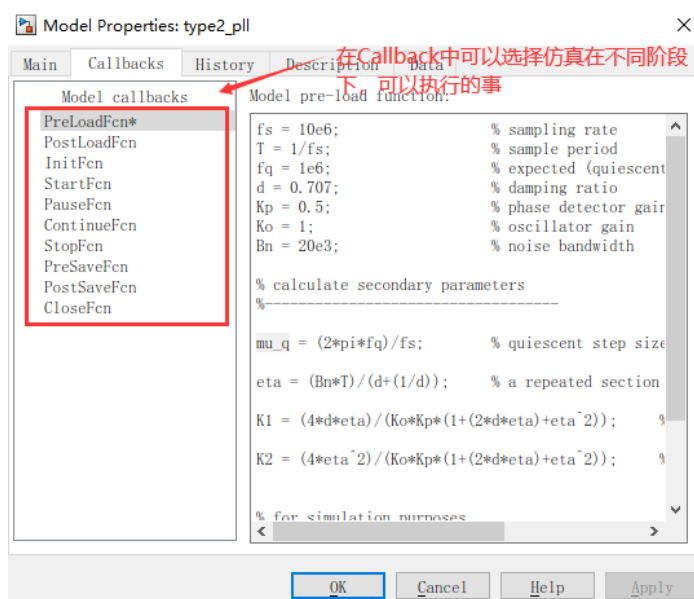


图 6-26

3) 搭建仿真模型

PLL 的仿真模型如图 6-27 所示，其中鉴相器和环路滤波器在前面的已经介绍过，下方是 NCO 的仿真模型，我们可以进到子模块当中查看其中的结构。

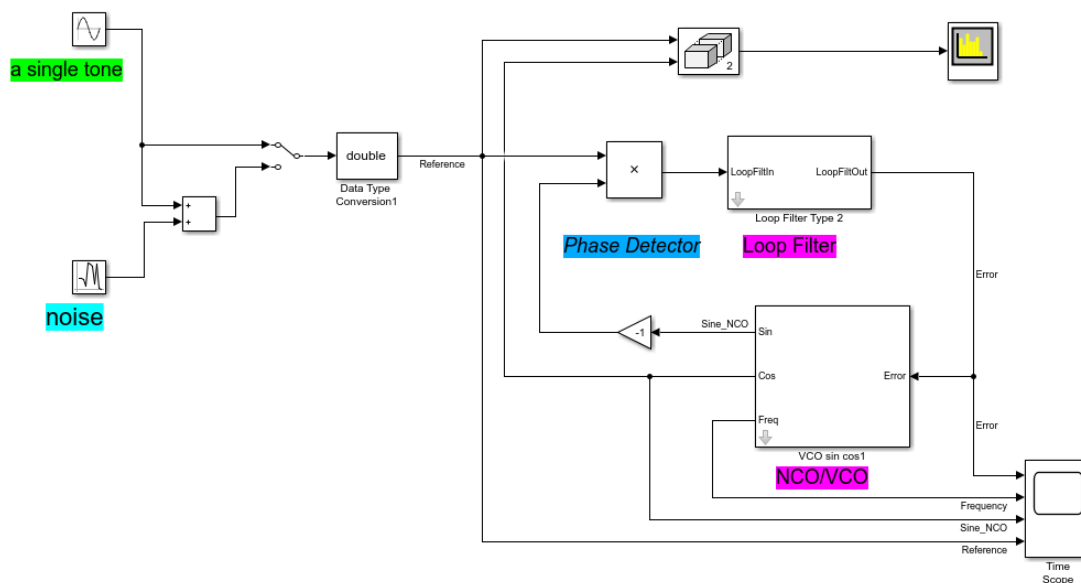


图 6-27 PLL 仿真模型

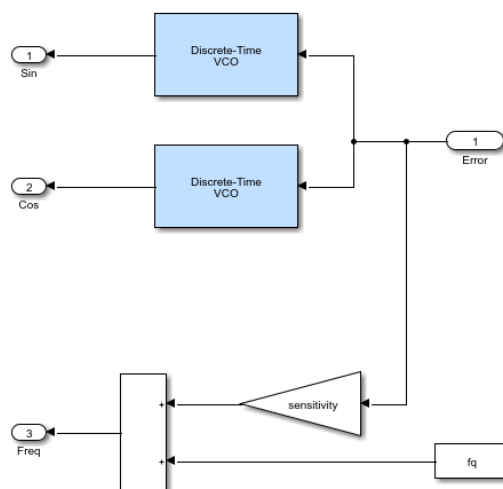


图 6-28 NCO 结构

在 NCO 当中，使用了两个离散的 VCO，这个模块的作用就是根据输入的误差信号来产生对应的频率，对这个模块需要设置期望的静态频率。

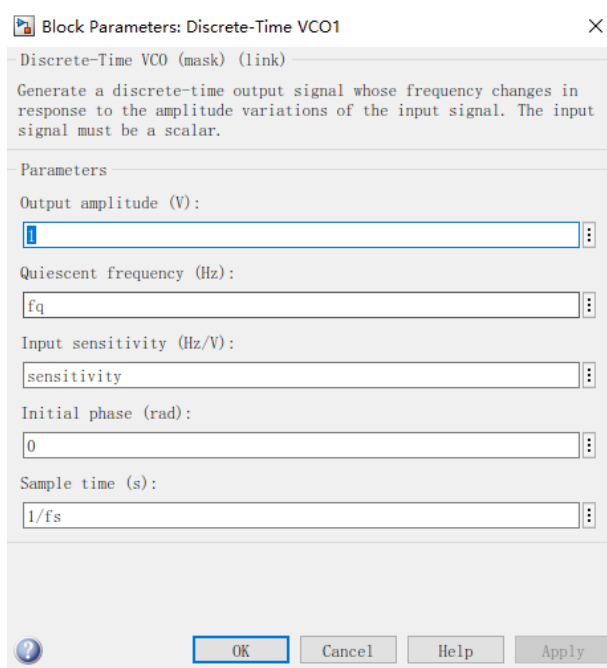


图 6-29 NCO 设置

然后点击运行仿真模型，来观察运行之后的结果，首先我们选择输入的信号是一个单音信号，然后运行仿真。

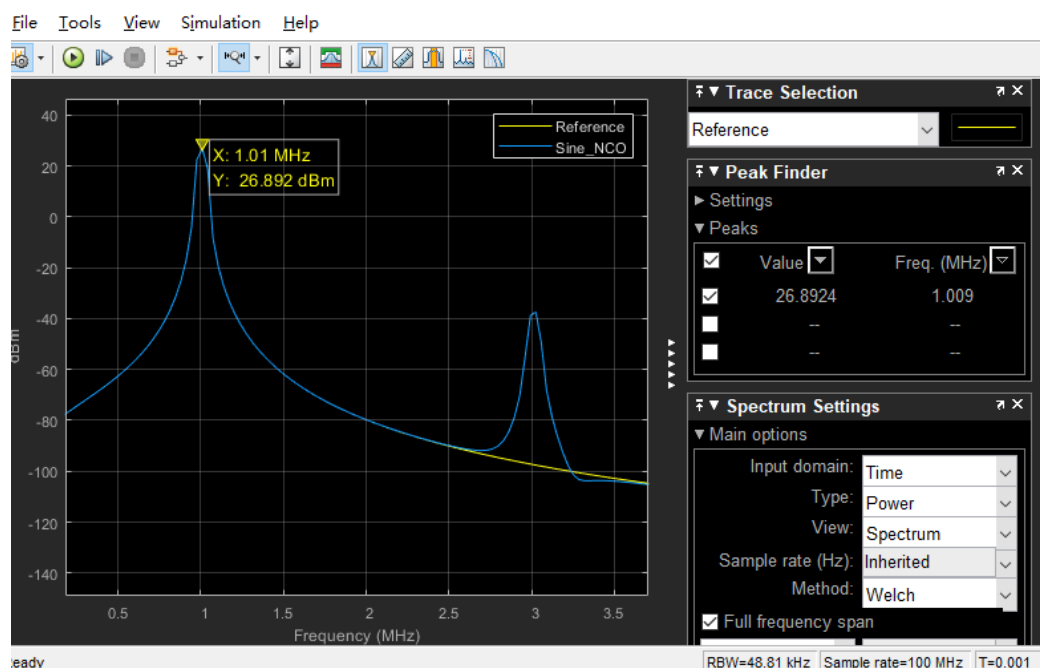


图 6-30 频谱仪结果

从频谱仪上来看，生成的正弦信号的频率和输入的参考频率之间能够很好的重合，然后我们观察示波器当中的结果，来查看这一锁定的过程。

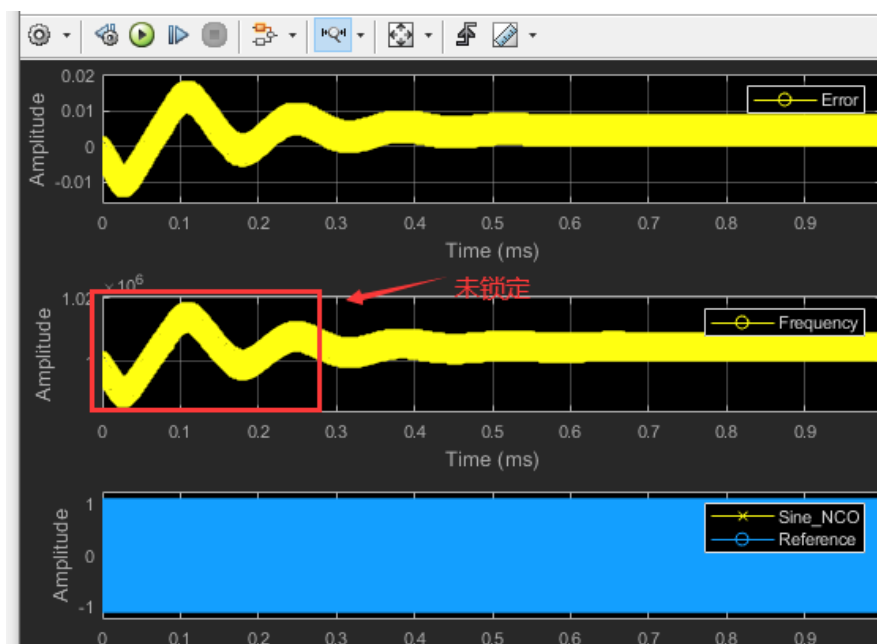


图 6-31PLL 锁定过程

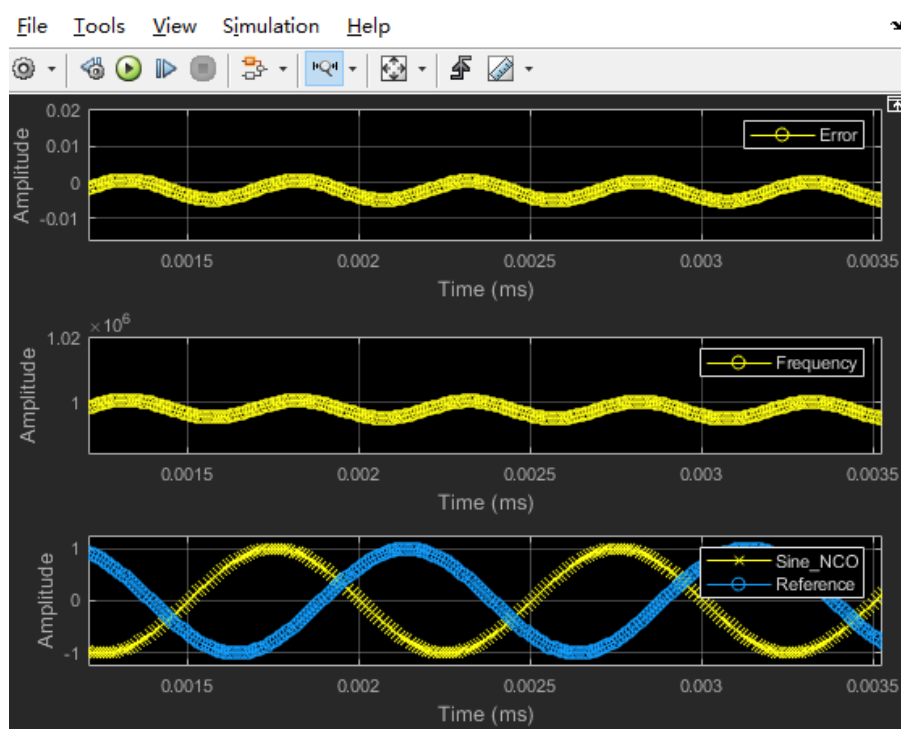


图 6-32 开始阶段信号未锁定

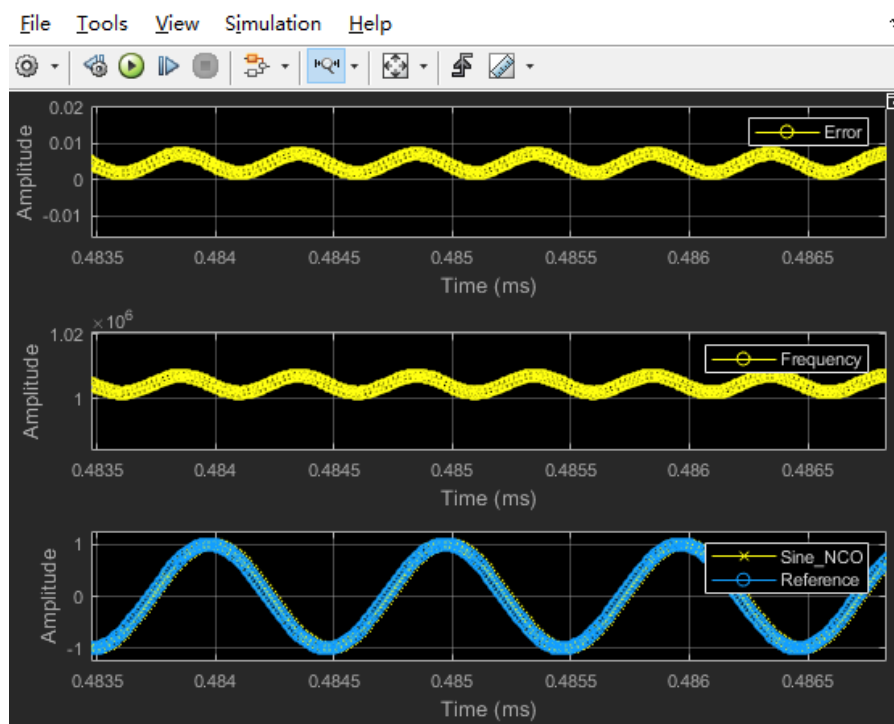


图 6-33 PLL 完成锁定

在一开始阶段，PLL 还没有锁定，可以看到这个时候生成的信号的频率和参考输入之间有一定的差别，然后 PLL 会不断地调整输出地信号，最终能够使输出的信号和参考信号锁定。

接下来，尝试在输入的参考信号之间加如噪声，然后看 PLL 能否完成锁定。

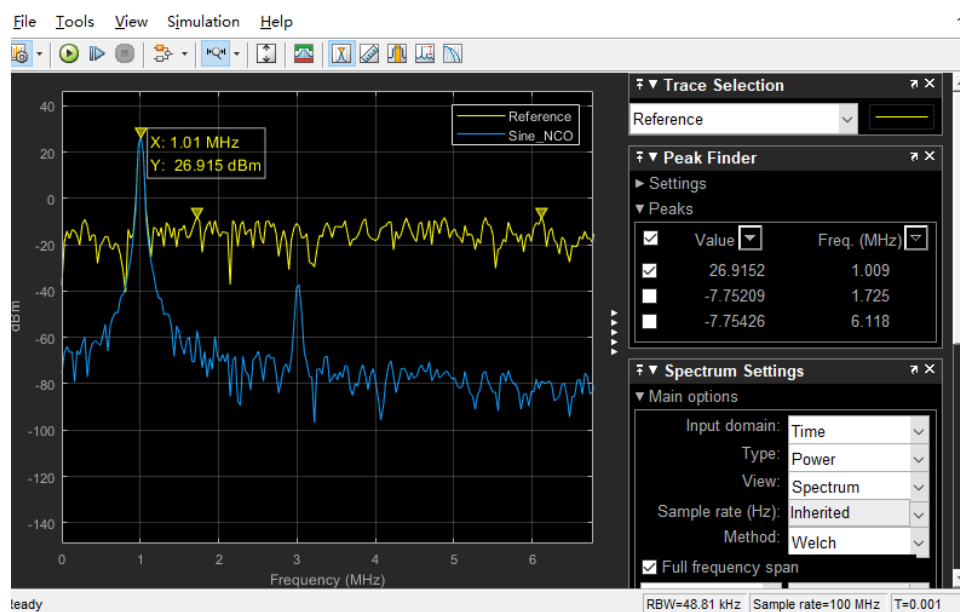


图 6-34 加入噪声之后的结果

可以看到在输入的参考信号中加入噪声之后，最终生成的频率和信号的频率

还是相同的。从示波器上，也能看出这一锁定的过程。

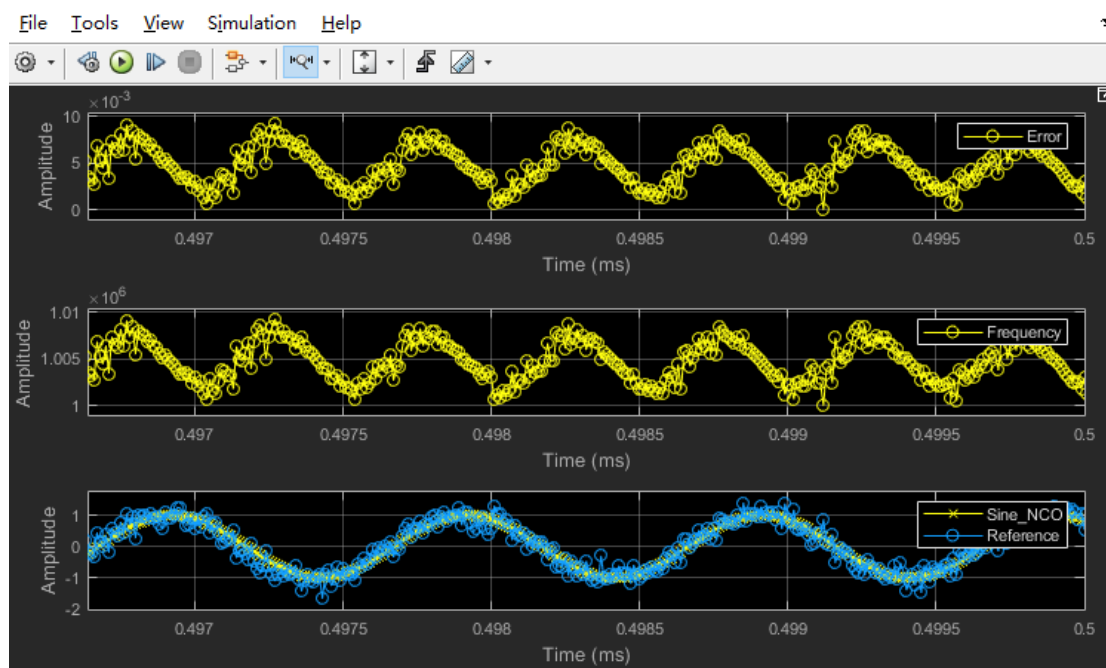


图 6-35 信号最终锁定

6.3 常规调幅的相干解调

在前面介绍了 PLL 实现同步的原理，PLL 是实现相干解调的非常重要的一个模块，在相干解调当中经常会使用到。现在我们可以来实现使用 PLL 来实现常规调幅的相干解调。

打开仿真模型 `dsb_pll_demod.slx`，如图 6-36 所示：

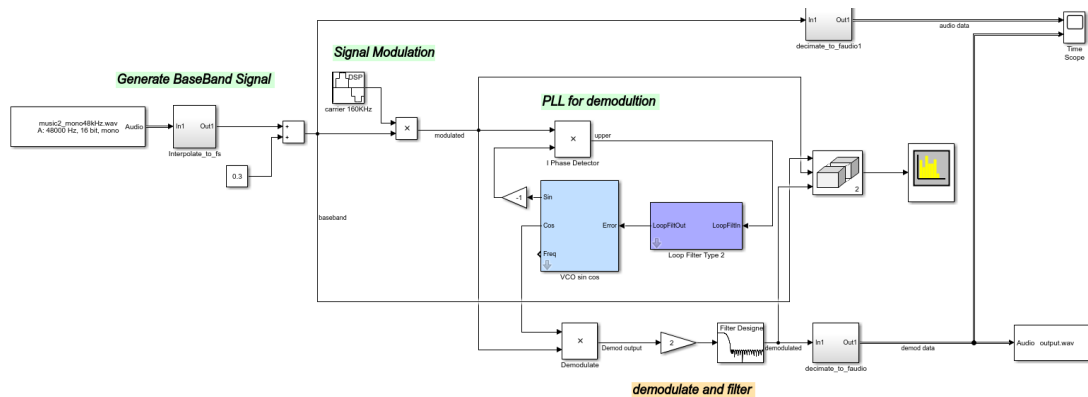


图 6-36 PLL 调制解调仿真模型

基带数据由音频数据提供，然后进行一个经过一个插值模块，提升信号的采

样率，在之后进行 AM-DSB-TC 调制，调制载波的频率为 96KHz，再之后将已调信号提供给 PLL 模块，也就是依次进行鉴相，环路滤波和压控振荡器，产生两路信号，其中一路信号用于和输入的信号鉴相，另外一路信号用于信号的解调。解调出来的信号再经过滤波和下采样，给到输出音频文件模块。

点击运行仿真，观察基带信号和解调信号的波形，并且观察频域图，确定解调是否正确。

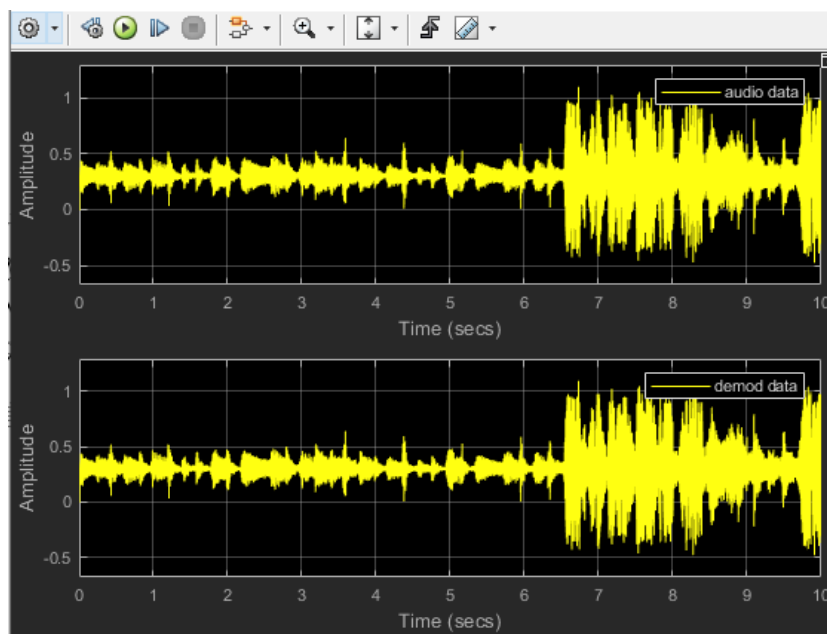


图 6-37 解调出来的信号的时域波形

从图 6-37 上可以看到，解调出来的信号波形和基带信号的波形基本一致。

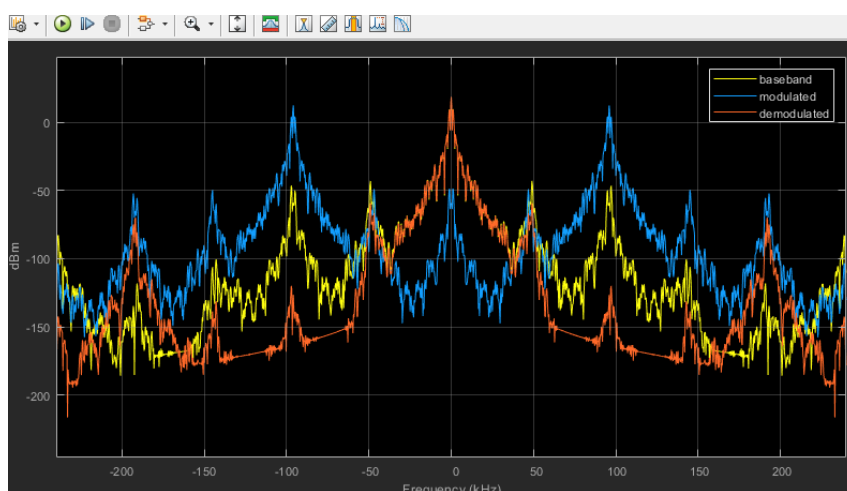


图 6-38

从频域图上也可以看到，基带信号经过调制过后，能够成功地被解调出来。

程序运行完成之后，你可以在 `simulink` 所在工程目录发现一个音频文件，这个音频文件就是解调出来的音频。

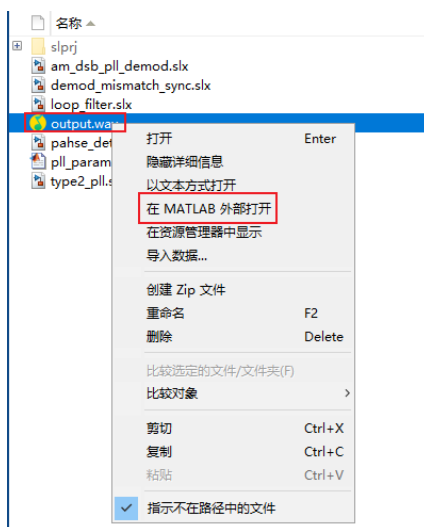


图 6-39 音频文件

选中它之后，然后选择在外部打开，就可以用播放器将音频文件播放出来。

6.4 ANTSDR 相干解调 AM-DSB-TC

在上一章的实验当中，我们使用 ANTSDR 用非相干的方式成功地解调出来了音频信号，在本章中，我们将使用 ANTSDR 进行非相干地解调出常规调幅的信号。

打开仿真模型 `antsdr_am_pll_demod.slx`，如图 6-40 所示：

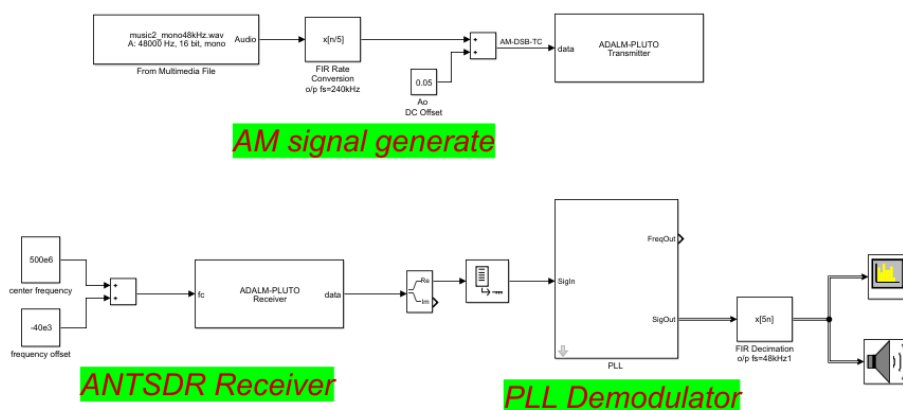


图 6-40 ANTSDR 解调模型

可以看到本次实验的模型和前面包络检波的模型基本一致，唯一的区别就是使用了 PLL 来进行解调，点击小箭头可以进入 PLL 的内部，如图 6-41 所示，这个模块和前面小节在仿真 PLL 模块的时候基本一致。这个模块被我们参数化了，

可以在使用的时候，可以通过修改参数，来调整 PLL 的锁定能力。如图 6-42 所示。

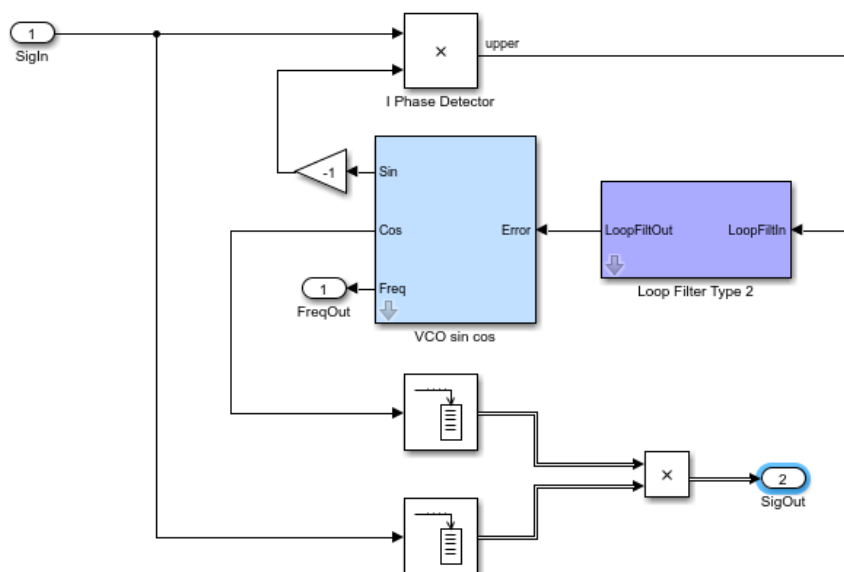


图 6-41 PLL 内部结构

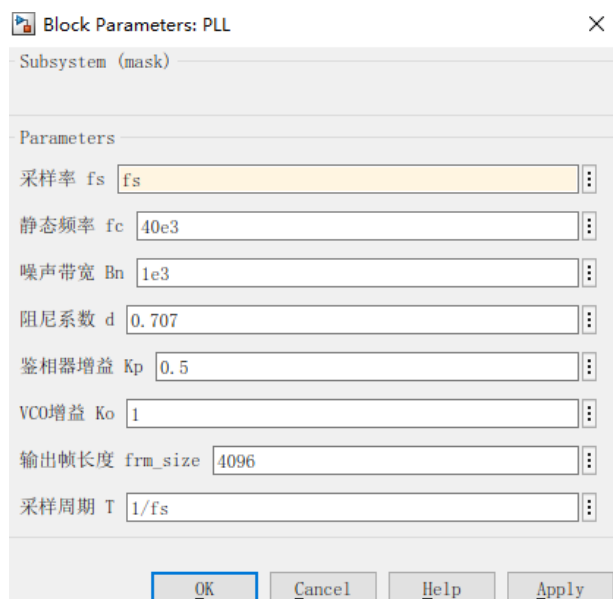


图 6-42 PLL 设置

运行这个仿真模型，我们就能够从 ANTSDR 发出音频信号到基带，通过 AD936x 进行调制过后经天线发射出去。在接收机当中，接收到天线上的信号，然后经过 AD936x 采样之后传递给 Simulink，最终经过相干解调之后，我们能够听到音频信号。

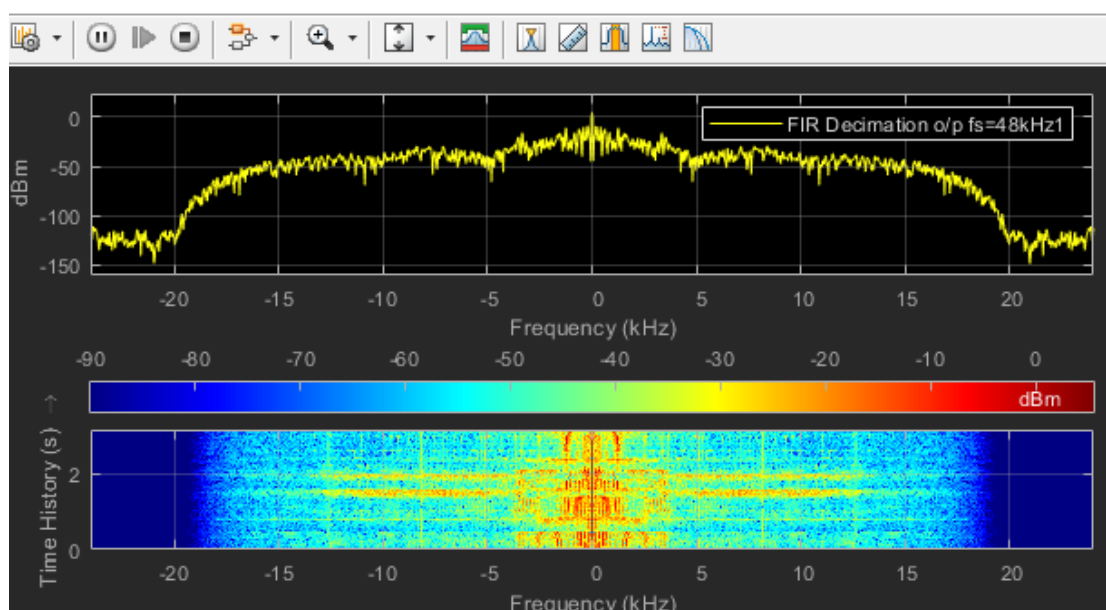


图 6-43 接收到的音频信号

6.5 总结

在本章的实验当中,我们首先介绍了 PLL 的原理,知道了 PLL 内部由鉴相器,环路滤波器和 NCO/VCO 组成。并且介绍了如何根据参数来设计 PLL,给出了其中关键的环路滤波器的参数的计算公式。并且通过 Simulink 里面的模型对各个模块都有一个直观地认识,最终完成了一个 PLL 的设计。

在设计完成 PLL 之后,我们使用 PLL 来进行了相干解调的仿真,并且最终通过 ANTSDR 完成了一个相干解调的仿真。

PLL 在相干解调中经常会使用到,这是同步解调当中一个常用的模块。

第7章 AM-DSB-SC 调制

在前面的实验中，我们实现了使用 ANTSDR 完成常规调幅，并且使用包络检波和相干解调两种方式对已调信号进行解调。在本章的实验中，我们会介绍抑制载波的双边带调幅，给出其调制和解调的仿真模型。

AM-DSB-SC 的调制会比前面我们说介绍的常规调幅更加简单，但是，相对而言解调就会更加麻烦，因为不能使用非相干的方式对 AM-DSB-SC 信号进行解调，在本章中我们会介绍 Costas 环来解调 AM-DSB-SC 信号，Costas 环在同步问题中是比较常见的一种结构。在 DQPSK 的解调和扩频通信中经常会使用到。

7.1 AM-DSB-SC 调制的原理

从数学上来讲，最简单的 AM 就是抑制载波的双边带调幅，原始消息信号 $s_i(t)$ 和载波 $s_c(t)$ 混频，产生了已调信号 $s_{am-dsb-sc}(t)$ 。

$$s_{am-dsb-sc}(t) = s_i(t) \times s_c(t)$$

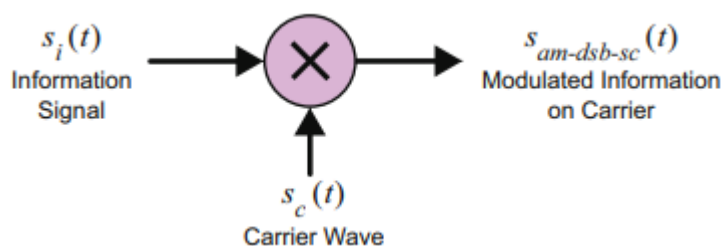


图 7-1 抑制载波的双边带调幅示意图

和前面的章节类似，本章中还是会先使用 AM-DSB-SC 来调制一个单音信号，确定其在时域和频域上的表现形式，然后会使用 AM-DSB-SC 来调制一个基带信号，观察基带信号被调制过后再时域和频域上的表现。

7.1.1 AM-DSB-SC 调制单音信号

首先消息信号消息信号振幅为 A_i ，频率为 f_i ：

$$s_i(t) = A_i \cos(2\pi f_i t) = A_i \cos(\omega_i t)$$

然后是调制的载波信号

$$s_c(t) = A_c \cos(2\pi f_c t) = A_c \cos(\omega_c t)$$

将两者进行混频之后得到的结果如下：

$$s_{am-dsb-sc}(t) = A_i \cos(\omega_i t) A_c \cos(\omega_c t)$$

$$s_{am-dsb-sc}(t) = \frac{A_i A_c}{2} \left(\cos(\omega_c - \omega_i)t + \cos(\omega_c + \omega_i)t \right)$$

可以看到，使用了抑制载波的双边带调制之后会在载波附近产生上下两个边带。

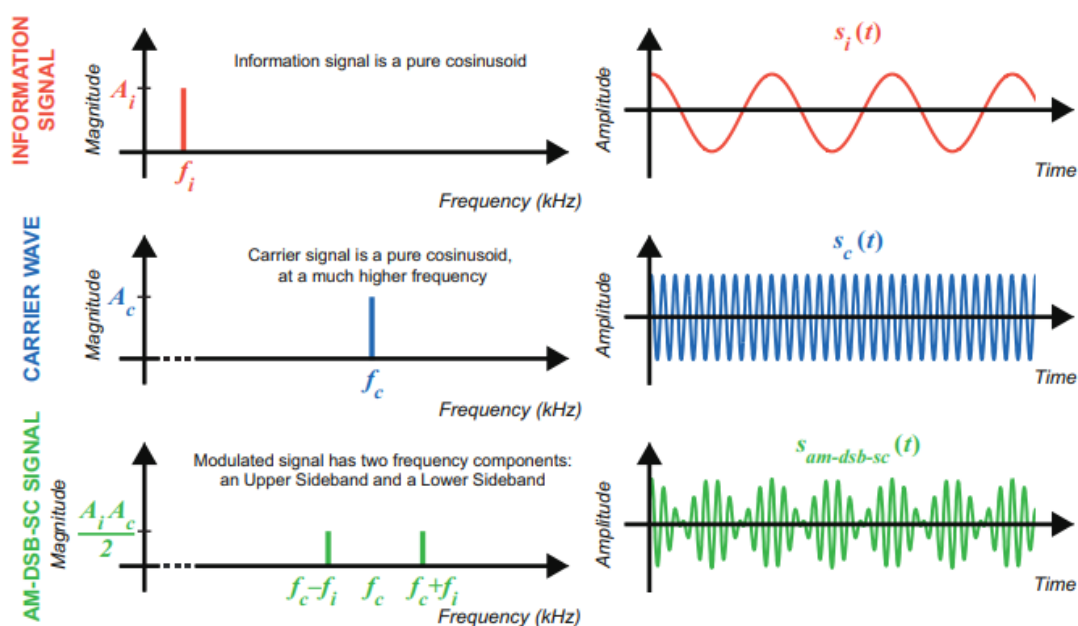


图 7-2 AM-DSB-SC 调制

7.1.2 AM-DSB-SC 调制多个单音信号

前面实现了使用 AM-DSB-SC 调制一个单音信号，可以看到产生了上下两个边带，现在来使用 AM-DSB-SC 调制多个单音信号。其实调制的结果和在前面前面的章节中实现的 AM-DSB-TC 的调制是类似的，在这里我们给出调制前后的示意图如图 7-3 所示。

基带信号通过调制，被搬移到了上下两个边带上，这两个边带关于载频 f_c 对称。

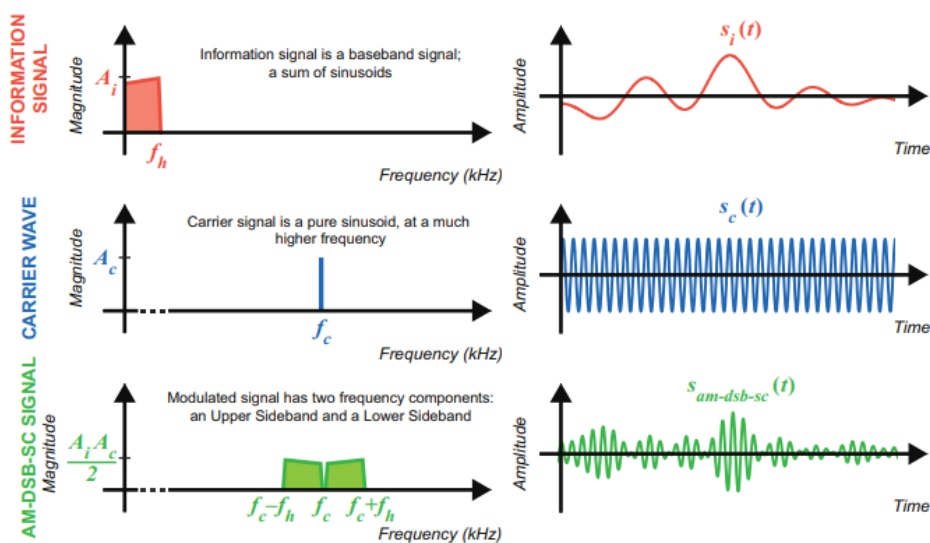


图 7-3 AM-DSB-SC 调制基带信号

7.2 使用 Simulink 仿真 AM-DSB-SC

为了更加直观地展示 AM-DSB-SC 的调制的过程，我们可以使用 Simulink 来仿真这一过程，AM-DSB-SC 的仿真比较简单，比上一章当中的常规调幅更加简单。打开仿真模型 `am_dsb_sc_mod.slx`，仿真模型如下图所示：

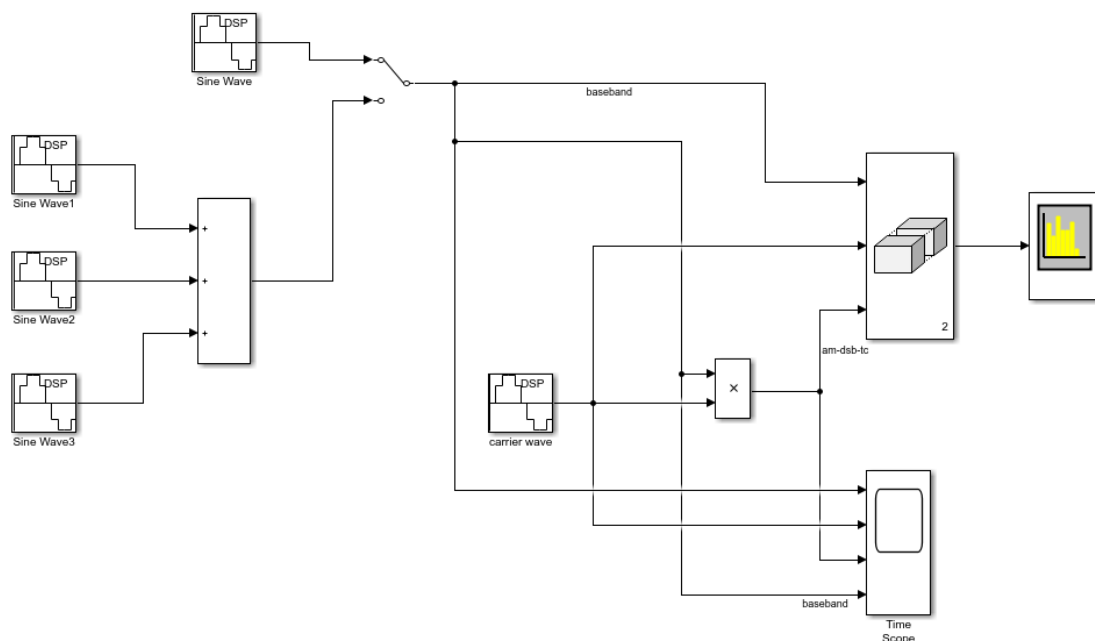


图 7-4 AM-DSB-SC 仿真模型

仿真的结果如下，基带信号被搬移到高频处，可以看到和常规调幅相比，已调信号中没有载波分量。

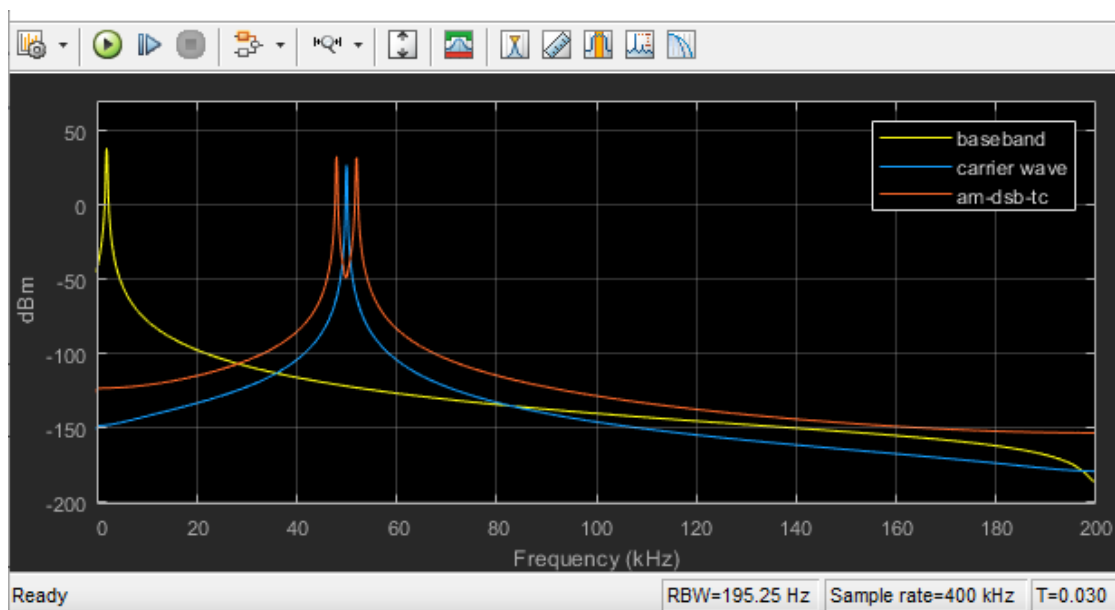


图 7-5 调制的频域分析

调制过后的频域结果如下图所示：

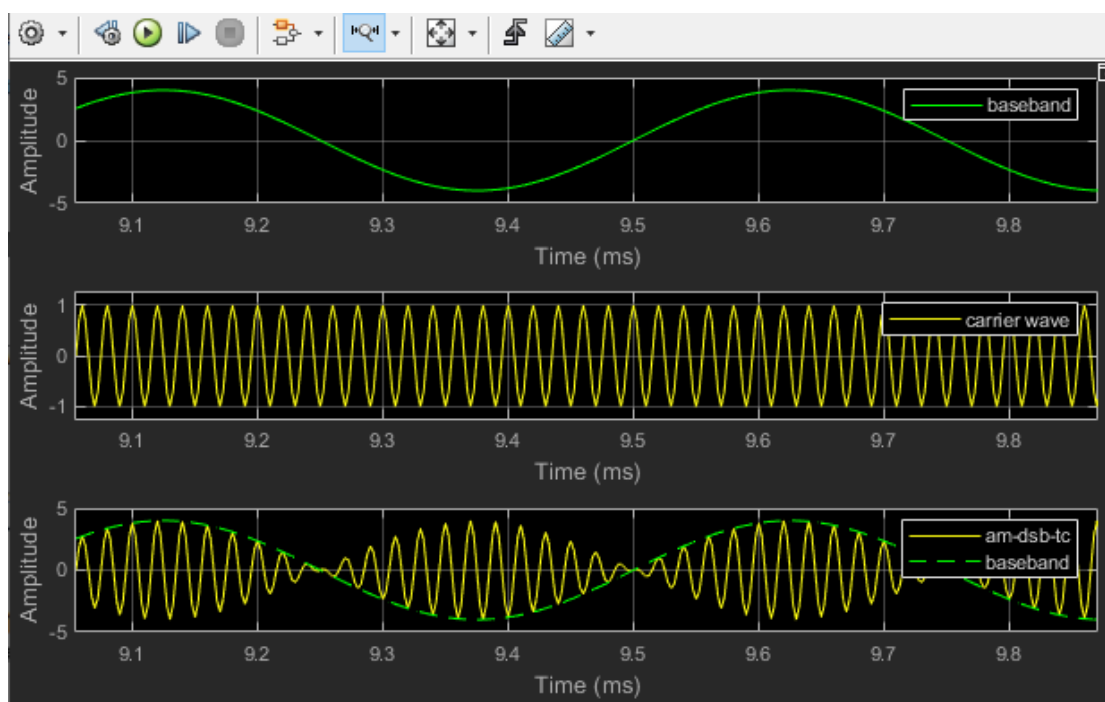


图 7-6 调制的时域

7.3 AM-DSB-SC 解调

1 AM-DSB-SC 比 AM-DSB-TC 更难以相干解调。因为没有明显的载波信号。因

此，接收机必须从接收到的调制后的信号里恢复出载波，然后再用它解调。一般来说，这种情况下要使用 **costas loop** 结构，如下图所示：

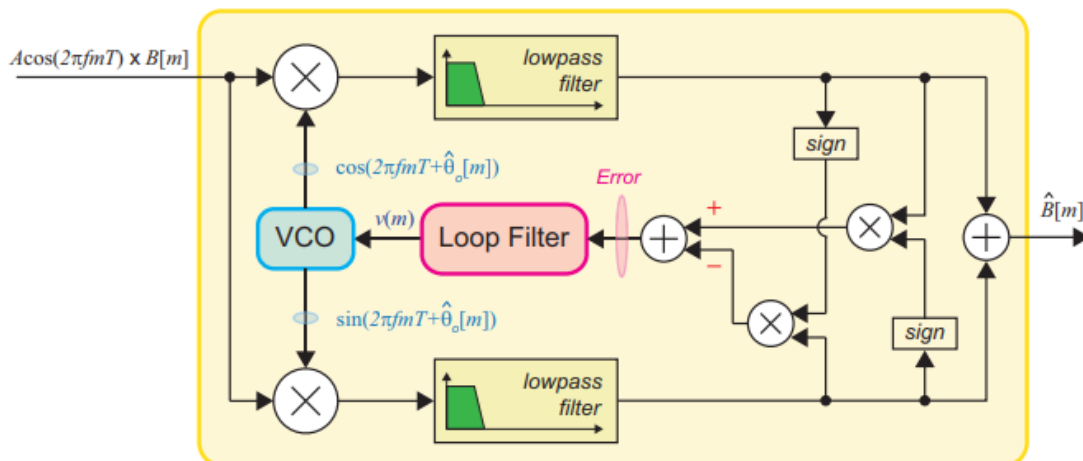


图 7-7 Costas Loop 结构

从 Costas 环的结构来看，会比前面介绍的锁相环的结构要复杂一点，但是我们还是能够从中看出其结构，主要部分还是环路滤波器和 VCO。其工作原理是，由 VCO 产生的正余弦信号分别与输入的信号进行鉴相，通过一个低通滤波器之后来提取误差信号的符号，为了后续求得整个误差信号做准备，最终给到环路滤波器一个误差信号，环路滤波器滤波之后，给到 VCO 一个控制信号，从而动态调整输出的信号的频率。

最终，整个科斯塔斯环的输出是两个之路上的信号之和。

由于在之前的实验当中，已经知道了 PLL 是如何设计的，因此对环路滤波器和 VCO 在这里就再做介绍了。

7.4 使用 Costas 环解调 AM-DSB-SC 信号

接下来会使用 Costas 环来解调 AM-DSB-SC 信号，在这里会用到一个新的模块 Costas 环，首先打开仿真模型，**am_dsb_sc_demod.slx**，如所示。Costas 环的结构和 PLL 的结构很类似，除此之外 Costas 环的参数，其实可以按照 PLL 的参数来设计，关于 PLL 的参数，用于可以参考前面章节中，如何设计 PLL 来进行参考，首先确定 Costas loop 的参数，然后根据这些指定的参数，来确定环路滤波器的系数。这样就能够设计出这个 Costas 环了。

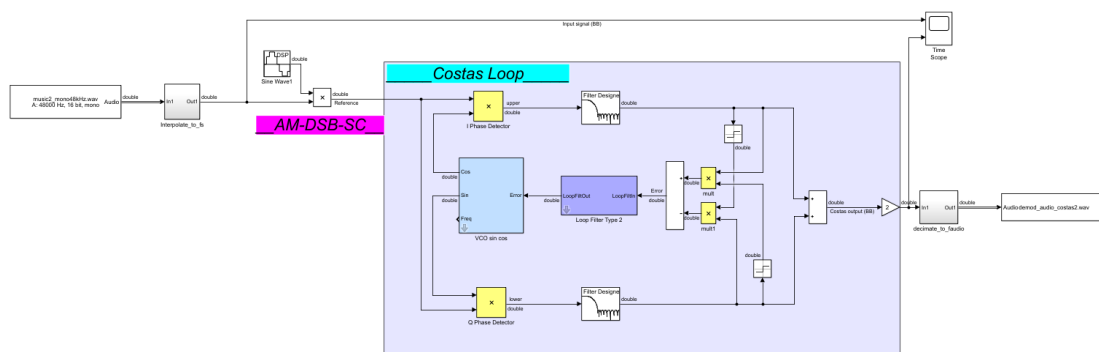


图 7-8 Costas 解调模型

从解调的模型中可以看出，图中灰色区域就是 costas 环的结构，用户可以和图 7-7 进行比较，观察在 Simulink 当中的这个模型和在图 7-7 当中的结构是否一致。

通过示波器来查看解调出来的信号和原始信号，如所示

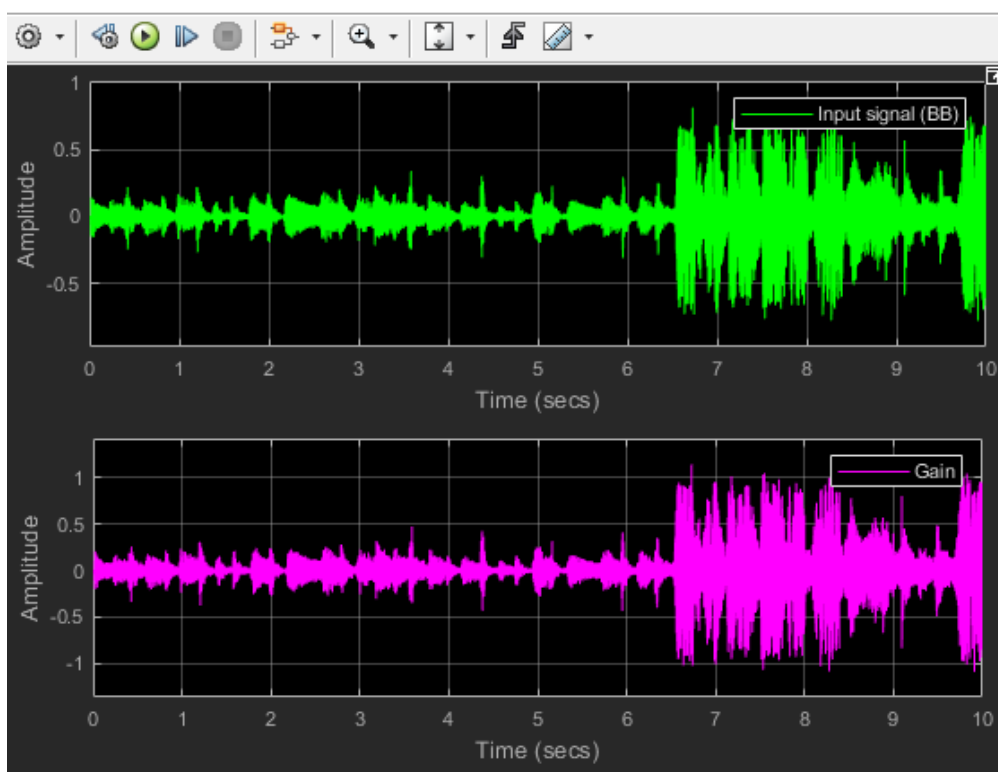


图 7-9 解调出来的信号

仿真结束之后，你将在当前目录下发现一个解调出来的音频，你可以使用第三方播放器来播放这个音乐。

7.5 ANTSDR 进行 AM-DSB-SC 调制与解调

7.5.1 使用 Costas 环进行解调

在前面介绍了 AM-DSB-SC 的调制和解调方法,接下来,我们就会基于 ANTSDR 来进行真实的信号的发送和接收了。对于发送而言,使用抑制载波的双边带调幅比常规调幅会更加简单,但是在解调的时候就会稍微麻烦一点。

打开仿真模型 `antsdr_am_dsb_sc_costas_demod.slx`, 本次实验的仿真模型如图 7-10 所示:

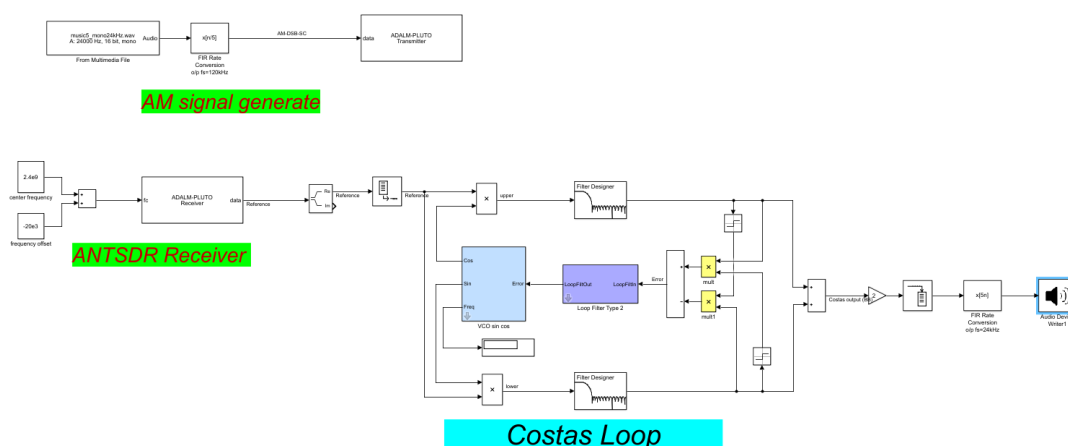


图 7-10 AM-DSB-SC 的调制与解调

从仿真模型可以看到,发射机的结构十分的简单,只需要将基带的信号进行上采样到基带信号的采样频率就可以了。然后使用 Pluto 的发射模块就能将信号发送到信道中。

而在接收机当中,需要的模块就相对来说多了很多。其中最重要的模块就是 Costas 环模块,这个模块在前面的小节当中就已经对其进行了仿真,在这里不再具体介绍它的具体功能,最终解调出来的信号经过下采样到 24KHz,然后经过音频播放器输出。

值得注意的是,本次实验所使用的音频信号源为 24KHz 的信号,这样经过上变频后能够达到 120KHz。笔者在完成本章实验的时候,发现如果使用音频信号源为 48KHz 的信号,那么进行 5 倍插值到 240KHz 的时候,不能够有效的进行处理这个应该和笔者的电脑性能有关。将采样速率降低一半之后,就没有这个问题了。

运行这个仿真模型,你应该就能听到播放出来的声音了。

7.5.2 直接得到基带信号

由于使用 AM-DSB-SC 信号, 直接将基带信号进行调制的, 由于我们的 ANTSDR 使用的 ad936x 系列芯片是一个零中频的芯片, 因此可以直接得到基带的信号, 在下面这个例子当中, 我们可以看到, Pluto 接收机模块接收到的信号直接进行播放, 也是能够听到音乐信号的。

打开仿真模型 `antsdsr_am_dsb_sc_mod_demod.slx`, 本次实验的仿真模型如下:

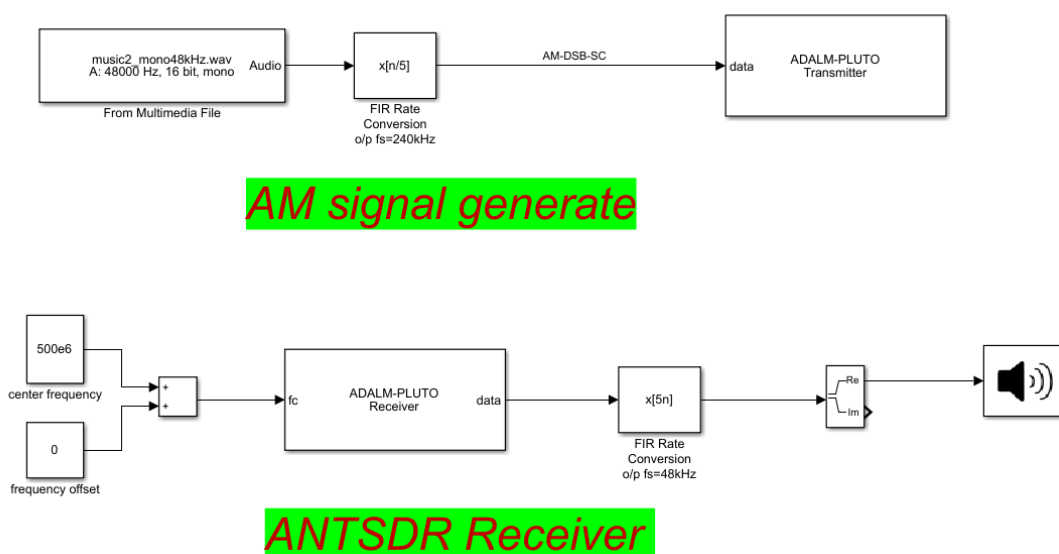


图 7-11 使用 ANTSDR 直接得到基带信号

可以看到本次实验的仿真模型特别简单, 将接收机和发送机设置到同一个频率下面, 就能够直接得到基带的信号了。通过播放器就能够直接将音频信号播放出来了。

7.6 总结

在本章当中, 我们学习了 AM-DSB-SC 调制, 也就是抑制载波的双边带调幅。介绍了其调制的原理, 并介绍了通过相干解调的方式对这样的已调信号进行解调。在相干解调的时候, 介绍了 Costas 环, 并给出了 Costas 环的仿真模型, 直观地感受了一下 Costas 环是如何来实现相干解调的。

Costas 环在通信系统中经常会使用到, 其结构和 PLL 类似, 也主要由三个部分构成, 分别是鉴相器, 环路滤波器和 VCO/NCO, 但和 PLL 在内部还是由结构上的不同。使用 Costas 环的计算量和占用的资源相较于 PLL 会更多。

最终我们通过 ANTSDR 来实现了发送 AM-DSB-SC 信号，并且在 Simulink 里面成功地将其进行了解调，并最终通过音频播放器将其播放出来。

第8章 实现 AM 的多路复用

在前面的实验当中，我们已经实现了 AM 的常规调幅和抑制载波的双边带调幅，在这一章当中，我们会实现 AM 的多路复用，也就是使用同样的频率承载多个信号，将基带信号按照不同的频率进行划分，每个信号占据不同的频带，依次来实现信号的多路复用。

8.1 音频信号 AM 复用

FDM（频分复用）是一种能够提升频谱利用率的一个简单的方法。FDM 能够使多个信号通过同一个线缆或者信道进行发送，把信道的带宽分为多个子信道。MPX 信号的产生过程比较简单。如图 8-1 所示：

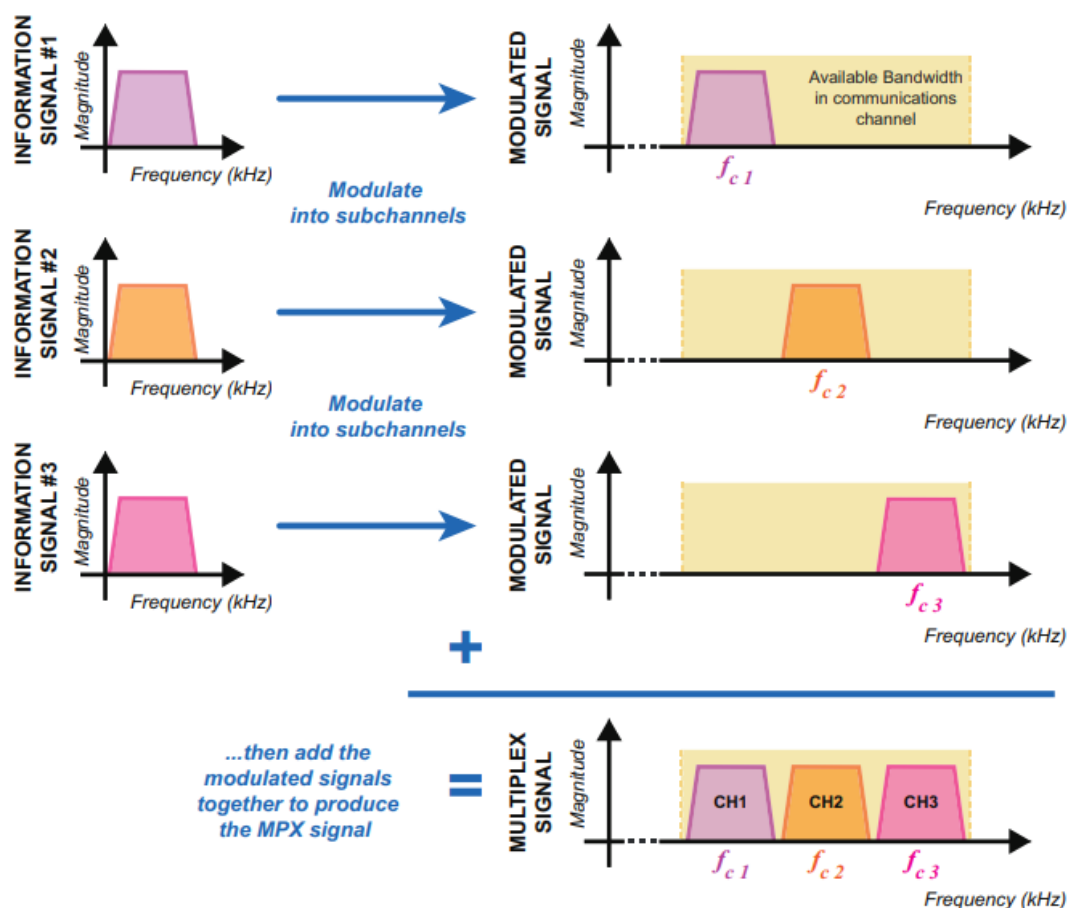


图 8-1 FDM 信号产生过程

多余几个基带信号，先分别用较低的频率进行调制，使其分布在基带上的不同频段上，然后将这几个经过调制过后的基带信号进行累加。累计之后组成一个

新的基带信号，这个信号的带宽是前面三个信号带宽的和。然后将这个组合的 MPX 信号使用高频载波进行调制，发送到信道之中。

举一个简单的例子，首先我们定义一个基带通信的信道带宽为 15KHz，使用 AM-DSB-TC 来调制两路基带信号，这两路基带信号的带宽为 5KHz。其中一个信号留在基带，其频带宽度为 5KHz，另外一个基带信号使用频率 $f_c = 10\text{KHz}$ 的载波先进行调制，那么这个基带信号经过调制之后所占的带宽是 5~15KHz，这个过程如所示：

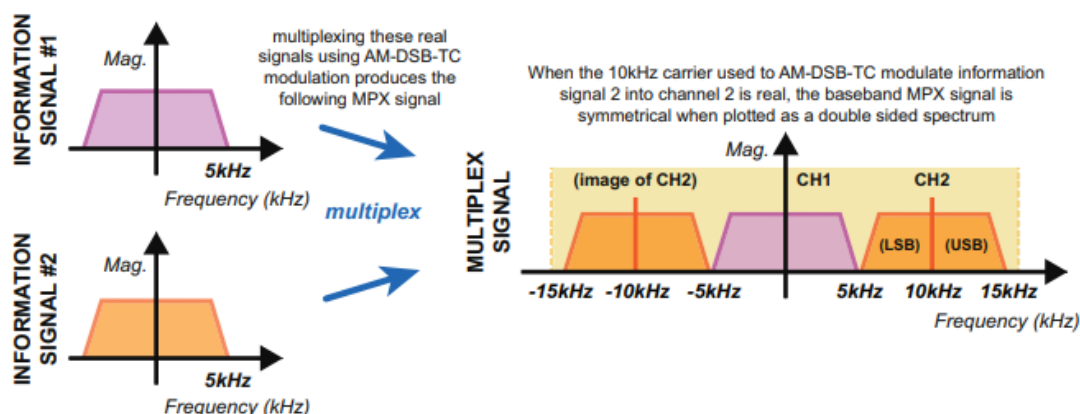


图 8-2 组成 MPX 信号的过程

可以看到，第二个基带信号经过调制后其带宽变为了原来的两倍，这是因为采用的调制方式是 AM-DSB-TC，在这里，我们采用的调制载波是实载波，所以我们观察调制出来的频谱可以看到，在负频率部分会有着两个基带信号的镜像。

在前面我们介绍复数信号的时候，可以知道使用实数载波进行调制的时候，相当于只取了复数载波调制的实数部分，因此会产生一个镜像，如果我们采用频率为 $f_c = 10\text{KHz}$ 的复载波来进行调制的话，那么就没有位于负频率部分的这个镜像。

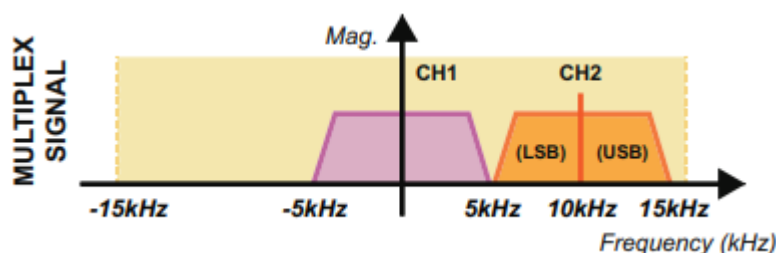


图 8-3 两路信号的复调制

使用复载波，我们甚至可以将第三个信号调制到基带信号当中，如果我们使用一个频率为 $f_c = -10\text{KHz}$ 的复载波来调制第三个信号，那么它将会被搬移到频率为 -10KHz 的位置。

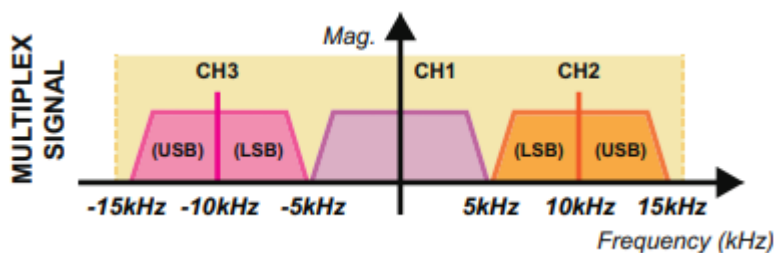


图 8-4 三路信号的复调制

由于我们的 ANTSDR 发送和接收的都是复数基带信号，因此是可以来实现上面的这种三路信号的复数调制的发送和接收的。

8.2 AM FDM 仿真

下面将会通过一个简单的仿真模型来介绍如何使用来调制多路 AM 信号。打开仿真模型 `am_fdm.slx`，仿真模型如图 8-5 所示：

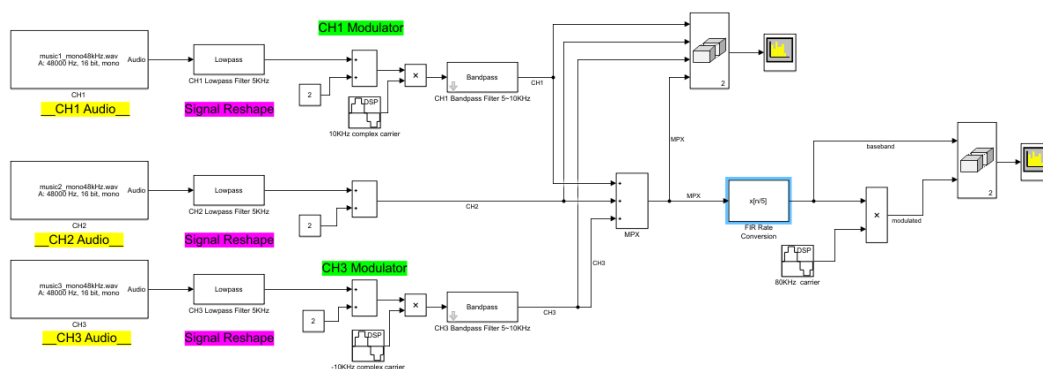


图 8-5 AM FDM 仿真模型

仿真模型使用到了三个通道的音频信号，首先经过一个低通滤波器，用来限制这三个基带信号的带宽为 5KHz。

然后使用复数载波来对 CH1 和 CH3 的信号进行复数调制，需要注意的是，这里的调制载波是复数载波。

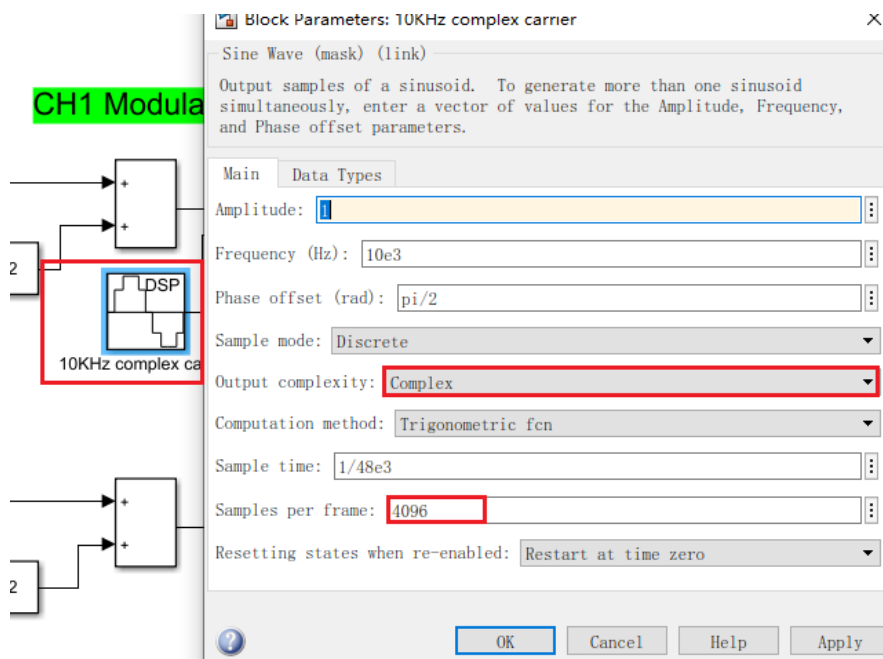


图 8-6 基带调制载波为复数载波

经过调制之后对 CH1 和 CH3 的信号进行带通滤波滤除带外噪声,然后将 CH1, CH2, CH3 通道的信号进行累加得到 MPX 信号。通过示波器,我们能够发现当前各个信号所在的频带。

首先我们来看各个信号所占用的频带,如图 8-7 所示:

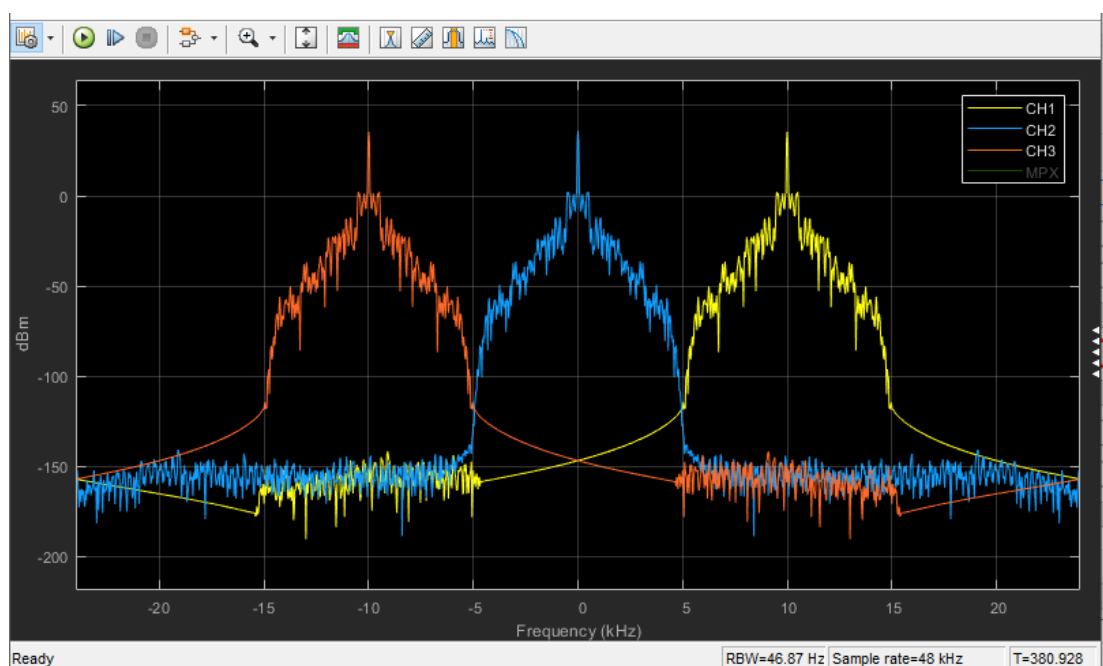


图 8-7 各个信号所在的频段

然后我们再观察累加的之后的信号，也就是 MPX 信号

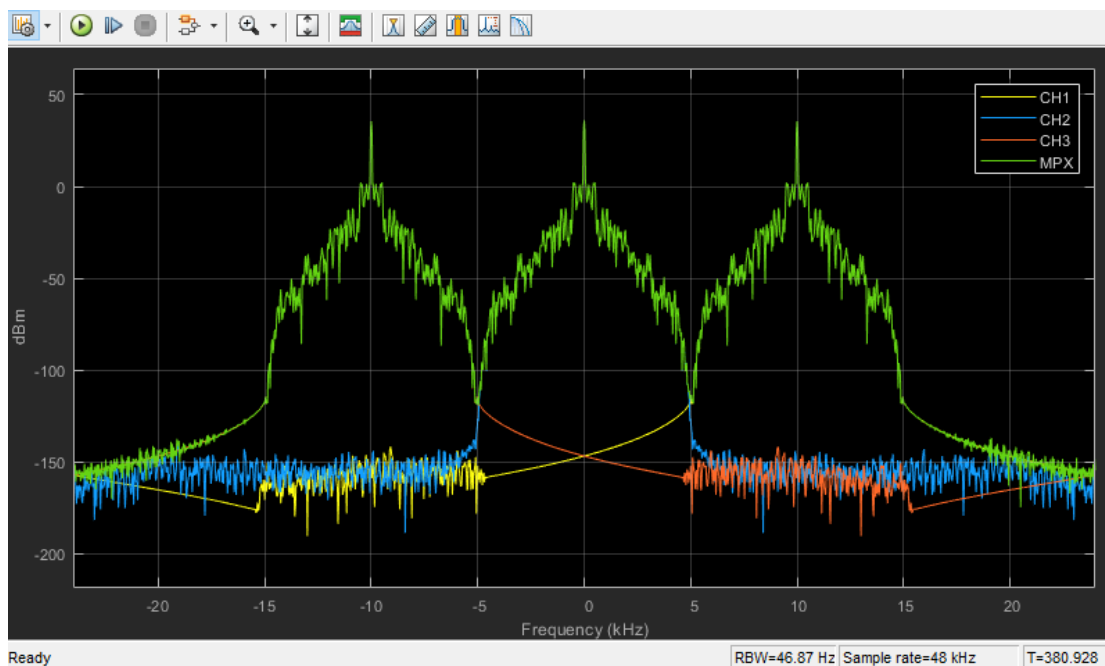


图 8-8 可以看到累加信号是这三个通道的信号的和

最终这个 MPX 信号经过了一个高频载波进行了调制，可以看到调制过后的信号频谱如下图所示，基带信号已经成功地被搬移到了高频载波附近。

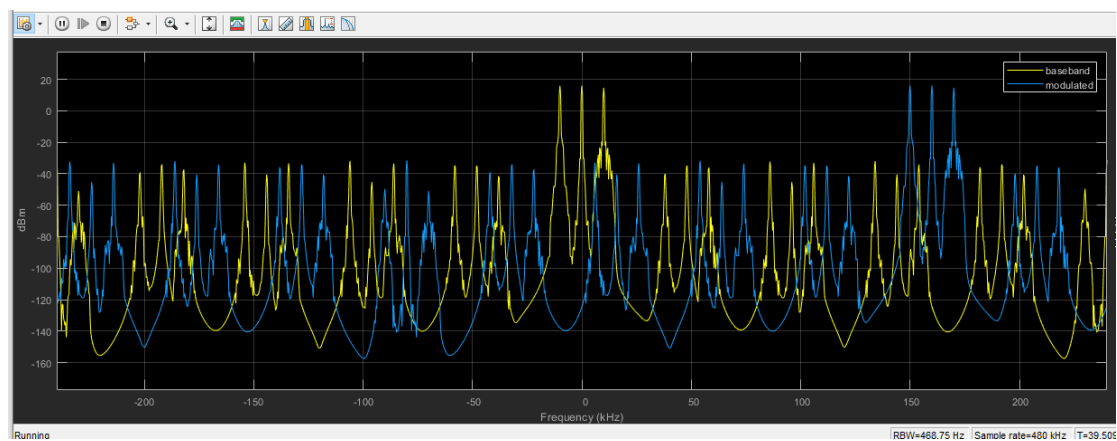


图 8-9 经过调制过后的 MPX 信号

8.3 音频信号 AM 解复用

音频信号的解复用的过程比较简单，在接收机当中，首先使用一个较低的中心频率对信号进行下变频，将信号搬移到基带附近，然后再使用带通滤波器将各个频带上的信号分别取出，最后采用包络检波就能对各个频带上的音频信号进行

解调了。如图 8-10 所示

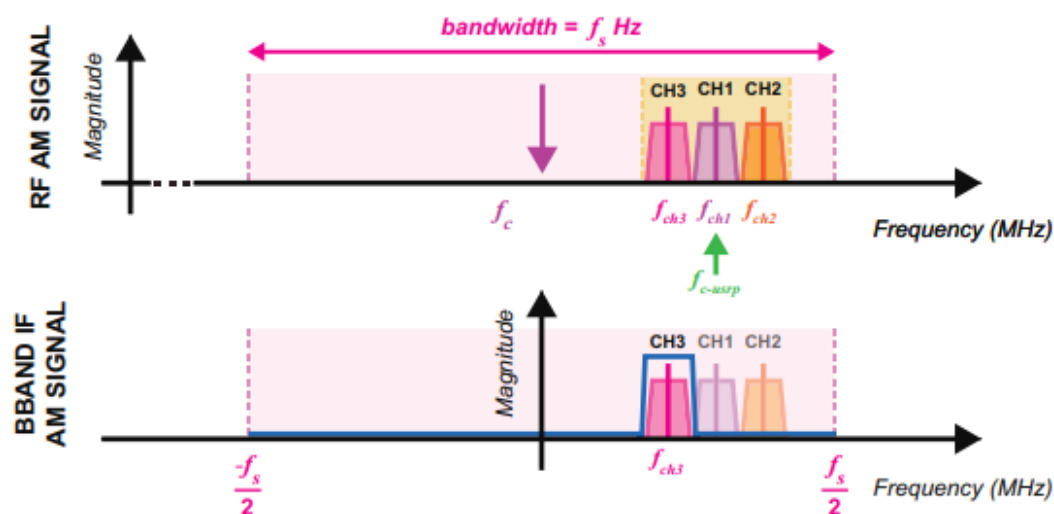


图 8-10 FDM 解调过程

8.4 音频信号 AM 解复用

接下来我们来设计设计一个仿真模型，完成 AM 的解复用的实验，将各个频带的音频信号进行解调。

打开仿真模型，`am_fdm_demod.slx`，如所示：

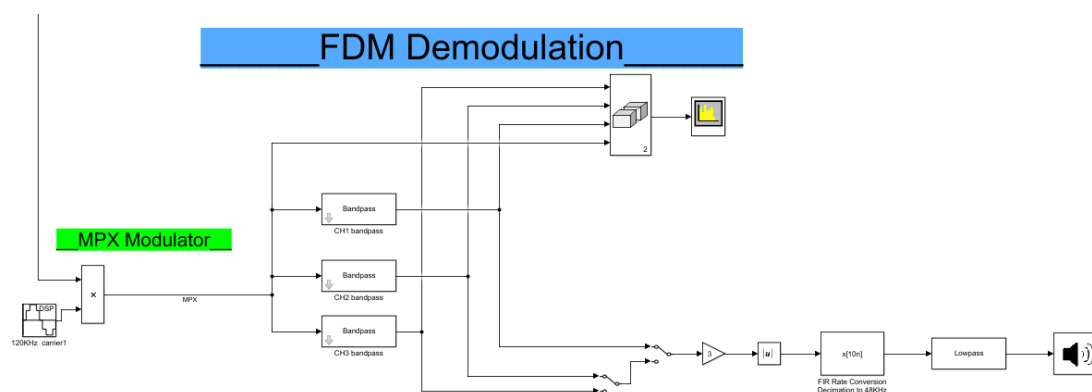


图 8-11 AM FDM 解调

如上图所示，每个在对已调信号进行下变频之后，使用一个带通滤波器来获取每个频带上的数据，最后通过包络检波的方式将信号提取出来，点击开始仿真，你应该可以听到音乐，如双击开关，可以切换到不同的频带上，这样你就可以来收听不同的音频了。

我们还是来观察一下，进行下变频之后的各个信号的频谱。

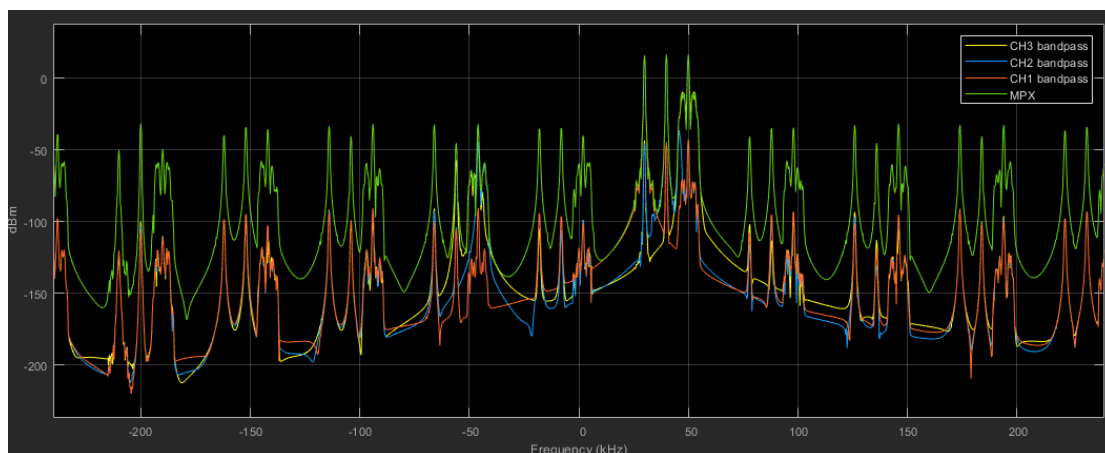


图 8-12 解调模块当中各个信号的频谱

可以看到，经过下变频之后，绿色的信号是 MPX 信号，已经被成功地搬移到 40kHz 附近。这时候，我们再来观察带通滤波器滤除各个频带的信号。

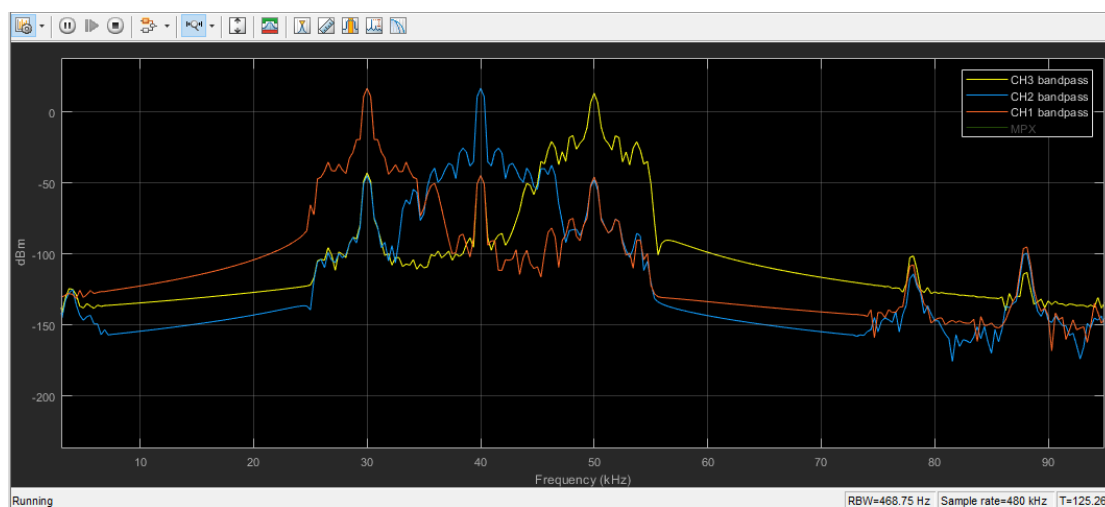


图 8-13 各个带通滤波器滤波之后的结果

可以看到经过带通滤波器之后，各个滤波器的输出，对应的就是每个频带上的信号。所以我们能够通过使用滤波器将各个频带进行区分，从而实现 FDM 信号的解调。

8.5 使用 ANTSDR 完成 AM FDM 的调制与解调

在前面的内容当中，我们介绍了了 AM FDM 的原理，并且给出了仿真模型，通过仿真模型，进一步认识到了 AM FDM 的实现的原理。并且最终能够解调出不同频带上的信号。

接下来，我们会来玩一点真实的信道，会在 ANTSDR 上，来完成前面我们实现的仿真，从而实现向真实信道中发送 AM FDM 信号和解调 AM FDM 信号。

打开仿真模型 `antsdr_am_fdm.slx`，其中 FDM 发送模块如图 8-14 所示：

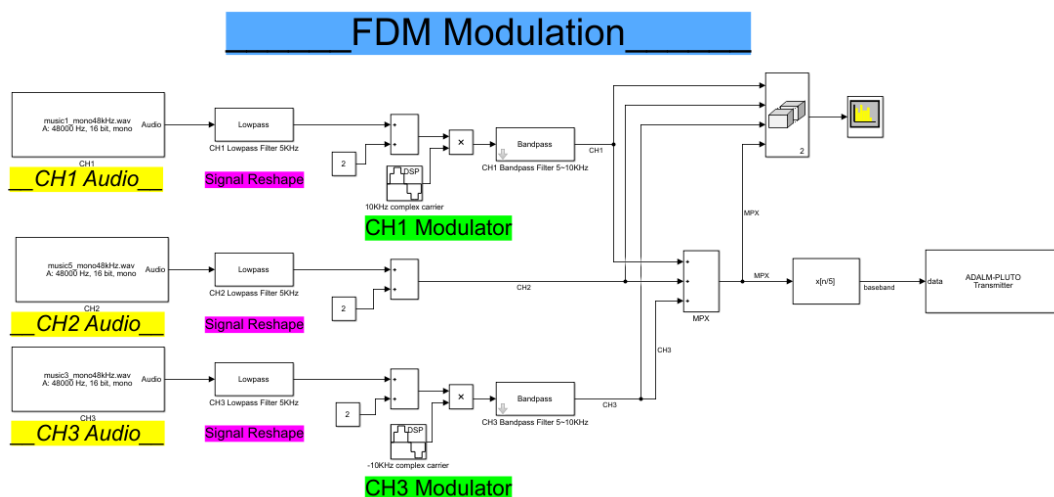


图 8-14 AM FDM 发送模块

解调模块如图 8-15 所示：

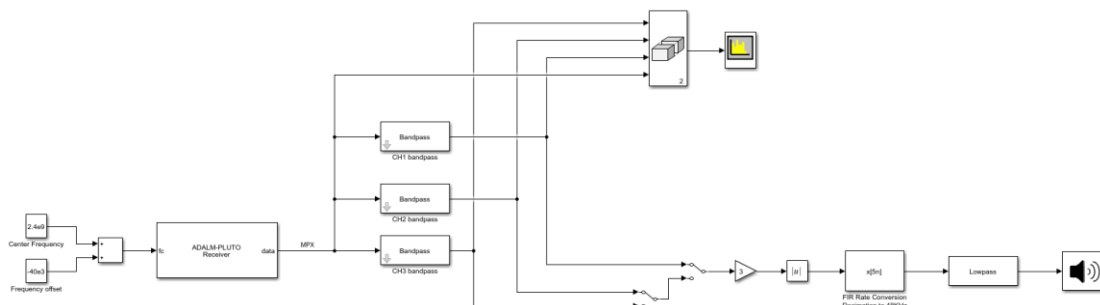


图 8-15 AM FDM 解调模块

点击运行仿真，我们来查看一下在真实的信道当中接收到的信号是什么样子的，如图 8-16 所示：通过 ANTSDR 接收到的信号在图中是绿色标识的，可以看到在经过下变频之后在 40KHz 附近有一个明显的信号。

接下来会让这个信号通过带通滤波器，滤波之后的各个新号如所示：

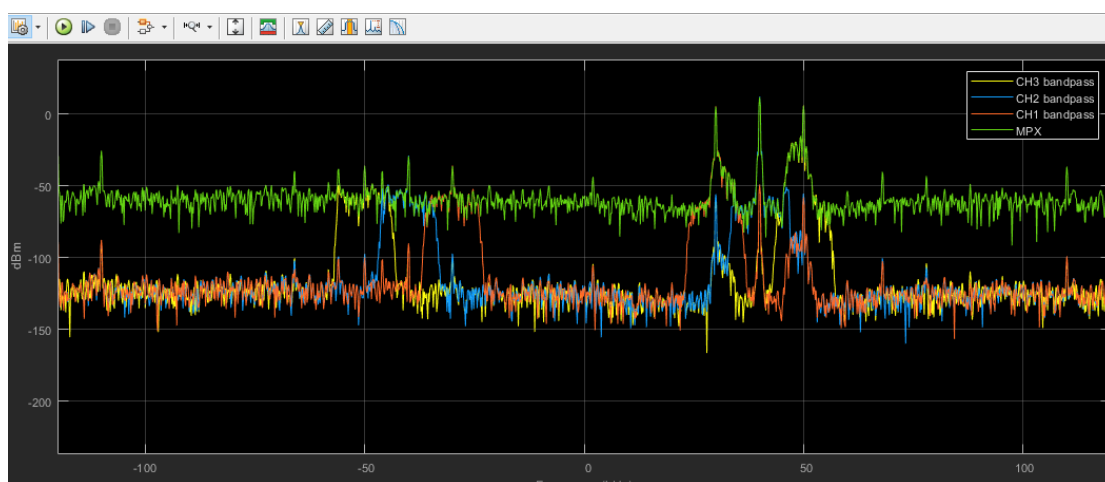


图 8-16 接收到的 FDM 信号

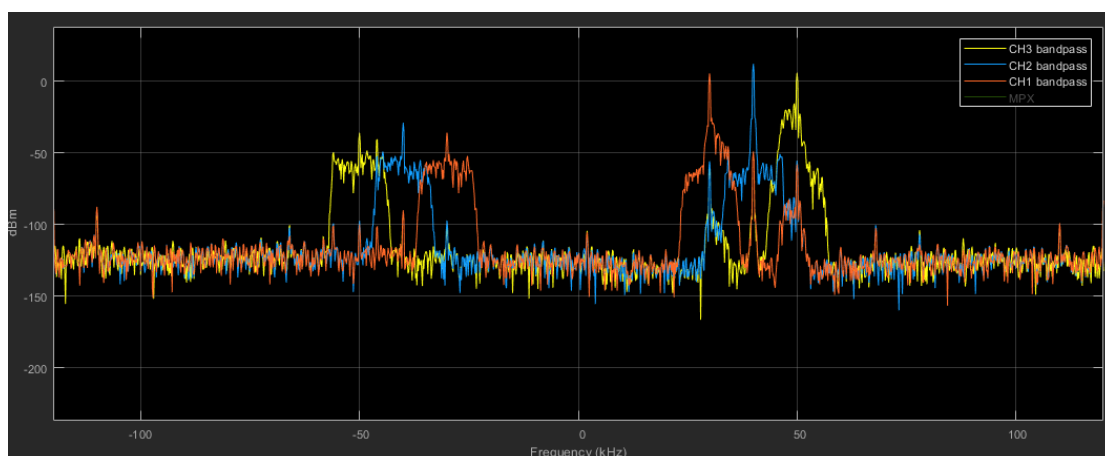


图 8-17 经过带通滤波之后的结果

可以看到经过带通滤波器之后，各个频带上的音频信号已经被分割开。因此 FDM 进行解调才能够生效。

最终，你能够通过调整开关，来收听不同的音频信号。

8.6 总结

在本章当中，以 AM 的频分复用介绍了简单的复用的概念，通过复用，在进行通信的时候，能够使用单个高频载波来承载更多的信息，从而提升通信的效率。

在本章当中通过仿真和真实的信道当中的 FDM 的调制和解调，直观地演示了在实际当中该如何来生成 FDM 信号和解调 FDM 信号。

第9章 FM 调制与解调（微分器）

在前面的实验当中，我们介绍了有关 AM 的常见的调制解调方式，与此同时，介绍了 AM 的相干解调和非相干解调方法。在相干解调当中，重点介绍了两种模型，一个使锁相环 PLL，另外一个 Costas 环。

在本章中，我们会介绍一种新的模拟调制方法，也就是频率调制 FM，我们会首先介绍 FM 调制与解调的理论，然后通过仿真来直观地展示如何从理论到模型的建立，最终我们还能实现接收现实中真实的 FM 广播。

9.1 FM 调制的数学原理

9.1.1 FM 信号产生的过程

FM 就是频率调制，就是使用载波的频率来承载消息信号，当消息信号的幅值较大的时候，产生的频率比较大，当消息信号较小时，产生的频率比较小。在我们之前介绍过的 VCO 这个结构，就是一个最简单的 FM 的调制器。

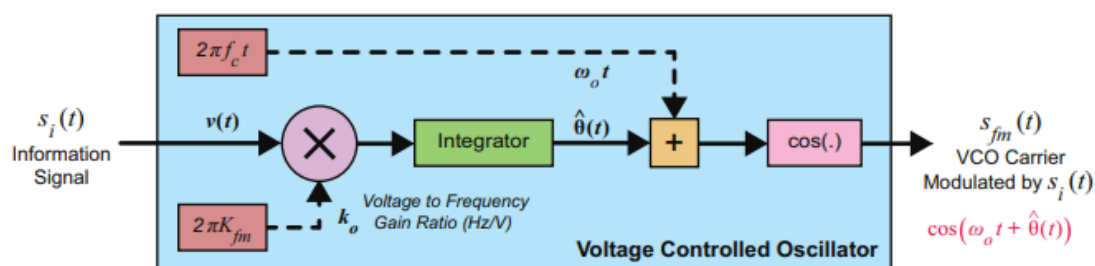


图 9-1 VCO 结构

控制信号进入了 VCO 之后，首先需要乘上一个增益，这个增益表示电压（消息信号的幅值）到频率（FM 调制的结果）的增益值， $v(t)$ 与 k_o 的乘积会经过一个积分器。积分之后的信号为：

$$\hat{\theta}(t) = k_o \int_{-\infty}^t v(t) dt$$

由 VCO 产生的正弦波会有一个初始的振幅 A_0 和频率 f_0 ，正弦波的相位取决于前面这个积分信号的瞬时值。将这个值带入到产生的正弦波当中。

$$\begin{aligned} c(t) &= A_o \cos\left(2\pi f_o t + \hat{\theta}(t)\right) \\ &= A_o \cos\left(\omega_o t + k_o \int_{-\infty}^t v(t) dt\right) \end{aligned}$$

其中这个 k_o 可以表示为 $k_o = 2\pi K_{fm}$,因此 VCO 得到的输出信号为:

$$s_{fm}(t) = A_c \cos\left(\omega_c t + \theta_{fm}(t)\right) = A_c \cos\left(\omega_c t + 2\pi K_{fm} \times \int_{-\infty}^t s_i(t) dt\right)$$

VCO 输出信号的频率就是对相位的微分, 可以表示为:

$$f_{fm\ inst}(t) = f_c + K_{fm} s_i(t) \text{ Hz}$$

可以看到使用 FM 调频之后, 已调信号将会包含很多的频率分量。

9.1.2 FM 调制一个单音信号

和前面 AM 调制类似, 我们首先来看使用 FM 调制一个单音信号, 并且观察其时域和频域波形, 从而对 FM 调制有一个更加直观地认识。

首先定义一个消息信号振幅为 A_i 频率为 f_i , 则这个消息信号的表示为:

$$s_i(t) = A_i \cos(2\pi f_i t) = A_i \cos(\omega_i t)$$

由此我们可以算出调制后的信号的瞬时相位, 就是使用这个信号与增益系数相乘然后进行积分:

$$\begin{aligned} \theta_{fm}(t) &= 2\pi K_{fm} A_i \times \int_{-\infty}^t \cos(\omega_i t) dt \\ &= 2\pi K_{fm} A_i \times \frac{\sin(\omega_i t)}{\omega_i} \\ &= \frac{K_{fm} A_i}{f_i} \sin(\omega_i t) \\ &= \frac{\Delta f}{f_i} \sin(\omega_i t) \\ &= \beta_{fm} \sin(\omega_i t) \end{aligned}$$

在上面的这个式子中 Δf 叫做频率变化量, β_{fm} 叫做 FM 的调制系数。频率变化量增益系数和当前信号的振幅的乘积, 在上面这个消息信号是一个单音信号例子中, Δf 也就是频率分辨率, 调制系数就是频率变化量和这个消息信号频率的比值。

因此一个单音信号经过调制之后就可以使用如下方式来进行表示:

$$s_{fm}(t) = A_c \cos(\omega_c t + \beta_{fm} \sin(\omega_i t))$$

$$\text{where: } \omega_c = 2\pi f_c \text{ and } \omega_i = 2\pi f_i$$

9.1.3 FM 调制一个基带信号

基带信号一般都是由很多的频率分量所组成的，相较于单音信号，要考虑的东西更多。在本章中，我们还是采用和前面章节当中类似的例子，用三个单音信号来表示一个基带信号，然后采用 FM 的方式来调制这一个基带信号。

假设有三个频率不同的单音信号组成的基带信号，其表示如下：

$$\begin{aligned} s_i(t) &= A_{i1} \cos(2\pi f_{i1} t) + A_{i2} \cos(2\pi f_{i2} t) + A_{i3} \cos(2\pi f_{i3} t) \\ &= A_{i1} \cos(\omega_{i1} t) + A_{i2} \cos(\omega_{i2} t) + A_{i3} \cos(\omega_{i3} t), \end{aligned}$$

$$\text{where: } \omega_i = 2\pi f_i.$$

采用 FM 进行调制，有了前面对单音信号的调制过程的介绍，我们很容易得到这个基带信号进入 VCO 产生的瞬时角度如下：

$$\begin{aligned} \theta_{fm}(t) &= 2\pi K_{fm} \times \int_{-\infty}^t \left[A_{i1} \cos(\omega_{i1} t) + A_{i2} \cos(\omega_{i2} t) + A_{i3} \cos(\omega_{i3} t) \right] dt \\ &= 2\pi K_{fm} \left[A_{i1} \int_{-\infty}^t \cos(\omega_{i1} t) dt + A_{i2} \int_{-\infty}^t \cos(\omega_{i2} t) dt + A_{i3} \int_{-\infty}^t \cos(\omega_{i3} t) dt \right] \\ &= \frac{K_{fm} A_{i1}}{f_{i1}} \sin(\omega_{i1} t) + \frac{K_{fm} A_{i2}}{f_{i2}} \sin(\omega_{i2} t) + \frac{K_{fm} A_{i3}}{f_{i3}} \sin(\omega_{i3} t). \end{aligned}$$

最终产生的一条信号的表达式如下：

$$s_{fm}(t) = A_c \cos \left(\omega_c t + \left[\frac{K_{fm} A_{i1}}{f_{i1}} \sin(\omega_{i1} t) + \frac{K_{fm} A_{i2}}{f_{i2}} \sin(\omega_{i2} t) + \frac{K_{fm} A_{i3}}{f_{i3}} \sin(\omega_{i3} t) \right] \right)$$

可以看到这个信号包含了很多的频率分量，但是现在还不能很直观地表示出来，在之后的仿真模型中，我们会直观地来认识这些频率分量，并且能够计算出已调信号的带宽。

9.1.4 宽带调频与窄带调频

在前面的 FM 调制单音信号和 FM 调制基带信号的结果中，不同于 AM 调制，FM 调制很难看出已调信号具有哪些频率分量，占用多少带宽。其实 FM 的带宽也是可以确定的，其中有窄带调频和宽带调频的区别。窄带调频一般用于业余无

线电，宽带调频一般用于商用广播。

FM 究竟是窄带调频还是宽带调频取决于调制系数 β_{fm} 。

9.1.4.1 窄带调频

窄带调频中，调制系数 β_{fm} 的值要远小于 1，我们再回顾一下前面单音信号的 FM 已调信号的表达式：

$$s_{fm}(t) = A_c \cos(\omega_c t + \beta_{fm} \sin(\omega_i t))$$

$$\text{where: } \omega_c = 2\pi f_c \text{ and } \omega_i = 2\pi f_i$$

在这里使用和差化积公式将这个式子展开：

$$\begin{aligned} s_{fm}(t) &= A_c \cos(\omega_c t + \beta_{fm} \sin(\omega_i t)) \\ &= A_c \cos(\omega_c t) \cos(\beta_{fm} \sin(\omega_i t)) - A_c \sin(\omega_c t) \sin(\beta_{fm} \sin(\omega_i t)) \end{aligned}$$

当 β_{fm} 很小的时候，我们可以对上面的式子进行一下近似：

$$\cos(\beta_{fm} \sin(\omega_i t)) \approx 1 \quad \text{and} \quad \sin(\beta_{fm} \sin(\omega_i t)) \approx \beta_{fm} \sin(\omega_i t)$$

于是我们可以得到

$$\begin{aligned} s_{fm-nfm}(t) &= A_c \cos(\omega_c t) - A_c \sin(\omega_c t) \beta_{fm} \sin(\omega_i t) \\ &= A_c \left[\cos(\omega_c t) + \frac{\beta_{fm}}{2} \cos(\omega_c + \omega_i)t - \frac{\beta_{fm}}{2} \cos(\omega_c - \omega_i)t \right] \end{aligned}$$

回忆一下之前在 AM-DSB-TC 调制的时候，进行调制之后的结果和上面这个窄带调频的结果。

$$s_{am-dsb-tc}(t) = A_c \left[A_o \cos(\omega_c t) + \frac{A_i}{2} \cos(\omega_c + \omega_i)t + \frac{A_i}{2} \cos(\omega_c - \omega_i)t \right]$$

可以看到，窄带调频的信号与 AM-DSB-TC 的频谱基本一致，只是下边带反向了，也就是说，下边带的信号和 AM-DSB-TC 相比有一个 180° 的相位差。

当我们调制的是基带信号而不是单音信号的时候，你可以得到同样的结果，只是整个下边带都反向了。

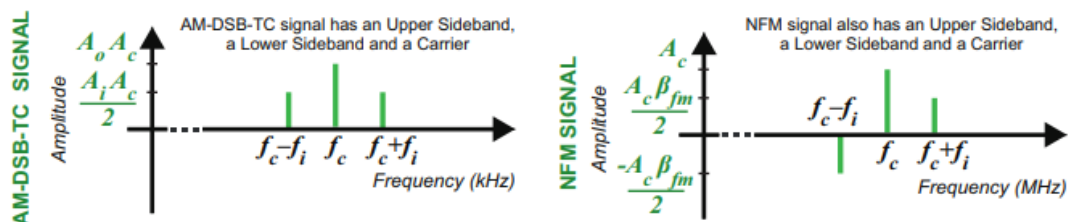


图 9-2 AM-DSB-TC 与 NFM

在使用频谱仪观察 NFM 的信号的时候，我们观察到的信号的频谱应该是幅度谱，所示观察到的结果应该是和 AM-DSB-TC 信号类似的。加上相位谱之后，才能正真的在频域中表示 NFM 信号。

9.1.4.2 NFM 仿真

接下来我们会通过在 Simulink 中搭建仿真模型，来实现对 NFM 的仿真。打开仿真模型，`fm_nfm.slx`，仿真模型如所示：

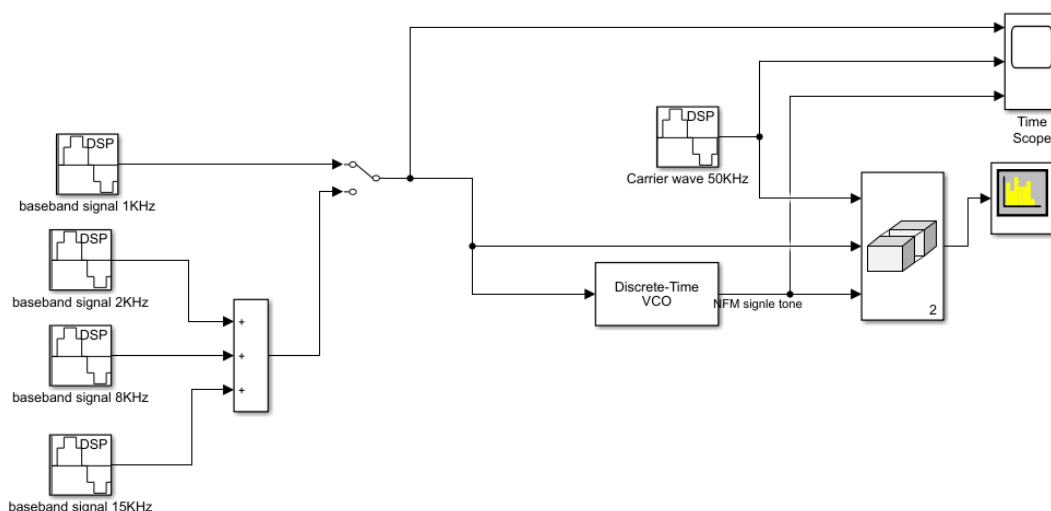


图 9-3 fm_nfm 仿真模型

开关可以选择基带信号输入的类型，输入的基带信号给到 VCO 进行 FM 的调制，VCO 的静态频率是 50KHz，增益为 $K_{fm} = 500$ ，然后我们可以通过示波器和频谱仪来观察调制后的信号。

首先我们会观察单音信号经过 NFM 调制之后的结果，根据前面的参数，我们可以知道调制系数

$$\beta_{max} = \frac{K_{fm} A_{max}}{f_i}$$

由于单音信号的频率为 1KHz，单音信号的振幅为 1，所以 β_{max} 的值小于 1，所以，该调制可以看作是窄带调制。

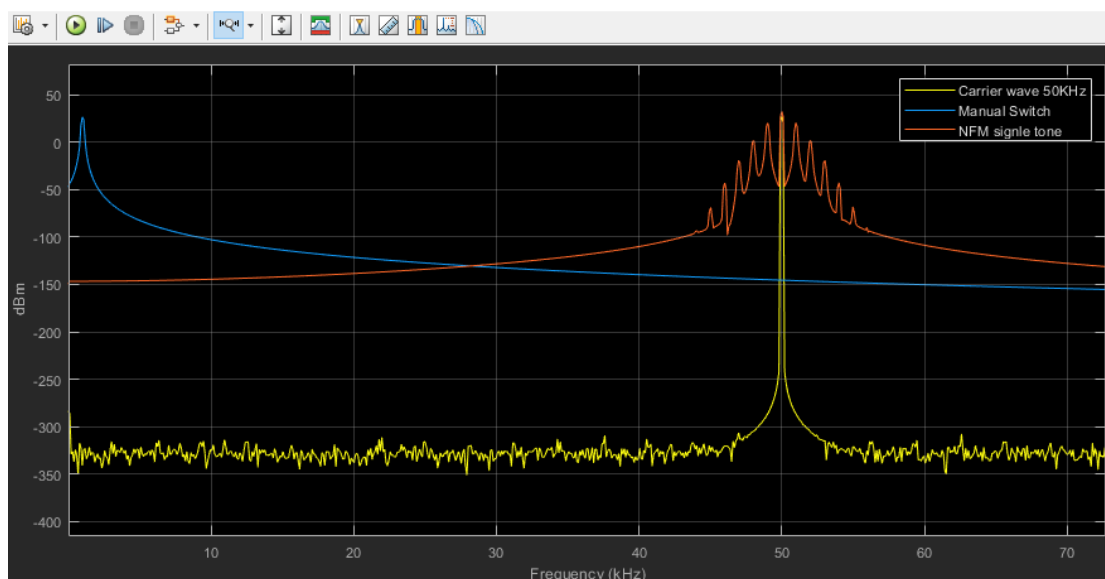


图 9-4 NFM 调制的频谱图

可以看到，使用了窄带 FM 调制之后，在载波附近生成了很多的频率分量，观察其中能量大于零的频率分量，可以看到，这部分的信号和 AM-DSB-TC 信号的频谱很相似。若这个 β_{max} 远小于 1 的话，可以看到这个 NFM 信号调制的单音信号和 AM-DSB-TC 调制单音信号的结果基本一致。

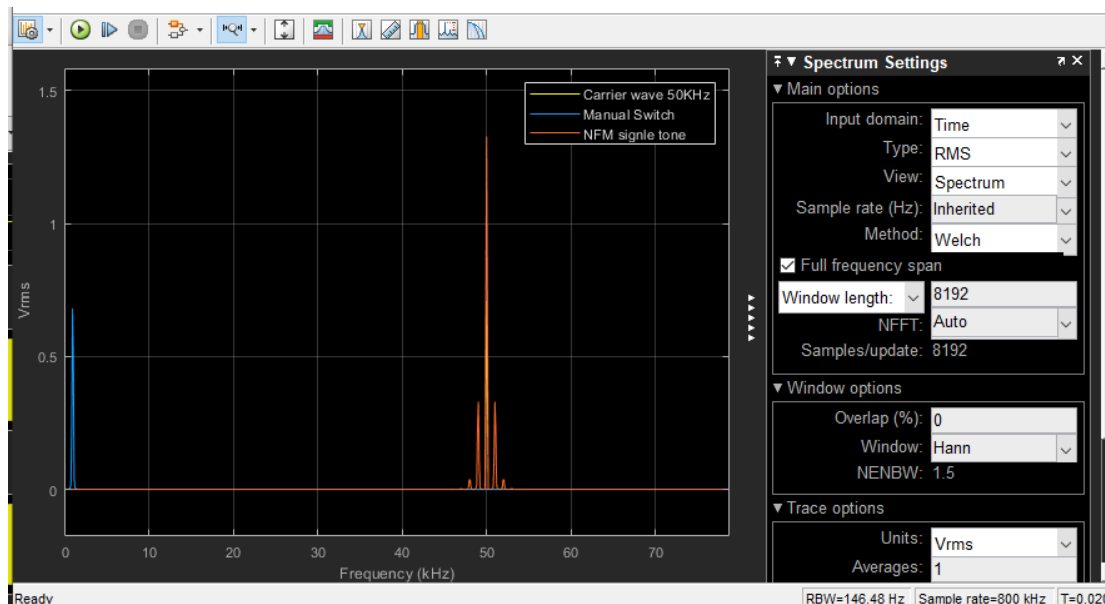


图 9-5

接下来我们再观察是使用一个基带信号输入之后的结果，将拨码开关拨动到基带信号处，然后运行仿真。

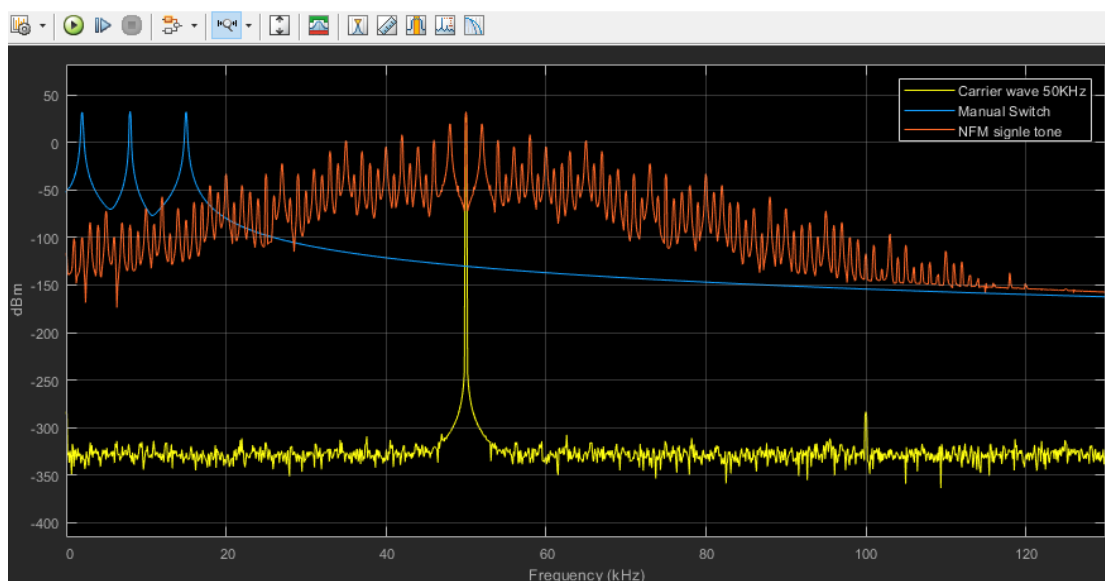


图 9-6 基带信号的 NFM 调制

可以看到使用基带信号进行 FM 调制的时候，会产生很多的频率分量，如果我们只关注大于零的频率分量，那么可以看到其调制的结果和 AM-DSB-TC 调制类似。

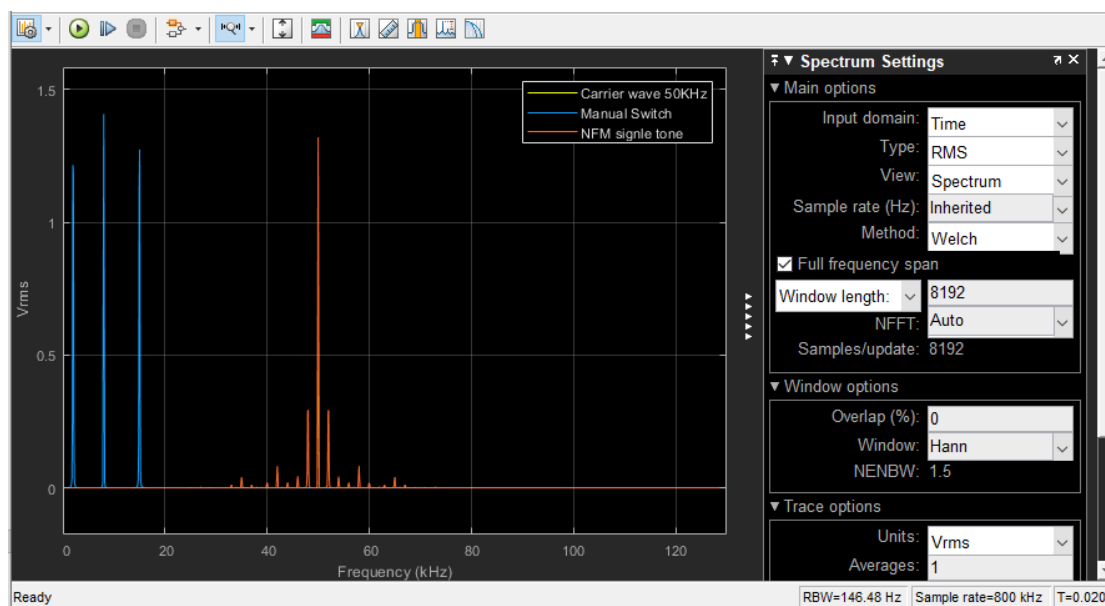


图 9-7

在 NFM 中，从时域波形上很难看出一条信号和调制载波的区别，我们可以查看一下示波器当中的波形，来观察一下时域的信号。

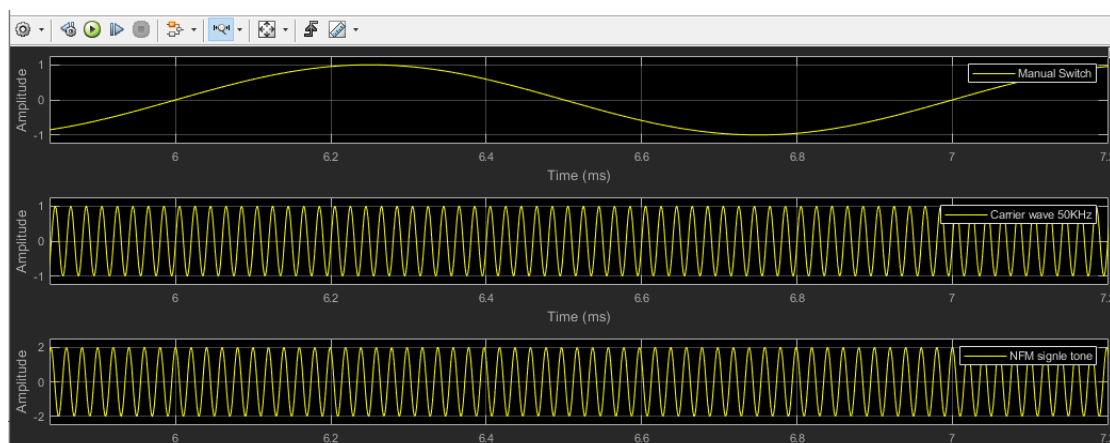


图 9-8 时域波形

9.1.4.3 宽带调频

绝大多数时候，所说的调频广播，都是指的宽带调频 WFM。WFM 是大多数商用电台所准寻的标准，其中最大频偏 Δf 是 75KHz，商用电台的带宽也会有限制，比如下图中，是使用 SDR#这个软件接收到的电台广播，可以看到每个电台广播都占有特定的频率。

在下图中所示的大约 2MHz 的频率范围内有 3 个电台，从瀑布图上可以看到每个电台大约占据的频谱的带宽。

在国内商用广播电台的频率一般是 88MHz~108MHz 之间。

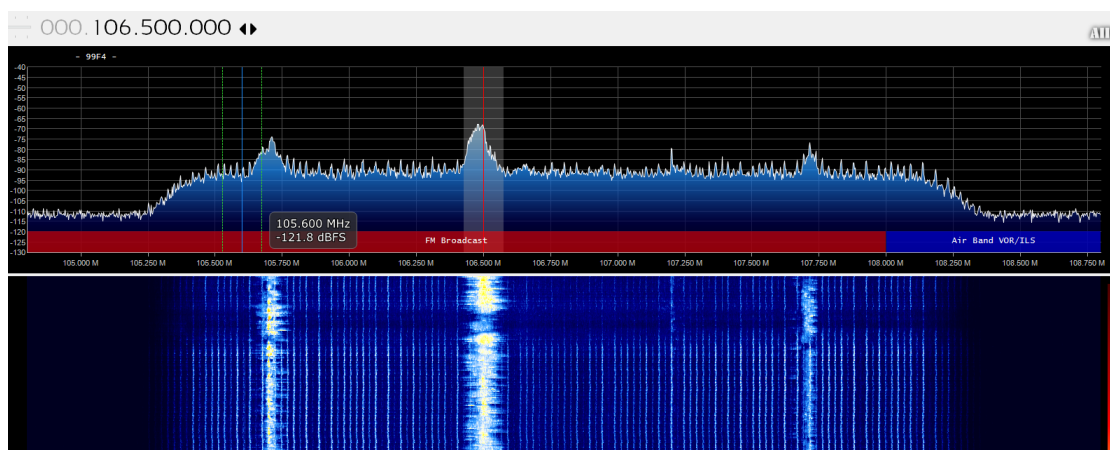


图 9-9 SDR# 接收到广播信号

如果信号的带宽不加限制，那么它在经过 WFM 调制之后将会在最终的载波频率附近产生无限多的频率分量，随着频率分量原理中心载波，其对应频率分量的信号的能量也会降低。

在前面我们介绍 NFM 的时候，使用和差化积公式，将已调信号进行了展开：

$$\begin{aligned}
 s_{fm}(t) &= A_c \cos(\omega_c t + \beta_{fm} \sin(\omega_i t)) \\
 &= A_c \cos(\omega_c t) \cos(\beta_{fm} \sin(\omega_i t)) - A_c \sin(\omega_c t) \sin(\beta_{fm} \sin(\omega_i t))
 \end{aligned}$$

对于含有信息信号的这两个分量，我们可以使用贝塞尔函数或者傅里叶级数将其展开：

$$\begin{aligned}
 \cos(\beta_{fm} \sin(\omega_i t)) &= J_0(\beta_{fm}) + 2 \sum_{n=1}^{\infty} J_{2n}(\beta_{fm}) \cos(2n\omega_i t) \\
 &= J_0(\beta_{fm}) + 2 J_2(\beta_{fm}) \cos(2\omega_i t) + 2 J_4(\beta_{fm}) \cos(4\omega_i t) + \dots \\
 &\equiv J_0 + 2 J_2 \cos(2\omega_i t) + 2 J_4 \cos(4\omega_i t) + \dots \\
 \sin(\beta_{fm} \sin(\omega_i t)) &= 2 \sum_{n=1}^{\infty} J_{2n-1}(\beta_{fm}) \sin((2n-1)\omega_i t) \\
 &= 2 J_1(\beta_{fm}) \sin(\omega_i t) + 2 J_3(\beta_{fm}) \sin(3\omega_i t) + 2 J_5(\beta_{fm}) \sin(5\omega_i t) + \dots \\
 &\equiv 2 J_1 \sin(\omega_i t) + 2 J_3 \sin(3\omega_i t) + 2 J_5 \sin(5\omega_i t) + \dots
 \end{aligned}$$

在上面的式子当中，我们 $J_n(\beta_{fm})$ 做了简化，使用 J_n 来代替，实际上这些数据都比较难以求得，我们在这里观察他们的规律。由傅里叶级数可以知道，频率越高的分量其前面的系数一般也越小。所以我们可以看到，随着频率的增加，其频率分量的大小也逐渐减小。

将上面的式子，带入到 FM 调制的表达式当中，就能够得到已调信号的表达式：

$$\begin{aligned}
 s_{fm-wfm}(t) &= A_c \cos(\omega_c t) [J_0 + 2 J_2 \cos(2\omega_i t) + 2 J_4 \cos(4\omega_i t) + \dots] \\
 &\quad - A_c \sin(\omega_c t) [2 J_1 \sin(\omega_i t) + 2 J_3 \sin(3\omega_i t) + 2 J_5 \sin(5\omega_i t) + \dots] \\
 s_{fm-wfm}(t) &= A_c J_0 \cos(\omega_c t) \\
 &\quad - A_c J_1 [\cos(\omega_c - \omega_i)t - \cos(\omega_c + \omega_i)t] \\
 &\quad + A_c J_2 [\cos(\omega_c - 2\omega_i)t + \cos(\omega_c + 2\omega_i)t] \\
 &\quad - A_c J_3 [\cos(\omega_c - 3\omega_i)t - \cos(\omega_c + 3\omega_i)t] \\
 &\quad + A_c J_4 [\cos(\omega_c - 4\omega_i)t + \cos(\omega_c + 4\omega_i)t] + \dots
 \end{aligned}$$

可以看到，最终一条信号的结果是在围绕着载波中心频率的地方，每隔基带信号的频率依次递减。

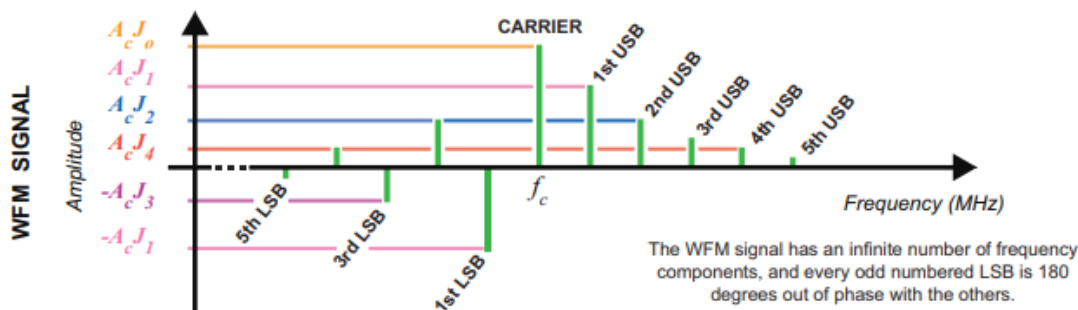


图 9-10 WFM 调制单音信号

对于 WFM 已调信号的带宽，可以根据其中能量比较显著的几个频率分量来确定，绝大部分的信息都承载在能量比较明显的这一区域。通常来说 WFM 已调信号的带宽可以通过卡森公式进行计算。

$$B = 2(\beta_{fm} + 1) f_i$$

当 WFM 调制的不是单音信号而是一个基带信号的时候，这个时候的频率可以用基带信号当中最大的频率分量 f_h 来表示。

9.1.4.4 WFM 仿真

接下来，我们在 Simulink 里面来建立仿真模型进行 WFM 的仿真。宽带 FM 和窄带 FM 的仿真模型基本一致，区别在于调制系数的不同，WFM 的调制系数要远远大于 1。调制系数的公式如下：

$$\beta_{max} = \frac{K_{fm} A_{max}}{f_i}$$

从调制系数的公式可以知道，改变调制系数，可以主要从 VCO 的电压频率转换增益和输入信号的振幅和频率进行入手，常用的更改 VCO 的增益就能够使之成为 WFM。仿真模型如图 9-11 所示：

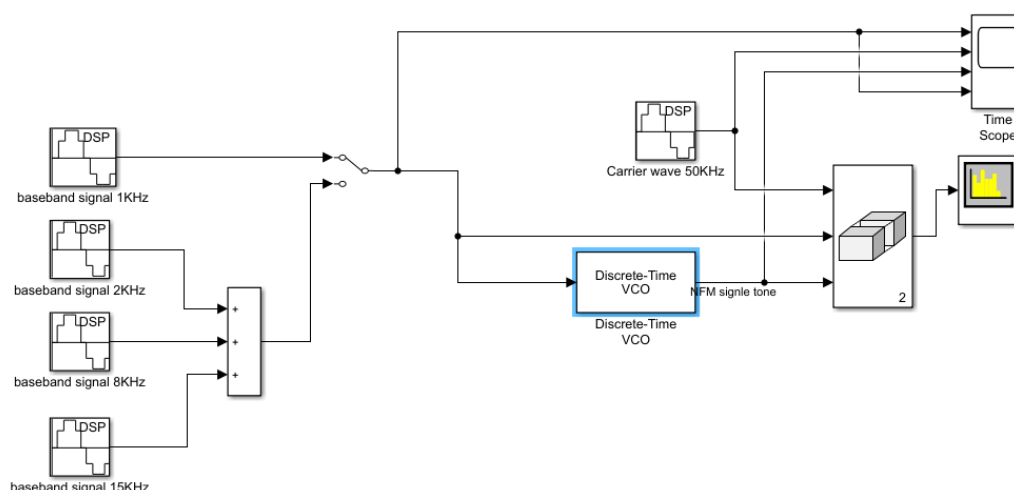


图 9-11 WFM 仿真模型

我们首先观察单音信号的调制，点击仿真，观察时域和频域波形：

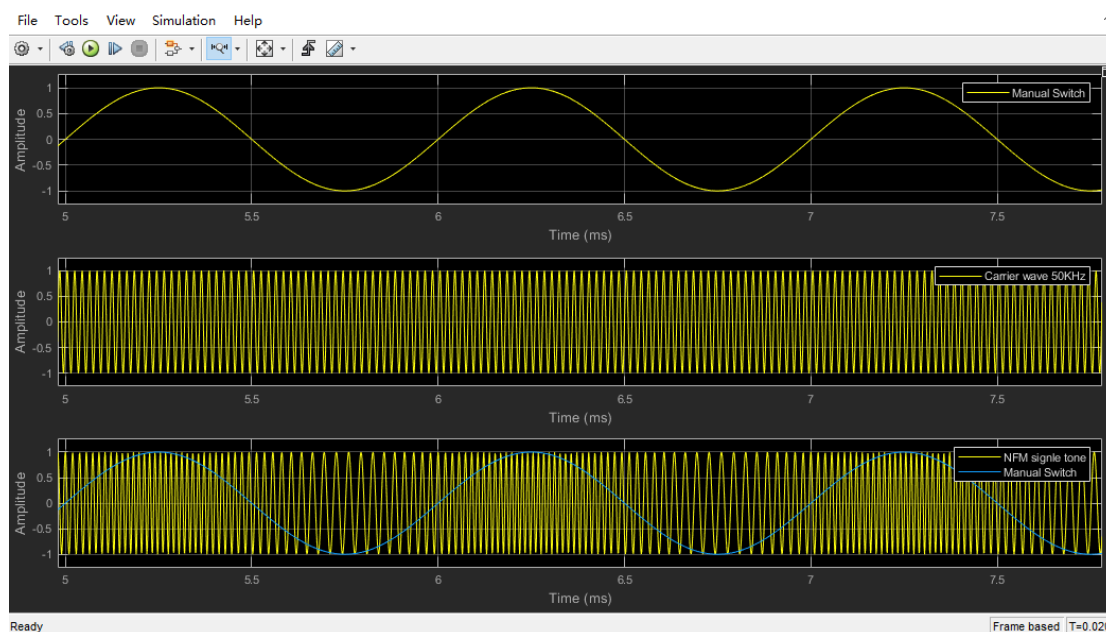


图 9-12 WFM 调制单音信号的时域波形

从时域波形上可以看出，在宽带调频下，能够比较明显地区分出基带信号对已调信号的频率的影响。

从频域上看，使用了 WFM 进行调制之后，将会产生很多的频率分量。

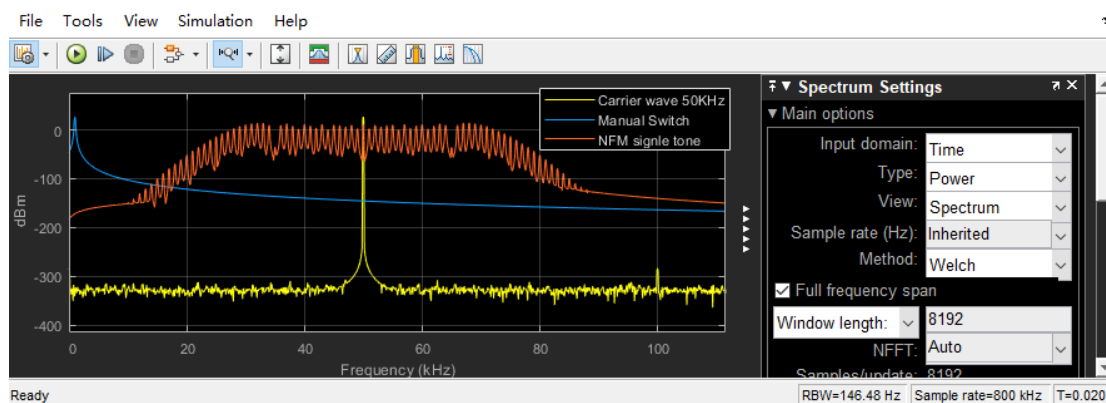


图 9-13 WFM 调制单音信号的频域波形

接下来我们再看看使用 WFM 调制一个基带信号的时域波形：

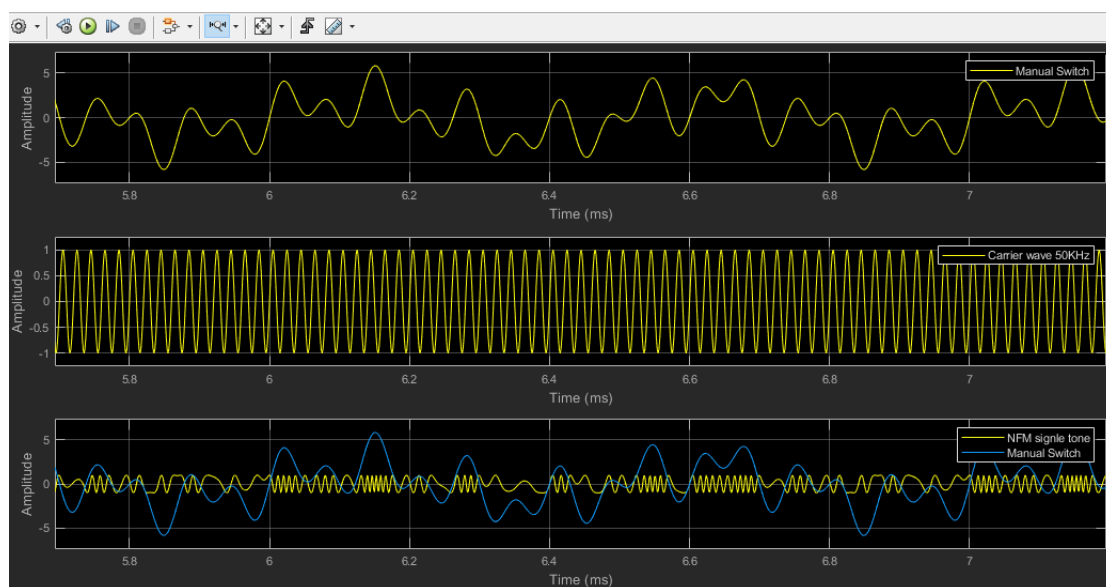


图 9-14 WFM 调制基带信号的时域波形

从 WFM 调制基带信号的时域波形上来看，可以更加直观地表现出基带信号对已调信号的影响。

9.2 使用微分的方式对 FM 信号进行解调

9.2.1 使用求导的方式对 FM 进行解调

假设我们现在成功地在接收机当中，接收到了由发射机发出的信号。

$$s_{fm}(t) = A_c \cos\left(\omega_c t + 2\pi K_{fm} \times \int_{-\infty}^t s_i(t) dt\right)$$

在接收机当中，我们对这个信号进行求导，可以得到：

$$\begin{aligned} s_{fm}'(t) &= \frac{d}{dt} s_{fm}(t) \\ &= -A_c \frac{d}{dt} \left[\omega_c t + 2\pi K_{fm} \times \int_{-\infty}^t s_i(t) dt \right] \sin\left(\omega_c t + 2\pi K_{fm} \times \int_{-\infty}^t s_i(t) dt\right) \\ &= -A_c \left[\underbrace{\omega_c + 2\pi K_{fm} s_i(t)}_{\text{amplitude component}} \right] \underbrace{\sin\left(\omega_c t + 2\pi K_{fm} \times \int_{-\infty}^t s_i(t) dt\right)}_{\text{high frequency component}}. \end{aligned}$$

从上面的式子可以看出，我们可以得到一个振幅发生变化的信号，而这个变化的振幅项，也正是这个信号的包络。

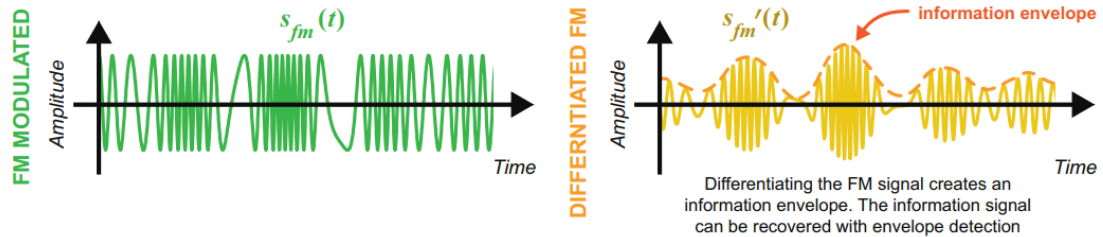


图 9-15 对 FM 信号进行求导

类比前面的 AM 的常规调幅，我们可以很容易地想到使用包络检波的方式对这个信号进行解调，尽管这个包络并不是基带的信号，包络包含有一个直流分量和一个增益系数。但是这个包络的变化和基带信号的变化是一致的。

9.2.2 下变频信号到复数基带

在前面介绍 AM 的解调的时候，我们就说过，使用 ANTSDR 和 Simulink 之间传输的是复数基带信号，ANTSDR 使用的 ad936x 在进行调制与解调的时候，采用的是复数调制和解调的方式。因此我们在 Simulink 中处理的数据是已经经过下变频之后的复数基带信号。

将一个 FM 的已调信号进行下变频到 ANTSDR 的复数基带的过程如下：

$$s_{bband}(t) = s_{fmRF}(t) e^{-j\omega_{lo}t}$$

其中 ω_{lo} 是 ad936x 的接收的本振频率。将 FM 的已调信号带入，可以得到：

$$\begin{aligned}
 s_{bband}(t) &= s_{fmRF}(t) e^{-j\omega_{lo}t} \\
 &= s_{fmRF}(t) \times (\cos(\omega_{lo}t) - j\sin(\omega_{lo}t)) \\
 &= A_c \cos(\omega_c t + \theta_{fm}(t)) \times (\cos(\omega_{lo}t) - j\sin(\omega_{lo}t)) \\
 \text{where: } \theta_{fm}(t) &= 2\pi K_{fm} \times \int_{-\infty}^t s_i(t) dt .
 \end{aligned}$$

将这个信号进行展开可以得到：

$$s_{bband}(t) = A_c \cos(\omega_c t + \theta_{fm}(t)) \cos(\omega_{lo}t) - jA_c \cos(\omega_c t + \theta_{fm}(t)) \sin(\omega_{lo}t)$$

接下来对上面的式子使用和差化积公式，可以得到：

$$\begin{aligned}
 s_{bband}(t) &= \frac{A_c}{2} \left[\underbrace{\cos(\omega_c t + \theta_{fm}(t) - \omega_{lo}t)}_{\text{baseband components}} + \underbrace{\cos(\omega_c t + \theta_{fm}(t) + \omega_{lo}t)}_{\text{high freq components}} \right] \\
 &\quad - j \frac{A_c}{2} \left[\underbrace{\sin(\omega_c t + \theta_{fm}(t) + \omega_{lo}t)}_{\text{high freq components}} - \underbrace{\sin(\omega_c t + \theta_{fm}(t) - \omega_{lo}t)}_{\text{baseband components}} \right]
 \end{aligned}$$

可以看到，经过一个低通滤波器之后，能够将高频分量滤除，从而只剩下低频分量，而这个低频分量也就正是我们所希望得到的复数基带信号。

$$s_{bband}(t) = \frac{Ac}{2} [\cos((\omega_c - \omega_{lo})t + \theta_{fm}(t)) + j \sin((\omega_c - \omega_{lo})t + \theta_{fm}(t))]$$

若我们使用的是理想的解调方式，那么 $\omega_c = \omega_{lo}$ ，此时基带信号就变为：

$$s_{bband}(t) = \frac{Ac}{2} [\cos(\theta_{fm}(t)) + j \sin(\theta_{fm}(t))]$$

$$s_{bband}(t) = \frac{Ac}{2} e^{j\theta_{fm}(t)} = \frac{Ac}{2} e^{j2\pi K_{fm} \int_{-\infty}^t s(t) dt}$$

9.2.3 使用复数微分鉴别器解调 FM

实现复数微分鉴别器就是对复数基带的 IQ 路信号分别进行处理，使用微分的方式来解调信号，下面我们会详细地介绍这一种解调方式。在这里我们以解调一个单音信号为例。

在前面地复数基带中包含实数分量和虚数分量,这两个分量对应着信号的 IQ 两路数据,我们分别用 $s_{ip}(t)$ 和 $s_{qp}(t)$ 来表示着两路信号,于是可以将这两个信号表示为:

$$s_{ip}(t) = \frac{A}{2} \cos(\theta_{fm}(t))$$

$$s_{qp}(t) = \frac{A}{2} \sin(\theta_{fm}(t))$$

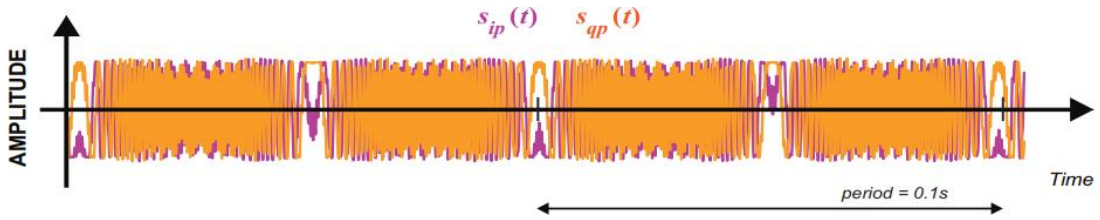


图 9-16 单音信号 IQ 原始信号

分别对 IQ 路信号进行求导,我们可以得到一个具有包络的信号

$$s_{ip}'(t) = -\frac{A}{2} \theta_{fm}'(t) \sin(\theta_{fm}(t))$$

$$s_{qp}'(t) = \frac{A}{2} \theta_{fm}'(t) \cos(\theta_{fm}(t))$$

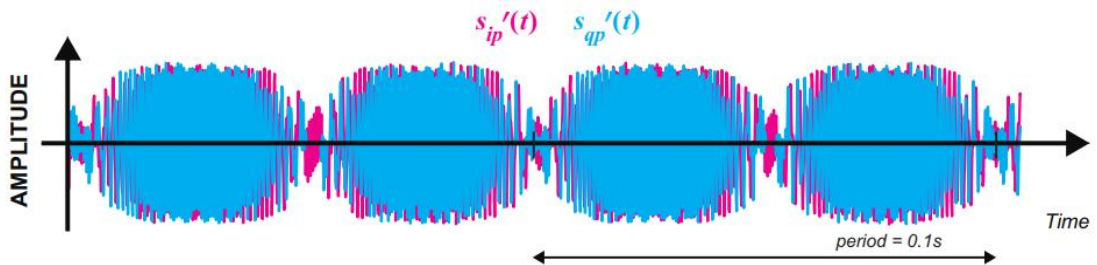


图 9-17 对 IQ 路信号求导

经过求导之后, IQ 路的分量看上去有点类似与 AM-DSB-SC 信号,没有明显的包络。但是可以看到一个被调制信号的轮廓和一个明显的载波(虽然载波的频率在变化)。上面求得的式子还需要进一步处理,我们将求导过后的 IQ 路信号和原来的 QI 信号相乘。

$$s_{ip}'(t) \times s_{qp}(t) = -\frac{A^2}{4} \theta_{fm}'(t) \sin^2(\theta_{fm}(t))$$

$$s_{qp}'(t) \times s_{ip}(t) = \frac{A^2}{4} \theta_{fm}'(t) \cos^2(\theta_{fm}(t))$$

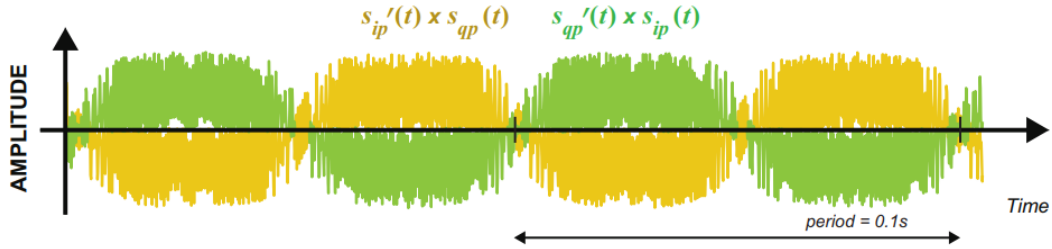


图 9-18 处理之后的各路信号

将上面这两个信号进行相减，可以得到只含基带信号的表达式：

$$\begin{aligned} s_{bb}(t) &= s_{qp}'(t) \times s_{ip}(t) - s_{ip}'(t) \times s_{qp}(t) \\ &= \frac{A^2}{4} \theta_{fm}'(t) \left(\cos^2(\theta_{fm}(t)) + \sin^2(\theta_{fm}(t)) \right) \\ &= \frac{A^2}{4} \theta_{fm}'(t) \end{aligned}$$

由前面的 FM 信号可以知道：

$$\theta_{fm}(t) = 2\pi K_{fm} \int_{-\infty}^t s_i(t) dt$$

因此可以得到

$$\theta_{fm}'(t) = 2\pi K_{fm} s_i(t)$$

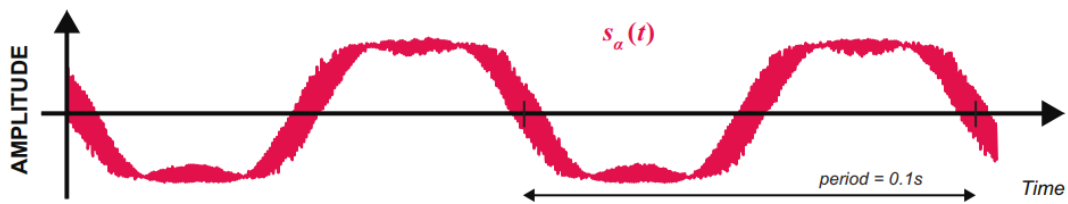


图 9-19 IQ 得到的包络

从上面的这个包络来看，这个并不是单音信号的包络，原因是当前的信号的包络与 $\frac{A^2}{4}$ 成正比，而不是与信号的振幅成正比，我们可以对这个信号进行归一化操作，比较简单的就是消除将这个结果和输入基带的 IQ 路数据的平方和求比值。

$$s_p(t) = s_{ip}(t) \times s_{ip}(t) + s_{qp}(t) \times s_{qp}(t)$$

$$= \frac{A^2}{4} \left(\cos^2(\theta_{fm}(t)) + \sin^2(\theta_{fm}(t)) \right)$$

$$= \frac{A^2}{4}$$

最终将两者进行相比可以得到：

$$s_n(t) = \frac{s_{bb}(t)}{s_p(t)}$$

$$= \frac{s_{qp}'(t) \times s_{ip}(t) - s_{ip}'(t) \times s_{qp}(t)}{s_{ip}^2(t) + s_{qp}^2(t)}$$

$$= \theta_{fm}'(t)$$

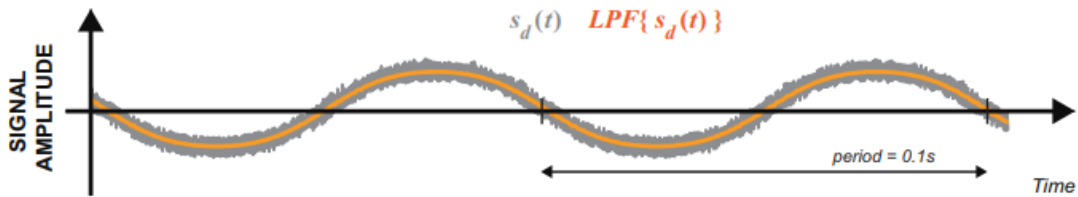


图 9-20 归一化之后的解调信号

这样我们就完成了复数基带信号的解调了。

9.2.4 数字复数微分鉴别器

前面介绍的解调过程是在模拟域下进行的，当 ANTSDR 将数据传递到 Simulink 的时候，已经是数字信号，因此我们只能够在数字域下来进行数据的处理和解调。

在数字域当中可以使用差分，来替代微分，我们回顾前面的复数微分的解调出来的最终的基带信号的表达式：

$$s_n(t) = \frac{s_{qp}'(t) \times s_{ip}(t) - s_{ip}'(t) \times s_{qp}(t)}{s_{ip}^2(t) + s_{qp}^2(t)}$$

在上面的式子中，我们只需要将微分替换成差分就可以了。

$$s_n[n] = \frac{(s_{qp}[n] - s_{qp}[n-2]) \times s_{ip}[n-1] - (s_{ip}[n] - s_{ip}[n-2]) \times s_{qp}[n-1]}{s_{ip}^2[n-1] + s_{qp}^2[n-1]}$$

常见的微分器的结构如下图所示：

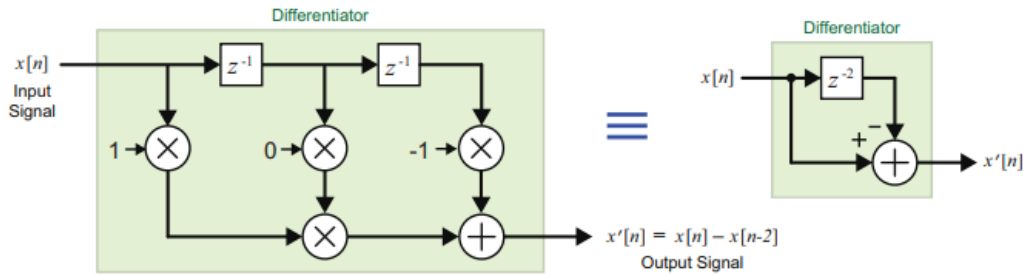


图 9-21 微分器结构

使用上面的结构，我们就能够完成 FM 的复数基带信号的解调。

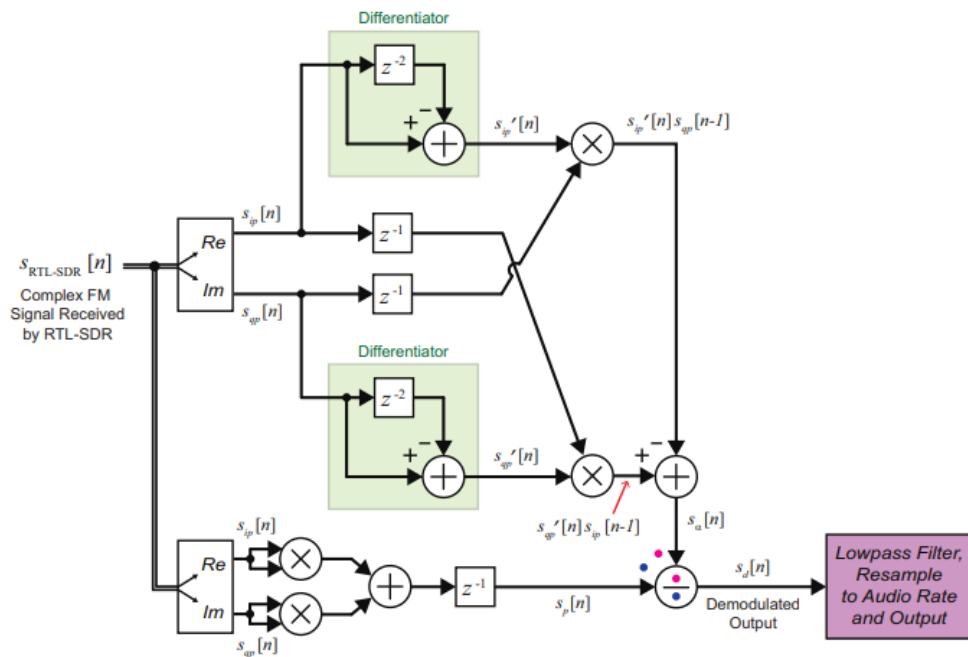


图 9-22 FM 差分解调器

9.3 使用 ANTSDR 发送 FM 信号

9.3.1 ANTSDR 发送产生 FM 信号原理

在进行本次实验之前，需要先郑重的声明，使用 SDR 的时候需要遵守当地的无线电法律，切记不要使用大功率的发射机向信道中发射信号。

在介绍了上面的相关的理论之后，我们就可以来实际实现一个 FM 的发射机和接收机了，首先我们会介绍如何使用 ANTSDR 来发送 FM 信号，发送 FM 信号的 Simulink 模型相较于前面繁琐的公式会显得格外简单。

首先我们还是需要先回顾一下 ANTSDR 的发射的架构：

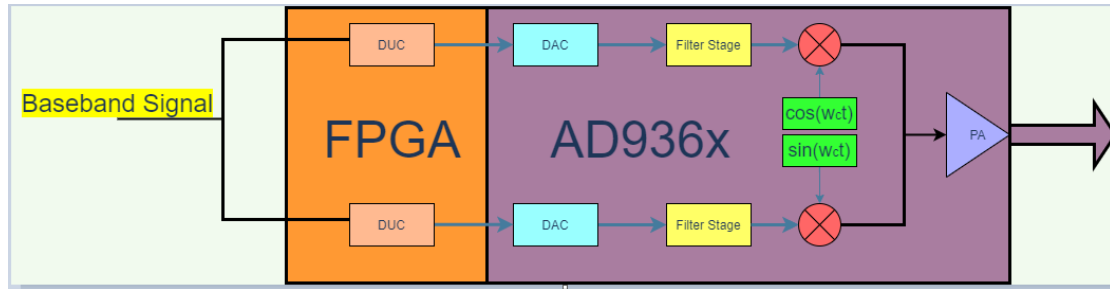


图 9-23 ANTSDR 的架构

在 ANTSDR 发射机内部，在 FPGA 内部会先将从上位机发来的数据上采样到与 AD936x 基带采样率相同的采样率，然后再 AD936x 之间依次执行数字到模拟的转换，然后是 AD936x 内部的一些列滤波器，最终经过 AD936x 内部的本振进行正交调制。

可以看到再 AD936x 内部会帮我们进行将基带信号调制到射频部分。于是我们使用 ANTSDR 发送一个 FM 调制的信号就很简单了，我们在 Simulink 之中可以使用复数基带信号，从而使这个调制过程更加简单。

在前面介绍 VCO 产生 FM 调制信号的时候，我们介绍过进入 VCO 的信号，首先需要与乘上一个增益常数，然后还要进行积分。这个积分后的结果作为调制信号的相位，

$$\theta_{fm}(t) = 2\pi K_{fm} \int_{-\infty}^t s_i(t) dt$$

其实我们可以在基带就先生成这个信号，在 Simulink 当中，我们可以生成一个这样的复信号。

回顾复信号的表达式，复信号可以表示成 e 指数的形式，也可以表示成幅度加相位的形式：

$$\begin{aligned} s_{fm-baseband}(t) &= A_c e^{-j\theta_{fm}(t)} = A_c \angle -\theta_{fm}(t) \\ &= A_c \angle -2\pi K_{fm} \times \int_{-\infty}^t s_i(t) dt, \end{aligned}$$

我们使用下面的结构就能够很轻易的生成这么一个信号了：

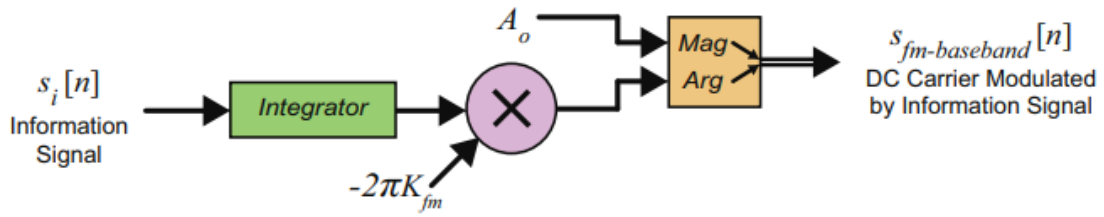


图 9-24 基带 FM 信号的生成

生成完成的基带信号，最终需要和 AD936x 进行混频，从而调制到射频的频率上，这一步的操作如下所示：

$$\begin{aligned}
 s_{tx}(t) &= K_p [\Re(s_{fm_baseband}(t)) \cos w_c t + \Im(s_{fm_baseband}(t)) \sin w_c t] \\
 &= K_p [A \cos(-\theta_{fm}(t)) \cos w_c t + A \sin(-\theta_{fm}(t)) \sin w_c t] \\
 &= \frac{K_p A}{2} [\cos(w_c t + \theta_{fm}(t)) + \cos(w_c t - \theta_{fm}(t)) + \cos(w_c t + \theta_{fm}(t)) \\
 &\quad - \cos(w_c t - \theta_{fm}(t))] \\
 &= K_p A \cos(w_c t + \theta_{fm}(t))
 \end{aligned}$$

可以看到经过 AD936x 混频之后，我们就能够成功地得到 FM 已调信号的表达式。

9.3.2 使用 ANTSDR 发送 FM 信号

有了前面的理论，接下来我们就可以来实现 ANTSDR 发送 FM 信号了。打开仿真模型 `antsdr_fm_mod.slx`，仿真模型的结构如图 9-28 图 9-25 所示，其中我们自己使用一个类似于 FIR 滤波器的结构实现了一个和前面 VCO 中相似功能的一个积分器。

积分器的设置如图 9-26 所示，在这里需要设置这个滤波器的转移函数的分子和分母的系数使之成为一个积分器。

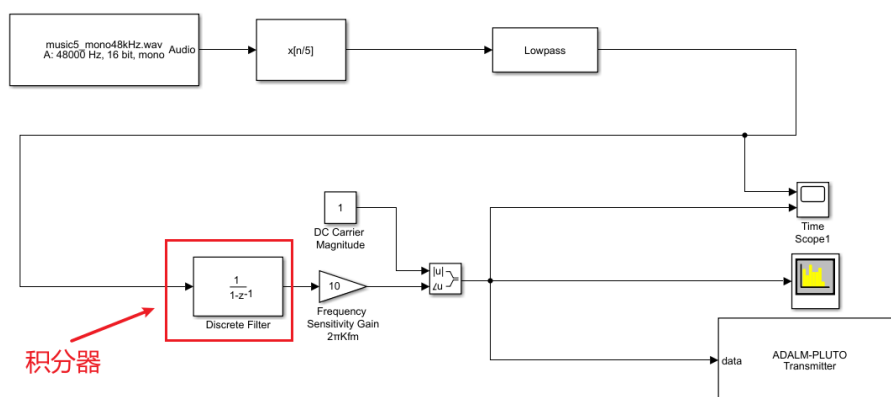


图 9-25 ANTS DR 发送 FM 信号仿真模型

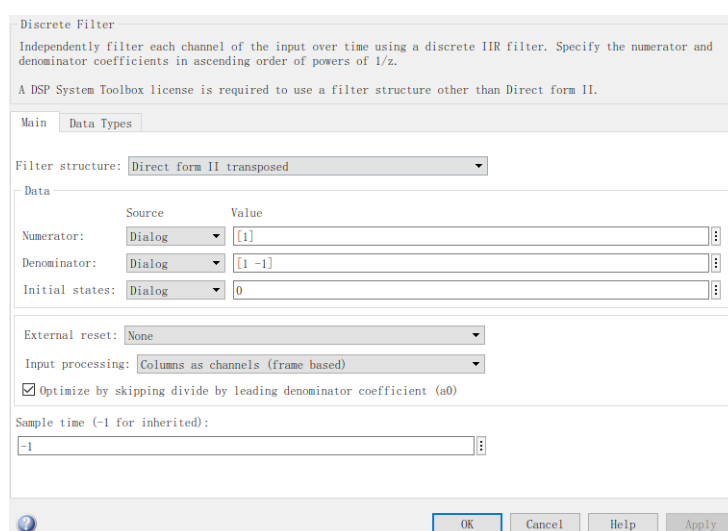


图 9-26 积分器设置

点击开始仿真，我们就可以来观察形成的基带的信号，首先我们观察时域波形，时域波形能够直观地看出 FM 调制的作用，可以看到生成的基带的 FM 信号，在消息信号的振幅不同时，产生的频率是不一样的。

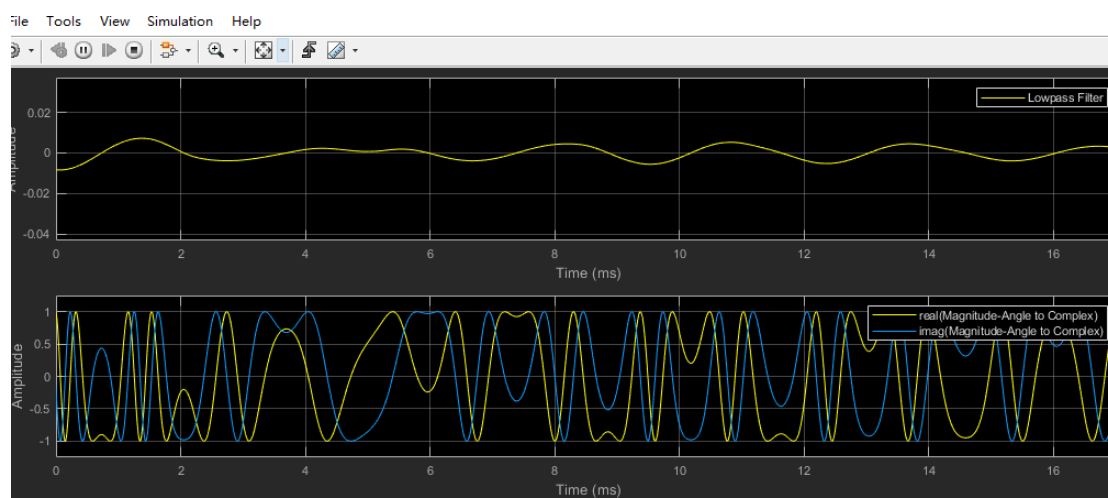


图 9-27 FM 调制的时域波形

这个时候如果用户手上有另外一个 ANTSDR 或者其他 SDR 设备比如 USRP 或者 RTL_SDR,那么用户就可以通过其他设备来接收我们通过 ANTSDR 发送的这个信号了。我们将 ANTSDR 的本振设置为 1.4GHz(正常情况下 FM 的频带为 88MHz~108MHz), 注意这里需要设置 ANTSDR 发射机的增益为-10dB,保证不会干涉大气当中的信道。

下图就是通过使用另外一个 ANTSDR 在 HSDR 上接收到的信号频谱图和瀑布图。可以看到在 1.4GHz 位置有一个明显的信号, 若这个时候选择使用 FM 来对这个信号进行解调的话, 那么现在就可以听到我们发送的音频了。

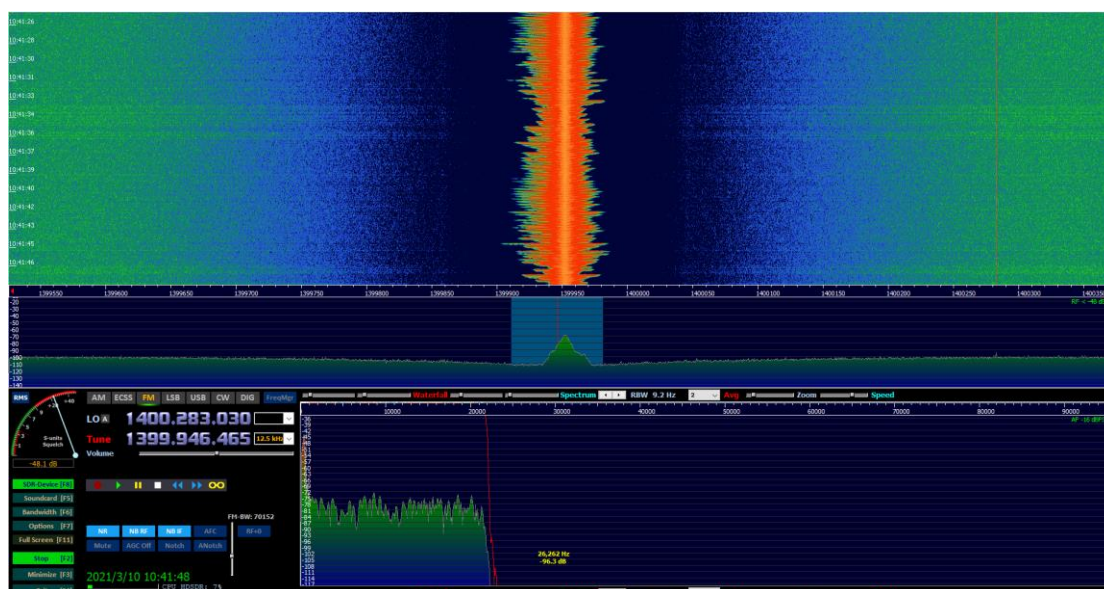


图 9-28 HSDR 在 1.4GHz 接收到的 Simulink 发送的音频信号

9.4 ANTSDR 使用差分鉴相器解调 FM 信号

在前面我们使用 ANTSDR 来发射一个 FM 信号，并且通过软件来接收了这个 FM 信号，并且能够接收到音频信号，接下来我们会自己来实现一个 FM 的解调器，来解调这些音频信号，最终，我们能在 Simulink 中完成解调商用广播电台，这样你就能够收听你周围的电台节目了。

打开仿真模型 `antsdr_fm_complexdiff_demod.slx`，仿真模型的结构如图 9-29 所示：

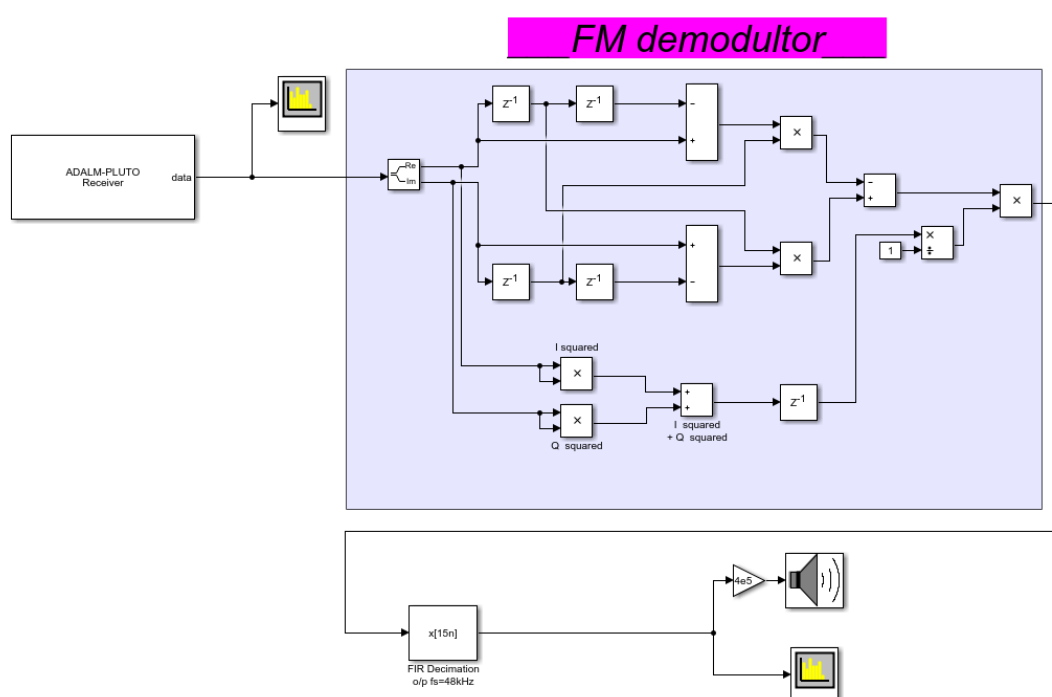


图 9-29 ANTSDR 解调 FM 信号仿真模型

在上面的仿真模型当中，最重要的部分就是差分解调的模块了，我们可以回忆一下前面进行差分解调的公式：

$$s_n[n] = \frac{(s_{qp}[n] - s_{qp}[n-2]) \times s_{ip}[n-1] - (s_{ip}[n] - s_{ip}[n-2]) \times s_{qp}[n-1]}{s_{ip}^2[n-1] + s_{qp}^2[n-1]}$$

我们可以将这个式子和上面的差分解调器进行比较，可以发现这个差分解调器的实现的其实就是上面的这个式子。对 IQ 路信号分别进行处理，然后就能够得到最终的消息信号。

在收听广播之前，你可能先确定一下你周围的电台的频率，然后在 Pluto Receiver 模块设置一下中心频率，比如附近的 100.1MHz 电台信号很强烈，我可以将中心频率设置为 100.1MHz。

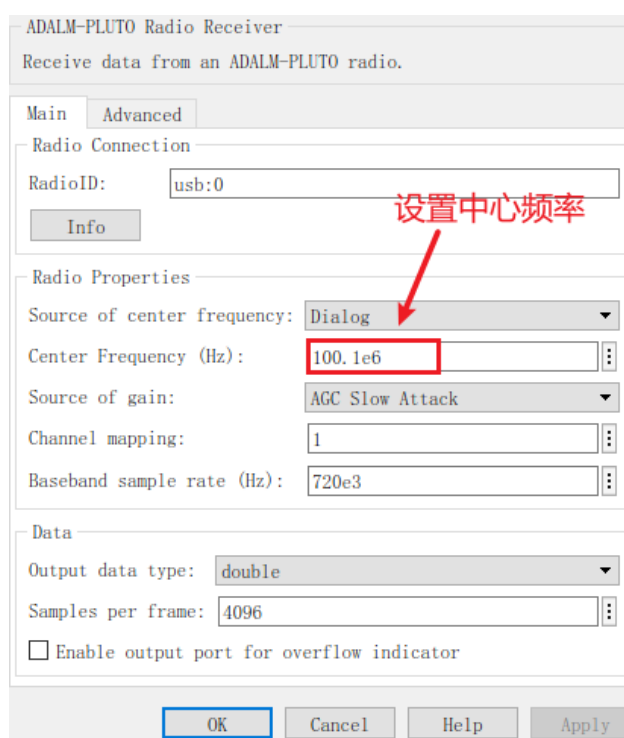


图 9-30 设置中心频率

然后可以点击仿真开始运行，首先我们观察一下频率仪，看看当前设置的频率下有没有一个较强的信号。

接收到信号之后，使用微分鉴相器进行解调，然后将解调出来的信号进行播放，就可以听到电台广播了。

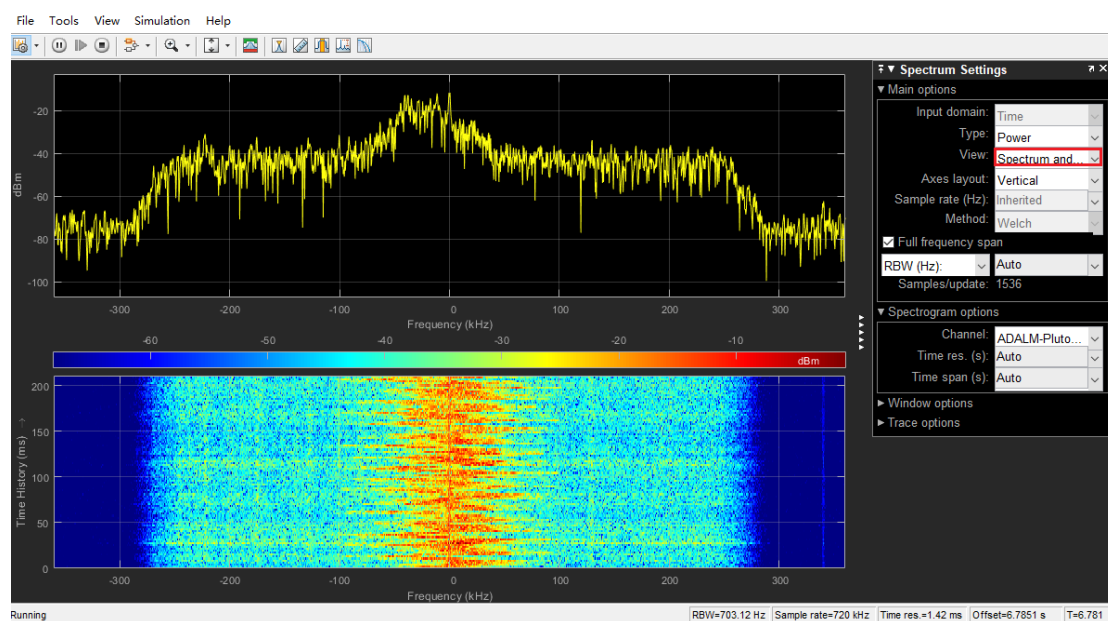


图 9-31 频谱仪上接收到的信号

9.5 总结

在本章中，我们介绍了 FM 调制和解调的原理。首先我们从 FM 调制的原理讲起，详细地介绍了 FM 调制的数学原理，并且以 VCO 为了，介绍了如何来产生一个 FM 信号，简单来说，FM 信号就是将消息信号用调制载波的频率来承载，将消息信号的振幅用载波频率的快慢来表示。

然后我们介绍了如何使用微分鉴相器来实现 FM 信号的非相干解调。从数学原理上解释了微分鉴相器工作的每个步骤，为之后我们自己实现一个微分鉴相器提供了理论依据。

我们使用 ANTSDR 发送了 FM 信号，并且能够在非 FM 广播频段上，使用其他接收机对 FM 信号进行解调，并播放出音乐。

最后，我们实现了使用 ANTSDR 来解调商用广播信号，能够在 Simulink 中将解调的信号信号通过播放器播放出来，这样我们就能够收听周围的广播了。

第10章 FM 非相干解调（延迟器）

在前面的实验当中，我们实现了对 FM 信号的解调，并且能够收听周围的电台广播。在前面的解调过程中，我们使用的是复数微分鉴相的方法，在本章的实验当中，我们会介绍另外一种非相干的 FM 解调方法，也叫做复数延迟鉴别器。

10.1 复数延迟鉴别器原理

在前面我们介绍了使用复数微分鉴别器实现对 FM 解调的原理，在本章我们会介绍一种更加简单 FM 解调器，也就是复数延迟鉴别器。

为了更加直观地来介绍如何使用复数延迟鉴别器来解调这个信号，我们还是以解调一个单音信号的 FM 已调信号为例。

FM 信号经过 ANTSDR 进行下变频之后在 Simulink 中的复基带信号如下：

$$s_{iq}(t) = \frac{A}{2} e^{j(\theta_{fm}(t))}$$

$$s_{ip}(t) = \frac{A}{2} \cos(\theta_{fm}(t))$$

$$s_{qp}(t) = \frac{A}{2} \sin(\theta_{fm}(t))$$

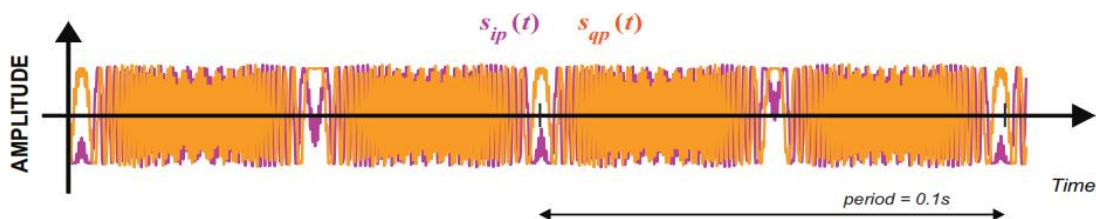


图 10-1

接下来，我们对这个复基带信号进行处理，首先使用一路信号来对这个复基带信号进行取共轭的操作，取共轭后的信号如下：

$$s_{conj}(t) = \frac{A}{2} e^{-j(\theta_{fm}(t))}$$

然后对没有取共轭之前的原始信号进行延时操作，延时过后的信号如下：

$$s_{delay}(t) = \frac{A}{2} e^{j(\theta_{fm}(t-\tau))}$$

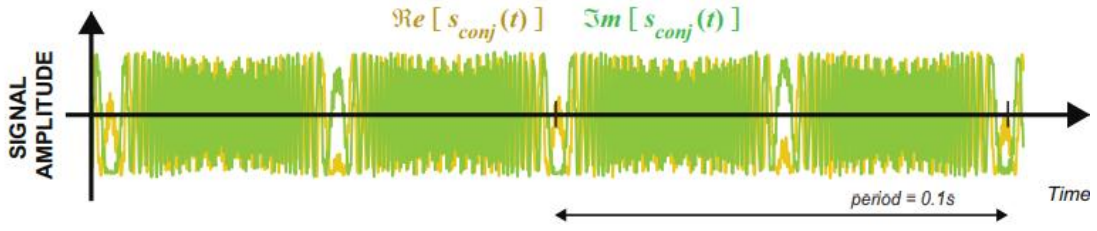


图 10-2 取共轭的信号

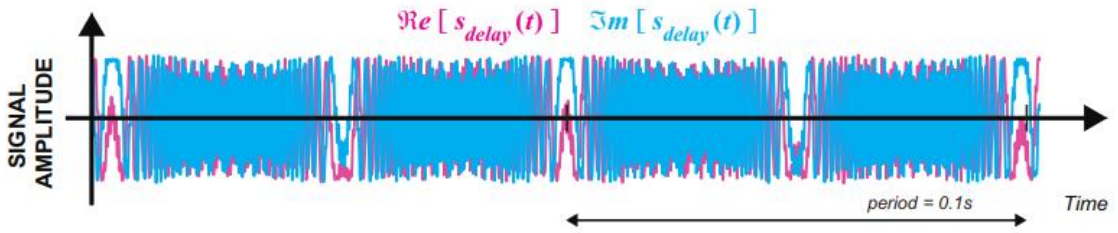


图 10-3 延时之后的信号

然后将这个延时之后的信号和共轭的信号进行相乘：

$$\begin{aligned}
 s_{pd}(t) &= s_{conj}(t) \times s_{delay}(t) \\
 &= \frac{A^2}{4} e^{-j(\theta_{fm}(t) - \theta_{fm}(t-\tau))}
 \end{aligned}$$

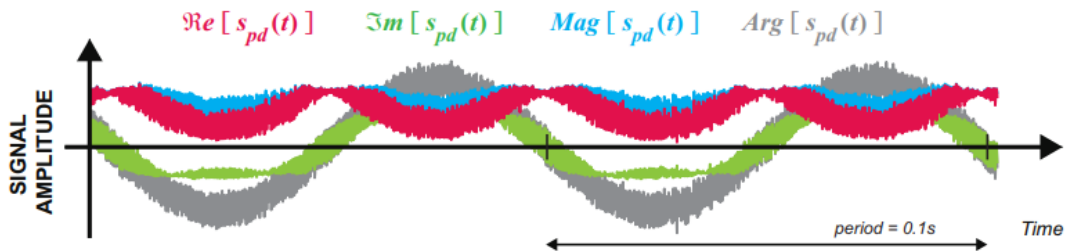


图 10-4 相乘之后的信号

可以看到，经过鉴相后的信号，其角度中包含有消息信号的有效信息，此时我们可以求取信号的角度。

$$s_d(t) = \angle s_{pd}(t) = -(\theta_{fm}(t) - \theta_{fm}(t - \tau))$$

当这个延时很小的时候，我们可以将这个角度看作近似微分的关系：

$$s_d(t) = \frac{d}{dt} \theta_{fm}(t) = \theta_{fm}'(t)$$

现在这个结果和前面使用复数微分鉴相器得到结果是一致的，所以我们通过

这种方法就成功地将消息信号从复基带信号中提取了出来。

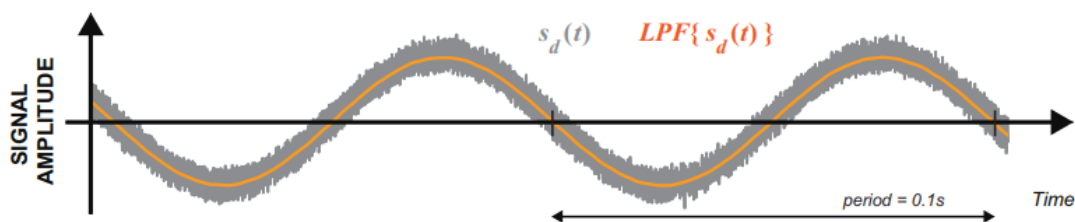


图 10-5

复数延迟鉴别器也被称作 FM 转 AM 操作，它将 FM 的频率变化转换到了幅度的变化。

10.2 ANTSDR 复数延迟解调

介绍完了复数延迟解调的原理之后，我们就可以在 ANTSDR 上来实现这种解调方式了。

打开仿真模型 `antsdr_fm_complex_delay_demod.slx`, 仿真模型的结构如下图所示：

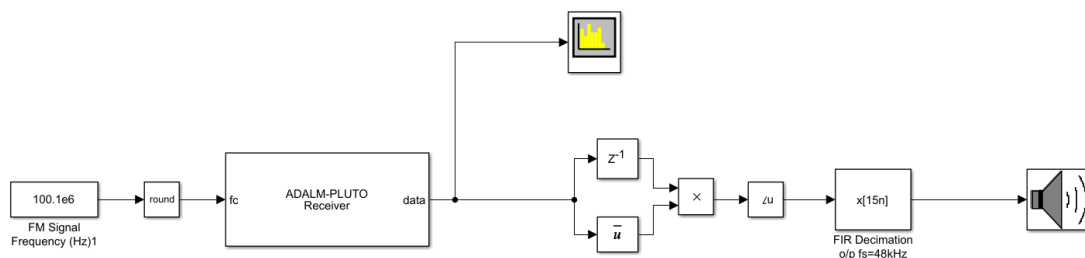


图 10-6 ANTSDR 解调的模型

可以看到使用这种方式，从结构上看十分简单，其中需要使用到一个求共轭的模块，可以使用下图种所示的这个模块，选择其功能为求共轭。

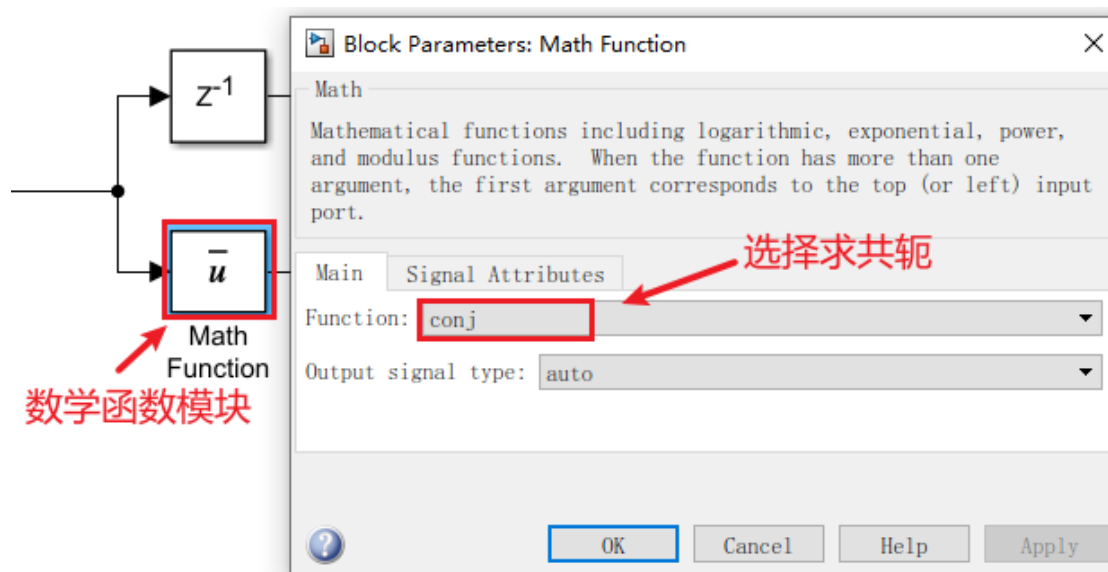


图 10-7 求共轭的模块

然后我们就可以通过这个模块完成 FM 广播的收听了。

10.3 总结

本章的实验比较简单，实现了另外一种 FM 的非相干解调的算法，也就是复数延迟鉴相器。

最终我们能够在 Simulink 种接收到来自电台的广播信号。

第11章 FM 调频立体声

11.1 FM 立体声的生成原理

FM 立体声生成的关键就是实现 FDM，也就是使用同一个载波同时调制多个基带信号。下图是一个立体声的基带信号的频谱。

在 FM 的 MPX 信号当中有一个十分重要的信号就是 19KHz 的正弦信号，也被叫做导频音，如果导频音存在于一个接收到的 FM 信号，那么硬件就会知道有立体声信息。

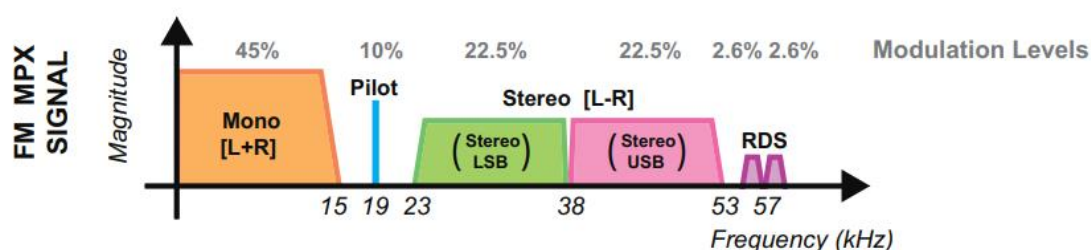


图 11-1 FM 立体声的基带信号

上图种的这个 FM 的 MPX 信号的具体生成过程如下：Mono 信号是把两个声道的信号相加产生的(L+R)位于基带。立体声分量是把左声道减去右声道(L-R)，然后将这个信号调制到 38KHz。选用 38KHz 的原因是 38KHz 的信号可以通过导频音的翻倍来实现，这样在 FM 的接收机当中，可以使用相干解调 AM 的方式将立体声分量搬移到基带。

再后来还加入了 RDS 信号这种信号的接入能够使接收机接收到数字信息，这样就可以实现接收电台的信息了。RDS 先用 BPSK 进行调制，然后再用 AM-DSB-SC 调制到 57KHz。这个 57KHz 的载波可以由导频信号频率扩大三倍来实现。

最终将这个三个分量合在一起组成 FM MPX 的基带信号，然后通过高频载波进行调制发送到无线信道当中。

图 11-2 反应了 FM MPX 基带信号的生成的过程，给出了在生成一个 FM 调频立体声的时候所需要的模块，这样在之后我们自己来实现这个模块的时候，就能够有一个参考。

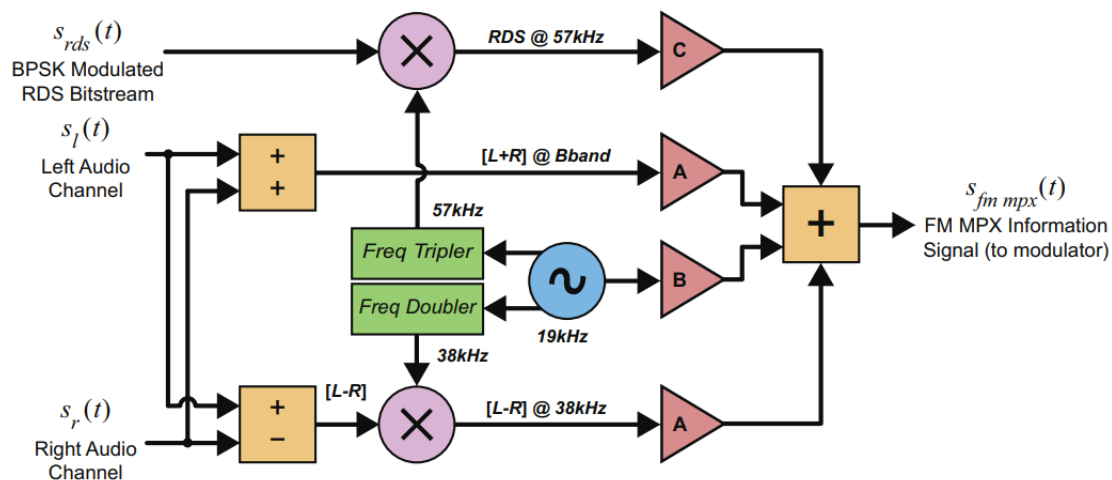


图 11-2 立体声信号生成的过程

当 FM 接收机在接收到 MPX 信号的时候，必须去执行解复用的工作，将左右声道的信号解调出来。实现这个操作的流程是：

先使用带通滤波器隔离各个 MPX 信号分量

导频音送入双倍和三倍频器合成 38KHz 和 57KHz 的载波

用合成的载波来使用相干 AM-DSB-SC 解调立体声和 RDS 分量，将其搬移到基带

然后进行低通滤波，滤除不需要的分量

把解调出来的 Mono 分量和 stereo 分量进行相加和相减得到左右声道信号。

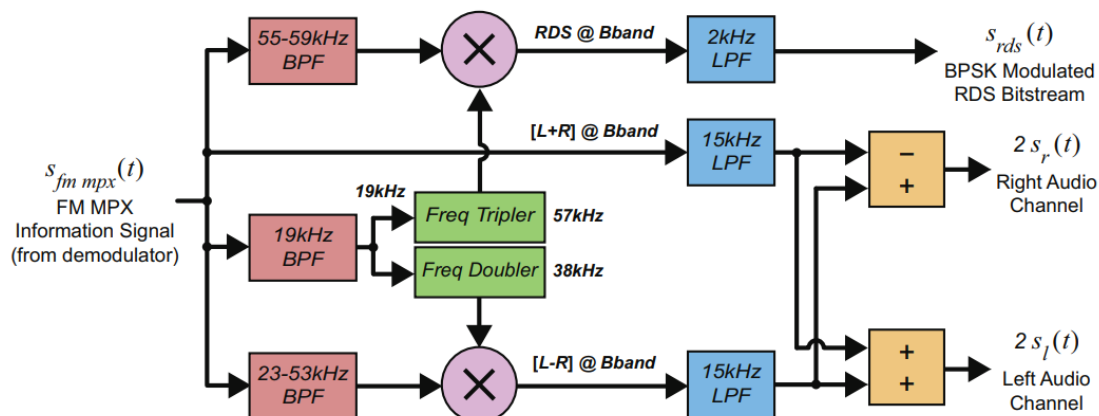


图 11-3 立体声信号解调

11.2 FM MPX 信号仿真

接下来我们会使用 Simulink 来仿真生成 FM MPX 信号的这一过程。打开仿真模型 `fm_mpx.slx`,

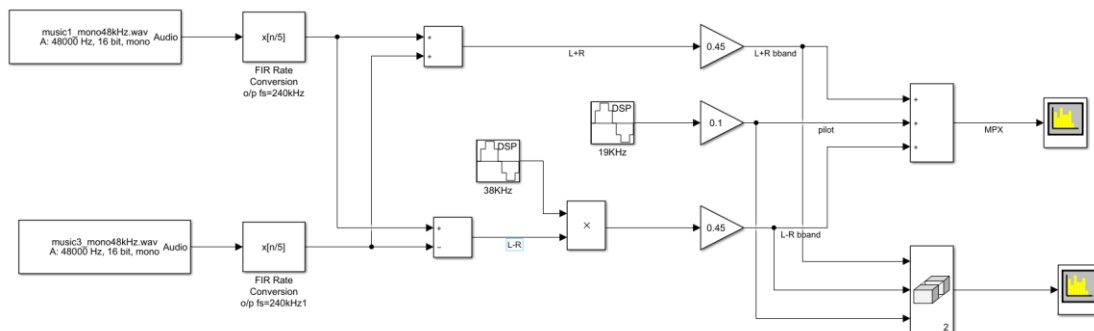


图 11-4 立体声信号生成仿真

在一开始使用了两个不同音频信号作为立体声信号的左右声道, 然后依次将这两个信号进行相加和相减, 分别得到 mono 分量和 stereo 分量, 然后使用 38KHz 的信号对 stereo 信号进行调制, 最后将得到的三个分量相加得到 MPX 信号。

在这个仿真模型当中, 使用了两个频谱仪, 其中一个用来观察各个 MPX 基带信号各个分量, 另外一个用来观察最终形成的 MPX 信号。点击运行仿真, 可以看一看运行的结果。

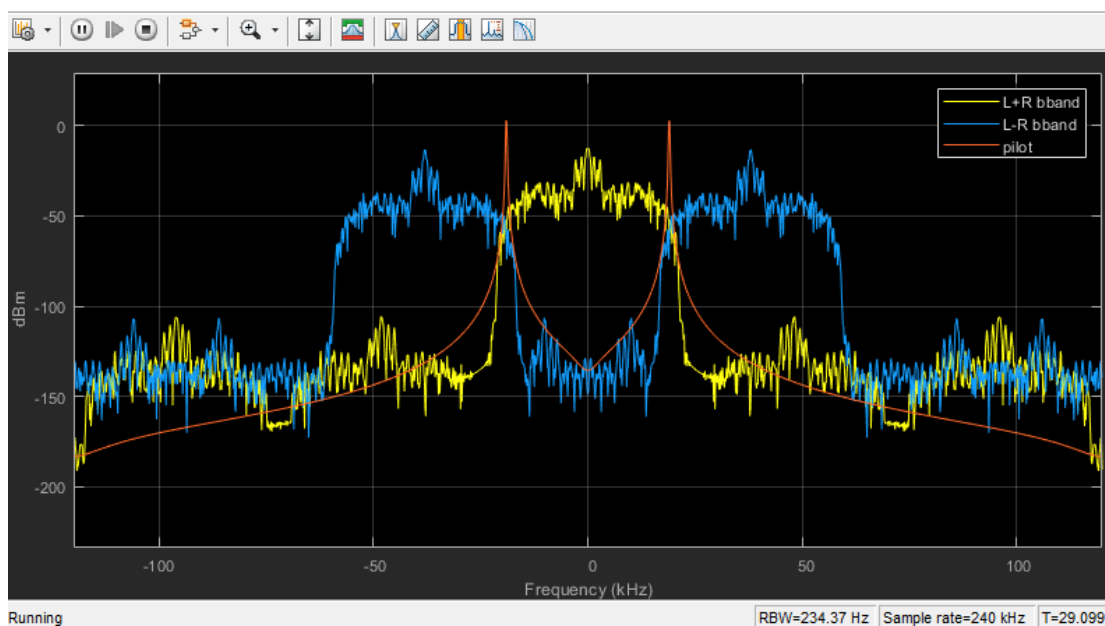


图 11-5 MPX 信号各个分量

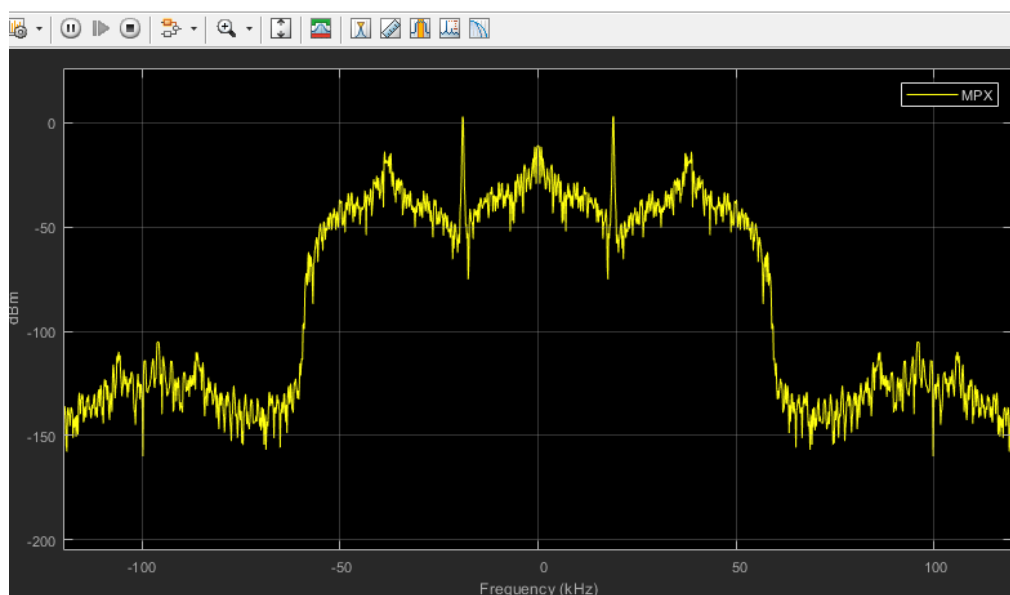


图 11-6 生成的 MPX 信号

11.3 使用 ANTSDR 发送立体声信号

在前面进行了调频立体声的仿真之后，我们知道了生成 FM MPX 信号的过程，然后我们就可以使用 ANTSDR 来发送一个调频立体声信号，只不过我们发送的不是立体声，其左右声道分别为不同信号，左右声道分别为两个不同的音频，这样在使用其他 SDR 软件进行接收并解调的时候，我们就能够听到两个完全不同的音乐了。

仿真模型如图 11-7 所示：

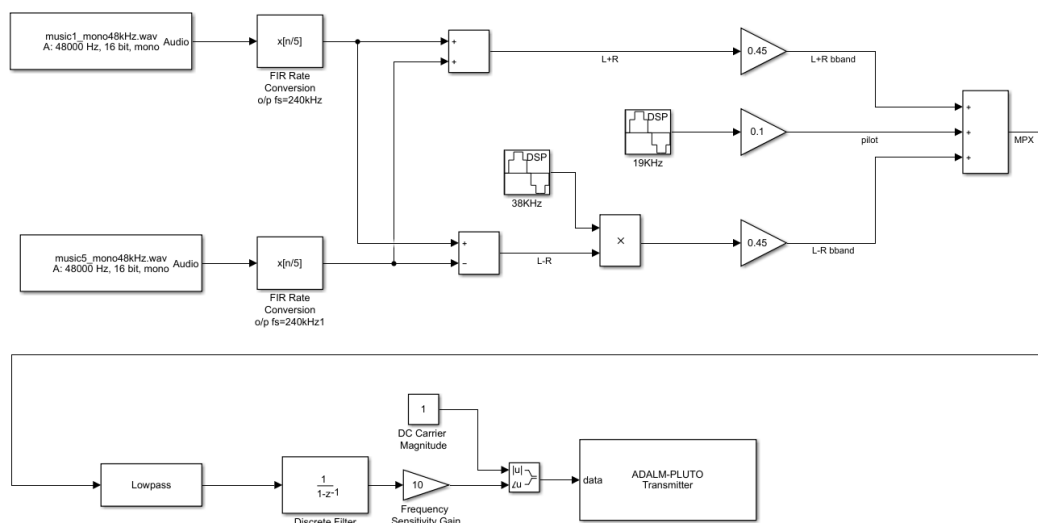


图 11-7 ANTSDR 发送调频立体声信号

不同的音频信号经过组成 FM 调频立体声的基带信号，然后通过 FM 调制发送到无线信道中。

点击仿真，开始运行，此时 Simulink 就会通过 ANTSDR 把数据发送往真实的信道当中。这个时候如果用户手上有另外一块 ANTSDR 或者其他 SDR 设备，比如 RTL_SDR USRP 等，就可以使用其他设备来接收这个立体声了。

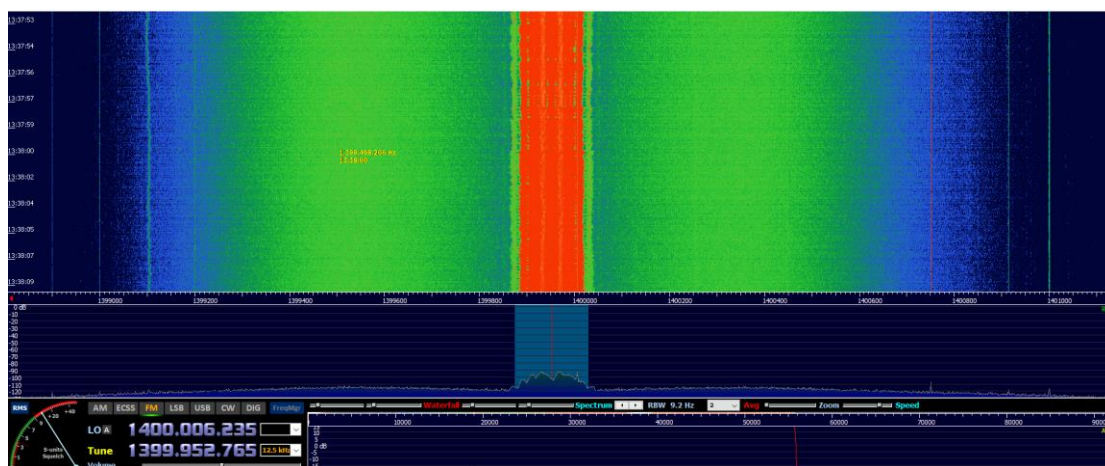


图 11-8 HDSDR 在 1.4GHz 接收到的 FM 信号

此时笔者使用 HDSDR 来接收这个信号，在 1.4GHz 的地方，接收到了由 ANTSDR 发送的数据，此时带上耳机，你应该能够同时听到两首歌。

我们可观察一下使用 HDSDR 接收到的信号解调出来的在基带的频谱：

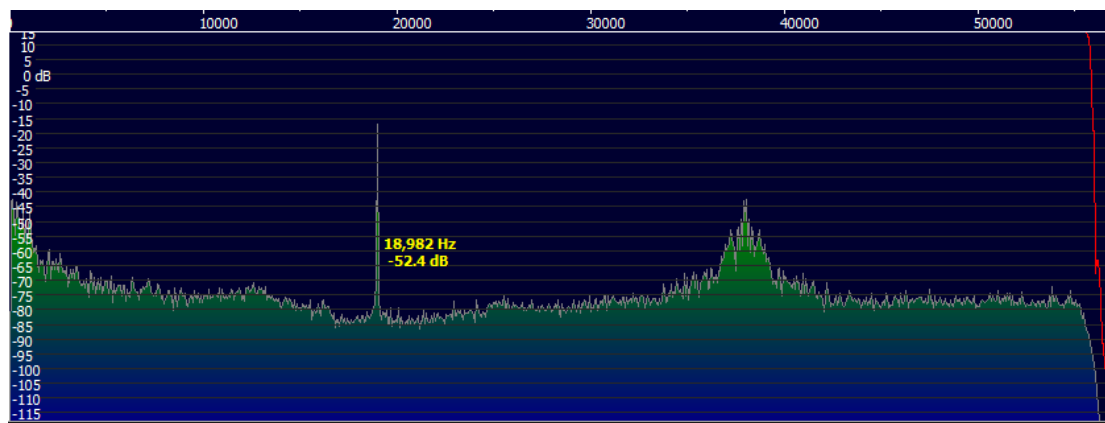


图 11-9 HDSDR 解调出来的基带信号

可以看到在 19KHz 处有一个导频音，这样 HDSDR 就会识别这个信号是一个调频立体声了。

11.4 ANTSDR 解调 FM 立体声

在前面的实验当中，已经实现了使用 ANTSDR 来发送调频立体声信号了，接下来我们介绍如何来解调调频立体声信号，回顾前面 FM MPX 信号解调的框图：

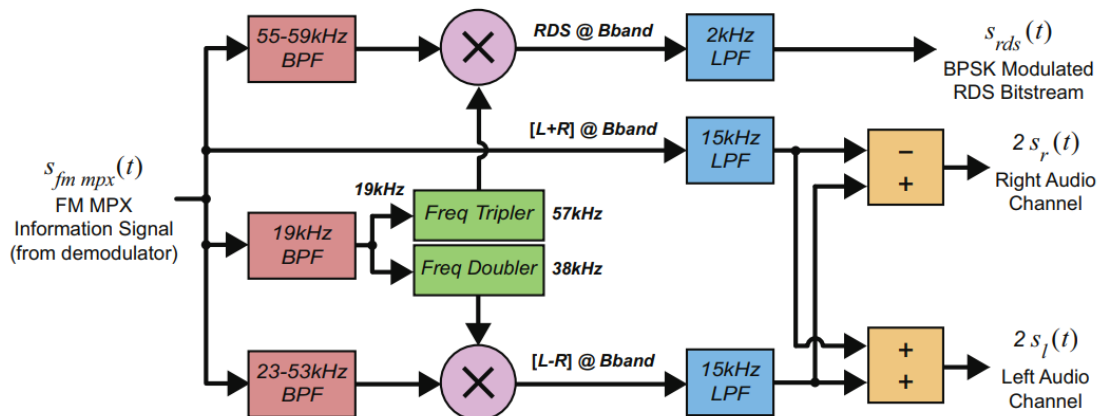


图 11-10 FM MPX 信号解调

打开仿真模型 `antsdr_fm_stereo_demod.slx`，可以看到，最开始的时候进行的和前面的章节当中的微分延迟鉴别器解调 FM 是相同的。

然后使用带通滤波器分别得到单声道分量和立体声分量，对立体声分量进行下变频，从而得到在基带的立体声分量。

在这之后，将单声道分量和立体声分量进行相加和相减，分别得到单独的左声道和右声道信号，然后将左右声道符合，收听最终解调出来的音乐。

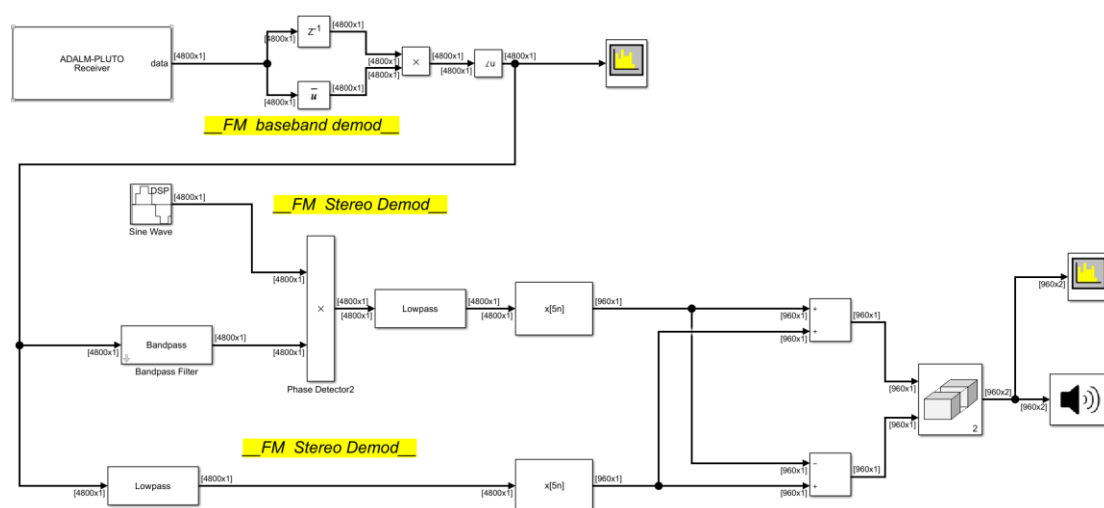


图 11-11 FM 解调立体声模块

本次实验 ANTSDR 接收的广播电台是 100.1MHz，这是一个音乐频道。点击

运行仿真，首先我们观察由差分延迟解调器解调出来的信号的频域波形：

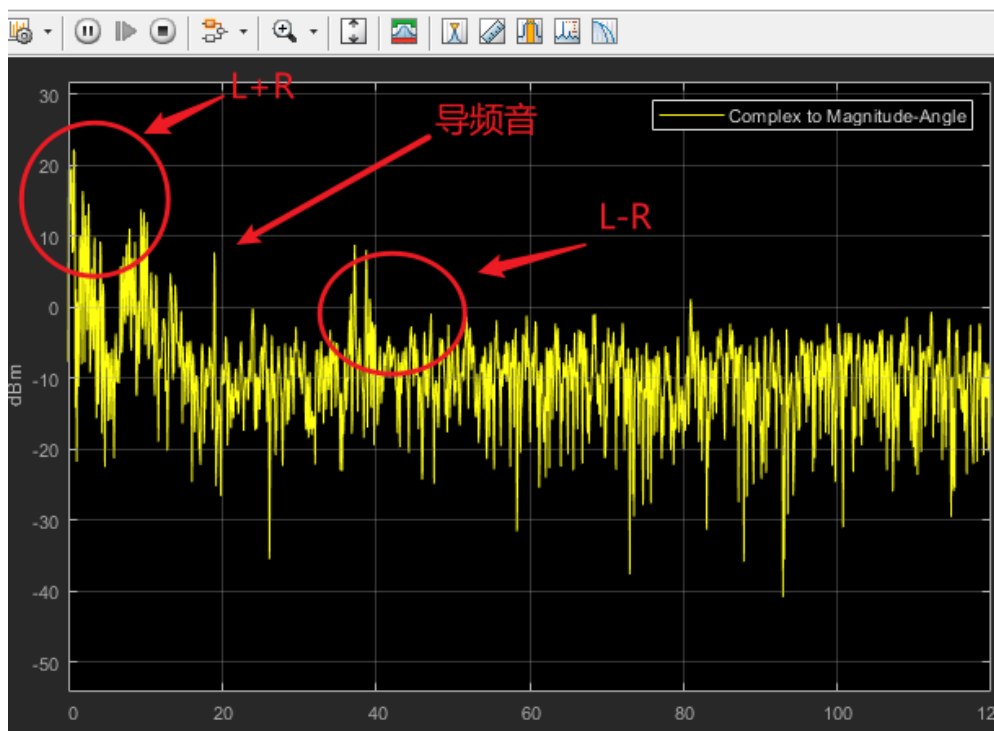


图 11-12 100.1M 电台解调频域

从上面的电台的解调波形中可以明显地看到在 19KHz 处有一个明显的单音信号，这个信号就是导频音。从而可以验证实际中的 FM 电台也是采用这种方式来发射 FM 立体声信号的。

最后我们可以看到解调出来的单个左声道和右声道的频谱。

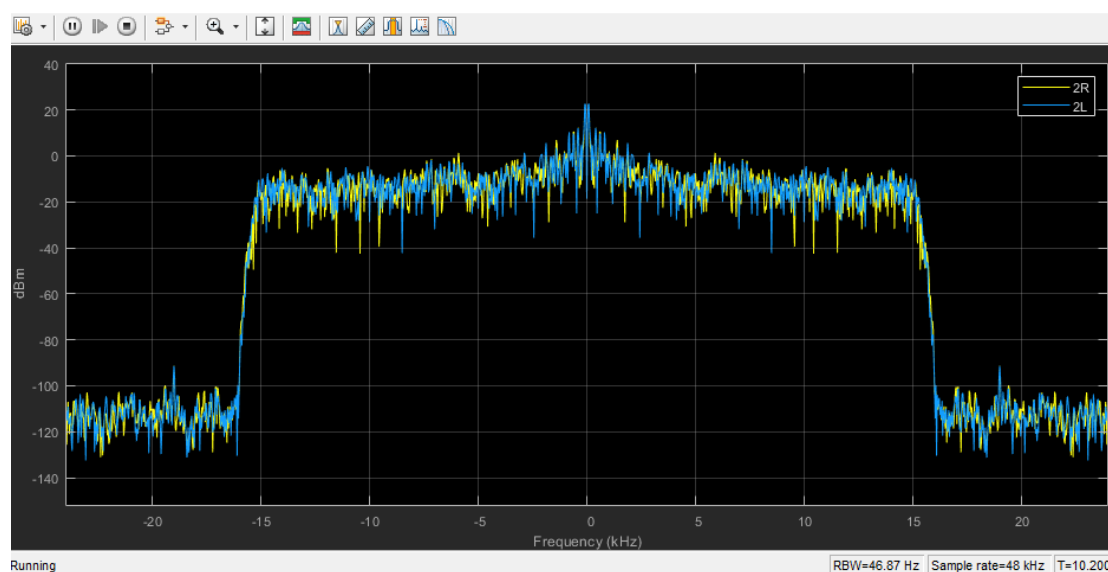


图 11-13 解调后的左右声道

11.5 ANTSDR 同时调制和解调 FM 信号

在前面的实验当中,我们实现了 ANTSDR 发送 FM 信号和 ANTSDR 解调信号,在接下来的这个实验当中,我们会使用 ANTSDR 发送一个调频立体声信号,并且使用 ANTSDR 来解调这个调频立体声信号。

打开仿真模型 `antsdr_fm_mpx_loop.slx`

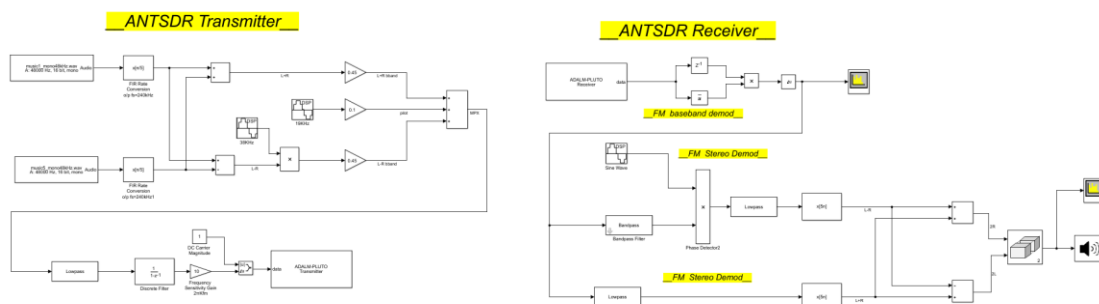


图 11-14 ANTSDR LOOP 仿真模型

从仿真模型上可以看到,左边是 FM MPX 信号生成和发送模块,发送的音频信号是两个不同的音频信号,然后形成一个 FM 立体声信号。

在右侧是一个接收和解调模块,接收机的结构和上一个实验是一致的。

点击开始仿真,你可以发现,你能够同时听到两个乐音。

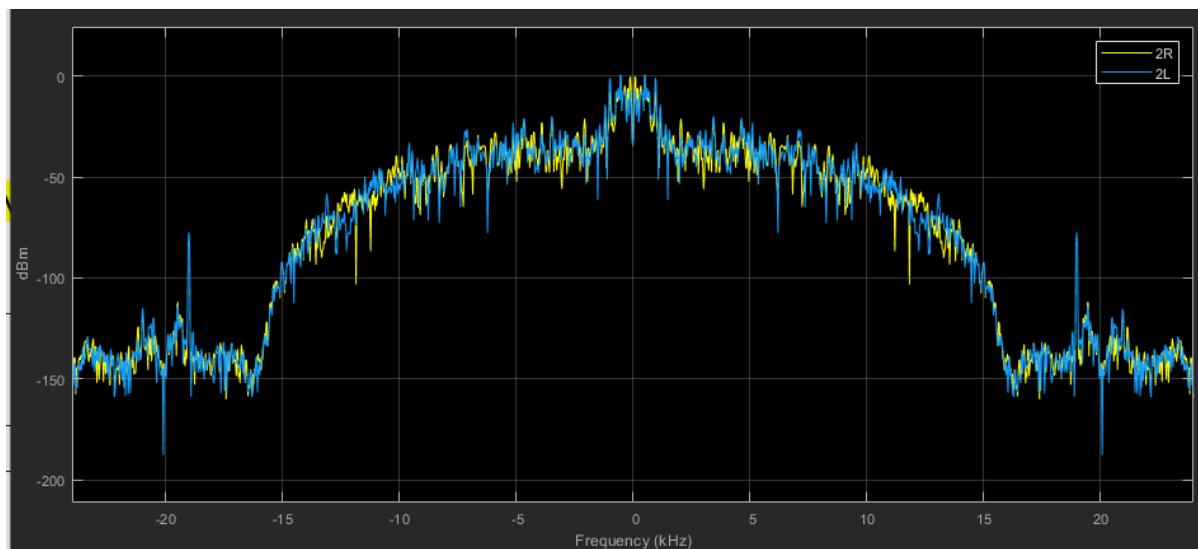


图 11-15 解调出来的立体声信号

从立体声解调的信号上看,接收到的两个信号的频谱,可以明显地看出两个信号的频谱不一致。

11.6 总结

在本章的实验当中，我们完成了 FM 立体声的调制和解调，并且使用 ANTSDR 发送了调频立体声，通过其他 SDR 软件进行了解调。

然后我们在 Simulink 当中，实现了解调 FM 立体声，成功地解调了在实际当中地商用电台的广播信号。

值得说明的是，在本次实验中对立体声信号进行解调的时候，直接使用了一个 38KHz 的信号来对立体声信号进行解调。这种方法是存在很大缺陷的，正确的做法应该是将导频音提取出来，然后通过 PLL 来锁定导频音，与此同时生成一个 2 倍频的信号，用这个信号来实现对立体声的相干解调。

第12章 FM 频分复用

在前面的实验当中，我们实现了调频立体声的发送和接收。通过上一个实验，其实我们已经接触了如何来进行 FM 的复用，在一个立体声当中，同时使用到了两个声道的信号。

在本章的实验当中，我们会学习使用 FM 进行频分复用的方法，提高信号传输的效率。

12.1 FM 频分复用原理分析

FM 立体声信号本省就是一个复用的信号，同时包含左右声道信号。FM 立体声的信号频率如图 12-1 所示：

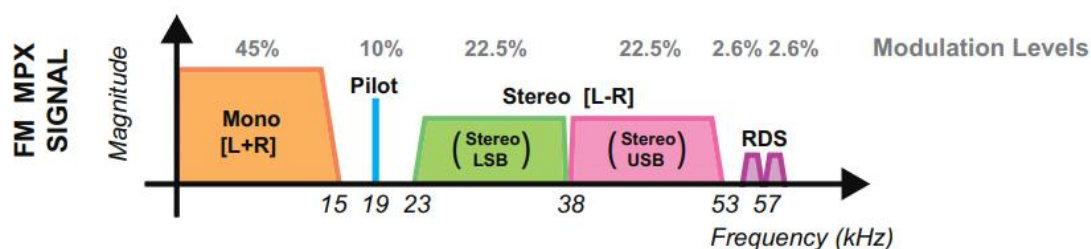


图 12-1 FM 立体声信号频谱

当前这个信号复用了两个信号，分别是左声道和右声道的信号。如果我们把单声道和立体声频谱看作两个 15KHz 宽的信道，那么我们就可以在这两个信道上实现频谱的复用了。

在前面我们介绍了 AM 信号的频谱复用，采用了复数调制的方法，将一个信号调制到了基带上。要显示 FM 的复用，要进行的操作是类似的。可以将两个 5KHz 的信号复用到单声道分量上，将另外两个 5KHz 的信号复用到立体声信道上。如图 12-2 所示：

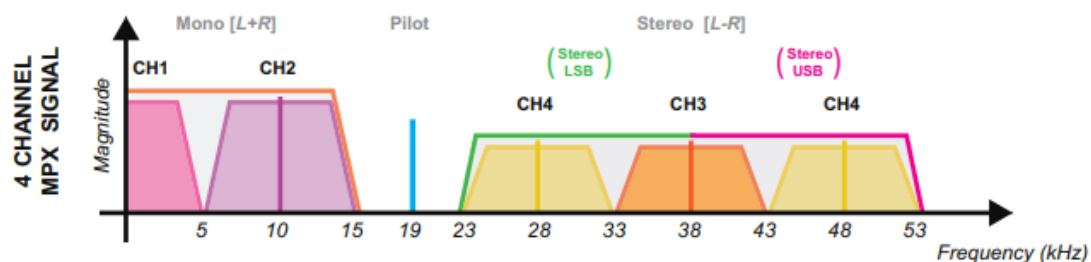


图 12-2 FM 复用后的频谱

可以看到复用后的频谱在 mono 声道上，有 CH1 和 CH2 的信号，在 Stereo 分量上有 CH3 和 CH4 信号分量。要实现这种复用，我们可以按照如下的方式进行。

首先我们有 4 个频带宽度为 5KHz 基带信号，CH1，CH2，CH3，CH4。

将 CH2 和 CH4 先用 10KHz 的余弦波进行调制，然后来生成左声道和右声道分量：

$$s_L(t) = s_{ch1}(t) + [1 + s_{ch2}(t)] \cos 2\pi f_{10K}t + [1 + s_{ch3}(t)] \\ + [1 + s_{ch4}(t)] \cos 2\pi f_{10K}t$$

$$s_R(t) = s_{ch1}(t) + [1 + s_{ch2}(t)] \cos 2\pi f_{10K}t - [1 + s_{ch3}(t)] \\ - [1 + s_{ch4}(t)] \cos 2\pi f_{10K}t$$

然后将左右声道信号分别组成单声道分量和立体声分量，如图 12-3 所示：



图 12-3 生成单声道和立体声信号

$$s_{mono}(t) = s_L(t) + s_R(t) = 2s_{ch1}(t) + 2[1 + s_{ch2}(t)] \cos 2\pi f_{10K}t$$

$$s_{stereo}(t) = s_L(t) - s_R(t) = 2s_{ch3}(t) + 2[1 + s_{ch4}(t)] \cos 2\pi f_{10K}t$$

然后将 stereo 信号通过 38KHz 的信号进行调制，最后再加上一个 19KHz 的导频音，这样我们就能够成功的实现这个复用之后的信号。

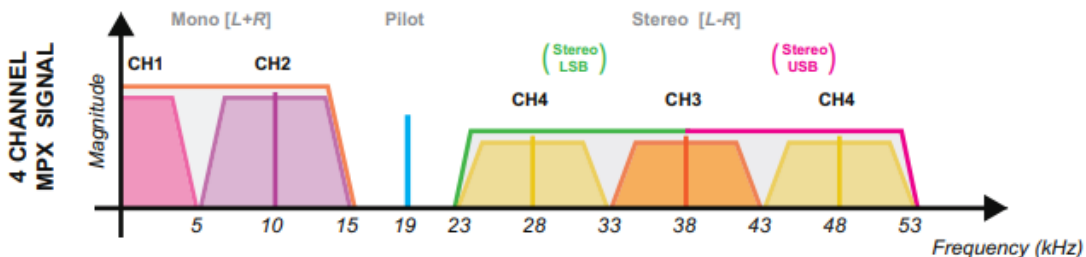


图 12-4 最终生成的复用信号

最终我们可以看一下，各个通道的信号在最终的频谱上所占用的频率范围：

CH1: 0~5KHz

CH2: 5~10KHz(LSB), 10~15KHz(USB)

CH3: 33~38KHz(LSB), 38~43KHz(USB)

CH4: 23~33KHz(LSB), 43~53KHz(USB)

最终我们得到的 fm MPX 信号为:

$$s_{fm_mpx}(t) = 2s_{ch1}(t) + 2[1 + s_{ch2}(t)] \cos 2\pi f_{10K}t \\ + \{2s_{ch3}(t) + 2[1 + s_{ch4}(t)] \cos 2\pi f_{10K}t\} \cos 2\pi f_{38K}t \\ + \cos(2\pi f_{19K}t)$$

12.2 使用 ANTSDR 发送 FM FDM 信号

在前面介绍了如何来生成一个 FM FDM 信号的基带信号，接下来我们就会实现这么一个基带信号，并且使用 ANTSDR 来将信号发送出去。打开仿真模型 `antsdr_fm_fdm.slx`，可以看到本次实验的仿真模型：

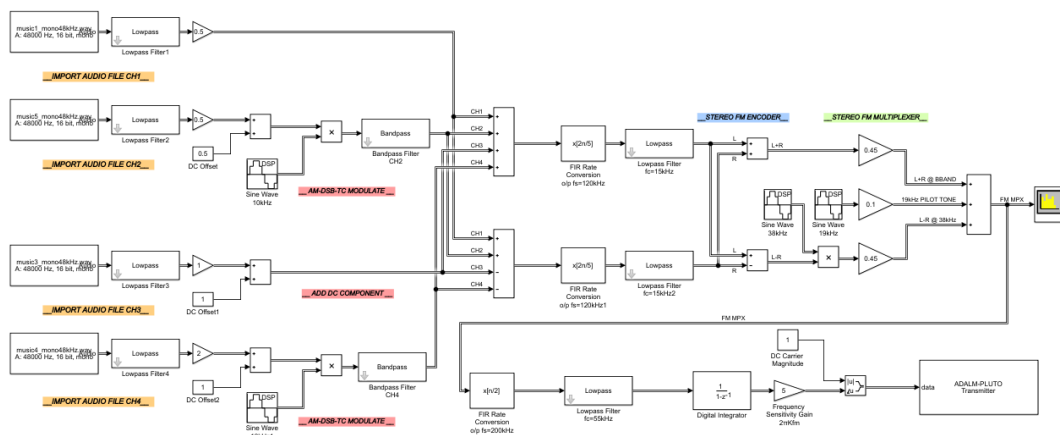


图 12-5 ANTSDR FDM 仿真模型

在模型的左边，是从几个不同的音频源当中获取数据，并进行 10KHz 的调制，然后形成左右声道信号。

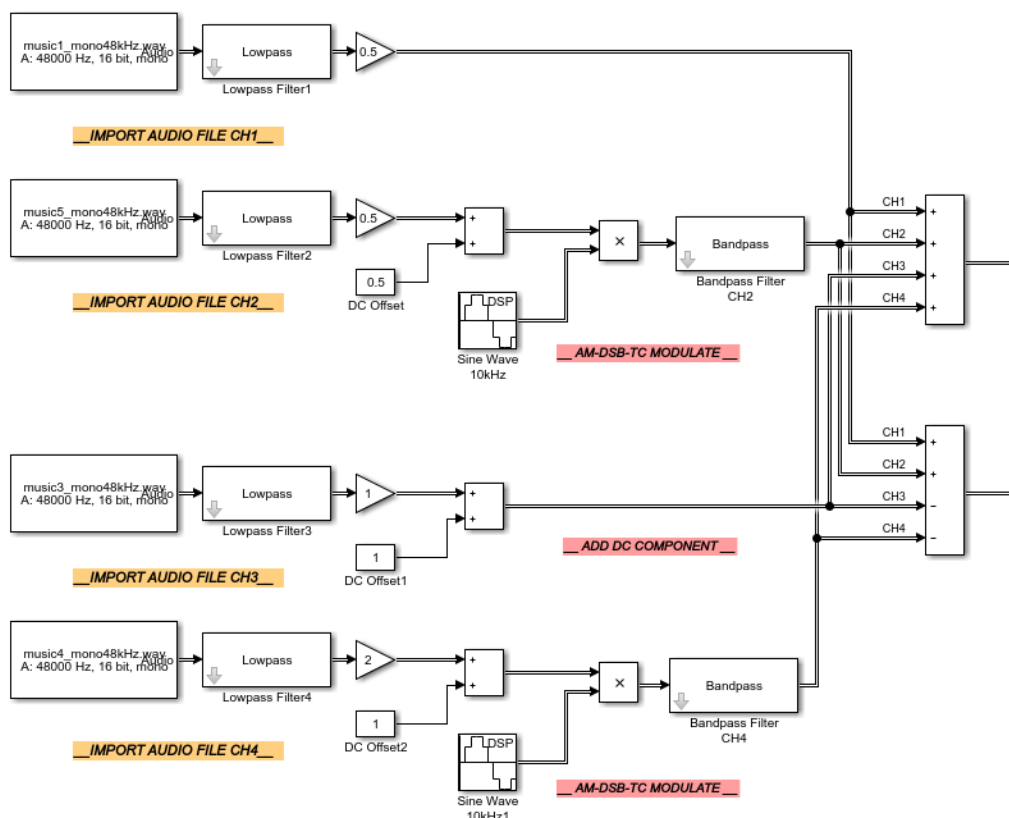


图 12-6 组成左右声道信号

组成左右声道之后，就可以使用前面实验中所知道的方法来形成调频立体声信号了。

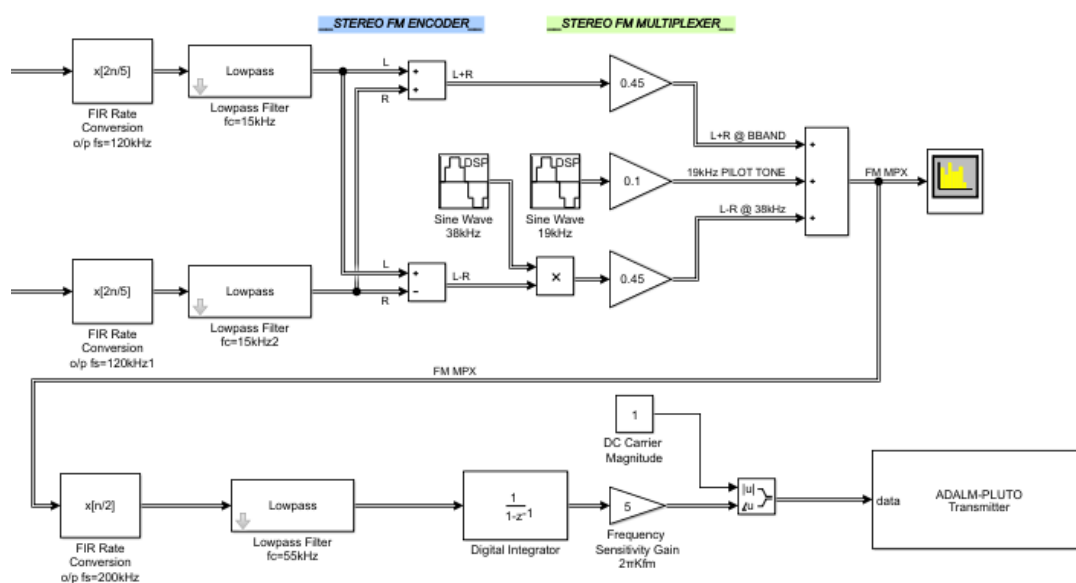


图 12-7 组成 FM 立体声信号

运行这个仿真模型之后，可以观察当前所获取到的频谱的信息，从频谱上能够很明显地看出，有 4 个明显的频带上有较强的信号。

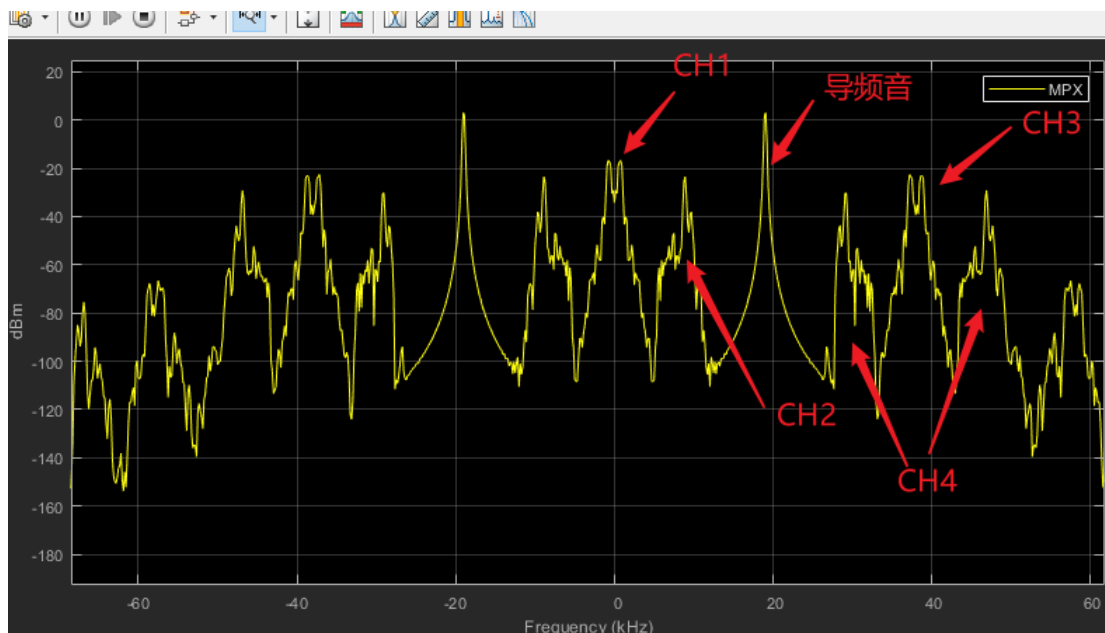


图 12-8 FM FDM 信号频谱

12.3 使用 ANTSDR 接收 FM FDM 信号

使用 ANTSDR 发出 FM FDM 信号之后，这时候如果我们再使用其他的 SDR 软件来解调这个信号的话，会听到十分奇怪的声音，这是理所当然的，其他 SDR 软件解调的是 FM 的调频立体声，我们在前面的调频立体声当中，单声道分量和立体声分量中都包含有两个通道的信息，因此使用其他软件很难将这 4 个通道的信号都解调出来。

这时候，就需要我们自己来实现一个接收机，从而能够将 4 个频道从 FM 上解调出来。

打开仿真模型，`antsdr_fm_fdm_demod.slx`

仿真模型如图 12-9 所示，在这个解调的模型当中，首先使用微分延迟鉴别器来对 FM 信号进行解调得到基带信号。

然后使用低通滤波器或者带通滤波器来对各个频带上的信号进行提取。对于 0~5KHz 的信号，它是 CH1 的基带信号，经过提取之后，可以直接给到音频播放器播放。

对于其它通道上的模块，由于在形成 FM FDM 信号的时候，对其他音频信号调制组合成一个 FM FDM 信号的时候，采用的是 AM-DSB-TC 调制，因此可以在

这里使用包络检波的方式来解调这些信号。

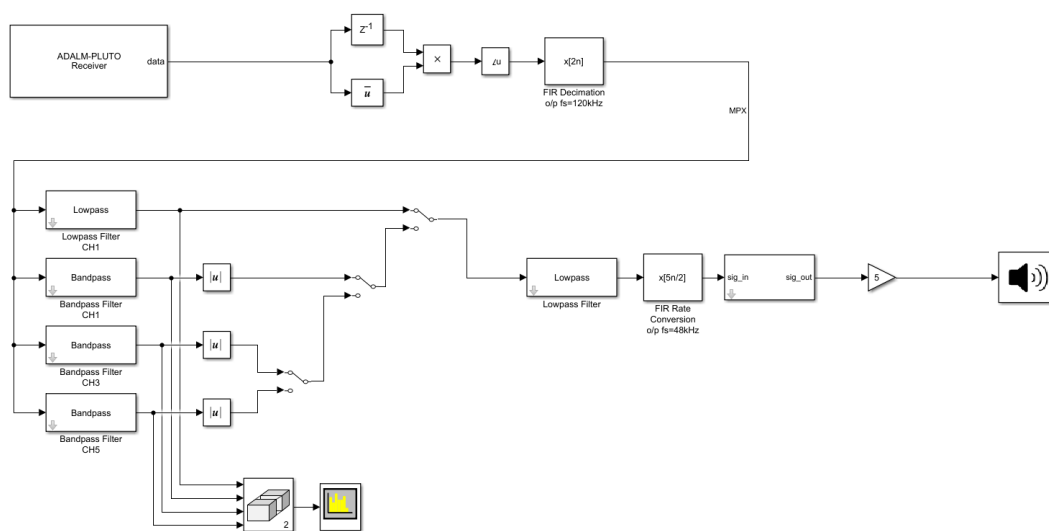


图 12-9 ANTSDR FM FDM 解调

在这个仿真模型当中，我么还会使用到在上一个实验当中完成的 FDM 的发送模块：

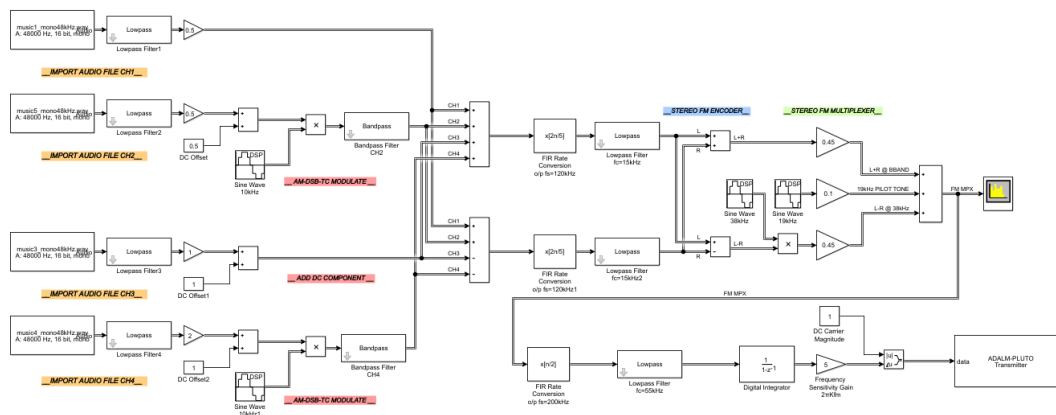


图 12-10 ANTSDR 发送 MPX 信号

点击运行仿真，你应该能够听到音乐声了，这个时候，如果你调节开关，应该能够听到各个频道上的信号了。

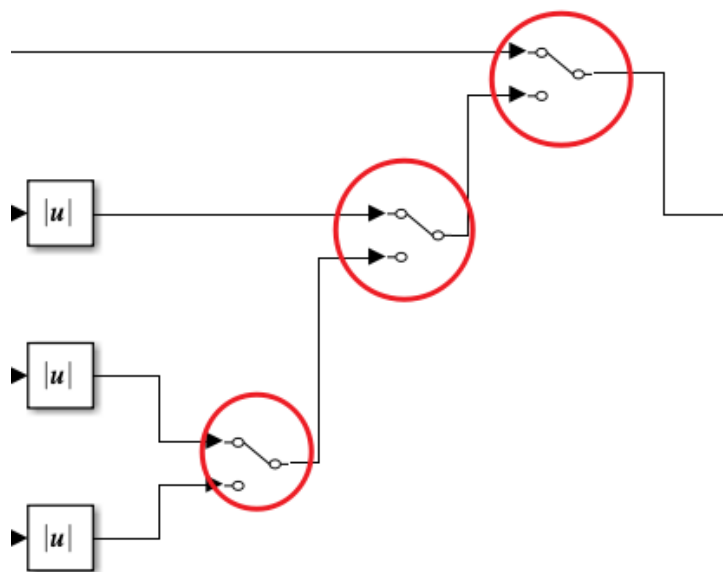


图 12-11 通过开关选择要播放的音乐

12.4 总结

在本章当中，我们完成了 FM 的信号复用，使得单个载波能够同时承载 4 个消息信号，并且能够在 ANTSDR 上完成发送这个信号，与此同时也能够使用 ANTSDR 和 Simulink 一起来配合，进行解调这个信号。

第13章 数字通信基础

在前面的实验当中，我们接触了 AM 调制和 FM 调制，这两种调制方法都是模拟调制方法，在现代通信当中，数字通信应用更加广泛，常见的蜂窝电话系统，WiFi，蓝牙等都是依靠数字通信发展起来的技术。在本章当中我们简单的介绍数字调制，并以 QPSK 为例，来介绍数字通信当中的调制，解调，符号和同步等概念。

13.1 数字调制

数字通信系统通过将二进制数据 bit 转换成符号来承载信息，将 bit 转换成符号的方法，根据所选择的调制方式来确定。

在本章当中我们会介绍两种调制方法，分别是以相位调制和幅度调制为基础，这两种调制方式的都按照 I 分量 Q 分量来确定，I 分量和 Q 分量也对应着复数中的实部和虚部。

其他的调制可能会同时采取幅度和相位来进行调制，根据时间的不同来改变 IQ 的值，比如 FSK 和 OFDM。

13.1.1 数字相位调制 PSK

一些典型的相位调制的方式有：

BPSK：每个符号表示一个 bit

QPSK：每个符号表示两个 bit

8PSK：每个符号表示三个 bit

在上面的这三种调制方式中，每个符号都会在星座图中对应一个点。下面的几幅图，分别表示了这三种调制方式的星座图。星座图一般都是对称的。

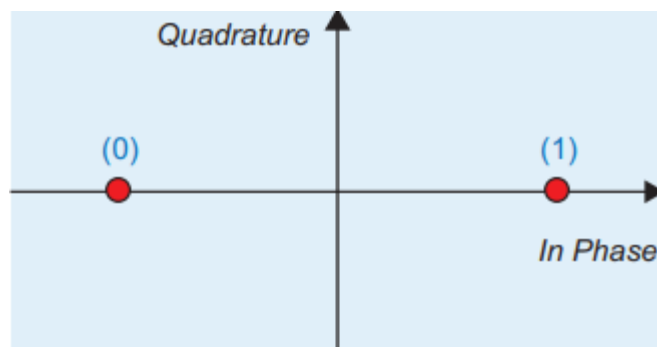


图 13-1 BPSK 星座图

BPSK 每个符号表示一个 bit，所以我们能够在其星座图上看到有两个点，并且每个点上表示的数据就是 1bit 的数据。

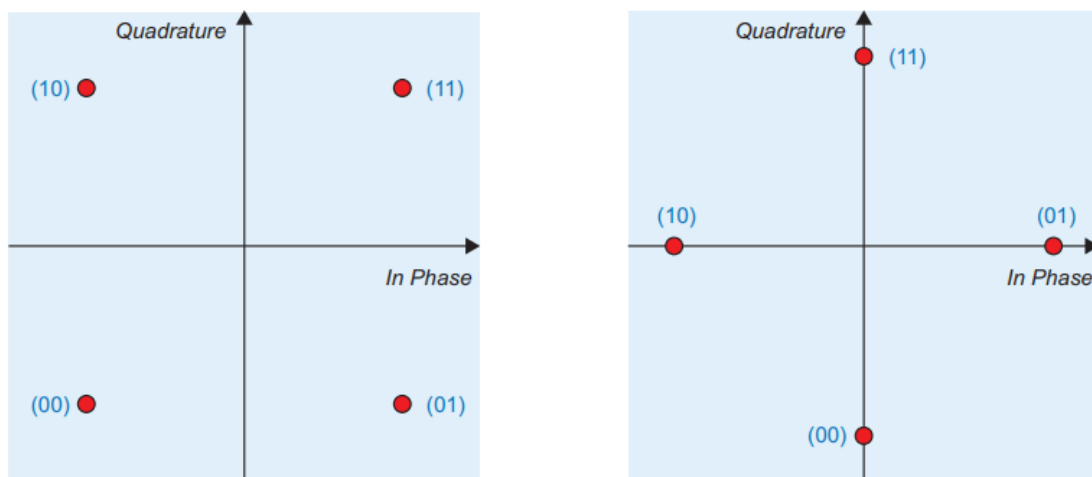


图 13-2 QPSK 星座图

QPSK 每个符号表示 2 个 bit，所以我们在上面的 QPSK 的星座图中可以看到，每个点上是用两 bit 来表示的。QPSK 的星座图上，没两个点之间的相位是固定的为 90° ，并且每个点原点的距离是一样的。

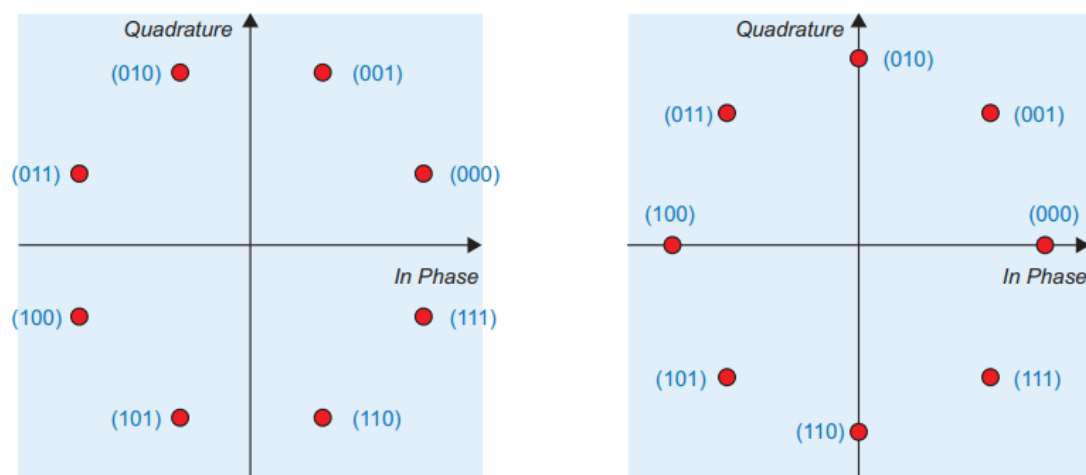


图 13-3 8PSK 星座图

在上面的 8PSK 的星座图当中，每个点可以表示 3bit，并且每个星座点之间的相位差为 45° ，且每个星座点到远点的距离是一样的。

还需要注意的是，上面的这些星座图中的点和实际对应的值可能不是一样的，一般开说星座图上的点都是遵循格雷码，也就是两个相邻星座之间只有一个 bit 不同。

13.1.2 数字幅度调制

最流行的数字幅度调制方式有以下几种：

4QAM：每个符号表示 2 个 bit

16QAM：每个符号表示 4 个 bit

64QAM：每个符号表示 6 个 bit

幅度调制一般会使星座图看起来像一个矩形，每个星座点也是采用和 PSK 相似的方式来表示一个点。

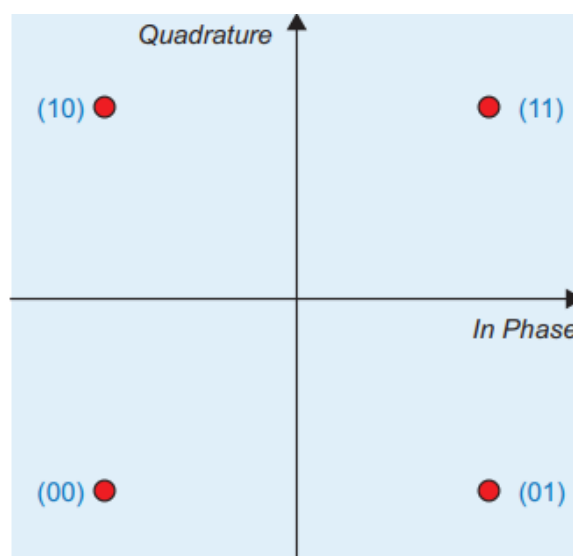


图 13-4 4QAM 星座图

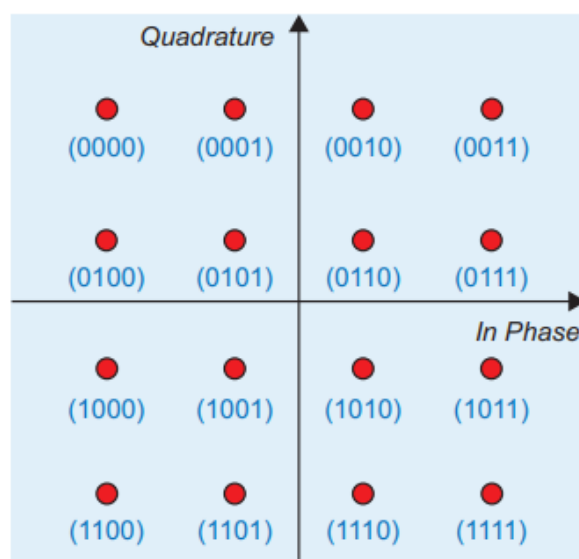


图 13-5 16QAM 星座图

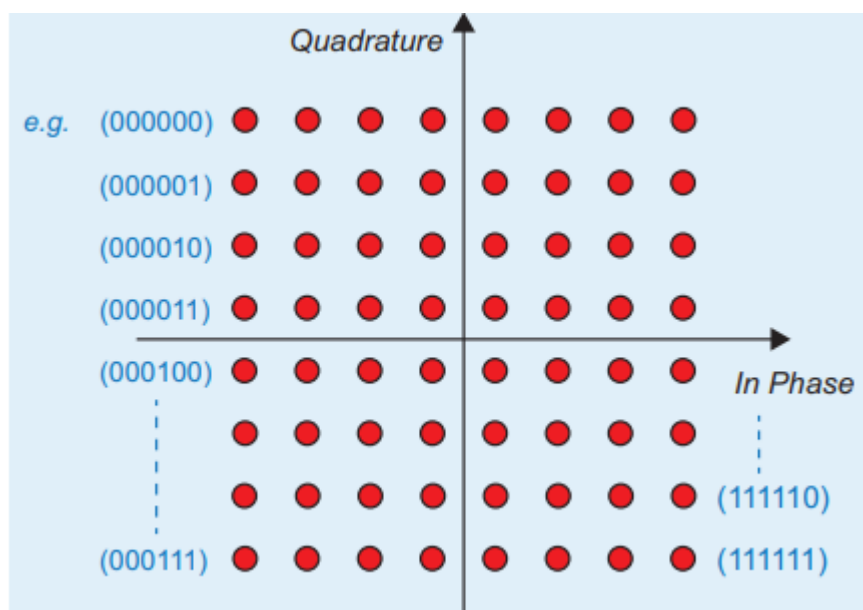


图 13-6 64QAM 星座图

在上面的星座图当中，我们可以看到 4QAM 的星座图和 QPSK 的一种星座图是相同的，虽然这两种是不同的调制方式，一个是相位调制，一个幅度调制。

对于 QAM 星座图，每个符号由 I 分量和 Q 分量组成的，用来表示在这个分量上的幅度。

13.1.3 Bit 与符号的映射与反射射

Bit 与符号的映射关系取决与调制的方式，可以是 BPSK 当中每个 bit 对应一个符号，也可以是 64QAM 当中，每 6 个 bit 对应一个符号。假设符号速率是相同的，那么 64QAM 的比特速率就是 BPSK 的比特速率的 6 倍。这样能够明显地提升传输的带宽。

Bit 与符号的映射关系可以看作是将独立的二进制数据组成一组二进制 bit 数据，然后使用使用所选的调制方式将这些数据发送出去。反映射的过程与映射正好相反，需要根据接收到的数据进行采样之后进行判决从而将符号反映射成对应的 bit。

由于信道的噪声的影响和定时误差的影响，一般接收到的数据都会有一定的错误。

符号的反映射可以将符号和对应的判决边界之间进行比较比较来实现，确定采样到的点和哪一个星座点更加接近，从而进行反映射。

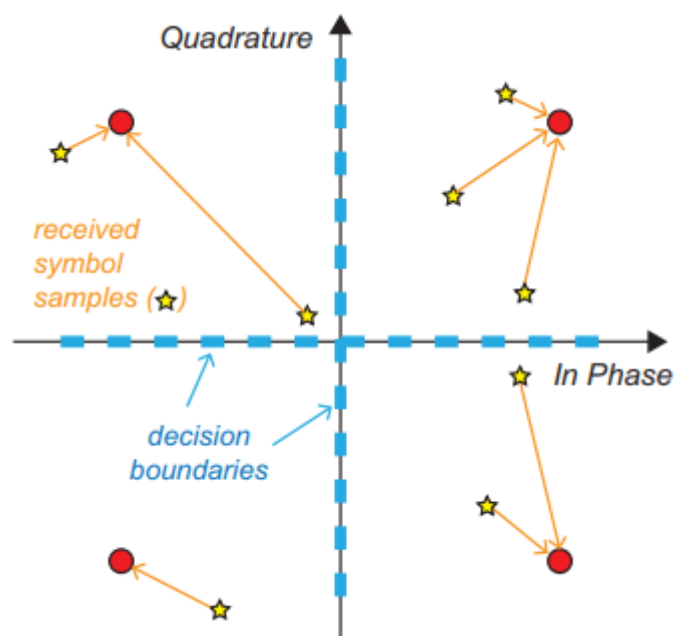


图 13-7 QPSK 边界与反射

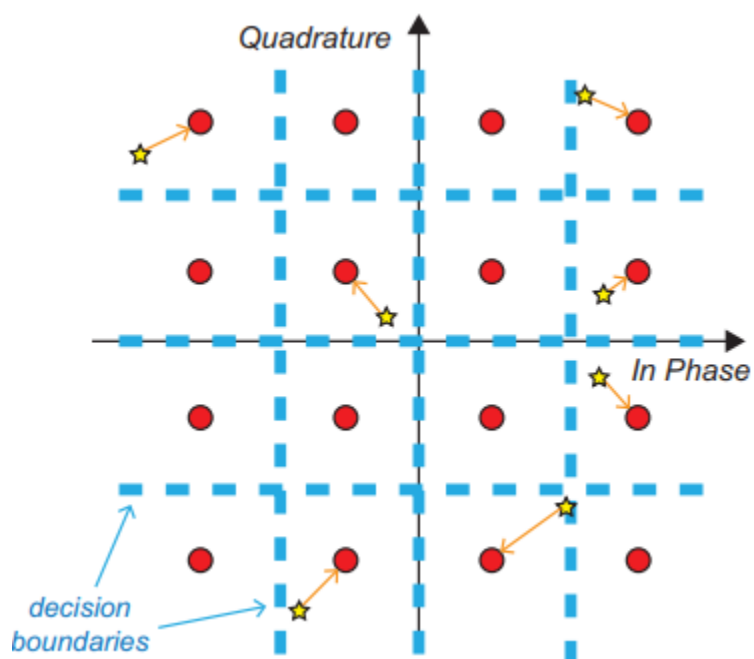


图 13-8 16QAM 判决边界

从上面的两种调制方式上可以看出，越是高阶的调制方式，其判决条件就越苛刻，收到信道的影响也就越大，出错的概率也就越大。

13.2 符号映射与反映射仿真

13.2.1 QPSK 映射与反映射

为了能够更加直观地展示出数字调制中符号映射与反映射的过程，我们可以使用 Simulink 工具来实现数字调制当中映射与反映射的仿真。

打开仿真模型 `qpsk_map_demap.slx`，仿真模型的结构如下：

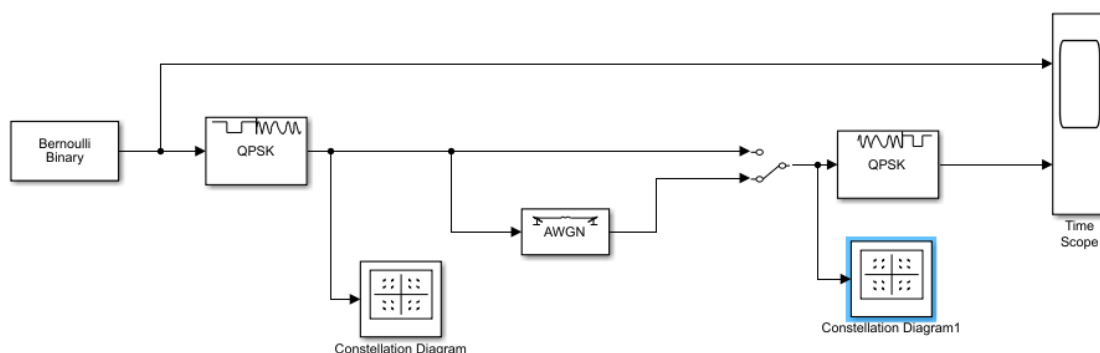


图 13-9

其中数据源模块是一个产生随机数的模块：

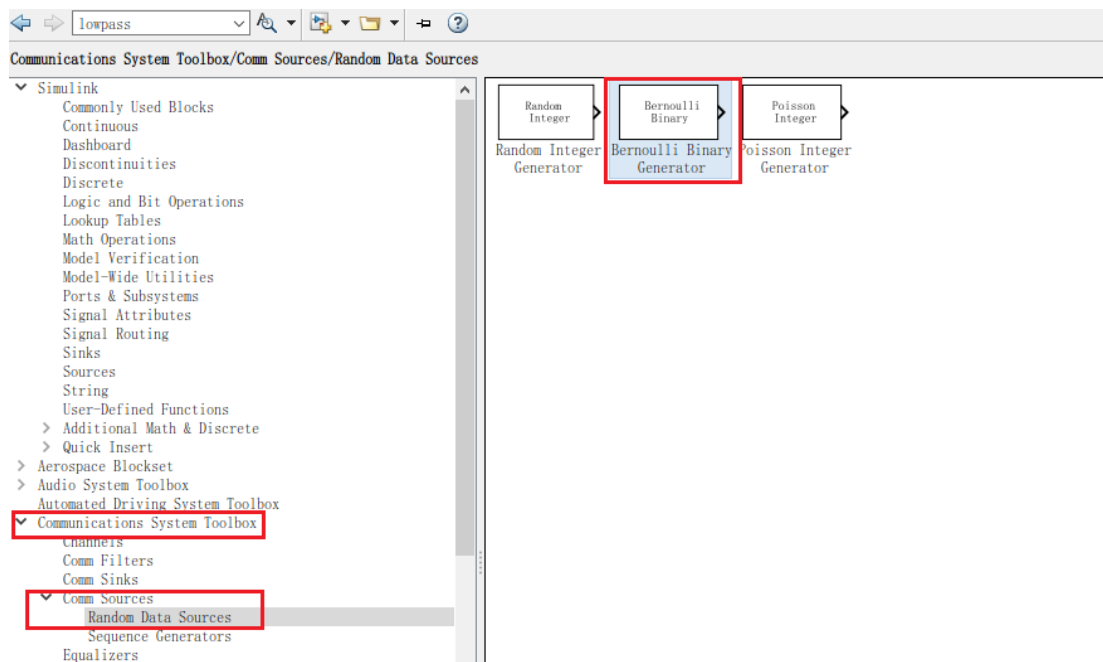


图 13-10 产生随机数的模块

这个随机数产生模块的设置如下：

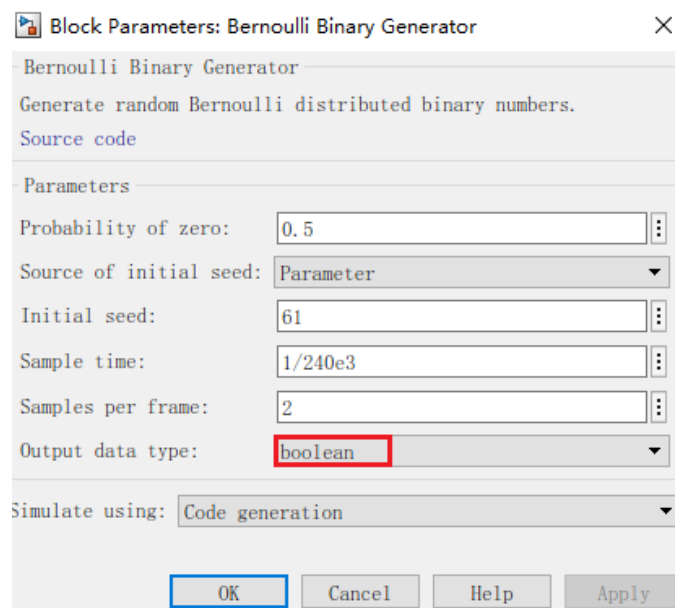


图 13-11 随机数产生模块的设置

其中需要设置采样周期，这个能够确定产生的数据的快慢，然后需要设置输出的数据类型是 Boolean 型的，这样得到的数据就是 0，1 的 bit，然后选择每一帧中有两个采样点，这刚好对应上了 QPSK 调制所需要的两个采样点。

准备好数据源之后，我们还需要使用一个 QPSK 的调制模块：

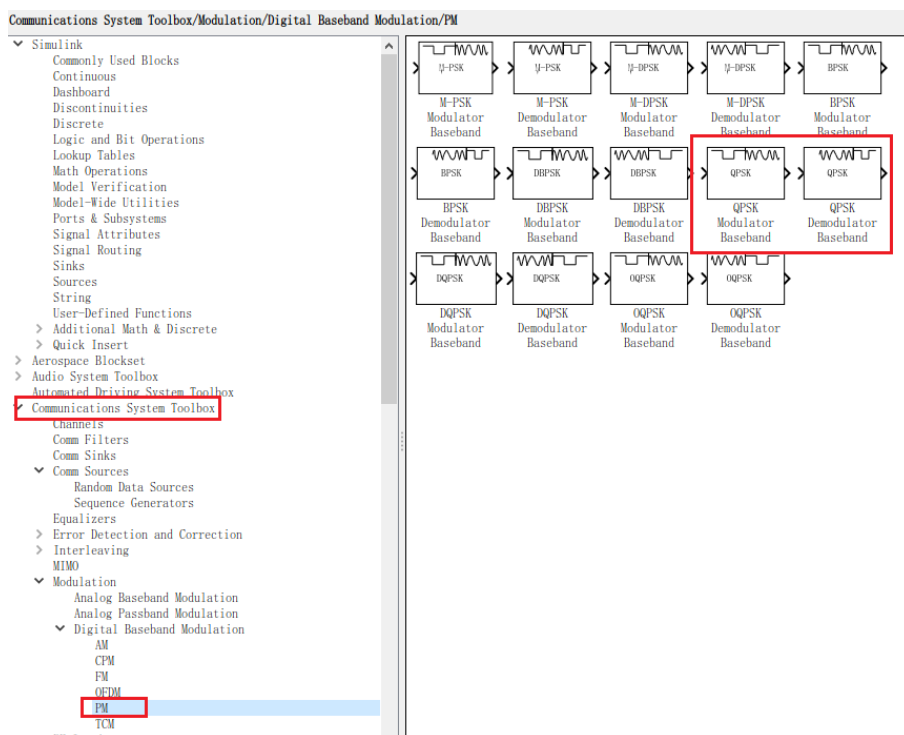


图 13-12 QPSK 调制和解调模块

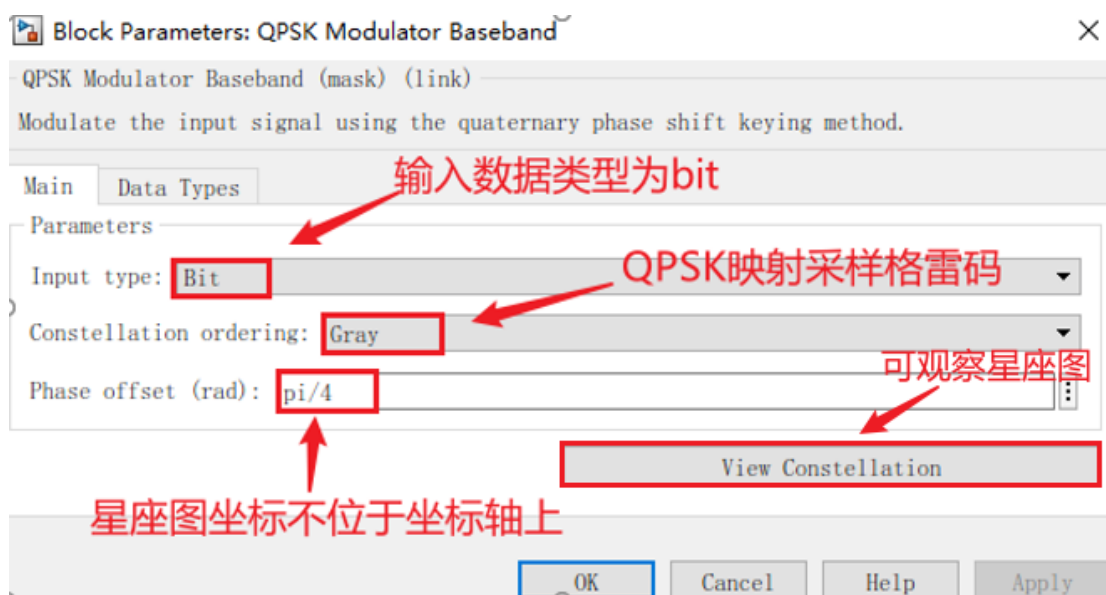


图 13-13 QPSK 调制模块

该模块能够对输入的 bit 数据进行映射，我们可以点击 view Constellation 来观察 QPSK 调制之后的星座图。

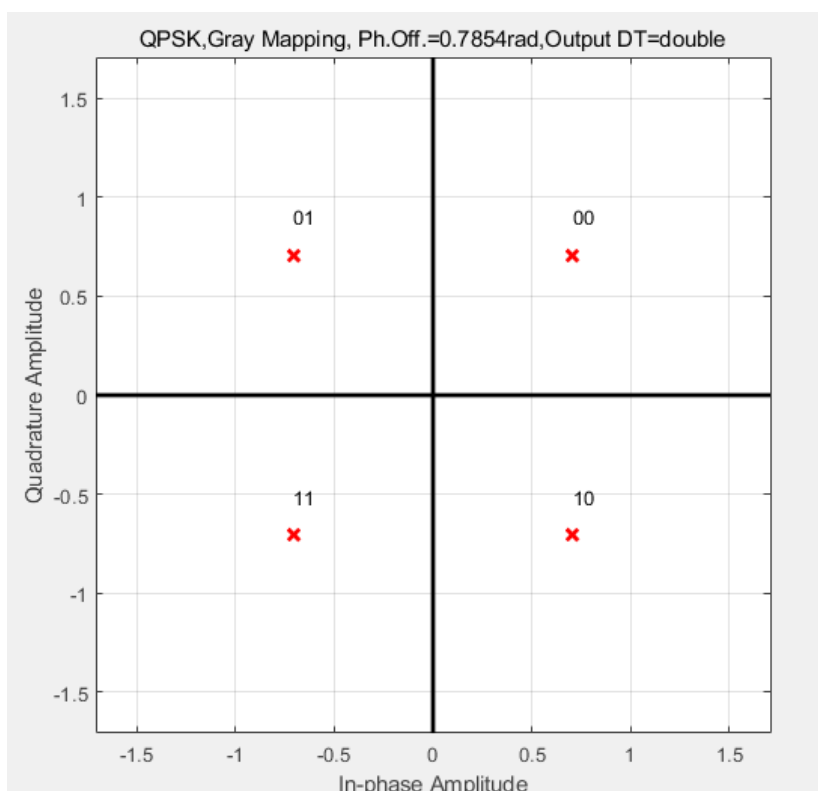


图 13-14 QPSK 调制星座图

可以看到经过格雷编码之后的星座图如图 13-14 所示, 相邻两个符号之间的编码只有一位不同。

经过 QPSK 映射之后, 我们就可以来观察星座图了, 在这里需要添加一个用于观察星座图的模块。

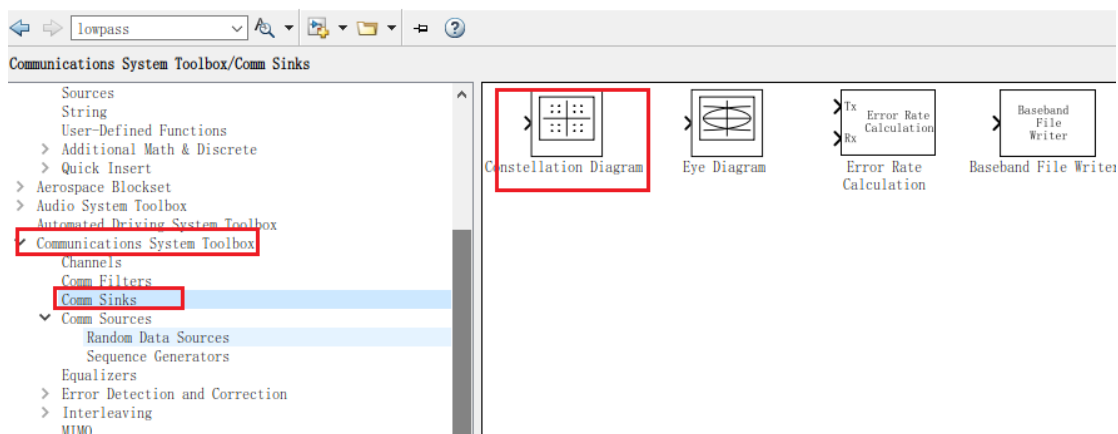


图 13-15 观察星座图

双击这个模块对其进行设置, 首先需要设置要显示的符号数量, 显示的符号数量这里设置为 4096, 便于我们观察符号映射的结果。

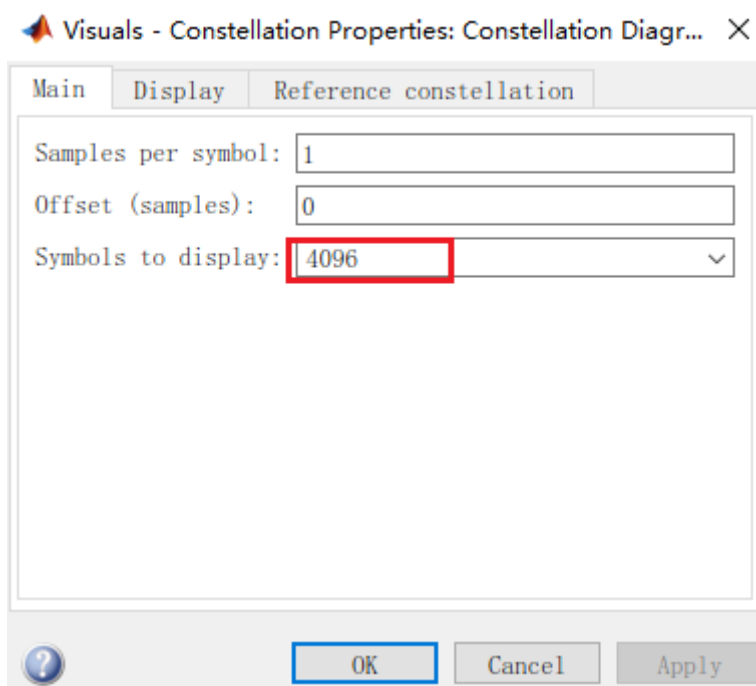


图 13-16 设置观察的符号数量

然后还需要设置星座图的参考类型，在这里选择 QPSK，这样我们之后观察的星座图的参考点就是以 QPSK 的星座图来进行定位的。

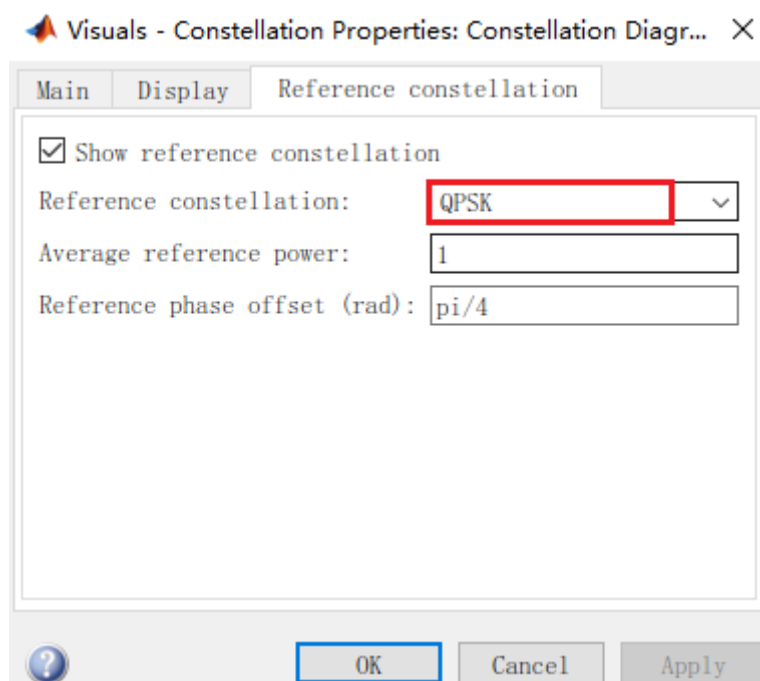


图 13-17 设置星座图参考为 QPSK

点击运行仿真，可以来观察一下映射之后的结果：

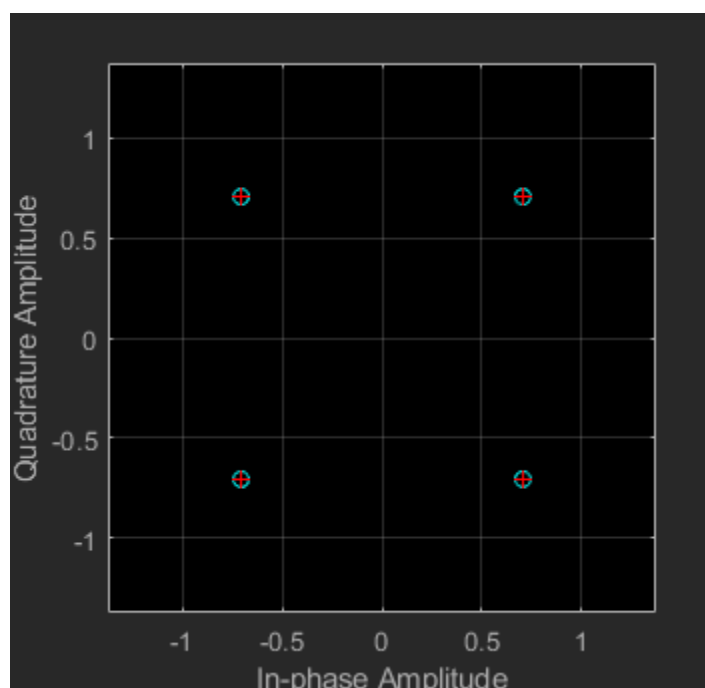


图 13-18 映射之后的星座图

可以看到，经过映射之后，这些发送的 QPSK 的符号全都能够对应到星座图上的一个点。

这时候我们观察 QPSK 反映射之后的结果与未经映射时候的数据源的对比：

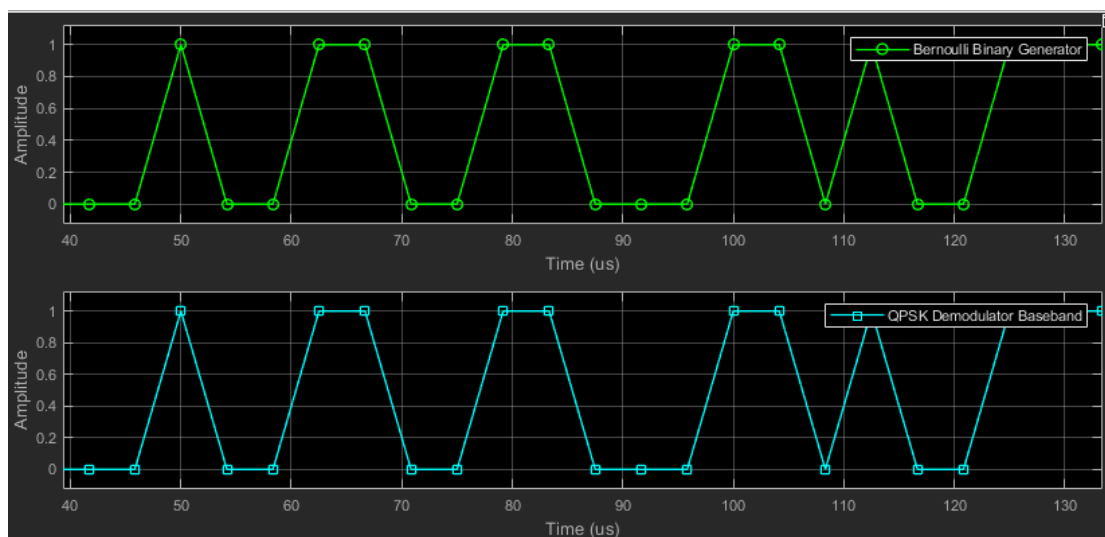


图 13-19 映射与反映射的结果对比

然后我们可以查看一下信道噪声对 QPSK 映射与反映射的影响，首先我们需要添加一个 AWGN 信道：

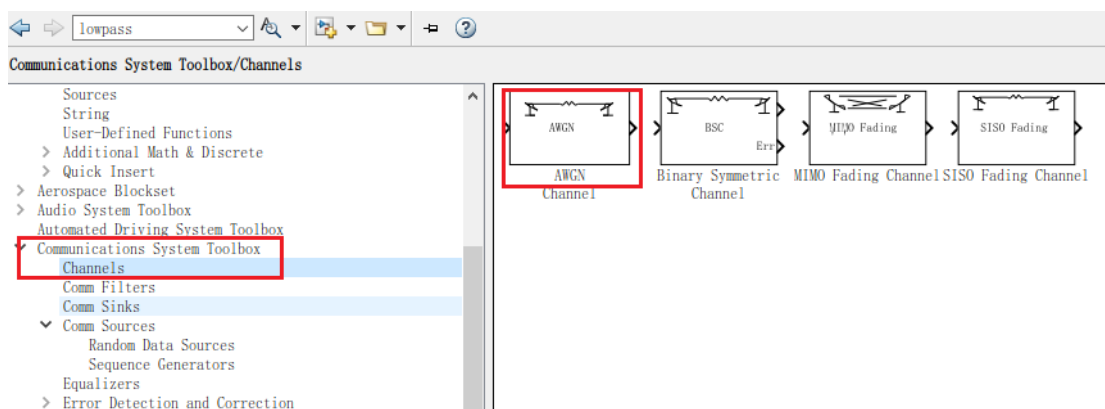


图 13-20 AWGN 信道模块

添加这个 AWGN 信道之后，我们可以对这个模块进行配置，首先设置一个大的信噪比 20dB，然后需要设置每个符号中含有的 bit 数为 2，因为我们采取的是 QPSK 调制，每个符号对应两个 bit。然后还需要设置符号周期，前面在数据源模块当中，产生 bit 的速率是 240KHz，两个 bit 组成一个符号，因此符号速率是 120KHz。

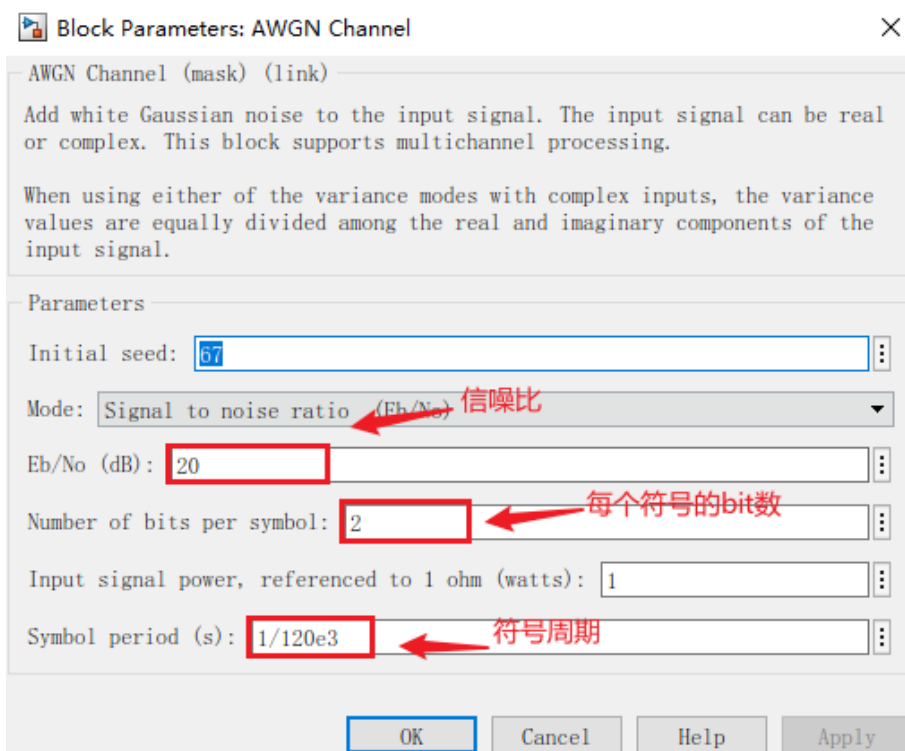


图 13-21 AWGN 信道设置

设置好 AWGN 信道之后，将开关选择到经过 AWGN 信道之后的结果，然后运行仿真：

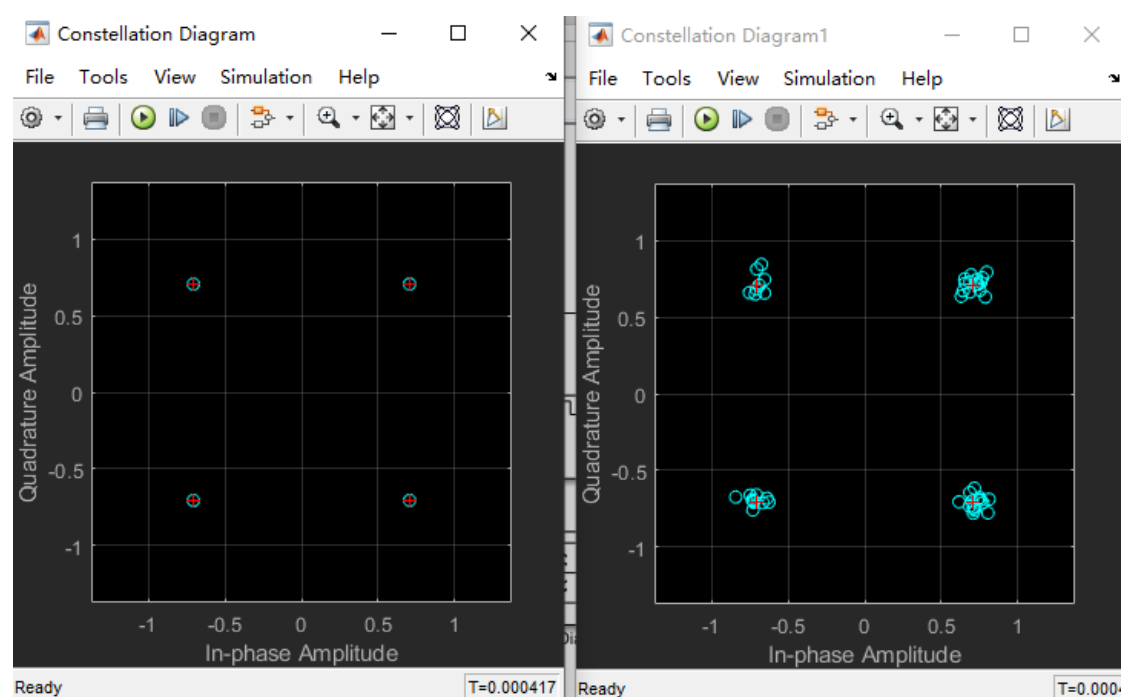


图 13-22 星座图对比

可以看到，经过一个 **awgn** 信道之后，在接收端的符号映射相较于发送端的符号映射而言出现了一些误差，这是由于 **AWGN** 信道引入了噪声，但是当前还是可以成功地进行反映射的。

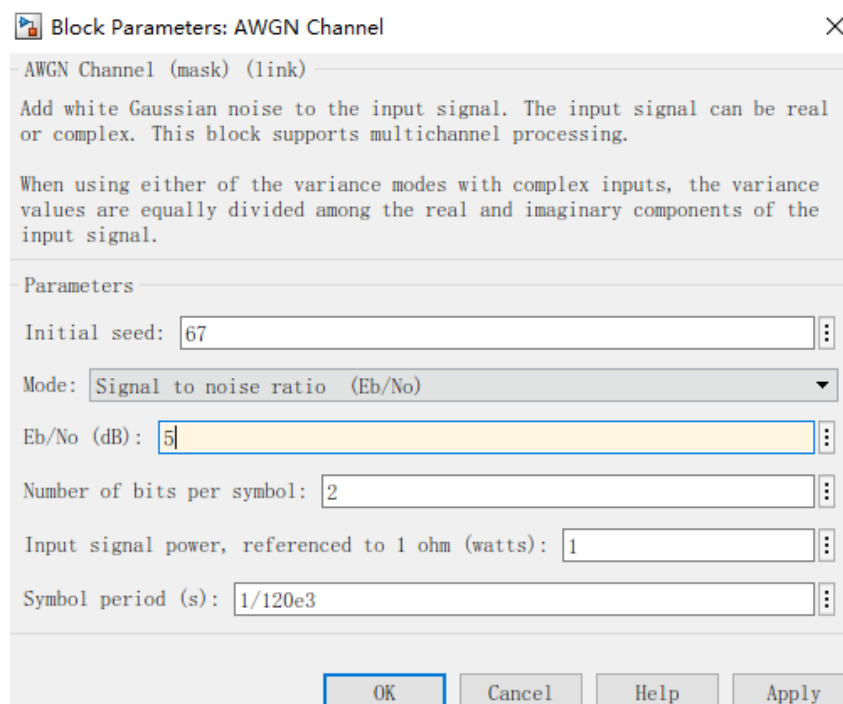


图 13-23 小信噪比 AWGN 信道

如果我们将 AWGN 信道的信噪比降低，然后再次运行仿真，可以看到在接收端的星座映射，比标准的结果偏离得更大。

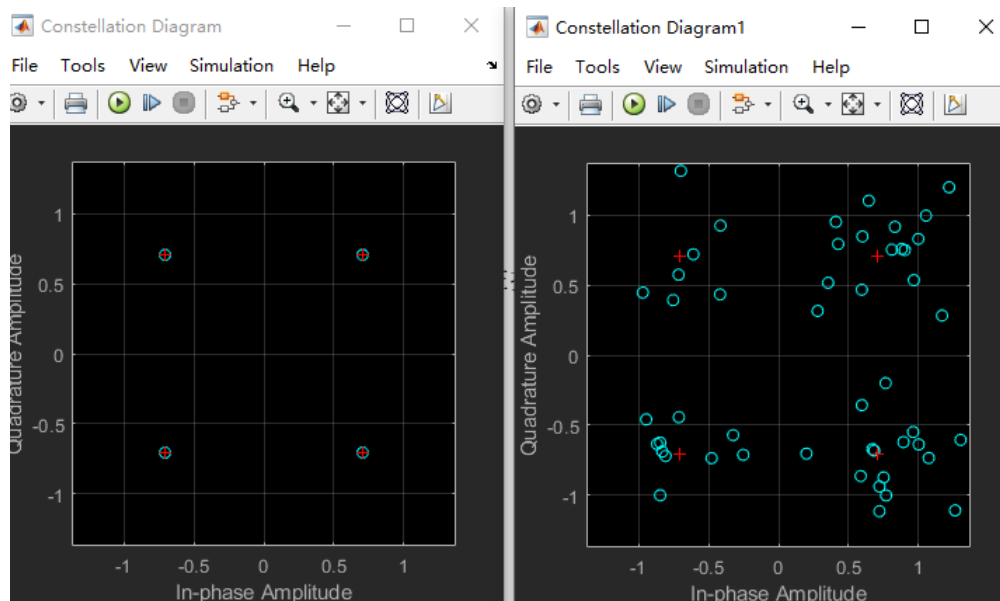


图 13-24 小信噪比的星座图对比

因此在经过一个大信噪比的信道的时候，想要恢复出原始的数据 bit 会需要进行其他的操作，这些操作在之后会介绍到。

13.2.2 16QAM 的映射与反射射

接下来我们来举一个数字幅度调制的映射与反射射，观察一下 16QAM 的星座图。打开仿真模型，QAM16_map_demap.slx。

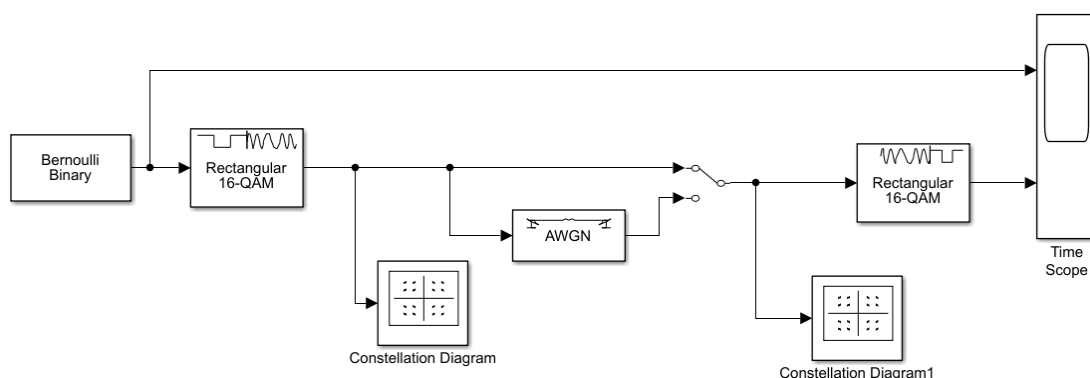


图 13-25 QAM16 仿真模型

首先，还是需要有一个数据源模块，在这里我们还是使用产生随机数的这个模块，其设置如下：

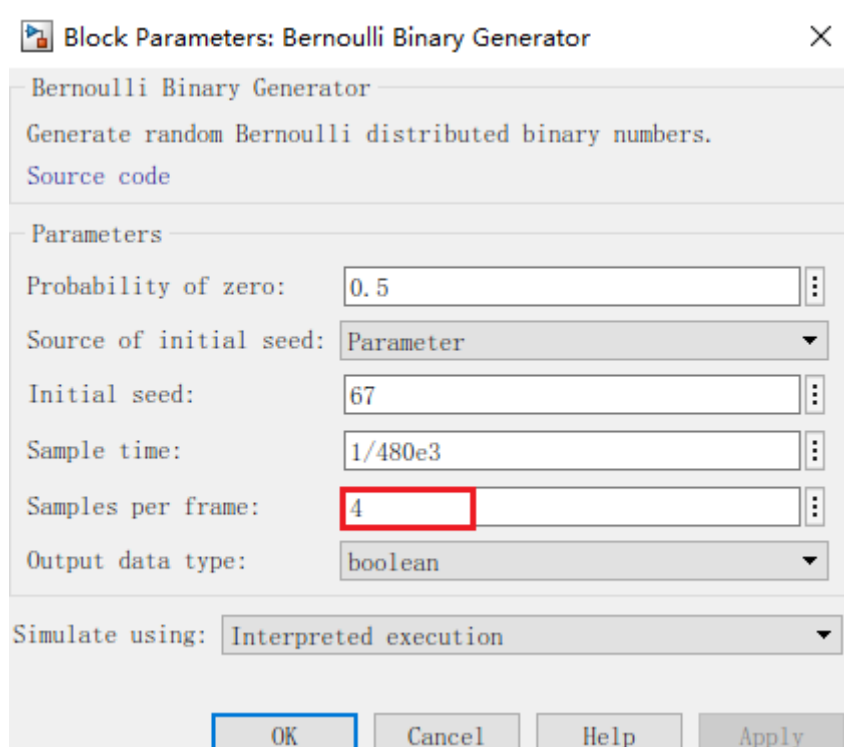


图 13-26 随机数生成模块

在随机数生成模块当中，设置每一帧当中有 4 个采样点，刚好和 16QAM 的一个符号对应起来。

在这之后需要进行 16QAM 的映射，因此我们需要添加一个 QAM16 的调制和解调模块。

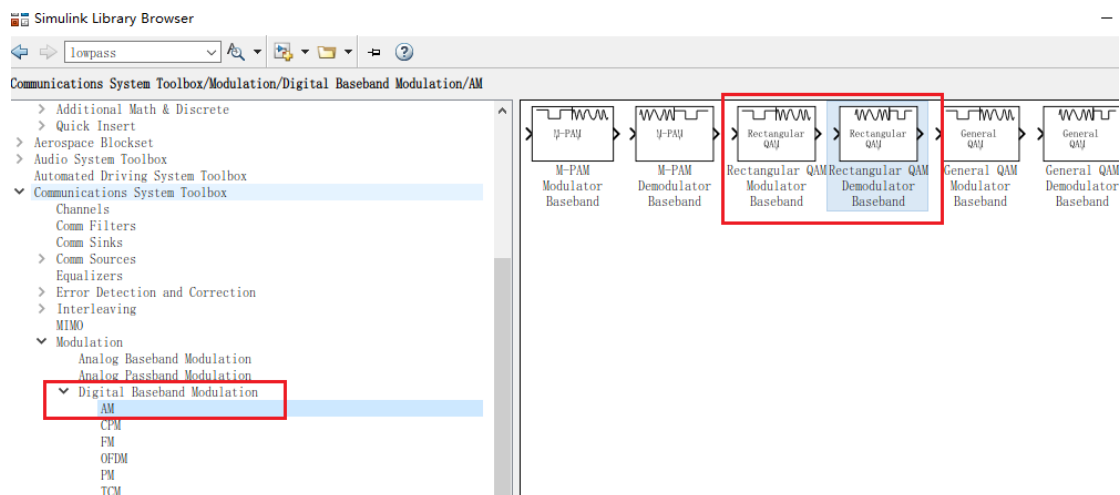


图 13-27 16QAM 调制和解调模块

添加之后进行如下设置：

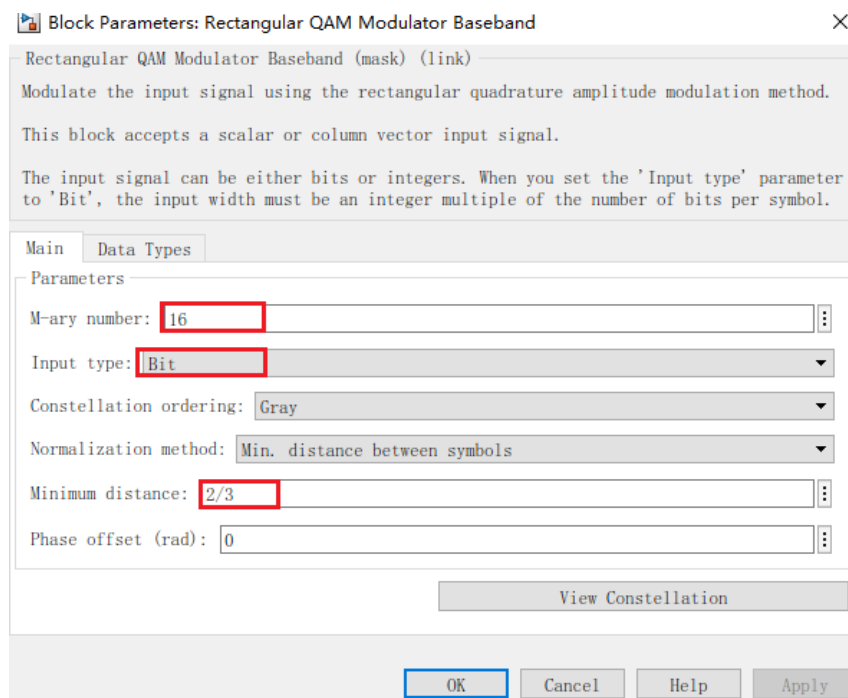


图 13-28 16QAM 调制设置

然后再添加星座图模块来观察星座图，其设置如下：

需要设置其参考点如下：

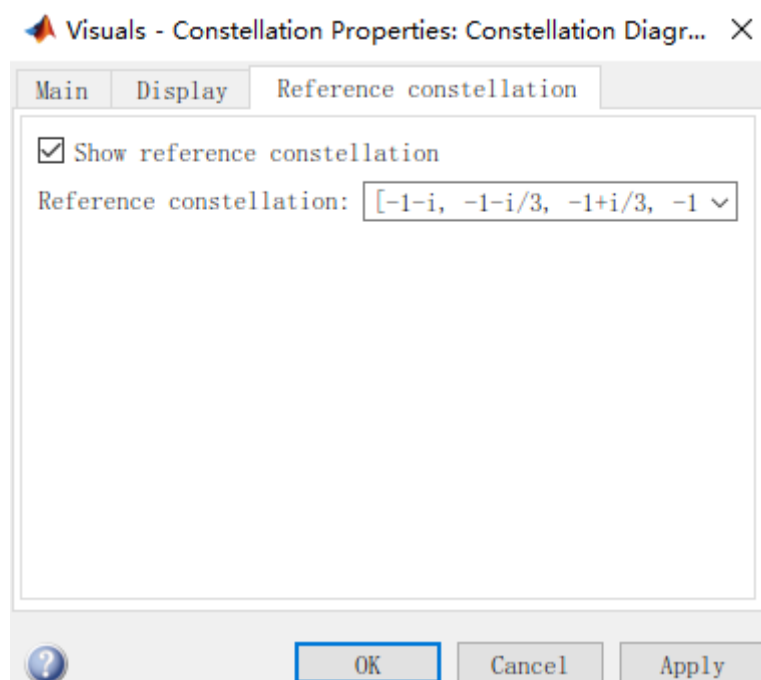


图 13-29 设置星座图的参考点

$[-1-i, -1-i/3, -1+i/3, -1-i/3, -1/3-i, -1/3-i/3, -1/3+i/3, -1/3+i, 1/3-i, 1/3-i/3, 1/3+i/3, 1/3+i, 1-i, 1-i/3, 1+i/3, 1+i]$

然后进行 16QAM 的反映射，添加反映射模块，并对其进行设置，其设置方法需要和映射模块继续对应。

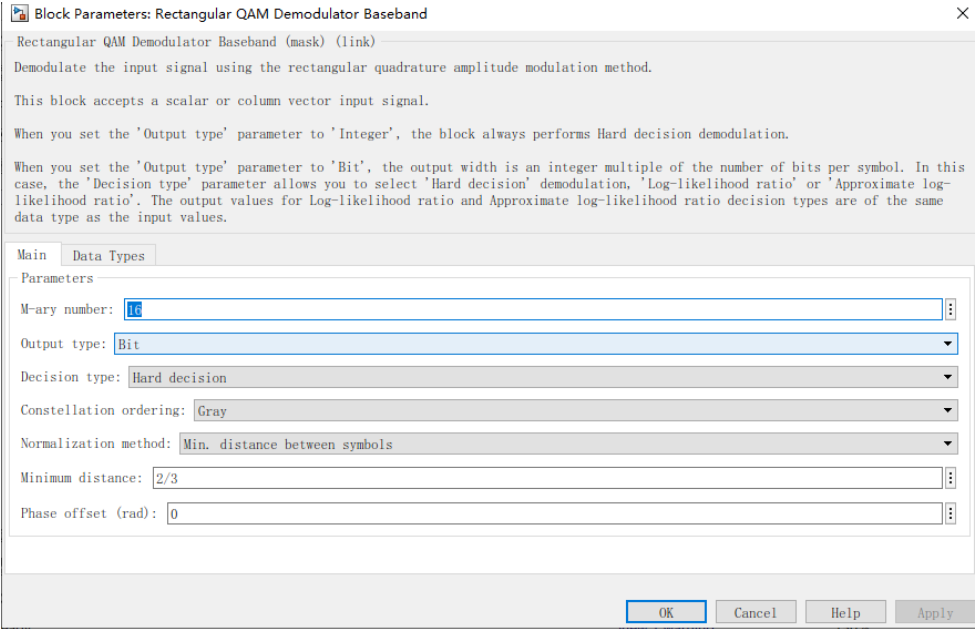


图 13-30 反映射模块设置

设置完成之后点击运行仿真模型，就可以来观察经过 16QAM 映射之后的星座图了。

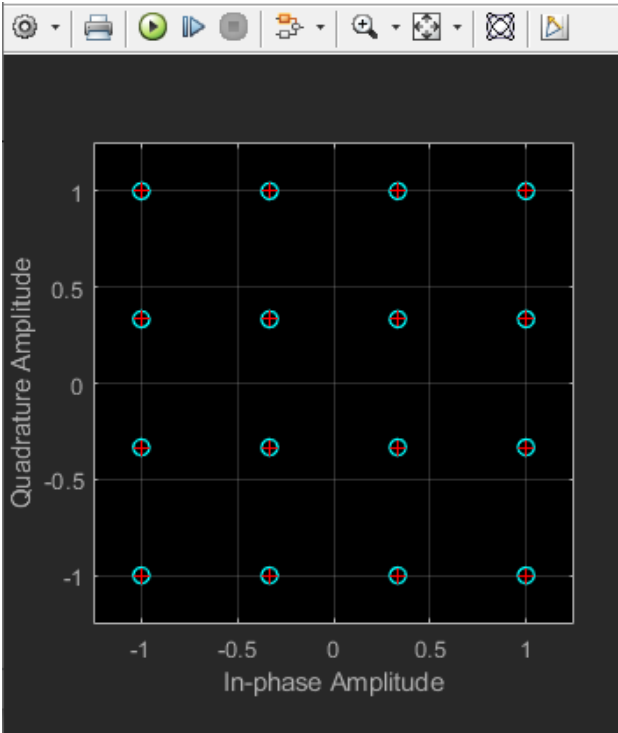


图 13-31 16QAM 星座图映射

从星座图中可以看到，16QAM 的星座图呈一个矩形，经过映射之后的符号对应到星座图上的一个点。

然后我们可以看一下经过反映射之后得到结果和原数据是否对应，观察 time scop 模块

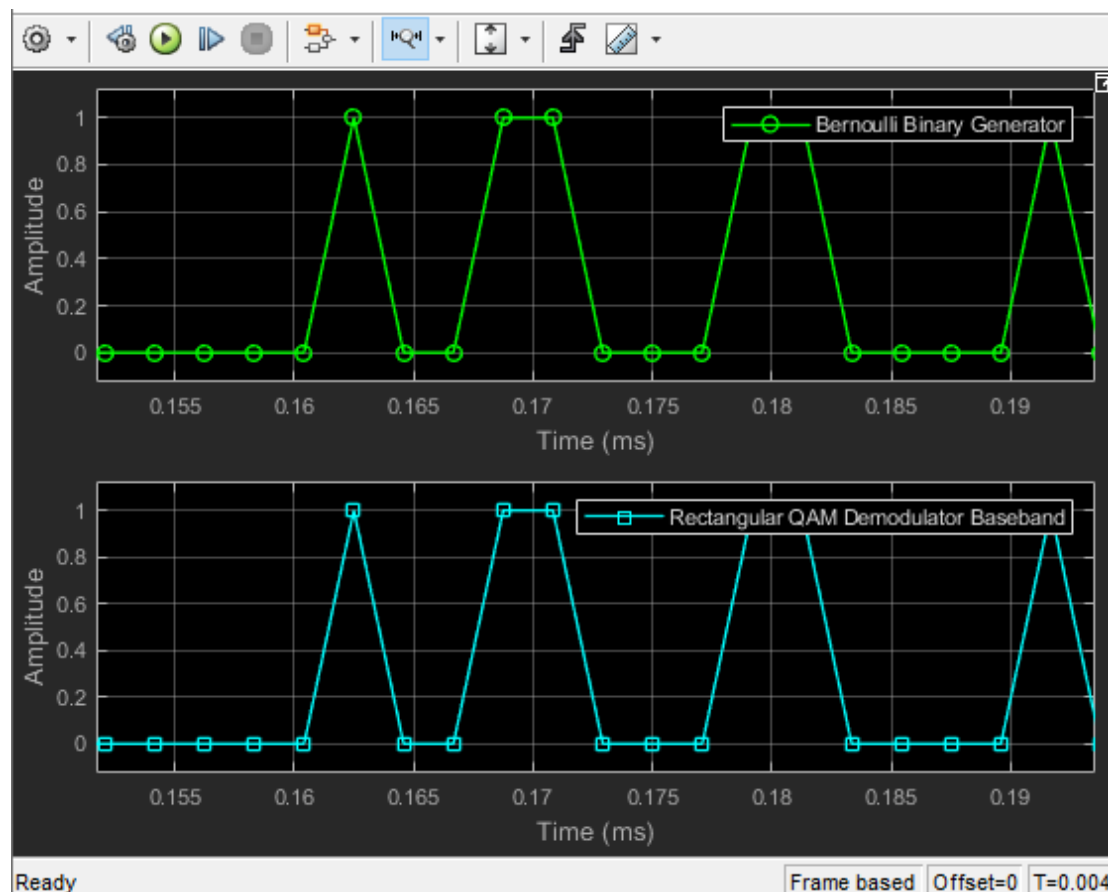


图 13-32 映射与反映射的结果

然后我们可以看一下噪声对 16QAM 映射与反映射的影响，将 switch 调整到使用 AWGN 信道，然后再次运行仿真。

可以看到，信号经过了 awgn 信道之后，对接收到的符号产生了影响从而使接收到的符号相较于标准参考点发生了偏移，但总体来说都还是该符号原本所在的位置上。

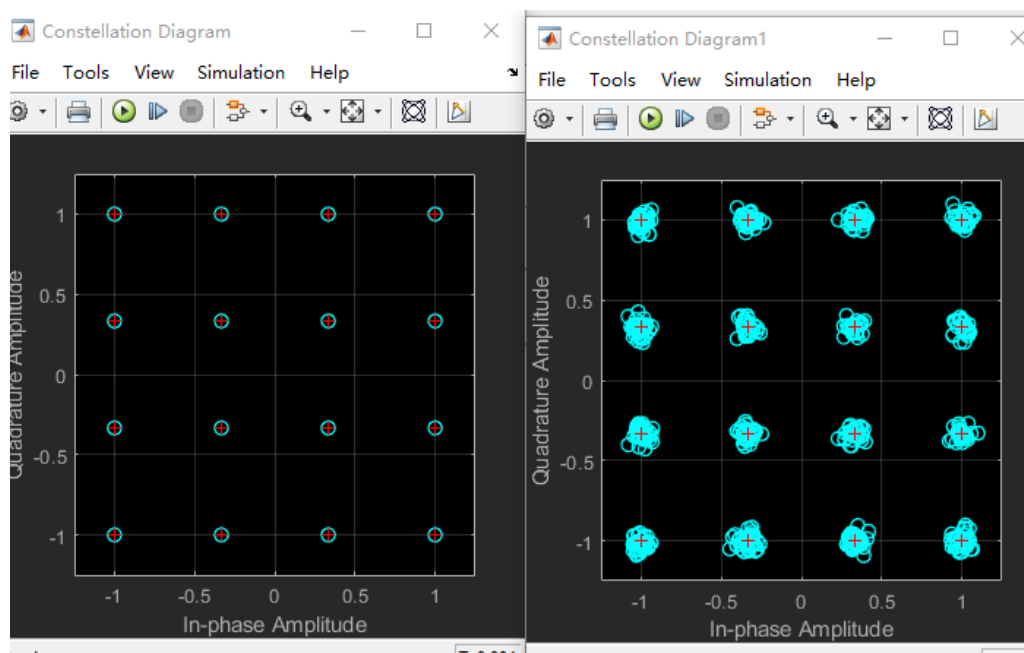


图 13-33 加入了噪声的星座图

13.3 脉冲成型

在进行无线信号的传输的时候，受限与各地的频谱法规，频谱的资源是十分紧缺的，因此在进行无线信号传输的时候，都需要对发射的信号进行严格的带宽控制，使得信号可以满足频谱遮罩的需求。

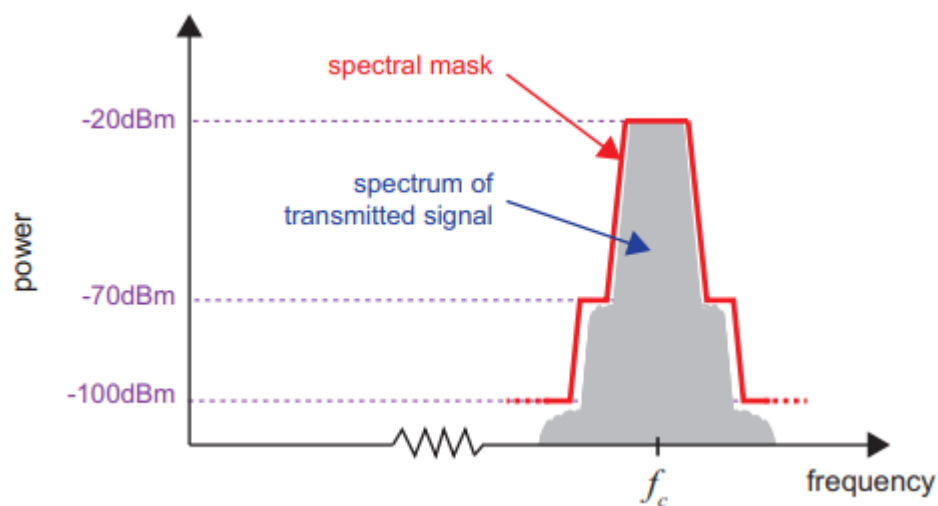


图 13-34 限制信号带宽

在进行对频谱带宽的限制的时候，还要避免引入符号间干扰 ISI，使得连续的符号之间不会相互干扰。常用的整形的滤波器是升余弦滤波器，升余弦滤波器可以实现零符号间串扰。尽管符号会延申到周围的好几个周期当中，但在对应符号

的位置不会产生影响。

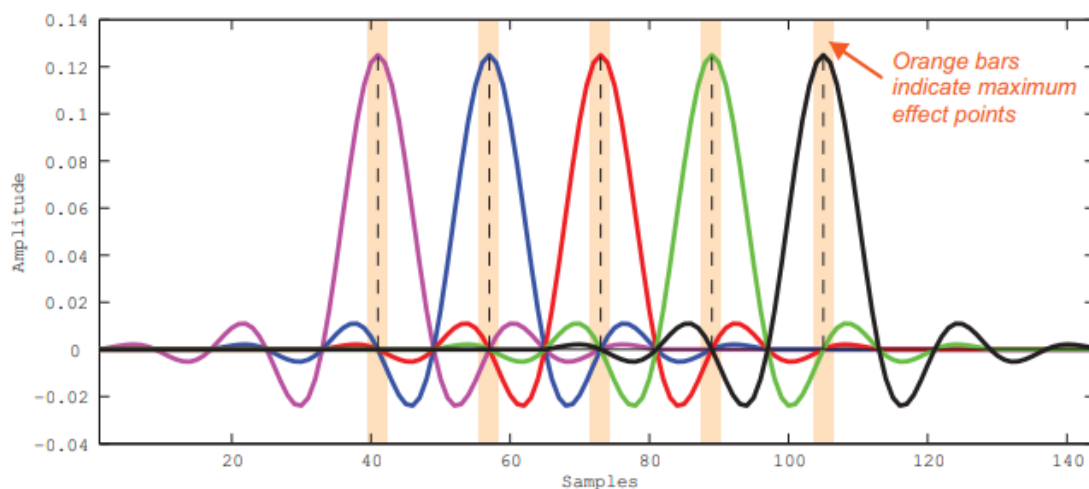


图 13-35 无符号间串扰的情况

通常情况下，升余弦滤波器不会只在一端来实现，一般需要在发射机和接收机当中都存在一个匹配滤波器来完成这个升余弦的整型过程，这个滤波器叫做根升余弦滤波器，也经常叫做匹配滤波器。

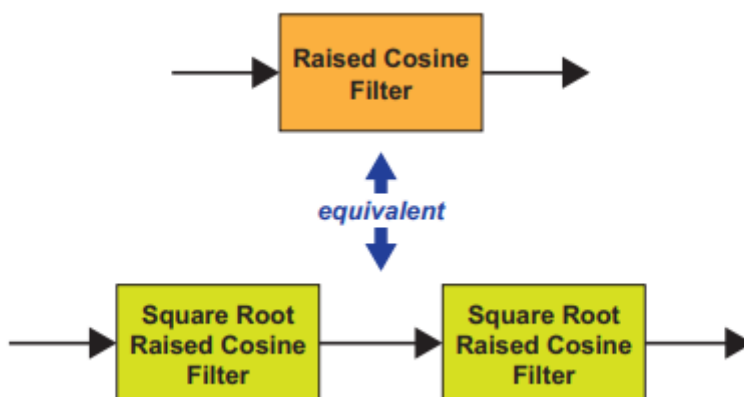


图 13-36 匹配滤波器

13.3.1 根升余弦滤波器的仿真

为了能够更直观地看出匹配滤波器的作用，我们可以使用 Simulink 来搭建一个仿真模型，看看匹配滤波器和常见的矩形脉冲成型滤波器之间的区别。打开仿真模型 filter_rrc.slx。

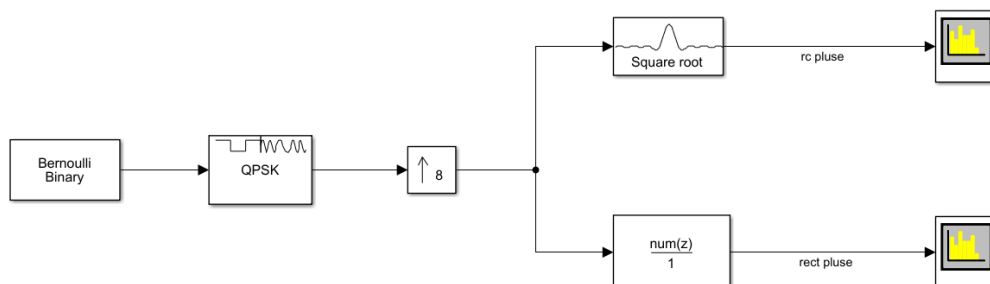


图 13-37 RC filter 与矩形脉冲成型

仿真模型如上图所示，其中我们来比较升余弦滤波器和普通的滤波器之间的区别，我们需要先添加一个用于发送的根升余弦滤波器。

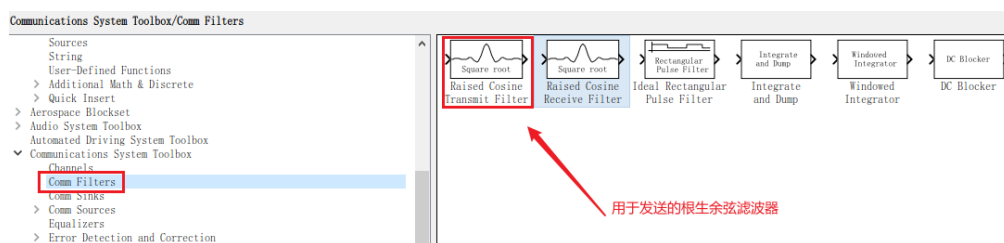


图 13-38 用于发送的根升余弦滤波器

根升余弦滤波器的设置如下：

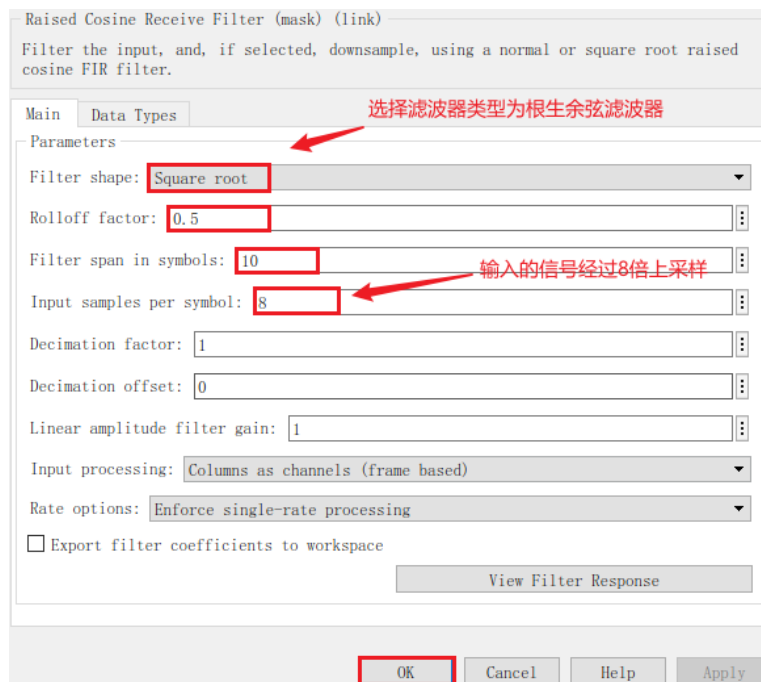


图 13-39 根升余弦滤波器设置

然后我们再添加一个矩形脉冲成型滤波器。

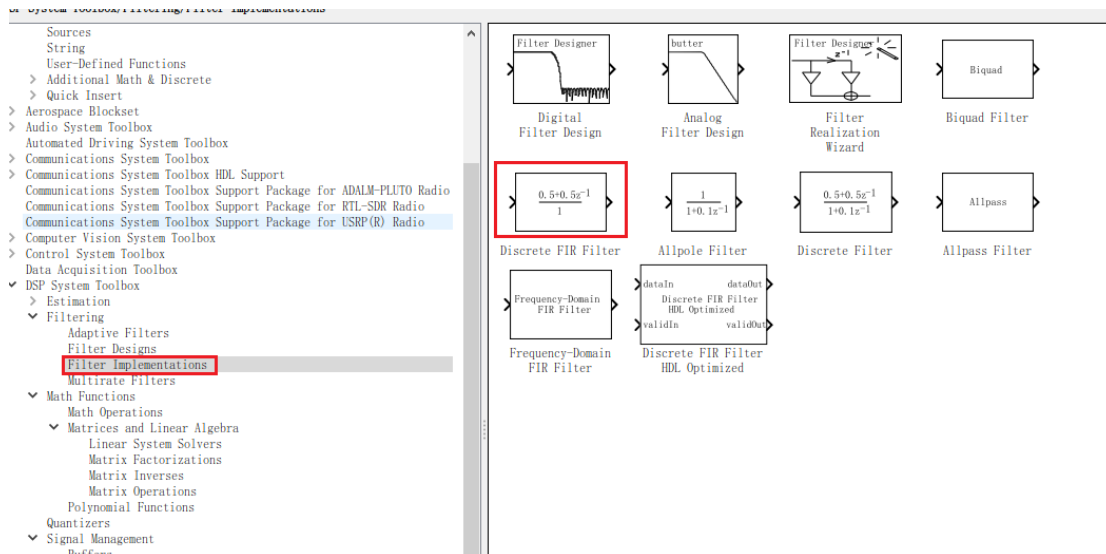


图 13-40 FIR 滤波器

然后我们设置这个 FIR 滤波器的系数：

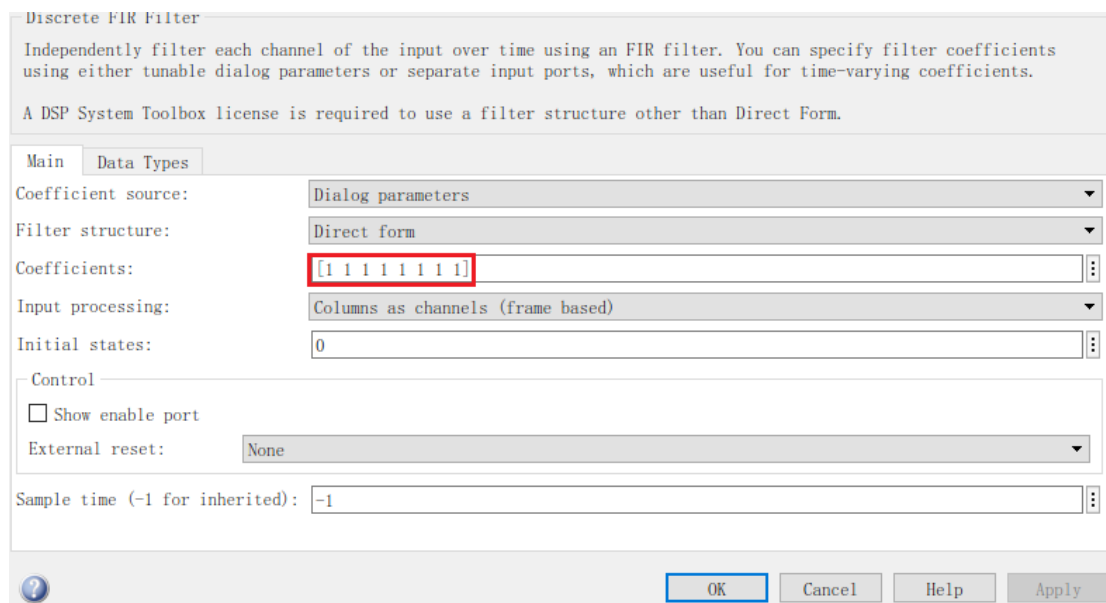


图 13-41 系数设置

点击仿真，可以来观察经过根升余弦滤波器和矩形脉冲成型滤波器之后的结果。

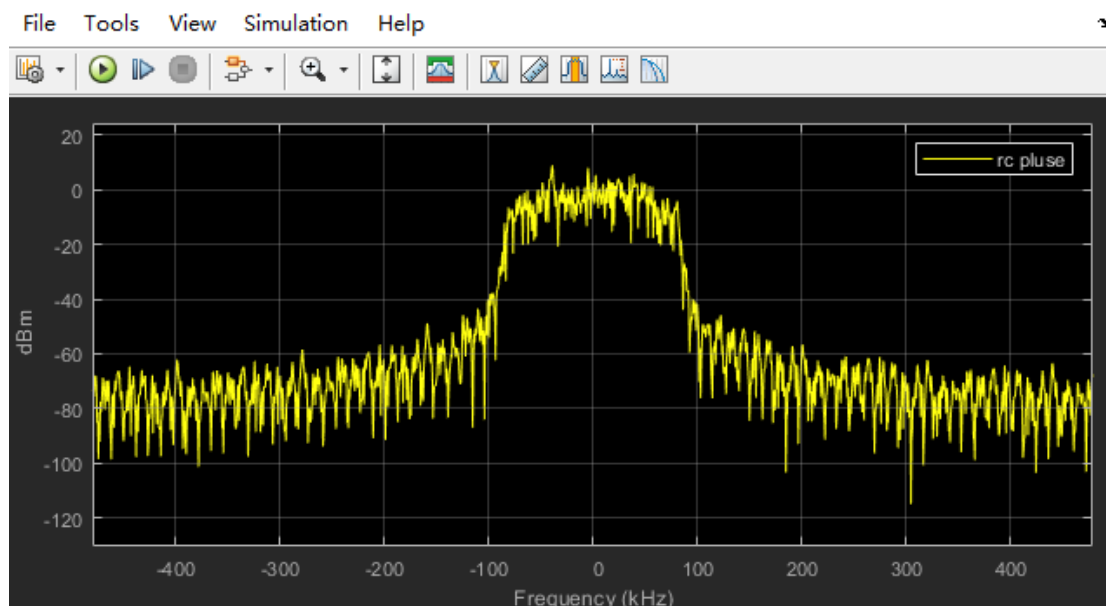


图 13-42 经过根升余弦后的信号

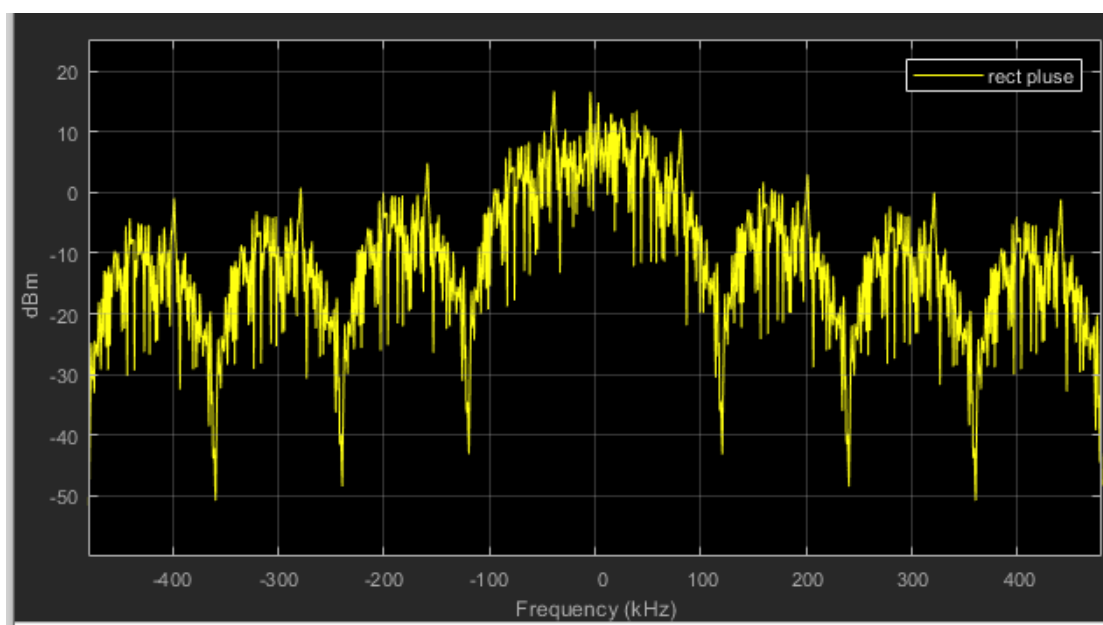


图 13-43 经过矩形脉冲成型之后的信号

可以看到，经过根升余弦滤波器之后的信号相较于经过矩形脉冲成型之后的信号，有更少的旁瓣，这样能够有效地减轻对邻近符号的影响。

13.3.2 根升余弦滤波器对符号之间的影响

前面我们介绍过，根升余弦滤波器滤波器一般需要再发射端和接收端来配合使用，接下来我们会再发送端和接收端各使用一个匹配滤波器来完成数据的收发

操作。

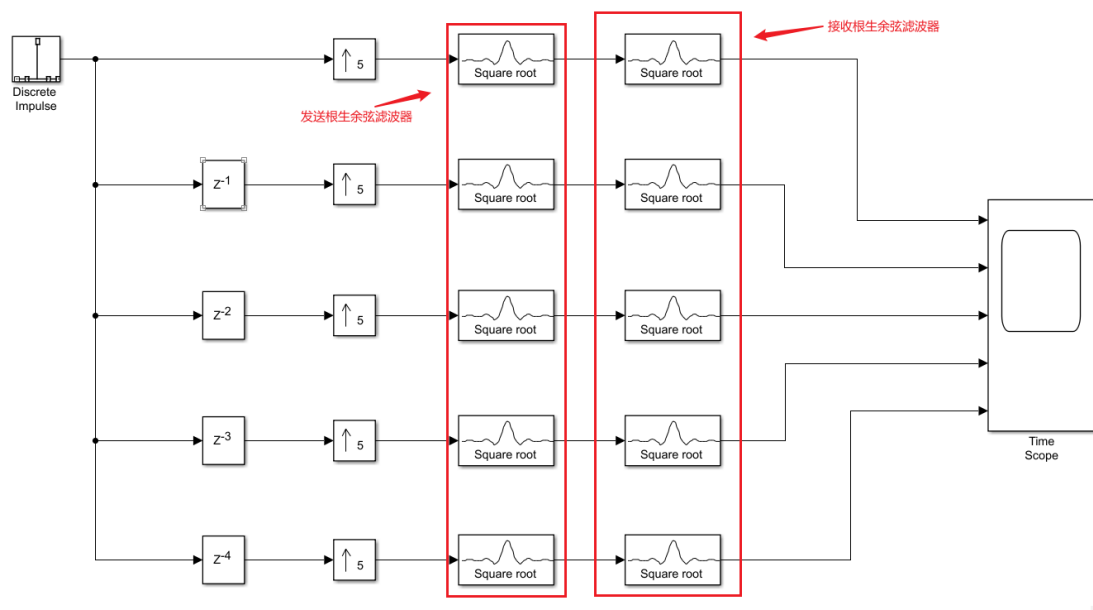


图 13-44 匹配滤波器仿真

本次实验需要使用到发送端和接收端的匹配滤波器：

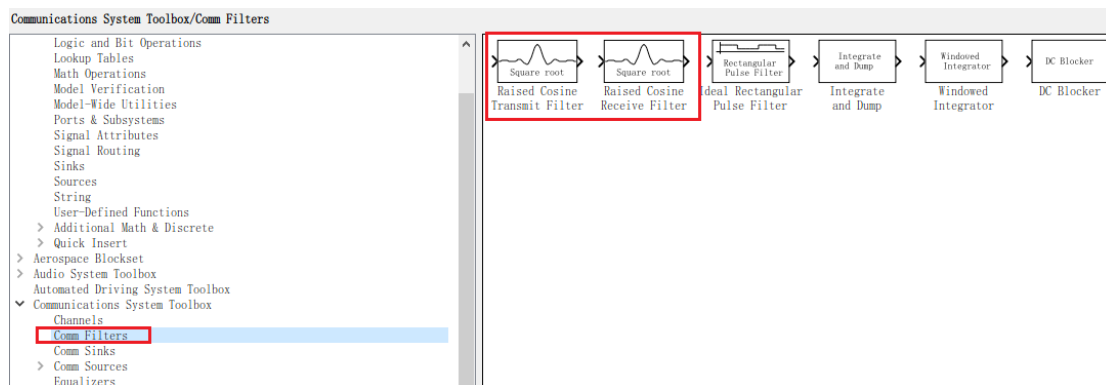


图 13-45 发送端和接收端的匹配滤波器

对于发送端，其参数设置如下：设置根升余弦滤波器滤波器的滚降系数为 0.25，设置每个符号有 5 个采样点，这是由于在前面使用了 5 被的上采样对符号进行了 5 倍的插值，也即在原来的符号序列中插零值。

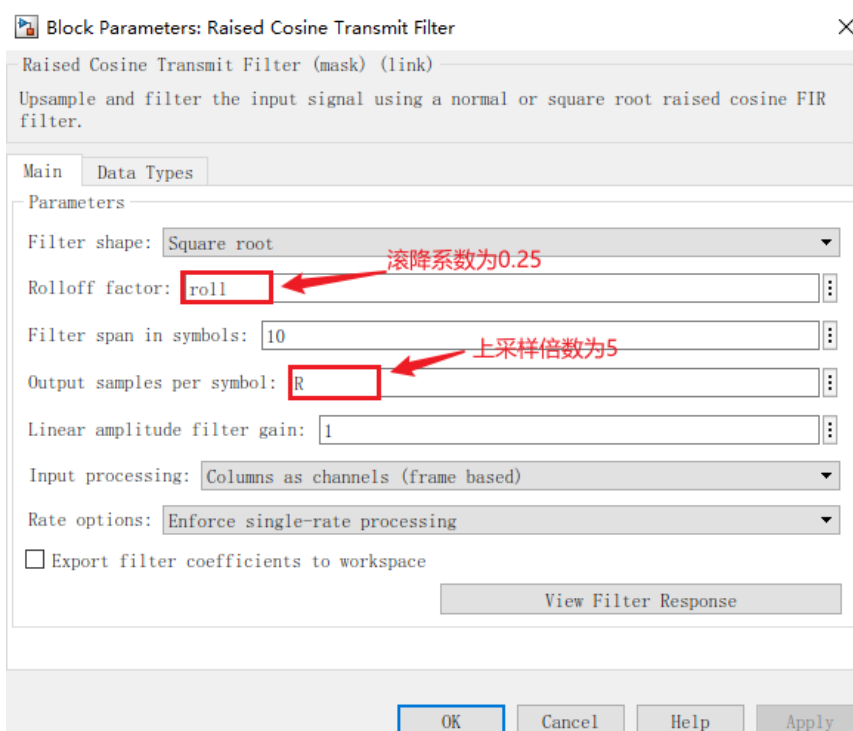


图 13-46 发送端匹配滤波器设置

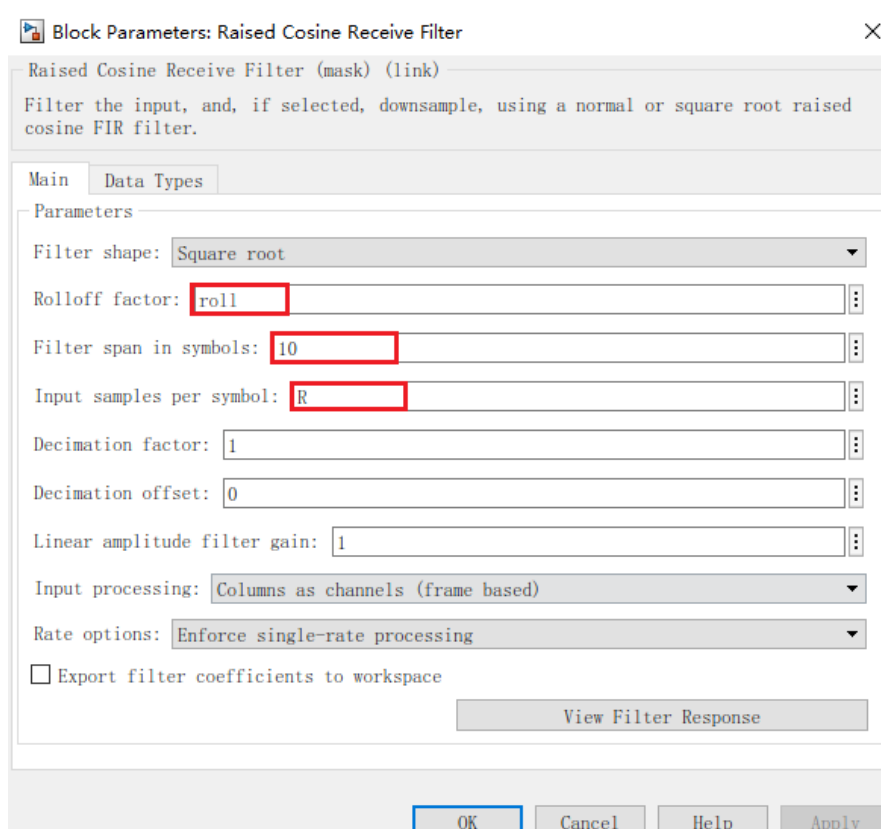


图 13-47 接收端匹配滤波器

在接收端，匹配滤波器的设置和发送端的滤波器设置需要满足一致，才能够

完成和发送根升余弦滤波器相匹配。

这个仿真模型完成的就是，由一个离散的单位阶跃信号进行延时，产生一系列的符号，然后将这个符号进行上采样，经过上采样之后，经过根升余弦滤波器进行符号的整型，然后发送出去，在接收端经过接收根升余弦滤波器进行匹配滤波，然后我们就可以在示波器当中来观察得到的结果。

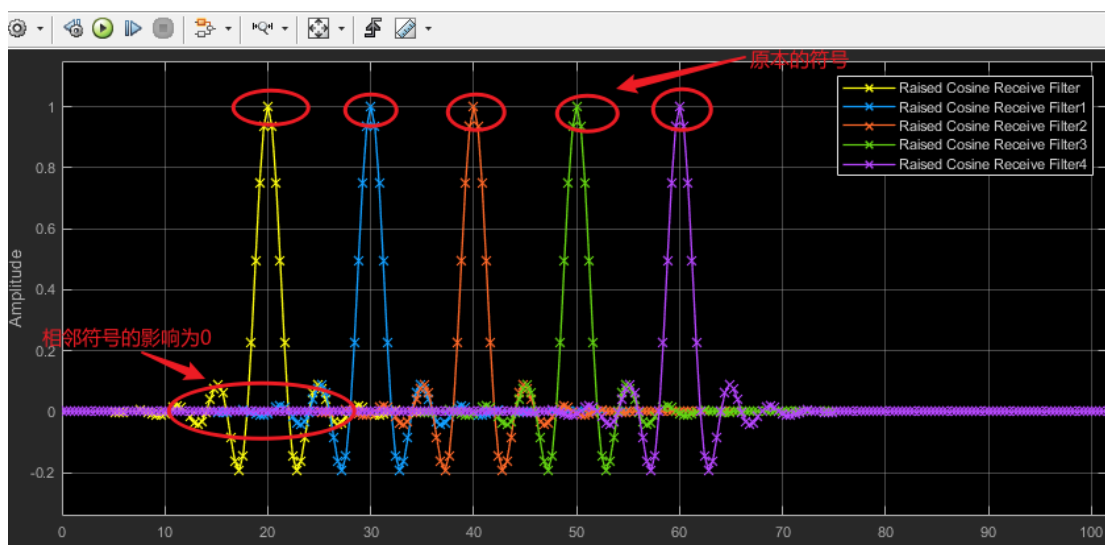


图 13-48 仿真结果

从仿真的结果上来看，每一个符号都会延申到其他的符号当中，但是在原来的符号所处的位置上，对原来的符号的影响为 0。

你可以调整上采样的倍数，获取在上采样的情况下，符号间干扰的情况。

13.4 粗频率校正

在进行 QPSK 解调的时候，经过 AD936x 调谐之后的信号，可能不是刚好将信号下变频到基带，甚至可能存在一个较大的频率误差，存在较大的频率误差的时候，会为后面的载波同步和符号同步产生很大的影响。因此，在进行 QPSK 解调的时候，首先需要做的就是粗频率校正。

在 Simulink 当中，提供有用于进行粗频率校正的模块，我们在设计接收机的时候，可以使用这个模块来进行粗频率校正，可以打开仿真模型 `coarse_frequency_correction.slx` 来查看这个模块的具体使用：

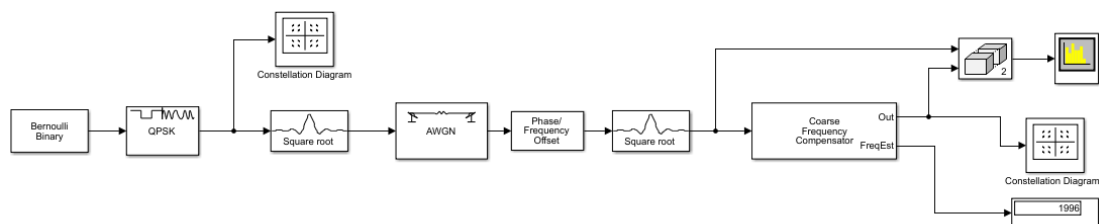


图 13-49 粗频率校正仿真

在进行粗频率校正的时候，我们首先需要生成一个 QPSK 信号，然后将这个信号，然后将 QPSK 信号经过根升余弦滤波器进行成型，之后经过 AWGN 信道，在仿真模型中，我们为 QPSK 传输增加了一个 2000Hz 的频率偏移，然后将这个信号在接收端通过根升余弦滤波器滤波器，最后需要将这个模块给到粗频率校正模块。

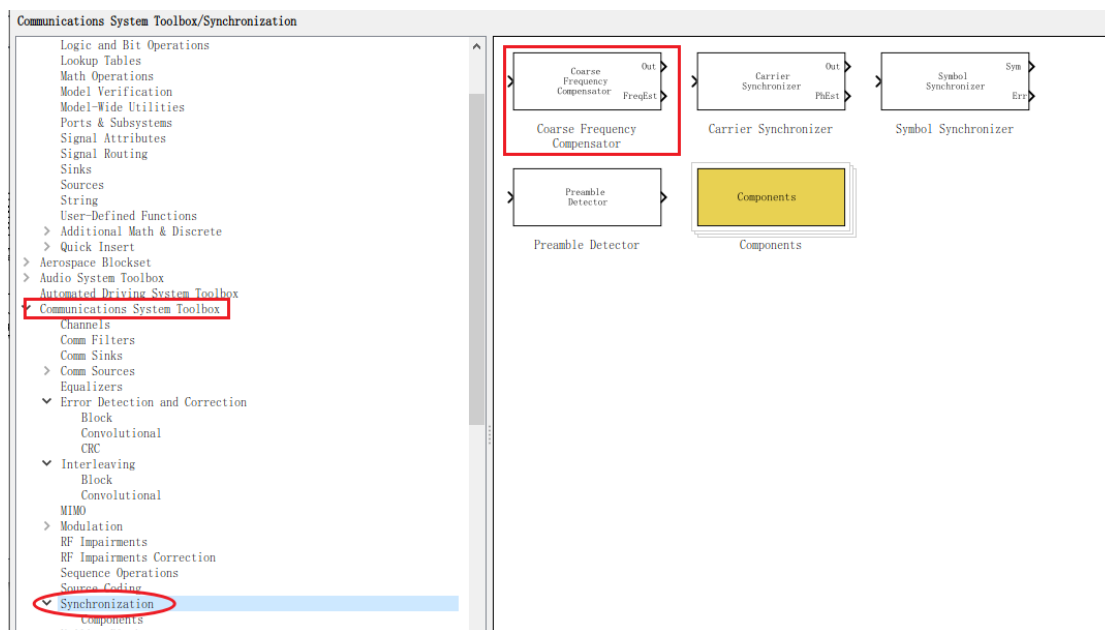


图 13-50 粗频率校正模块

对于该模块的设置如图 13-51 所示：

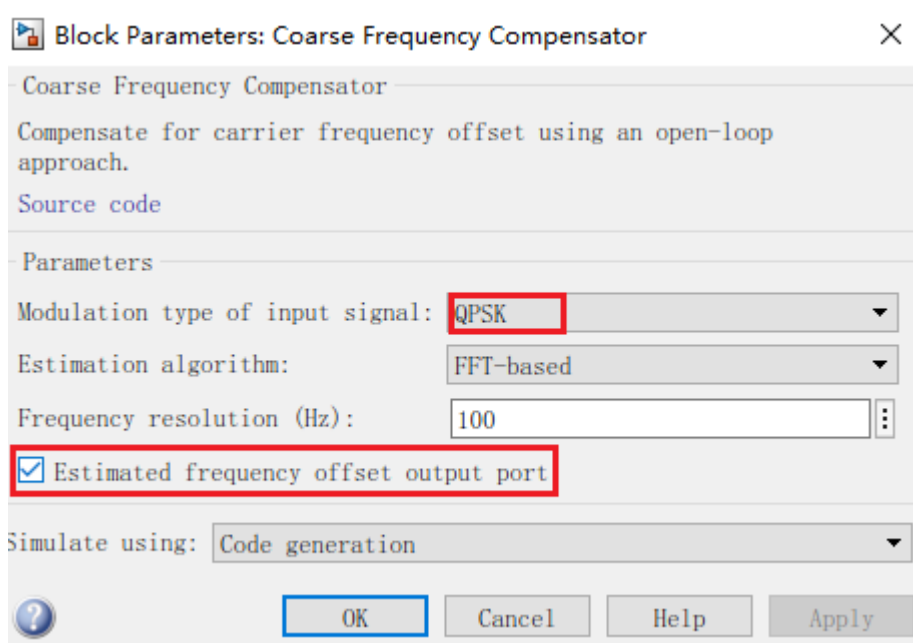


图 13-51 粗频率校正模块设置

设置好各个模块之后，开始运行仿真，在仿真的一开始，可以看到两个信号之间存在一个频率的偏移，这是因为我们在前面的频率和相位偏移模块当中，设置了 2KHz 的频率偏移。

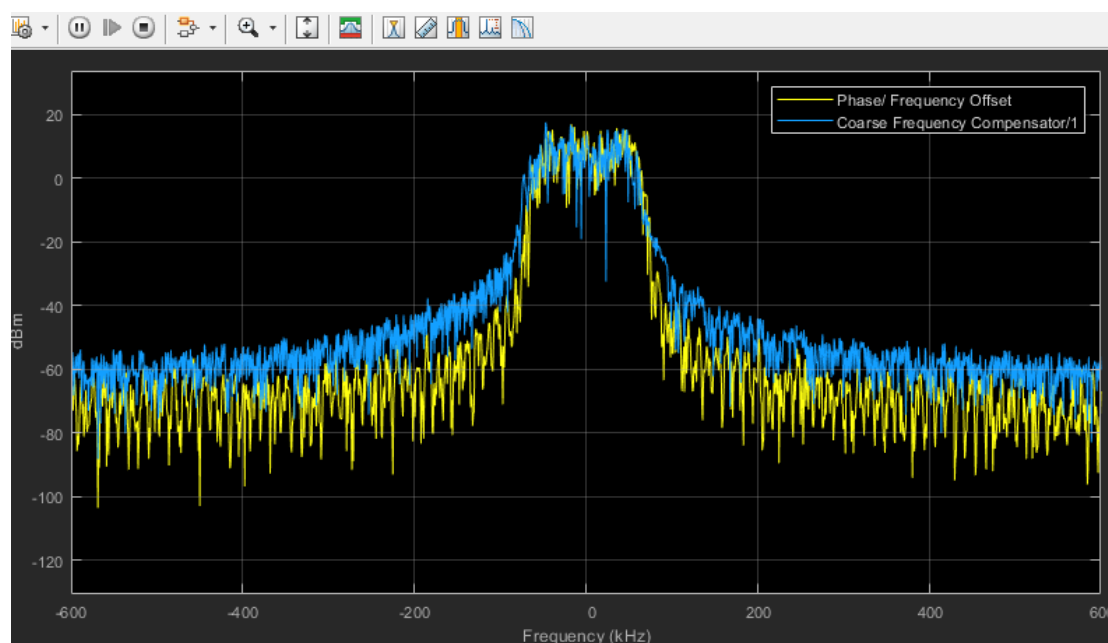


图 13-52 一开始两个信号之间存在较大频率偏移

当仿真模型运行了一段时间之后，可以看到接收到的信号基本和原始信号在相同的频率范围内。这时候我们也可以通过 display 这个模块上显示的数据，来查看当前的频率的偏移量。

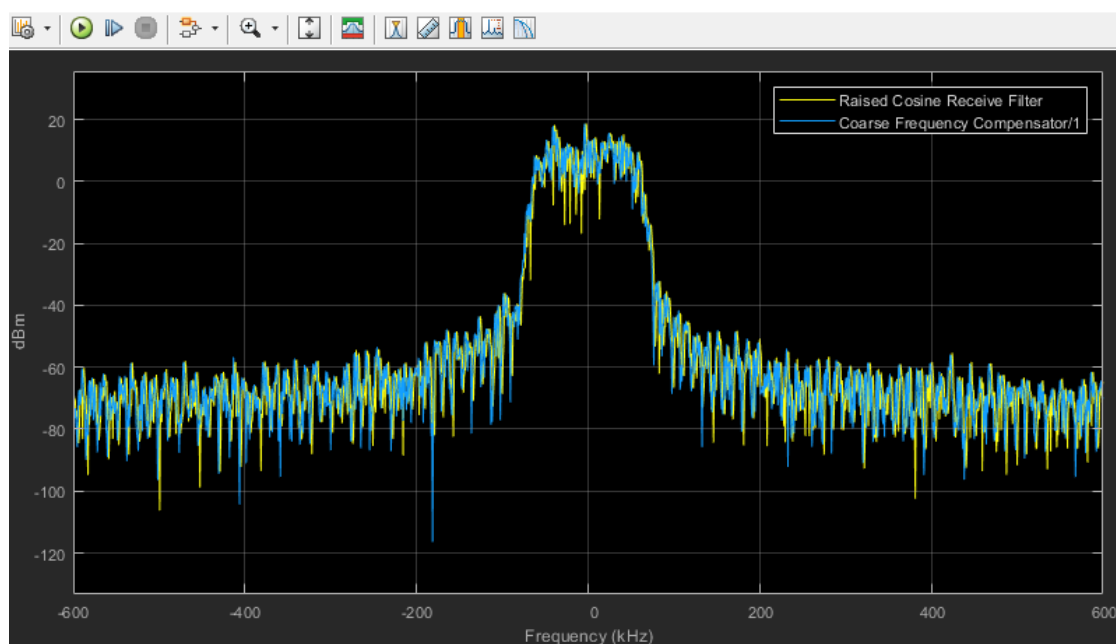


图 13-53 进行频率校正之后

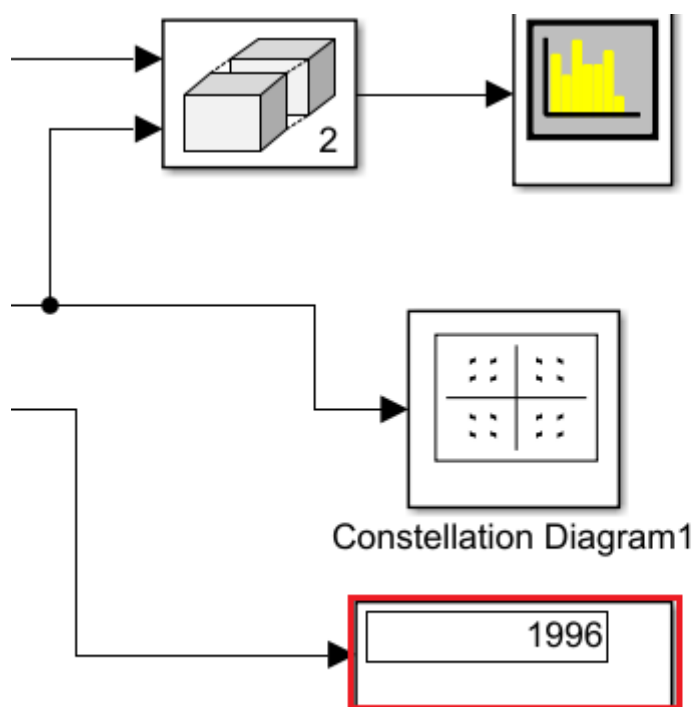


图 13-54 频率偏移量

可以看到，在进行了频率的校正之后，显示出当前按频率的偏移为 1996Hz，与我们前面设置的频率偏移相差不大，这样就完成了粗频率校正。

13.5 符号同步

由于缺乏一个共有的时钟参考，大多数通信系统需要在接收端实现符号定时同步。如果不做同步，那么就无法纠正相位和频率误差，导致接收到的数据出错。尤其是存在定时频率误差的时候，更可能还原不出数据。

定时同步器测量定时误差，并且产生了一个反馈控制信号来动态调节定时参数。因此符号同步器的大致框图与 PLL 很像。在这里，误差检测器测量出了定时误差，而不是两个正弦波的相位差（PLL），

另外还有一个可调节的部分叫做压控时钟 VCC 或者数控时钟 NCC，而不是 PLL 中用来生成正弦波的 NCO 或者 VCO。NCC 的作用时产生一个采样时钟参考，包含采样率和相位，它们都要与接收到的信号匹配。

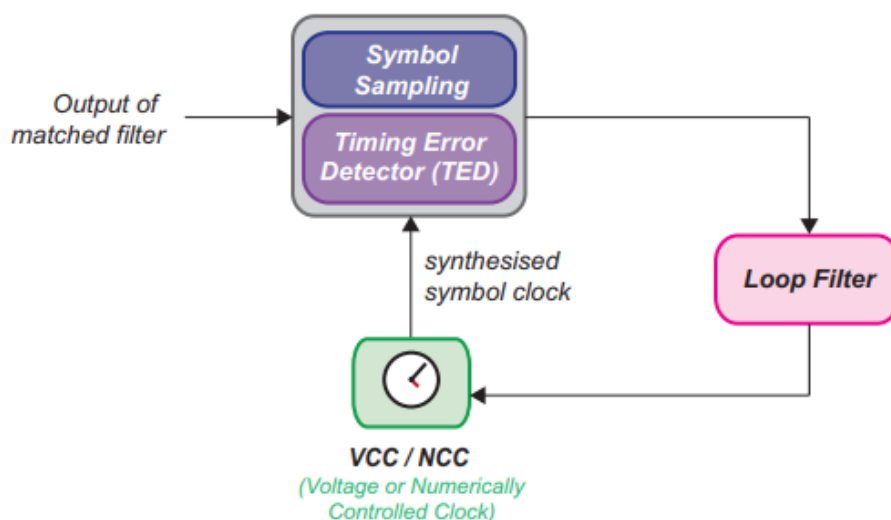


图 13-55 符号同步模块结构

在 Simulink 工具包当中，有能够进行符号同步的模块，符号同步器模块针对 PAM，PSK 或 QAM 调制方案校正符号定时时钟偏斜。

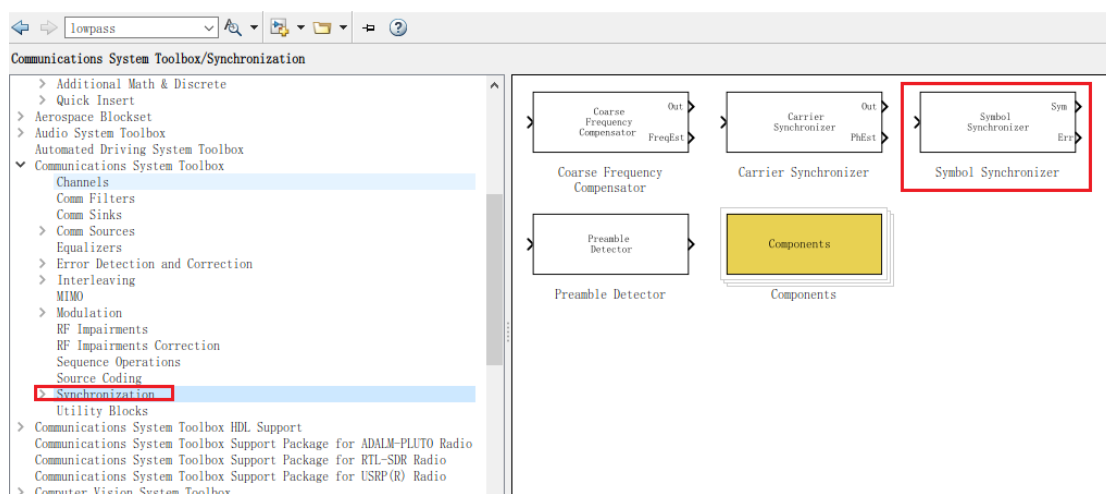


图 13-56 符号同步模块

接下来，我们来搭建一个简单的 simulink 模型来直观地感受一下符号同步对恢复信号的作用。

打开仿真模型：`symbol_sync.slx`，可以看到仿真模型如下图所示：

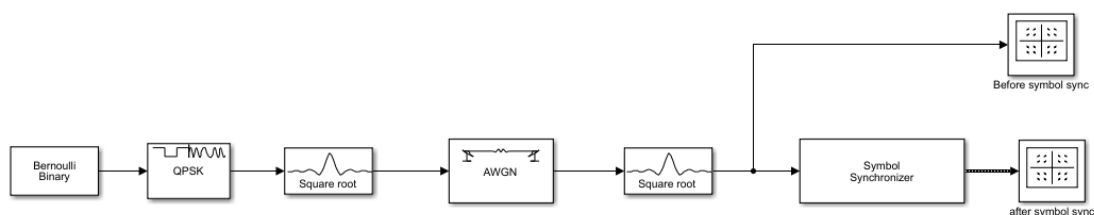


图 13-57 符号同步仿真模型

其中数据源模块和 QPSK 调制模块和前面的实验相类似，在这里不再赘述。在这里对信号通过了一个 AWGN 信道，该模块的设置如图 13-58 所示：

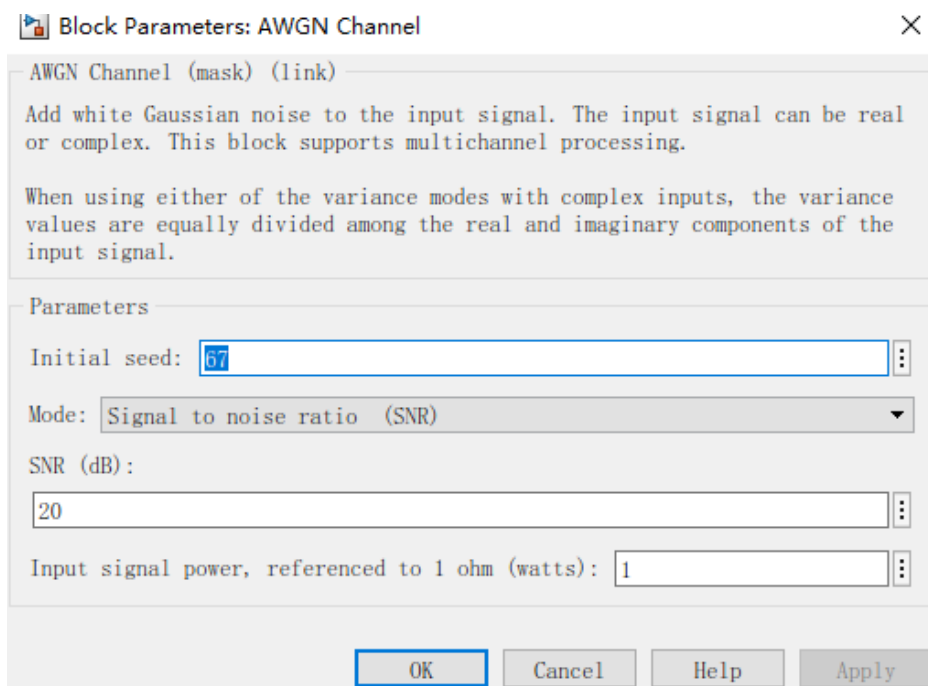


图 13-58 awgn 模块设置

然后需要注意两个匹配滤波器的参数需要保持相匹配,这样才能完成正确地匹配滤波。

在接收到的信号经过匹配滤波之后,需要对其进行符号同步,其设置如下:

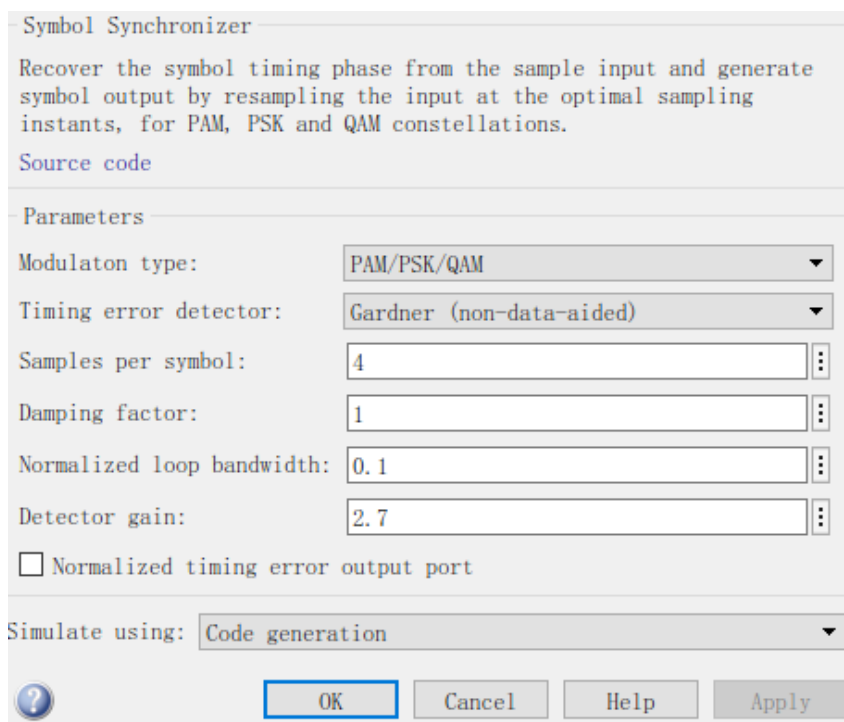


图 13-59 符号同步模块设置

然后我们可以使用两个星座图，来观察同步前后的星座图。点击仿真，开始运行这个仿真模型，然后观察星座图。

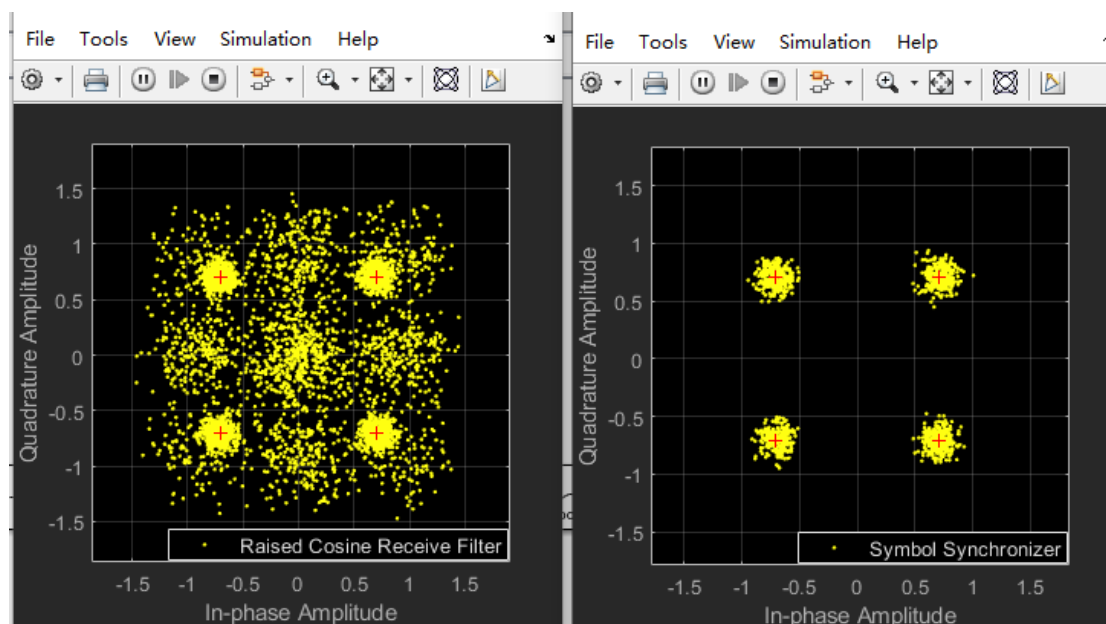


图 13-60 同步前后星座图对比

从星座图的结果来看，经过 AWGN 之后的信道中含有噪声，导致星座图上的点偏离了原始的位置，如果不经过处理，在进行数据获取的时候很容易出现误判。

而在经过符号同步之后，可以看到，星座图上的点，基本上都是在对应的星座参考点附近，这就为我们进行最终的抽样判决进行数据恢复提供了很大的便利。

13.6 载波同步

在接收机和发射机当中，用于发送的载波和用于解调的载波是很难保持完全一致的，这个在我们前面介绍模拟调制的时候，也介绍过相干解调，为的就是能够跟踪频率的变化，从而能够产生一个和调制载波相干的信号。

在数据接收机当中，我们也经常会遇到调制载波和解调载波不一致的情况，这时候就需要进行载波的同步。

当载波不同的时候，经常可能会有以下三种情况：

1. 有固定的相位偏移
2. 有固定的频率偏移
3. 有变化的频率偏移

这 3 中情况分别如图 13-61 图 13-61 图 13-63 所示：

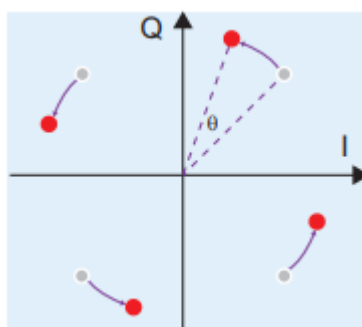


图 13-61 有固定的相位偏移

当接收机和发射机的载波存在一个固定的相位偏移的时候，在接收的星座图当中，可以看到接收到的符号和参考点的符号有一个固定的相位的偏移。

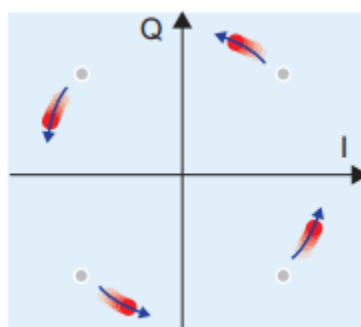


图 13-62 有固定的相位偏移

若发射机和接收机的载波之前存在一个固定的频率偏移，那么我们可以看到这个星座图上的点将会以一个固定的速率在进行旋转。

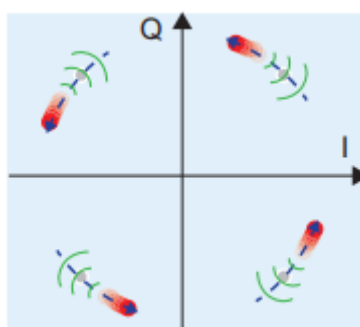


图 13-63 变化的频率偏移

当接收机和发射机当中载波频率存在频率偏移并且这个频率偏移在不断的变化的时候，那么从星座图上看，这个星座图上的点会不停旋转，并且当接收机和发射机的频率偏移变大的时候，旋转的速率加快。

在 Simulink 当中，也提供了用于载波同步的模块：载波同步器模块使用针对

BPSK, QPSK, OQPSK, 8-PSK, QAM 和 PAM 调制方案的闭环方法来补偿载波频率和相位偏移。

通过使用载波同步模块，能够同时解决载波的相位和频率的偏移。接下来就让我们来实现一个仿真模型，来完成载波同步的工作。

打开仿真模型 `carrier_sync.slx`，该仿真模型的结构如下：

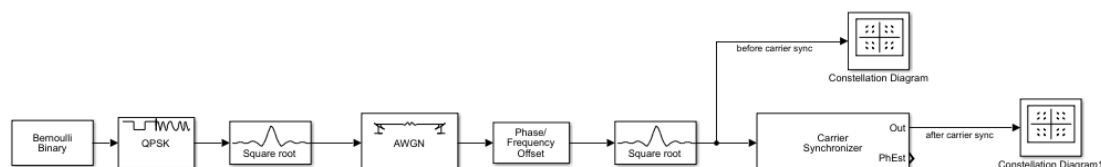


图 13-64 载波同步仿真模型

在上面的仿真模型当中，许多模块都和前面的实验类似，在这里不做过多的介绍。可以看到，首先由随机数产生模块来产生数据源，然后经过 QPSK 映射，随后将映射之后的信号进行根升余弦滤波器滤波，发送到 AWGN 信道，在 AWGN 信道中，我们人为的引入了频率误差和相位误差。在接收机当中，我们进行了匹配滤波，最后进行了载波的同步。

引入频率和相位偏移的模块的设置如下：

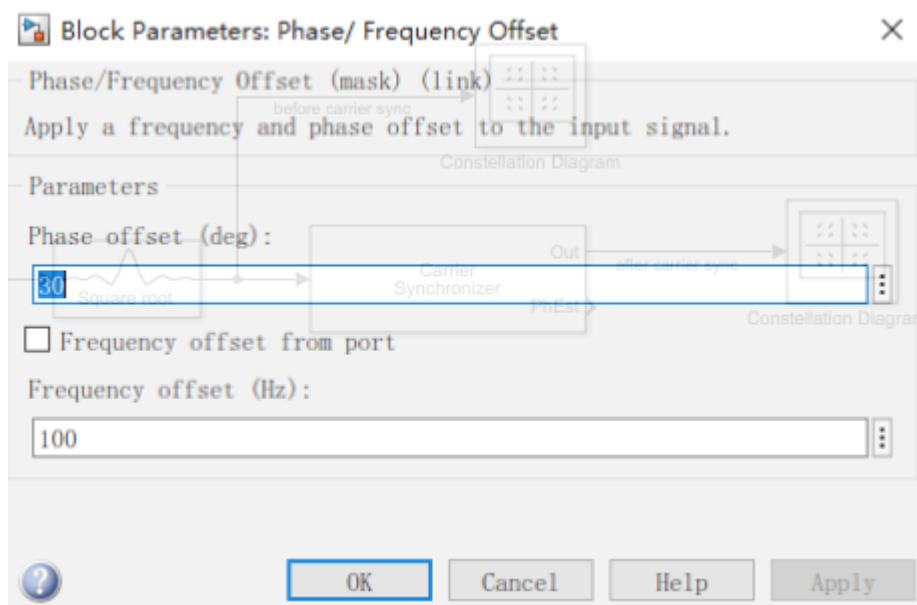


图 13-65 引入频率和相位误差

在载波同步模块当中，需要进行的设置如图 13-66 所示：

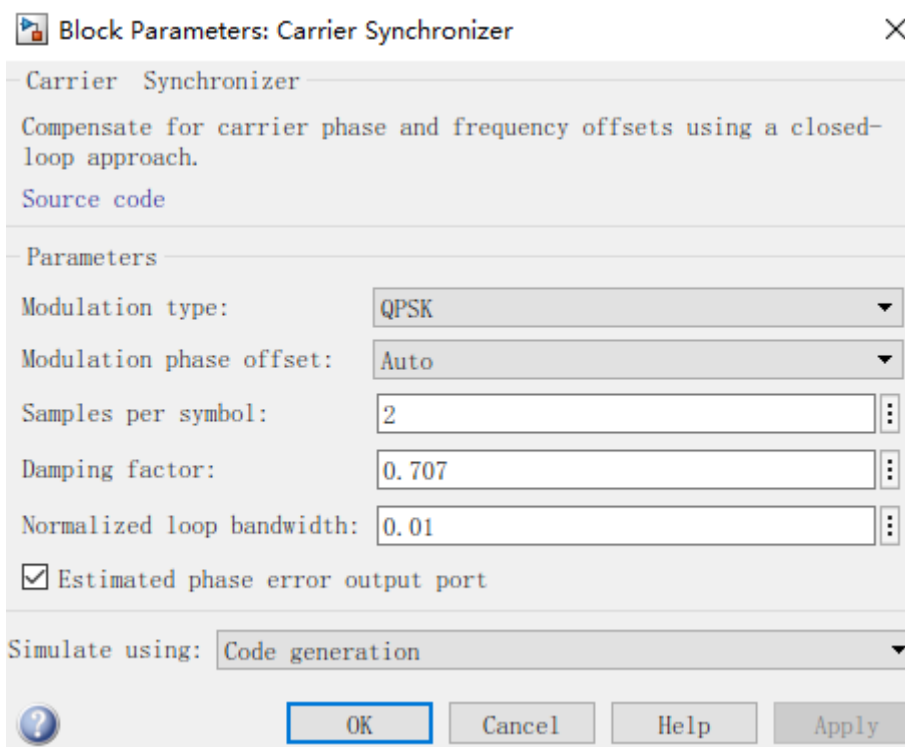


图 13-66 载波同步模块设置

准备好仿真模型之后，我们就可以开始进行仿真，观察星座图，来查看载波同步对存在频率偏移的信号是能否进行调整。

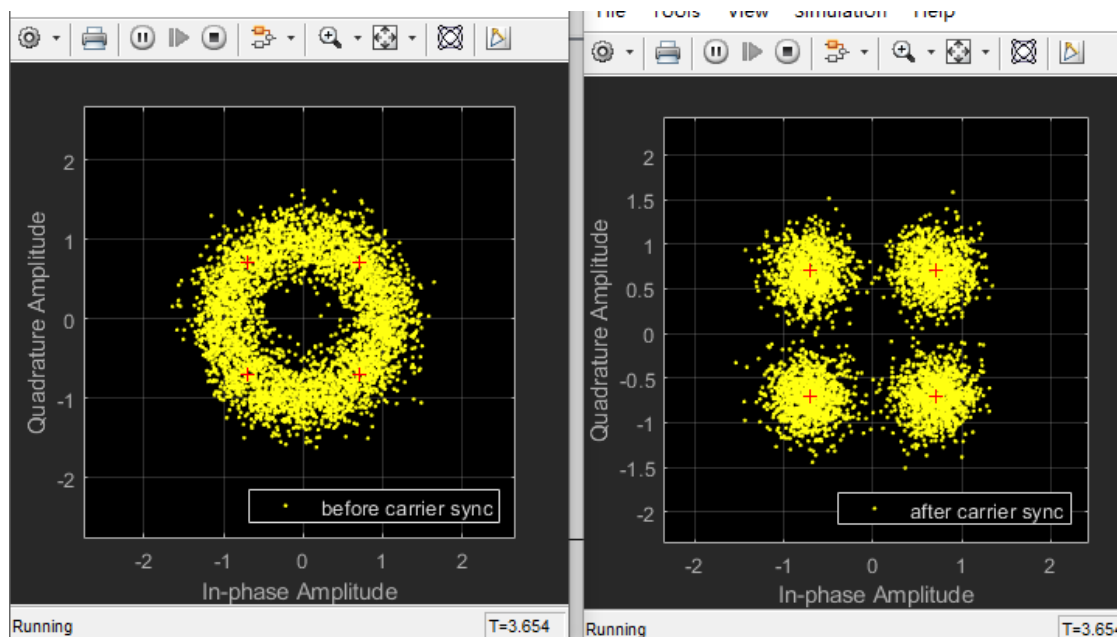


图 13-67 载波同步结果

可以看到在没有进行载波同步的时候，星座图上的点呈现一个圆心，并且在

不停的运动，这是因为我们在前面的频率和相位偏移模块当中，引入了误差，导致接收机中的频率和发射机当中存在频率误差，因此这个星座图会旋转。

然后经过载波同步之后，我们可以看到，这个时候星座图相较于没有同步之前，距离参考点的偏移会减小。

到这里，我们就完成了载波同步，在实际的通信系统中，为了能够减小误差，通常会将前面介绍到的在接收端的同步方法联合起来使用，将粗频率校正，载波同步和符号同步联合起来使用能够大大地减小误差，减小误码率，从而使通信系统的更加稳定可靠。

13.7 使用 ANTSDR QPSK 调制与解调

在前面介绍了很多关于 QPSK 信号的基础之后之后，我们现在可以来使用 ANTSDR 来进行真实的无线信号的发送与接收了。

打开仿真模型 `antsdr_qpsk_mod_demod.slx`，仿真模型的结构如图 13-68 所示：

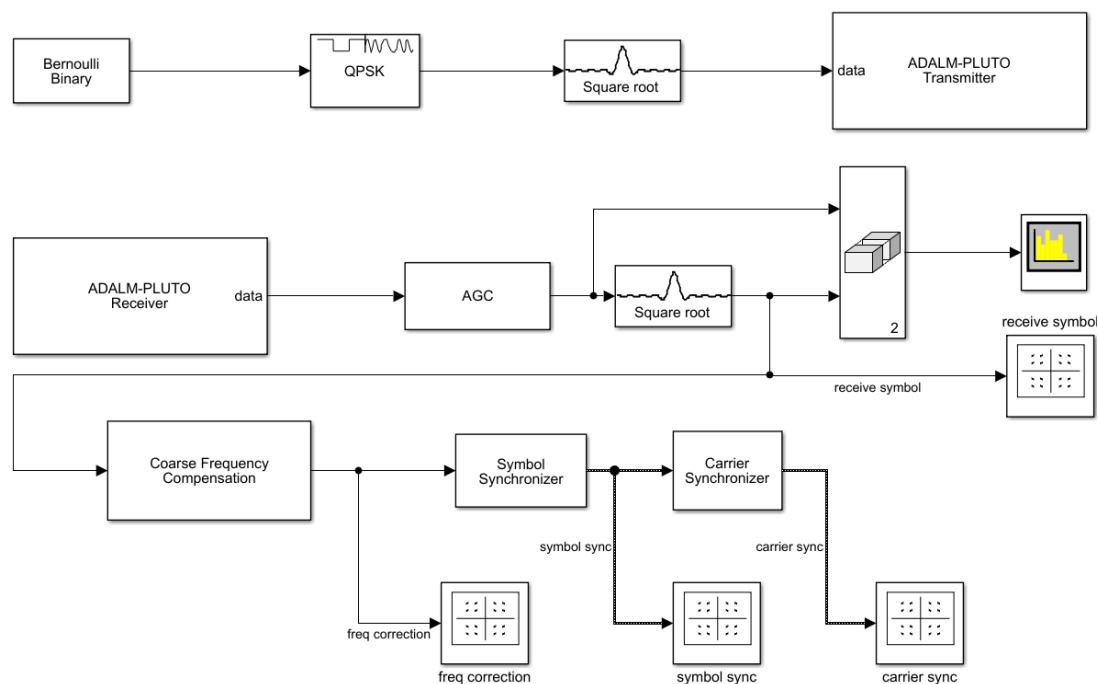


图 13-68 ANTSDR 发送 QPSK 信号模型

首先由随机数模块产生数据源，其设置如图 13-69 所示，该随机数产生模块的采样率是 20KHz，也就是产生的符号的频率是 20KHz，

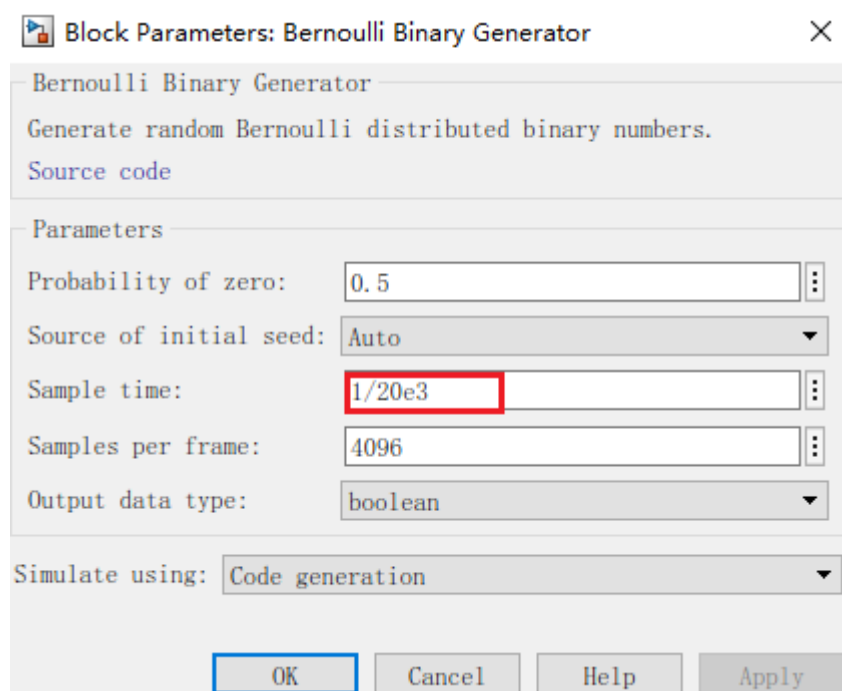


图 13-69 随机数产生模块

然后是进行 QPSK 的映射，由于采用 QPSK 映射，所以符号率会变为 10KHz。

再这之后进行根升余弦滤波器滤波，其设置如下：

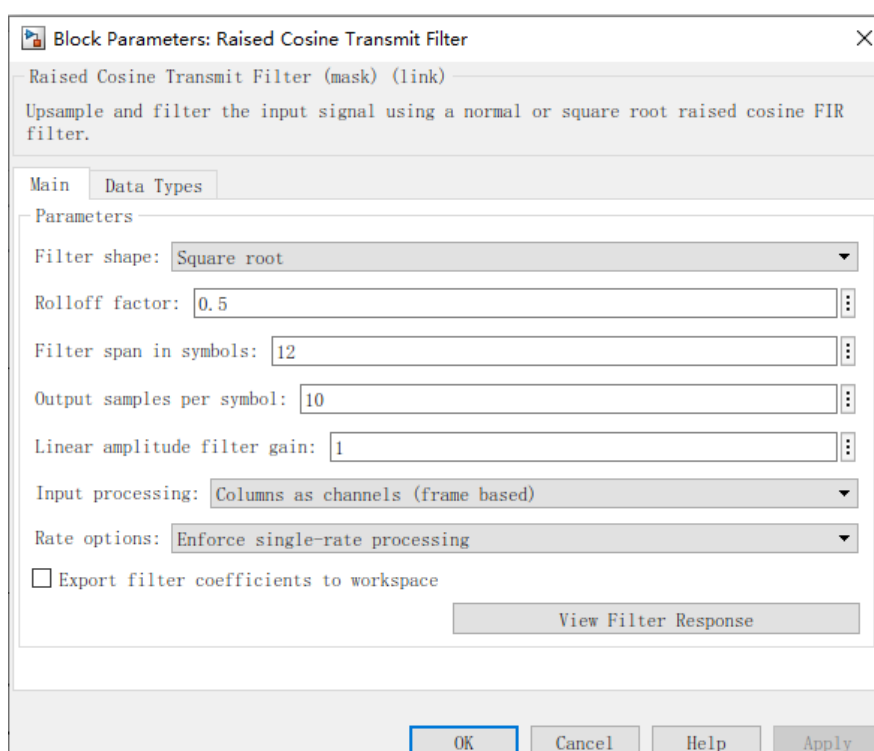


图 13-70 根升余弦滤波器设置

其中进行了 10 倍的插值，因此插值过后的符号速率为 100KHz，也就是采样率为 100KHz。

然后经过 PLUTO 发送模块进行发送，pluto 发送模块的基带采样率为 100KHz。

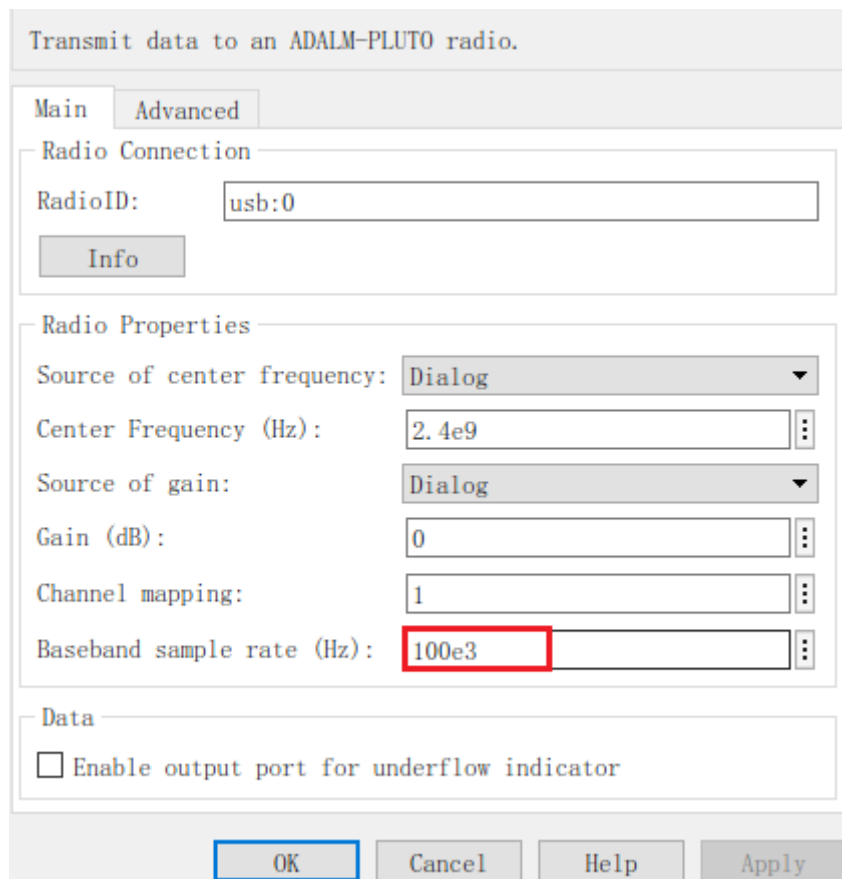


图 13-71 pluto 发送模块设置

到此就完成了使用 ANTSDR 来发送 QPSK 信号，在这之后就是实现 QPSK 的接收和解调了。

使用首先需要使用 PLUTO 接收模块，接收模块的设置需要和发送模块相对应。基带采样率为 100KHz，中心频率设置为 2.4GHz。

然后将接收到的数据进行一个自动增益控制，自动增益控制模块，能够调整输入信号的大小，便于后续的分析。

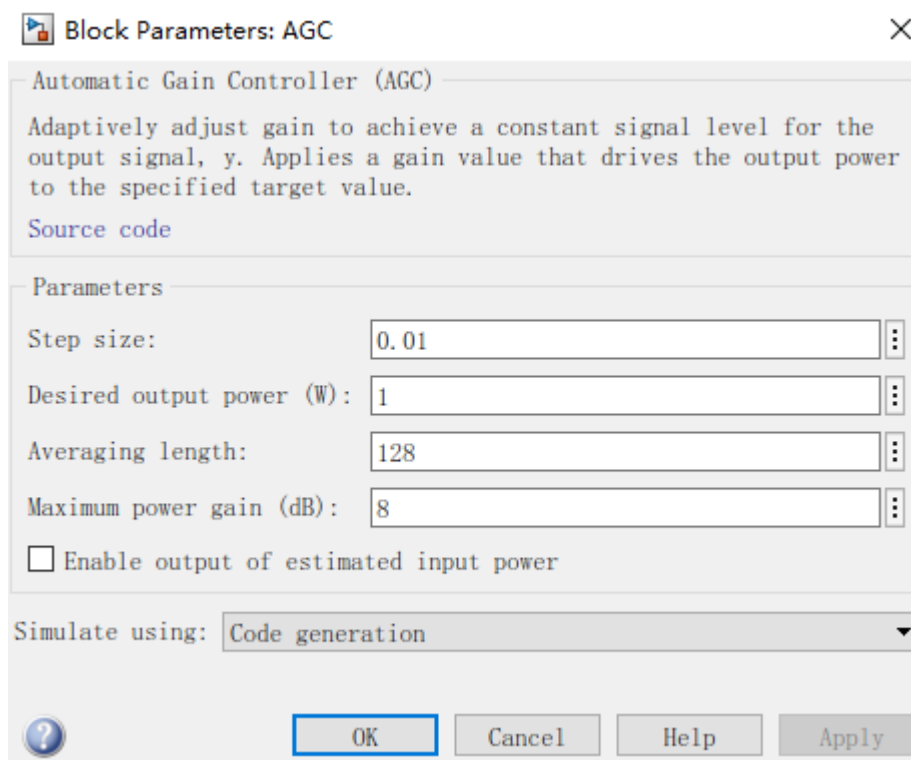


图 13-72 自动增益控制设置

然后是进行匹配滤波，接收机当中的匹配滤波器需要和发射机当中的匹配滤波器组合形成升余弦滤波器。

在经过匹配滤波之后，可以对接收到的信号进行粗频率校正，在这里使用到了一个子系统，检测出当前的频率误差，并使用相位频率偏移模块进行调整。

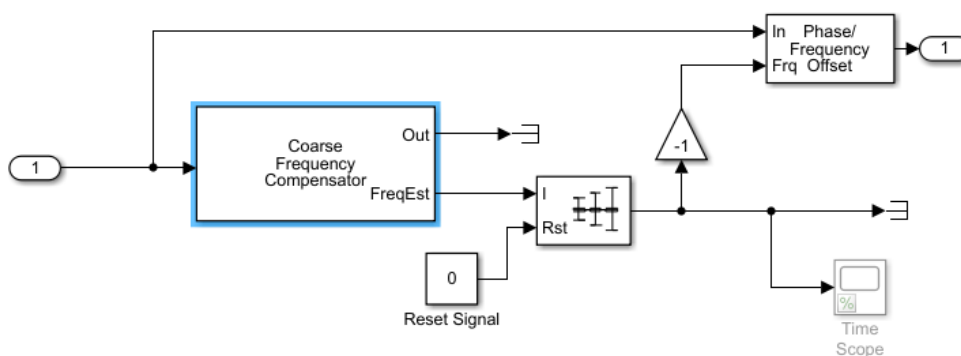


图 13-73 粗频率校正模块

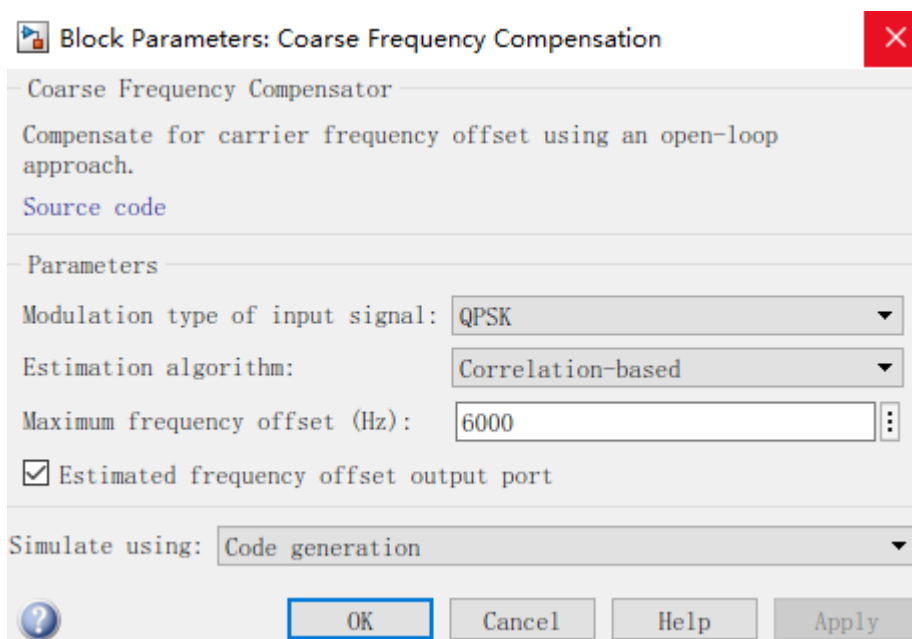


图 13-74 粗频率校正模块设置

最后进行符号同步和载波同步就可以完成所有的同步工作了，这两个模块的设置在前面的时候，已经有过介绍。

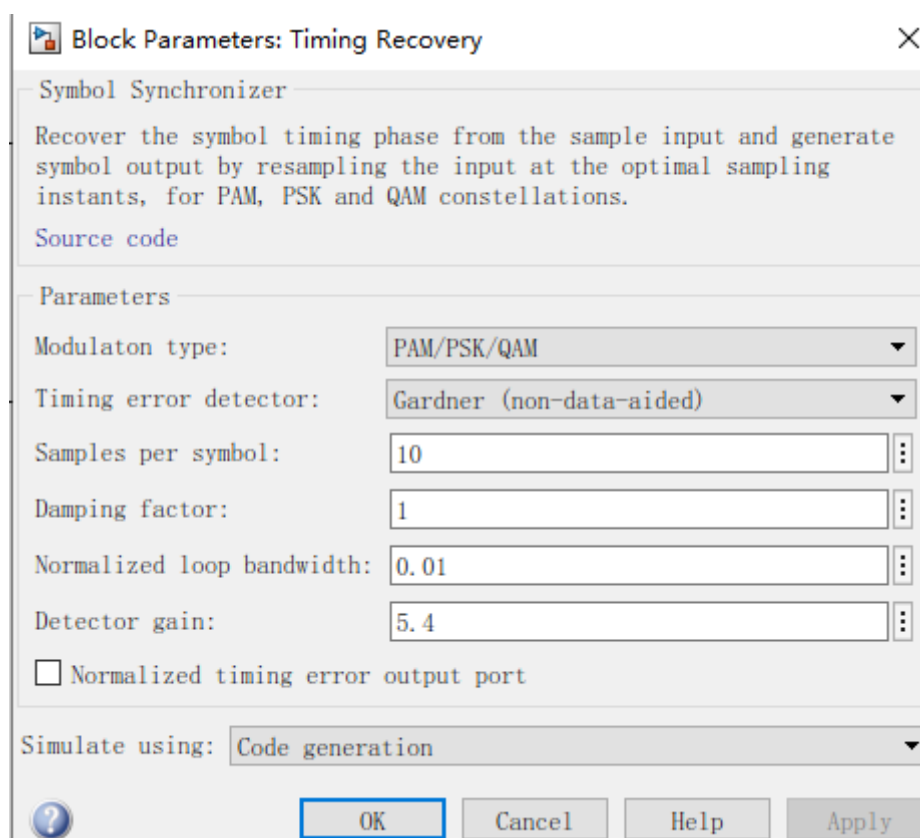


图 13-75 符号同步模块

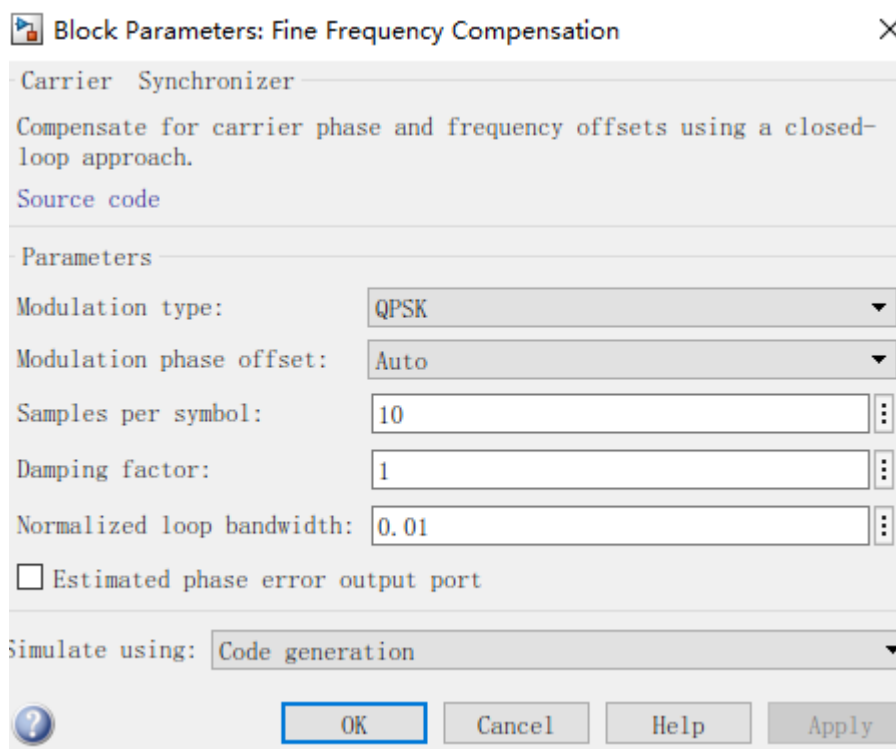


图 13-76 载波同步模块

全部模块都设置好了之后，就可以运行这个仿真模型了。

首先我们观察经过匹配滤波之后的信号的频谱图：

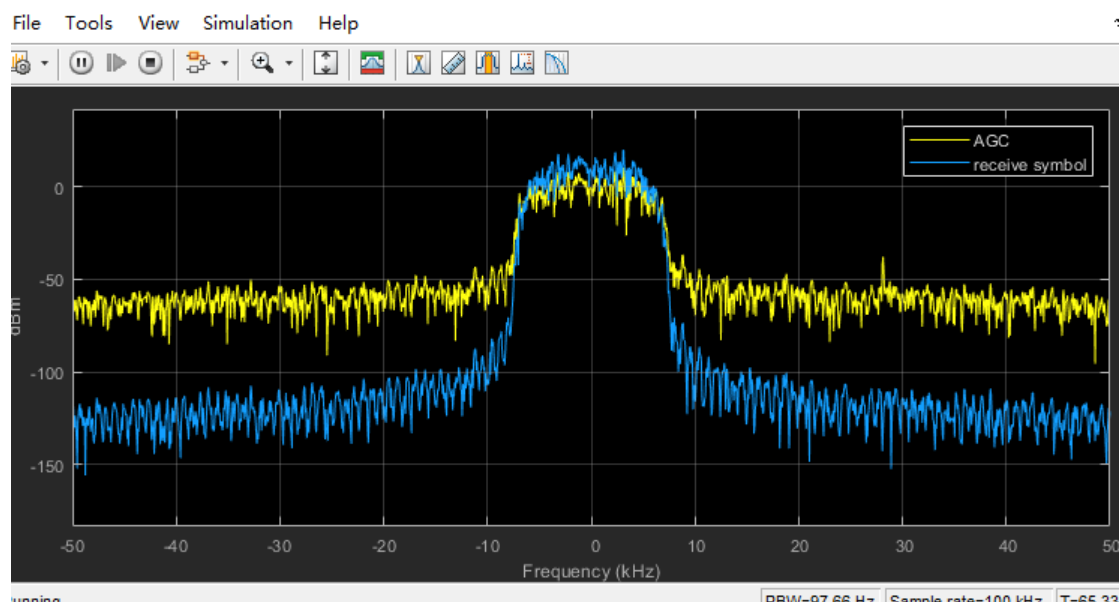


图 13-77

可以看到经过匹配滤波之后，能够接收到的信号进行很好的接收，并且消除周围频段信号的影响。

然后我们可以看当前接收到的星座图：

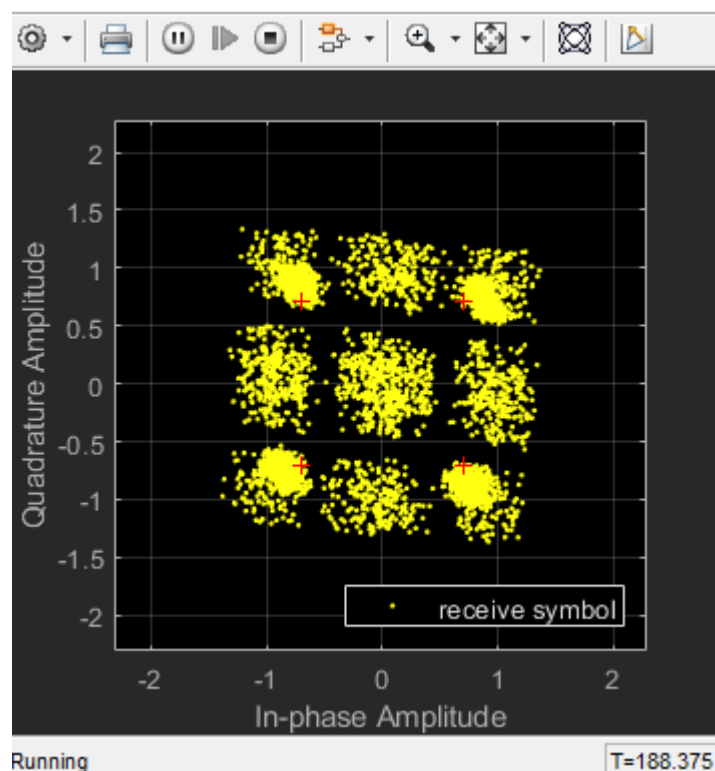


图 13-78 匹配滤波之后的星座图

可以看到，当前的星座图比较分散，中间的点比较难以进行判别区分。所以我们需要后续的操作来对接收到的信号进行优化。

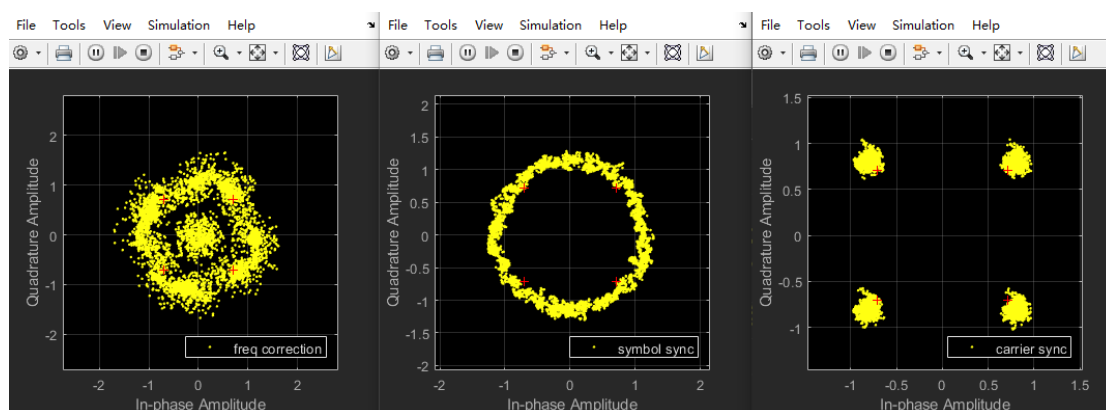


图 13-79 经过同步之后的星座图

可以看到在经过同步之后，星座图已经能够正确地分布在参考点周围，这样就能够很方便地来对这些符号进行判决了。

到这里我们就完成了使用 ANTSDR 来进行 QPSK 地发送和接收了。通过使用匹配滤波，频率校正，载波同步和符号同步的联合同步，最终恢复出了效果很好

的星座图。

13.8 总结

在本章中，我们介绍了数字通信的相关知识，接触到了数字通信当中常用的一些调制方法。并且实际地认识了数字调制当中 QPSK 和 QAM，相位调制和正交幅度调制是现代数字通信当中常用地调制方式。

在本章当中，我们还介绍了数字通信当中地成型滤波，也即是通过根升余弦滤波器来对符号进行成型，并在接收当中实现匹配滤波减小符号间的干扰。

随后我们介绍了在数字通信当中用于解调和同步的一些方法，具体包括，粗频率校正，符号同步和载波同步，在实际的同步问题当中，通常是这几种同步方法联合在一起使用。

最后我们使用 ANTSDR 成功地发送了 QPSK 的信号，并在接收当中完成了对这个信号的同步。

第14章 使用 QPSK 发送 ASCII 码

在前面的章节当中我们已经实现了使用 ANTSDR 来进行 QPSK 的发送和接收，完成了最基础的数据收发工作，在前面我们发送的都是随机数，这些数据并不包含任何信息。在数字通信里面，需要构建协议来使得发送出去的数据能够正确地承载信息。

在本章实验中，我们会使用到 Matlab 中自带的 QPSK 的例子，来完成使用 ANTSDR 来发送和解调 QPSK 信号。

14.1 打开官方 QPSK 仿真模型

打开 matlab 之后搜索 QPSK



图 14-1 搜索 QPSK

找到 QPSK Transmitter and Receiver 这个 Simulink 的仿真模型：

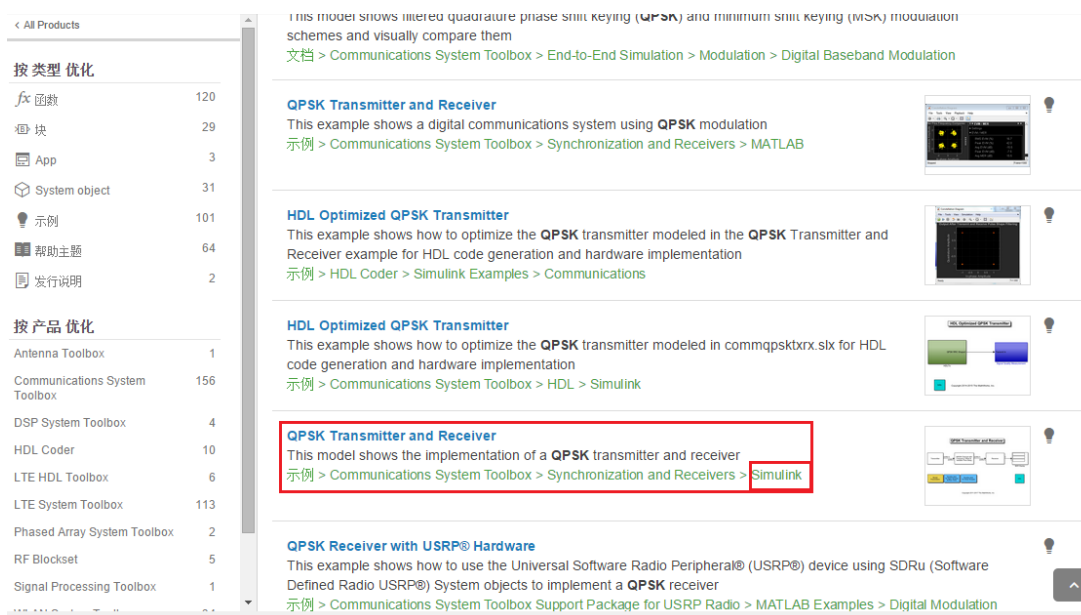


图 14-2 找到 QPSK 仿真模型

选择打开这个模型：

QPSK Transmitter and Receiver

This model shows the implementation of a **QPSK** transmitter and receiver. The receiver addresses practical issues in wireless communications, e.g. carrier frequency and phase offset, timing drift and frame synchronization. The receiver demodulates the received symbols and outputs a simple message (e.g., to the Diagnostic Viewer).

Available Example Implementations

This example includes both MATLAB and Simulink® implementations:

MATLAB script: [commQPSKTransmitterReceiver.m](#)

Simulink model: [commqpsktxrx.slx](#)

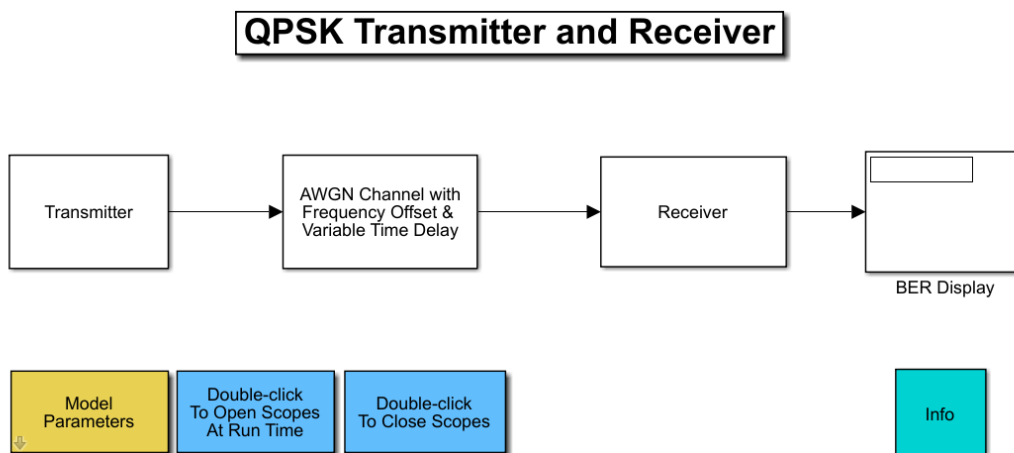
This example also uses:

[Simulink](#)

Open Model

图 14-3 打开模型

打开的仿真模型如下，这个仿真模型主要由 3 个部分构成，分别是用于发送的模块，用于接收的模块和中间的 AWGN 信道。



Copyright 2011-2017 The MathWorks, Inc.

图 14-4 打开的仿真模型

14.1.1 发射模块

对于发送机，其具体的结构如图 14-5 所示：

Transmitter

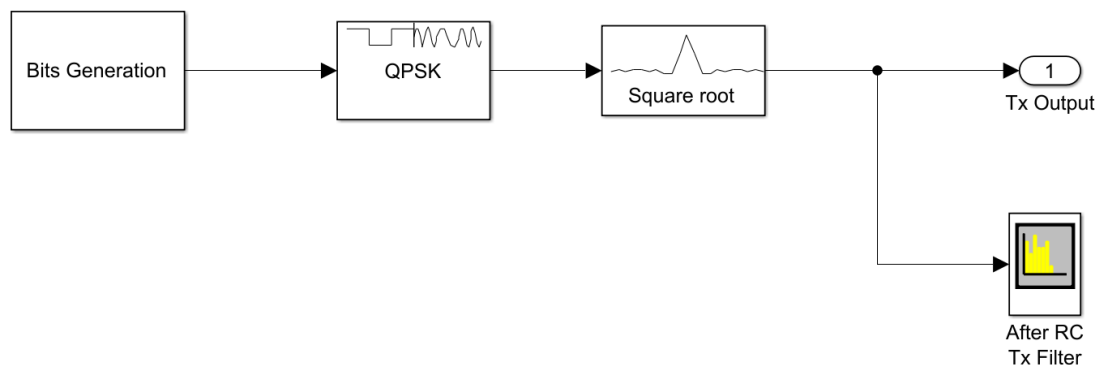
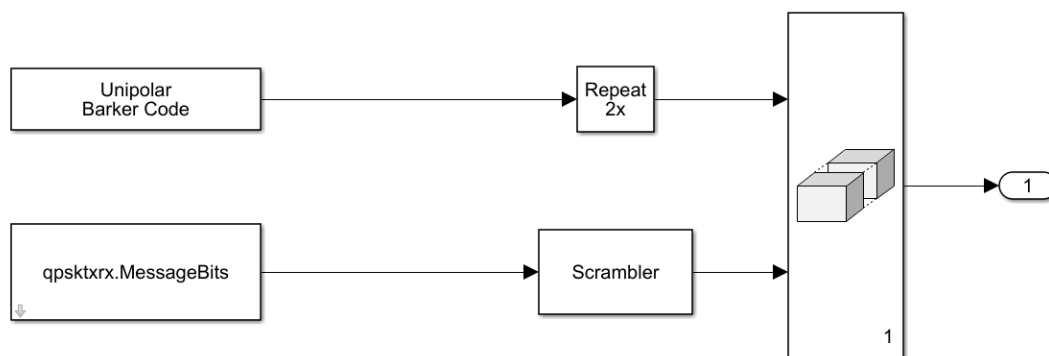


图 14-5 发射模块

首先有一个 bit 产生模块，该模块用来产生所需要的数据源，产生之后的数据经过 QPSK 进行调制，调制之后的数据通过根升余弦滤波器进行脉冲成型，然后进行发送，使信号进入信道。

在数据源产生模块中，会冲 Matlab 工作区读取要发送的消息序列，然后将这个消息序列进行加串之后和巴克码组成一个新的数据帧。巴克码是由 01 构成的一个特殊序列，常用于帧头，用于检测一帧的开始。



Each data frame contains 26 bits header (For Sync Purpose) and 112 bits data bits contain "Hello world ###" message. Scrambler is there to improve data transition density and frequency offset estimation.

图 14-6 数据产生模块

这个有效载荷对应序列“Hello world ###”的 ASCII 表示，其中###代表当前帧的序列，比如“Hello world 001”, “Hello world 002”等。

由数据源产生的符号速率是 50KHz，经过根升余弦滤波器之后，符号速率为

100KHz。

14.1.2 信道模块

在信道模块当中，首先对信号进行频率和相位的偏移，然后对信号加入一个可变的延时，最后将这个信号通过 AWGN 信道，从而进入到接收机。

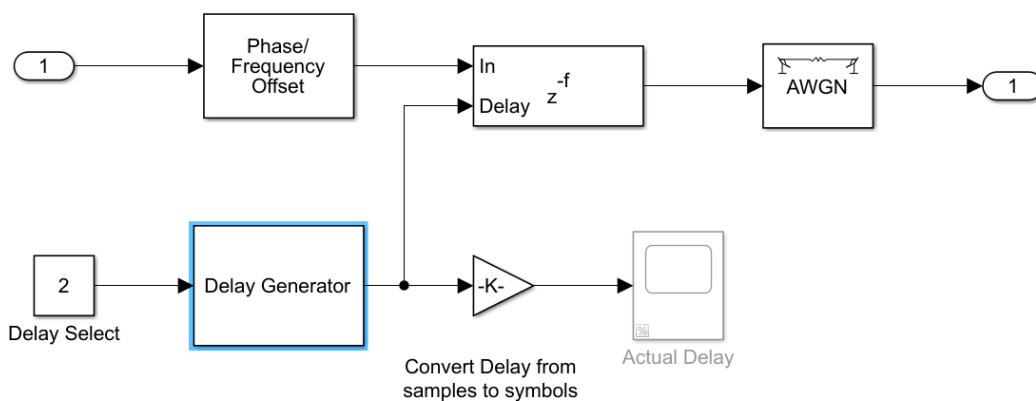


图 14-7 信道模块

14.1.3 接收模块

接收模块如图 14-8 所示：

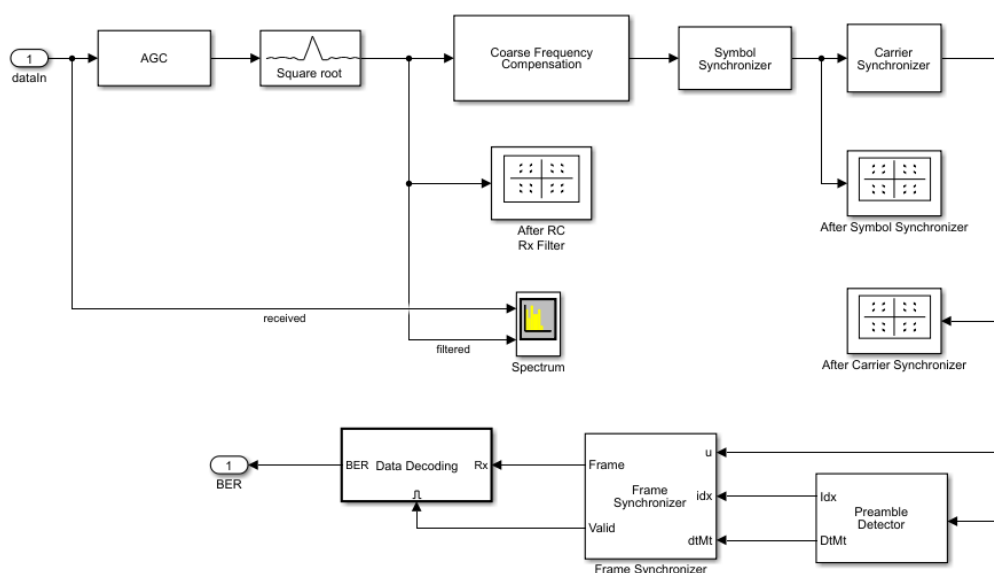


图 14-8 接收模块

其中接收模块中的很大一部分，我们在上一章当中的实验已经有过了解，

分别用于完成同步工作。

然后会使用前导码检测模块，前导码检测模块使用已知的帧头，也就是前面介绍的巴克码，与接收到的 QPSK 符号进行相关，用来找到一帧消息的边界。

帧同步模块使用前导码检测模块得到的结果来完成数据帧的对齐。并将符号转换为固定大小的帧，提供给下游模块，并且会给出一个 **valid** 信号来标志每一帧是否有效。

最后，同步帧将会进入到数据解码模块当中

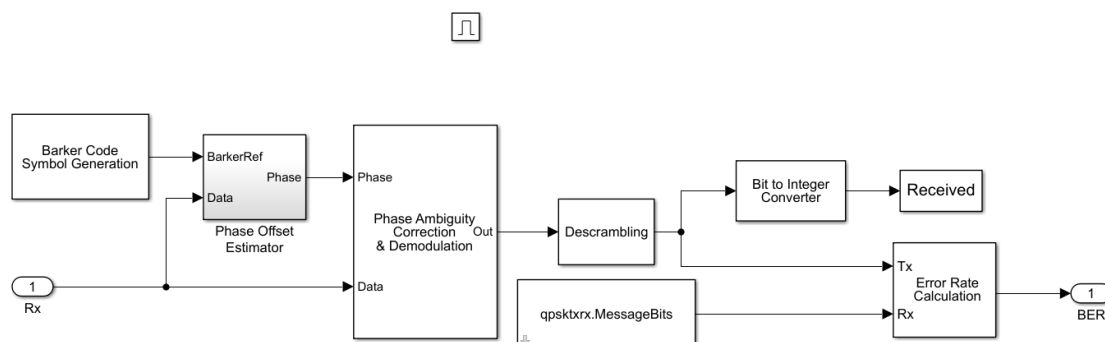


图 14-9 数据解码模块

最终，该模块能够将解码得到的数据保存到工作区，并且会计算出误码率。现在点击运行，就可以运行这个仿真模型。

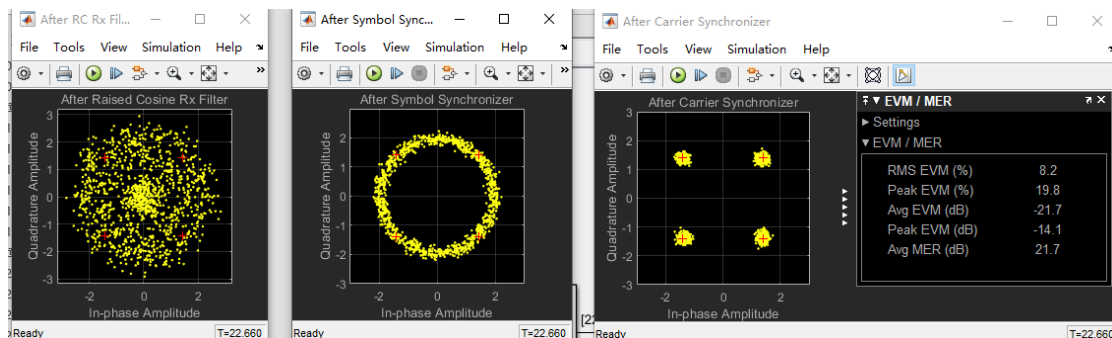


图 14-10 仿真结果

从仿真结果上看，经过同步之后，星座图上的点已经能够很好地分布在参考点附近了。

然后我们观察窗口中打印出来的信息，如图 14-11 所示，可以打开打印窗口，来查看打印的信息。

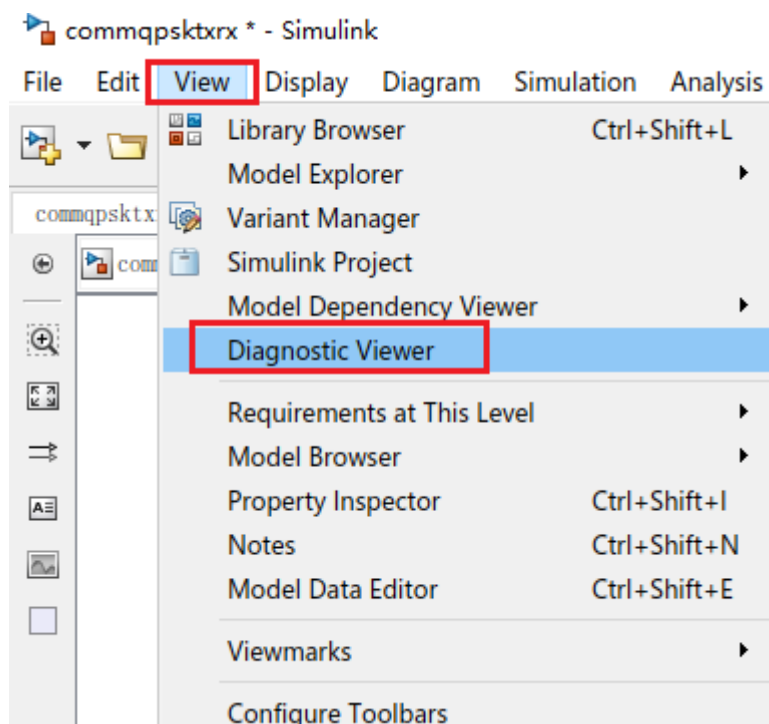


图 14-11

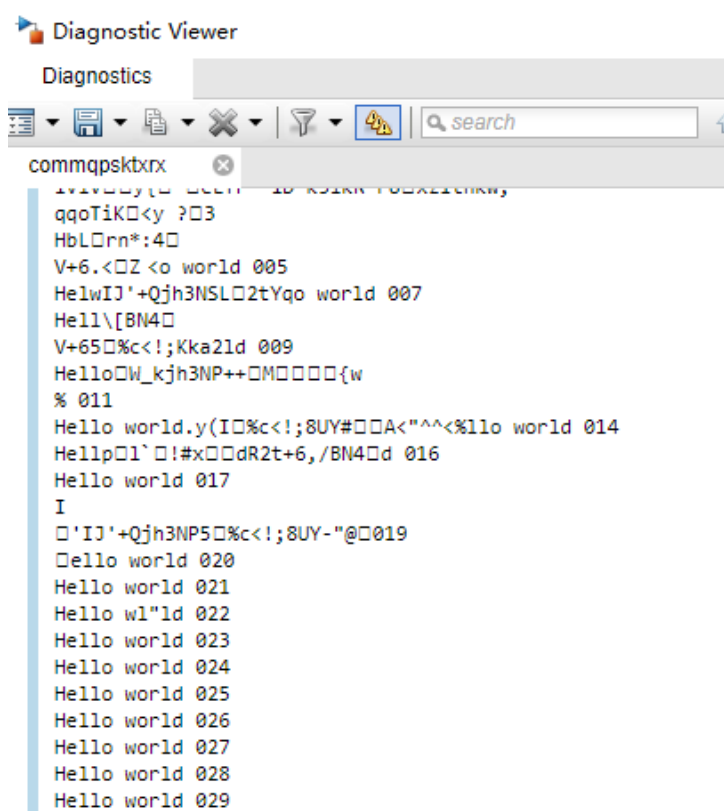


图 14-12

可以看到，在还没有完成同步的时候，解调出来的信息存在乱码，当整个接

收机对信号进行同步之后，就能够正确地解调出消息信号了。

14.2 使用 ANTSDR 来发送消息

在前面我们使用系统中自带的例子完成了 ASCII 字符的发送，在接下来我们就可以在 ANTSDR 来完成 QPSK 发送 ASCII 字符的实验了。我们在官方的参考例子上进行修改的话会十分的简单。

打开仿真模型，`antsdr_qpsk_ascii.slx`，你可以看到仿真模型和官方的参考例子基本一致。

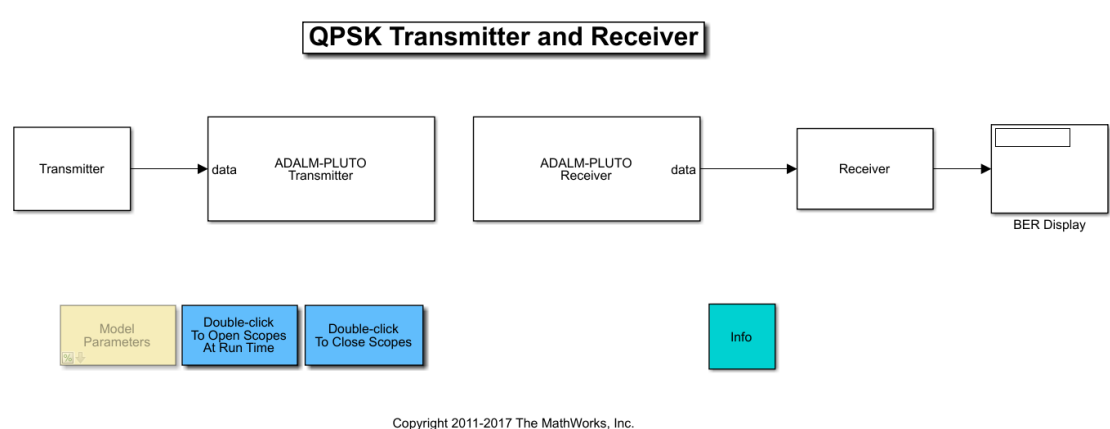


图 14-13 仿真模型

在这个仿真模型中，我们只需要对 `antsdr` 进行正确的配置就可以了，对于发射机，其配置如图 14-14 所示。

对于接收机，其配置如所示。

ADALM-PLUTO Radio Transmitter

Transmit data to an ADALM-PLUTO radio.

Main Advanced

Radio Connection

RadioID: usb:0

Info

Radio Properties

Source of center frequency: Dialog

Center Frequency (Hz): 2.4e9

Source of gain: Dialog

Gain (dB): -10

Channel mapping: 1

Baseband sample rate (Hz): 100e3

Data

☐ Enable output port for underflow indicator

OK Cancel Help Apply

图 14-14 发射机设置

ADALM-PLUTO Radio Receiver

Receive data from an ADALM-PLUTO radio.

Main Advanced

Radio Connection

RadioID: usb:0

Info

Radio Properties

Source of center frequency: Dialog

Center Frequency (Hz): 2.4e9

Source of gain: AGC Slow Attack

Channel mapping: 1

Baseband sample rate (Hz): 100e3

Data

Output data type: double

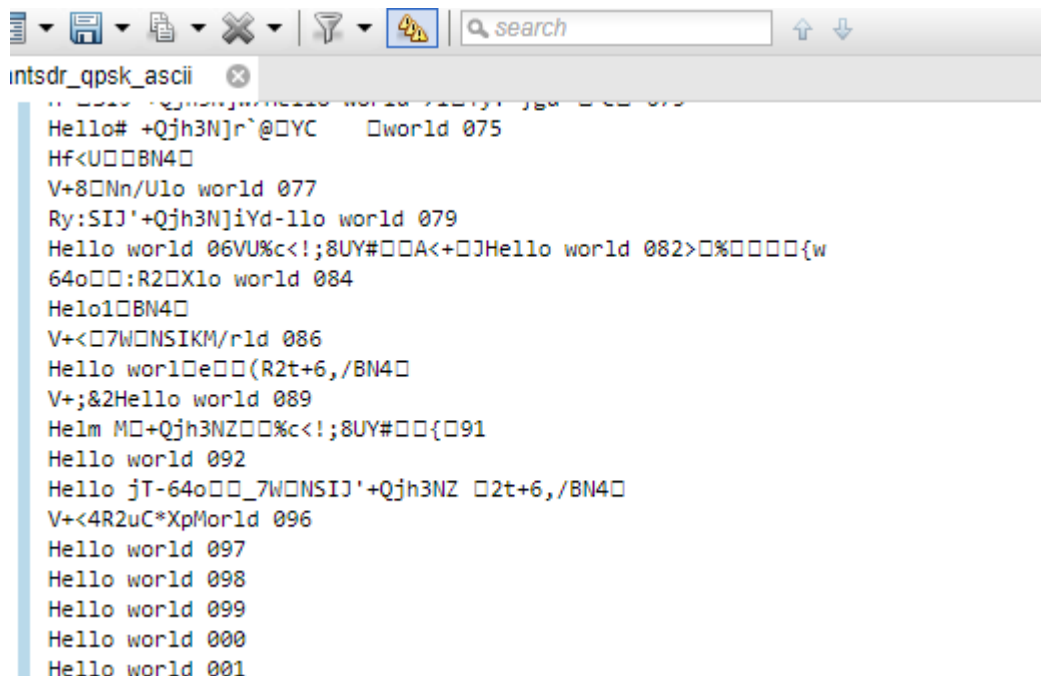
Samples per frame: 2266

☐ Enable output port for overflow indicator

OK Cancel Help Apply

图 14-15 接收机设置

设置完成之后，我们就可以来运行仿真模型了。从打印信息上看，使用 ANTSDR 进行 ASCII 字符传输的时候，在一开始的错误码会比较多，但随着同步的进行，可以看到最终能够正确解析出所发送的字符。



```

intsd_r_qpsk_ascii
Hello# +Qjh3N]r`@YC   world 075
Hf<U   BN4
V+8   Nn/Ulo world 077
Ry:SIJ'+Qjh3N]iYd-llo world 079
Hello world 06VU%c<!!;8UY#   A<+   JHello world 082>%   {w
64o   :R2   Xlo world 084
Helo1   BN4
V+<   7W   NSIKM/rld 086
Hello worlde   (R2t+6,/BN4
V+;&2Hello world 089
Helm M+Qjh3NZ   %c<!!;8UY#   {   91
Hello world 092
Hello jT-64o   _7W   NSIJ'+Qjh3NZ   2t+6,/BN4
V+<4R2uC*XpMorld 096
Hello world 097
Hello world 098
Hello world 099
Hello world 000
Hello world 001
  
```

图 14-16

14.3 总结

在本章当中，我们使用 Matlab 自带的 QPSK 的例子，实现了一个基于 QPSK 的文字发送系统，使用这个系统能够来模拟实际中的数字信号的发射和解调同步等。

然后我们使用 ANTSDR，基于上面的例子，实现在真实的无线信道之中来发射消息信号，并最终在 Simulink 中恢复除了所发送的例子。

在真实的信道当中，所接收到的消息信号在未同步的时候，会存在更多的错误信息，但是当同步之后，使用 ANTSDR 就能够稳定地运行了。

总 结

到这里基于 ANTSDR 的 Matlab & Simulink 开发手册就完结了，最为初次编写的文档，由于本人水平有限，文档中难免会出现错误与疏忽，若您发现了文档当中出现的错误还请与我们联系，作为一款开源的 SDR 设备，您的反馈将会对我们提供莫大的帮助。

参 考

- Software Defined Radio using MATLAB & Simulink and the RTL-SDR
- Software-Defined Radio for Engineers
- 深入浅出通信原理
- 大话无线通信